



// Master in Computer Engineering

Infrastructures for Cloud Computing and Big Data

Andrea Sabbioni, Antonio Corradi

by Matteo Fontolan



// 2025/2026

Disclaimer & Info

This document is a personal collection based on lectures and/or educational materials provided by professors and/or collected over the years by students.

! Attention

The author assumes no responsibility for any errors, omissions, or inaccuracies. The material is provided "as is" for educational support purposes only.

If you want to **add** useful material or **report** errors, please do so [here](#).

I hope this resource proves useful to you.
Good study and good luck! 🐛

If you'd like to support me with a chocolate 🍫, you can do so [here](#).

[Bento Profile](#)

Table of Contents

1. INTRODUCTION	18
1.1. The Cloud	18
1.2. DC Physical Architecture	19
1.3. DC Network Architecture	20
1.3.1. Traditional 3-Tier Architecture	20
1.3.2. Modern Leaf-Spine Architecture	20
1.4. Compute Architecture Inside a DC	21
1.5. Hyperscalers and Sustainability	22
1.6. Large Cloud Ecosystem	22
1.7. Big Data	24
1.7.1. The 5 (or 6) Vs	24
1.8. Application Scenarios	25
1.8.1. Smart Cities	25
1.8.2. Industry 4.0 and 5.0	25
1.9. Quality of Service	25
1.9.1. Old World vs New World	26
1.9.2. Supporting QoS	26
1.10. Middleware	26
1.10.1. Middleware Families	27
1.10.2. Major Use Cases	27
1.11. Cloud as an Evolution	27
1.12. Roadmap	27
2. GOALS, BASICS, MODELS	28
2.1. Modern Distributed Systems	28
2.2. ICCBD Approach: Resources and APIs	28
2.3. Properties of Large Systems	28
2.3.1. Abstraction	29
2.3.2. Transparency vs Visibility	29
2.4. TINA-C	29
2.4.1. Roles and Views	30
2.4.2. Architectural Layer View	30
2.4.3. Transparent vs. Non-Transparent Architecture	31
2.5. Monitoring and Observability	31
2.5.1. Minimal Intrusion Principle	32
2.6. Granularity	32
2.7. Service-Oriented Architecture (SOA)	33
2.7.1. The 3 Actors of SOA	34
2.7.2. Granularity problem	34
2.8. Enterprise Application Integration (EAI)	35
2.8.1. Traditional vs SOA-oriented Architecture	35
2.9. Technology Life Cycles: The Gartner Hype Cycle	35
2.10. Resources in Distributed Systems	36
2.10.1. Classification of resources	36
2.11. Objects, Components, and the Path to Services	36
2.11.1. From Objects to Components	36
2.11.2. Interfaces vs. Classes	37

2.12.	Containers	37
2.12.1.	Container Delegation	38
2.13.	DevOps	38
2.14.	Microservices	39
2.15.	Docker and Modern Containers	40
2.15.1.	Docker Architecture	40
2.15.2.	Docker Containers vs. Virtual Machines	41
2.15.3.	Docker Container Lifecycle	42
2.15.4.	Kubernetes	42

3. RESOURCE MANAGEMENT MODELS 43

3.1.	Quality of Service (QoS)	43
3.1.1.	Functional vs. Non-Functional Properties	43
3.1.2.	QoS Verification and Control	44
3.1.3.	The 3 Planes	44
3.1.4.	Service Level Agreement (SLA)	45
3.2.	General Resource Management Models	45
3.2.1.	Preventive vs. Reactive Models	46
3.2.2.	Static vs. Dynamic Models	46
3.3.	Application Deployment	46
3.3.1.	Process Allocation	47
3.3.2.	Static vs. Dynamic Resources in Applications	47
3.3.3.	Allocation Strategies	47
3.3.4.	Allocation Decision Models	47
3.3.5.	Deployment Support	47
3.3.6.	Modern Deployment: Application + Configuration Together	48
3.4.	Resource Services and Agent Management	48
3.4.1.	Agent-Based Management (DFS)	48
3.5.	Load Sharing and Load Balancing	48
3.5.1.	The Farm Pattern	49
3.5.2.	Static Load Sharing Strategies	49
3.6.	Load Balancing: Process Migration	50
3.6.1.	Internal Migration Problems	51
3.6.2.	External Migration Problems	51
3.6.3.	Migration Policies	51
3.6.4.	Sender vs. Receiver Initiative	52
3.6.5.	Centralized vs. Decentralized Migration	52
3.6.6.	Key Lesson from Migration	52
3.7.	Scalability Problems: Orchestrators	53
3.7.1.	Kubernetes	53
3.8.	Modern Microservice Support: Service Mesh	54
3.8.1.	Istio	54
3.9.	Continuous DevOps (CI/CD)	54
3.10.	Infrastructure as Code (IaC)	55
3.11.	Computational Models for Parallelization	55
3.11.1.	Speed-up and Efficiency	55
3.11.2.	Grosh Law and Loading Factor	56
3.11.3.	Amdahl's Law	56
3.11.4.	Overhead Time	57

3.11.5.	Case Study: Sum of N Integers	57
3.11.6.	More Real Indicators	58
3.11.7.	Iso-Efficiency	58
4.	UNIX FILES AND SINGLE PRIMITIVE ATOMICITY	60
4.1.	UNIX File Primitives	60
4.2.	UNIX File System Architecture	60
4.2.1.	i-nodes and Directories (recalls)	61
4.3.	Consequences of Per-Primitive Atomicity	62
4.3.1.	Non-Determinism Under Concurrent Writes	62
4.3.2.	Directory Listing Under Concurrent Modification	62
5.	COMPONENTS, MICROSERVICES, AND CONTAINERS	64
5.1.	From Monolith to Microservices	64
5.1.1.	The Monolith	64
5.1.2.	Microservices	64
5.1.3.	Benefits of the Microservice Decomposition	65
5.1.4.	Prerequisites Before Adopting Microservices	66
5.2.	Hosting Problems	66
5.3.	Process Isolation	66
5.3.1.	A Brief History of Container Isolation	66
5.3.2.	Linux Namespaces	67
5.3.3.	Linux Control Groups (cgroups)	68
5.4.	Containers	68
5.5.	Docker	69
5.5.1.	Docker Engine Architecture	69
5.5.2.	Docker Images and Layers	70
5.5.3.	Dockerfile	71
5.5.4.	Docker Networks	71
5.5.5.	Docker Plugins	71
5.5.6.	Configuration and Secrets	71
5.5.7.	Docker Compose	72
5.6.	Microservice Resilience Patterns	72
5.6.1.	Circuit Breaker	72
5.6.2.	Implementation	73
6.	MIDDLEWARE AND CLOUD MODELS	74
6.1.	What is Middleware?	74
6.1.1.	Middleware Support Functions	74
6.2.	Middleware in a Layered View	75
6.2.1.	Host Infrastructure Middleware	75
6.2.2.	Distribution Middleware	75
6.2.3.	Common Middleware Services	75
6.2.4.	Domain-specific Middleware Service	76
6.3.	Taxonomy of Middlewares	76
6.3.1.	RPC / RMI Middleware	76
6.3.2.	MOM	77
6.3.3.	DOC / OO Middleware	77
6.3.4.	DTP Monitor	77

6.3.5.	DB Middleware	78
6.3.6.	Adaptive & Reflective Middleware	78
6.3.7.	Self-* Middleware	79
6.3.8.	Specialized Middlewares	79
6.4.	Middleware Usage Scenarios	79
6.4.1.	Scenario 1: Minimum Cost Middleware	79
6.4.2.	Scenario 2: Middleware for Fast Applications	80
6.4.3.	Scenario 3: Middleware for Continuity	80
6.5.	Middleware Design Issues	80
6.6.	The Problem Space	81
6.7.	A Brief History: Before the Cloud	81
6.8.	What is a Cloud?	82
6.8.1.	How Cloud Differs from its Predecessors	82
6.9.	The SaaS / PaaS / IaaS Model	82
6.9.1.	Architecture Comparison: What the Customer Manages	83
6.10.	XaaS: Anything as a Service	84
6.11.	The NIST Cloud Definition Framework	84
6.12.	QoS-Related Properties	85
6.13.	Cloud Architecture	85
6.13.1.	Data Center Organization	85
6.13.2.	Data Center Network Topology	86
6.14.	Cloud Management and Monitoring	87
6.14.1.	Remote Management for QoS	87
6.14.2.	Areas of Cloud Management	87
6.14.3.	Cloud Monitoring	87
6.15.	Middleware for Cloud	87
7.	CLOUD AND DC GLOBAL STRATEGIES	89
7.1.	Cloud Data Center Architecture	89
7.1.1.	Level 1: Cloud Edge	89
7.1.2.	Level 2: Cloud Internal	89
7.1.3.	Two-Level Replication	90
7.2.	Sharding	90
7.3.	Consistency vs. Speed: The Core Tension	90
7.3.1.	Critical Paths and Asynchronous Effects	90
7.3.2.	Is Inconsistency Always Bad?	91
7.4.	! The CAP Theorem	91
7.4.1.	Consistency: Two Perspectives	92
7.5.	ACID Properties	92
7.5.1.	Nested Transactions and Two-Phase Commit	93
7.6.	BASE Properties	94
7.6.1.	BASE Effect in Practice	94
7.6.2.	ACID vs. BASE Summary	94
7.7.	Consistency Spectrum	95
7.7.1.	Eventual Consistency Implementation	95
7.7.2.	Technology Examples and CAP Position	95
7.8.	! eBay Principles: Five Commandments for Internet Scale	95
7.8.1.	Partition Everything	96
7.8.2.	Asynchrony Everywhere	96

7.8.3.	Automate Everything	97
7.8.4.	Everything Fails	97
7.8.5.	Embrace Inconsistency	98

8. CORBA, ADVANCED C/S MODELS, EVENTS AND MOM..... 99

8.1.	Remote References and the Proxy Pattern	99
8.1.1.	Proxy	99
8.2.	CORBA	100
8.2.1.	ORB: Object Request Broker	101
8.2.2.	Object Management Architecture (OMA)	101
8.2.3.	Static vs Dynamic Invocation	102
8.2.4.	Repositories and Inter-ORB Protocols	102
8.2.5.	CORBA IDL	103
8.2.6.	Data Types in CORBA IDL	104
8.2.7.	Language Mapping	104
8.3.	Advanced Client/Server Interaction Models	105
8.3.1.	Delegation Patterns	105
8.4.	Message Exchange	106
8.4.1.	Modes of Message Exchange	106
8.4.2.	Coupling and Decoupling	107
8.5.	Events and Publish-Subscribe	107
8.5.1.	From Local to Distributed Events	107
8.5.2.	Evolution of Events	107
8.5.3.	Publish-Subscribe	108
8.6.	Tuple Model: Linda (Gelernter)	109
8.7.	MOM: Message Oriented Middleware	109
8.7.1.	MOM Deployment	110
8.7.2.	MOM Infrastructure Components	110
8.7.3.	MQSeries IBM	111
8.8.	Apache Kafka	111
8.8.1.	Kafka Pub/Sub Model	112
8.8.2.	Topics, Partitions, and Brokers	112
8.8.3.	Kafka QoS	113
8.8.4.	Kafka Producers and Consumers	114
8.9.	MQTT: Message Queue Telemetry Transport	114
8.10.	OPC UA: Open Platform Communications Unified Architecture	115
8.11.	Group Communication and Multicast	115
8.11.1.	Group Communication Semantics	115
8.11.2.	IP Group Communication	116
8.11.3.	IGMP: Local Group Management	116
8.11.4.	Global Multicast Routing	117
8.11.5.	Multicast Spanning Tree	117
8.11.6.	Pruning and Grafting	117
8.11.7.	Multicast Protocols	118

9. KUBERNETES..... 119

9.1.	What is Kubernetes	119
------	--------------------------	-----

9.1.1.	From Docker to Kubernetes	119
9.2.	Cluster Architecture	120
9.2.1.	Control Plane	120
9.2.2.	Worker Node Components	120
9.3.	The Declarative Model	121
9.3.1.	Kubernetes Resources	121
9.4.	Controllers and the Control Loop	122
9.4.1.	Controller-ETCD Interaction	122
9.4.2.	The Operator Pattern	123
9.5.	etcd and RAFT	124
9.5.1.	etcd	124
9.5.2.	RAFT	124
9.6.	Nodes, Pods, and Workloads	126
9.6.1.	Nodes	126
9.6.2.	Pods	126
9.6.3.	Scheduler	127
9.6.4.	Kubelet	127
9.6.5.	Workload Resources	127
9.7.	Storage	129
9.8.	Networking	129
9.8.1.	The Kubernetes Network Model	129
9.8.2.	Kube-Proxy	130
9.8.3.	CNI Plugins	130
9.8.4.	Services	131
9.8.5.	Ingress	131
9.9.	Load Balancing and Autoscaling	132
9.9.1.	Horizontal Pod Autoscaler (HPA)	132
9.9.2.	KEDA: Kubernetes Event-Driven Autoscaling	133
9.10.	Service Mesh	133
9.10.1.	What is a Service Mesh?	133
9.10.2.	Service Mesh Architecture	134
9.10.3.	Traffic Management	135
9.10.4.	Observability	135
9.10.5.	Security	136
9.10.6.	Dev/Ops Capabilities	136
9.10.7.	Service Mesh at Meta (Case Study)	136
10.	OPENSTACK	137
10.1.	Cloud Data Centres and IaaS	137
10.1.1.	The Cloud Data Centre	137
10.1.2.	Cloud Resource Virtualization	137
10.2.	OpenStack Overview	137
10.2.1.	History and Positioning	137
10.2.2.	High-Level Architecture	138
10.2.3.	Logical Layers	138
10.2.4.	Design Guidelines	138
10.2.5.	Common Service Architecture	139
10.3.	OpenStack Core Services	139
10.4.	Nova - Compute Service	140

10.4.1.	What Nova Does	140
10.4.2.	Nova Internal Components	140
10.4.3.	Nova General Interaction Pattern	141
10.4.4.	VM Provisioning Workflow (Multi-Service)	141
10.5.	Swift - Object Storage	142
10.5.1.	What Swift Does	142
10.5.2.	Object Storage Data Model	142
10.5.3.	Swift Components	142
10.6.	Cinder - Block Storage	143
10.6.1.	What Cinder Does	143
10.6.2.	Cinder Components	143
10.7.	Glance - Image Service	144
10.7.1.	What Glance Does	144
10.7.2.	Glance Components	144
10.8.	Horizon - Dashboard	145
10.9.	Keystone - Identity Service	145
10.9.1.	What Keystone Does	145
10.9.2.	Keystone's Four Sub-Services	145
10.9.3.	Keystone Request Flow	145
10.10.	Neutron - Networking Service	146
10.10.1.	What Neutron Does	146
10.10.2.	Logical vs. Physical View	146
10.10.3.	Neutron Components	146
10.10.4.	Neutron Agents	147
10.10.5.	Tenant Networks	147
10.11.	Ceilometer and Heat	148
10.11.1.	Ceilometer - Telemetry	148
10.11.2.	Heat - Orchestration	148
10.12.	Putting It All Together	148

11. SERVERLESS COMPUTING 149

11.1.	Cloud Computing Service Models	149
11.1.1.	Serverless: Definitions	149
11.1.2.	Serverless as BaaS + FaaS	149
11.2.	Backend as a Service (BaaS)	150
11.3.	Function as a Service (FaaS)	150
11.3.1.	Definition and Properties	150
11.3.2.	FaaS as Finer Decomposition	151
11.3.3.	FaaS Advantages	151
11.3.4.	Positioning FaaS in the Cloud Stack	151
11.4.	FaaS Architecture	152
11.4.1.	FaaS Workflow	152
11.4.2.	Essential Components of a FaaS Platform	152
11.4.3.	The Trigger	152
11.4.4.	The Controller	153
11.4.5.	The Function Executor	153
11.4.6.	Platform Architecture (detailed)	154
11.5.	FaaS Workflows and Function Composition	154
11.5.1.	Workflows of Functions	154

11.5.2.	Core Workflow Components	154
11.5.3.	Composition Patterns	155
11.5.4.	Example: E-Commerce Order Workflow	156
11.6.	FaaS Platforms	156
11.6.1.	Hosted vs. Installable Platforms	156
11.7.	OpenFaaS	157
11.7.1.	Architecture Overview	157
11.7.2.	OpenFaaS Internal Flow	157
11.7.3.	OpenFaaS: Gateway	157
11.7.4.	OpenFaaS: Prometheus (Monitoring)	158
11.7.5.	OpenFaaS: NATS (Messaging)	158
11.7.6.	OpenFaaS: Providers	158
11.7.7.	OpenFaaS: Watchdog (Invoker)	159
11.7.8.	OpenFaaS: faasd	160
11.8.	Apache OpenWhisk	160
11.9.	Knative	161
11.9.1.	Overview	161
11.9.2.	Knative Serving Architecture	162
11.9.3.	Knative Serving: Resource Model	162
11.9.4.	Knative Serving: Orchestration via Revisions	163
11.9.5.	Knative Request Flow	163
11.9.6.	Knative Serving vs. Eventing	163
11.10.	Knative Eventing	164
11.10.1.	Motivation	164
11.10.2.	Eventing Components	164
11.10.3.	Eventing Interface Contracts	165
11.10.4.	Eventing: Broker and Trigger	165
11.10.5.	Eventing: Messaging Components	166
11.10.6.	Knative Eventing: Summary	166
11.10.7.	Knative Perspectives	166
11.11.	Choosing the Right Model	167

12. STREAM PROCESSING.....168

12.1.	Motivation and Challenges	168
12.1.1.	Why Streaming?	168
12.1.2.	Why Not MapReduce / Batch?	168
12.1.3.	Problems with Message-Passing Dataflows	168
12.2.	The Stream Processing Model	169
12.2.1.	Support Functions	169
12.3.	Spark Streaming	169
12.3.1.	Discretized Stream Processing	169
12.3.2.	DStream: The Core Abstraction	170
12.3.3.	DStream Operations	170
12.3.4.	Fault Tolerance in Spark Streaming	171
12.3.5.	Requirements Summary	171
12.4.	Apache Flink	171
12.4.1.	Core Features	171
12.4.2.	Flink Architecture	172
12.4.3.	Pipelining	173

12.5.	Streaming Semantics and Fault Tolerance	173
12.5.1.	Processing Guarantees	173
12.5.2.	Flink's Distributed Snapshots	173
12.6.	Comparison: Spark Streaming vs. Flink	175
13.	REPLICATION FOR DEPENDABILITY	177
13.1.	Core Concepts: Dependability and Faults	177
13.1.1.	Some Definitions	177
13.1.2.	Faults, Errors, and Failures	178
13.1.3.	Types of Faults	178
13.1.4.	Services Unavailability	178
13.1.5.	Service Unavailability Indicators	178
13.1.6.	Failure Costs	179
13.2.	Fault Identification and Recovery	179
13.2.1.	Fault Identification in Client/Server Systems	179
13.2.2.	Single Point of Failure (SPoF)	179
13.2.3.	Single Fault Assumption	180
13.2.4.	Fault Assumptions for Communicating Processors	180
13.2.5.	Fault Assumptions and Number of Processors	180
13.2.6.	Distributed Systems Fault Assumptions	181
13.3.	High-Level Goals and Formal Properties	181
13.3.1.	Availability and Reliability	181
13.3.2.	Correctness and Vitality	181
13.4.	Fault-Tolerance Architectures	182
13.4.1.	Replicated Components	182
13.4.2.	Stable Memory	182
13.4.3.	Stable Memory: Support Protocols	182
13.4.4.	TANDEM Systems	183
13.4.5.	RAID: Redundant Array of Inexpensive Disks	183
13.5.	Fault Tolerant Support: Costs and Principles	184
13.5.1.	Fault Tolerant Support Overview	184
13.5.2.	High Replication Costs	184
13.6.	Resource Management and Replication Architectures	185
13.6.1.	Resource Management with Replicated Resources	185
13.6.2.	Abstract Unique Resource Model	185
13.6.3.	Replication Architectures: Passive Model (Master-Slave)	185
13.6.4.	Active Replication Model	185
13.6.5.	Master-Slave: Architecture	186
13.6.6.	Checkpoint: Slave Checkpoint	186
13.6.7.	Master-Slave: Checkpoint Timing	187
13.7.	Active Replication: Models and Coordination	187
13.7.1.	Active Replication Model Details	187
13.7.2.	Active Copies Replication: Manager Strategy	187
13.7.3.	Active Copies Coordination	188
13.7.4.	Copies Coordination: Read/Write Actions	188
13.7.5.	Active Copies Updating	188
13.7.6.	Active Copy Agreement	188
13.8.	The Five Phases of Replication Operation	189
13.8.1.	Overview	189

13.8.2.	Phase 1: Client Request	189
13.8.3.	Phase 2: Copy Coordination	189
13.8.4.	Phase 3: Copy Execution	189
13.8.5.	Phase 4: Copy Agreement	189
13.8.6.	Phase 5: Result Delivery	190
13.8.7.	Observations on Active Copies Operations	190
13.8.8.	Active Copy Operations Classification	190
13.8.9.	Eager Primary Copy	190
13.8.10.	Lazy Primary Copy	190
13.8.11.	Update of Active Copies	191
13.8.12.	Optimistic Eager Update	191
13.8.13.	Pessimistic Eager Update	191
13.9.	Replication Forms and Widespread Models	191
13.9.1.	Replication Forms Summary	191
13.9.2.	Widespread Replication Models	192
13.10.	Industrial Operations and Evolution	192
13.10.1.	Industrial Safety Evolution	192
13.11.	Case Study: ALMA Web Service Architecture	192
13.11.1.	Small-Scale Example	192
13.11.2.	ALMA Web Service Architecture	192
13.12.	High Availability Clusters	193
13.12.1.	High Availability Cluster	193
13.12.2.	Cluster Support Operations	193
13.12.3.	Cluster: Failover and Heartbeat	194
13.12.4.	Storage Area Network (SAN)	194
13.12.5.	Red Hat Cluster	194
13.13.	Optimistic Lazy Policies and Eventual Consistency	194
13.13.1.	Optimistic Lazy Policies	194
13.13.2.	Eventual Consistency	195
13.13.3.	Amazon S3: Optimistic Lazy Policies	195
13.14.	Docker Swarm and Modern Replication	195
13.14.1.	Docker Swarm	195
13.15.	Apache ZooKeeper	196
13.15.1.	ZooKeeper Overview	196
13.15.2.	ZooKeeper Architecture	197
13.15.3.	ZooKeeper Reads and Writes	197
13.15.4.	ZooKeeper Leader Election	198
14.	GROUP ISSUES AND POLICIES	199
14.1.	Partitioning and Groups	199
14.2.	Group Communication	199
14.3.	Reliable Multicast	200
14.3.1.	Implementation Decisions	200
14.4.	Multicast Ordering	200
14.4.1.	No Ordering	200
14.4.2.	FIFO Ordering	201
14.4.3.	Causal Ordering	201
14.4.4.	Atomic Ordering	201
14.4.5.	Cost of Orderings	202

14.5.	Synchronization	202
14.5.1.	Clock Synchronization	202
14.5.2.	Synchronization Strategies	203
14.6.	Lamport Relationship	203
14.6.1.	Happened-Before Relationship	204
14.6.2.	Logical Clock and Timestamp	204
14.6.3.	Total Ordering	205
14.6.4.	Vector Clocks	205
14.7.	Mutual Exclusion and Synchronization	205
14.7.1.	Centralized Coordinator	206
14.7.2.	Lamport Synchronization	206
14.7.3.	Ricart-Agrawala Protocol	206
14.8.	Atomic Multicast	207
14.8.1.	ISIS: Atomic Multicast	207
14.8.2.	ISIS ABCast	207
14.8.3.	ISIS CBCast	208
14.8.4.	ISIS GBCast	208
14.8.5.	JGROUPS: Reliable Multicast in Java	208
14.9.	Apache ZooKeeper	209
14.10.	Token-Based Synchronization	209
14.10.1.	Recovery in a Ring	209
14.10.2.	Token Recovery	209
14.11.	Election Protocols	209
14.11.1.	Bully Algorithm	210
14.11.2.	Election in a Ring	210
14.12.	Global State	210
14.12.1.	Global States Consistency	211
14.12.2.	Global State via Snapshot	211
15.	QOS: QUALITY OF SERVICE	212
15.1.	QoS in Different Environments	212
15.1.1.	Application Classification	212
15.2.	QoS Indicators	213
15.2.1.	Latency	213
15.2.2.	Jitter and Skew	214
15.2.3.	User-Level QoS Indicators	214
15.3.	QoS Management	215
15.3.1.	Active Path	215
15.3.2.	SLA Negotiation Example	215
15.4.	Video Streaming Services	216
15.4.1.	Three Operational Planes	216
15.4.2.	RTSP: Real-Time Streaming Protocol	216
15.5.	IntServ: Integrated Services (RFC 2210)	217
15.5.1.	RSVP: Resource Reservation Setup Protocol (RFC 2205)	218
15.5.2.	RTP: Real-Time Transport Protocol (RFC 1889)	219
15.5.3.	RTCP: Real-Time Transport Control Protocol	220
15.6.	DiffServ: Differentiated Services (RFC 2474/2475)	220
15.6.1.	QoS Traffic Conditioning	221
15.6.2.	IntServ vs. DiffServ	222

15.6.3.	IntServ + DiffServ Together	222
15.7.	SIP: Session Initiation Protocol (RFC 2543/3261)	222
15.8.	Network Management	223
15.8.1.	SNMP: Simple Network Management Protocol	223
15.8.2.	RMON: Remote Monitor	224
15.8.3.	OSI Advanced Network Management	224
15.9.	Router QoS Policies	225
15.9.1.	Best-Effort Router (FIFO)	225
15.9.2.	Kleinrock Conservation Law	225
15.9.3.	Bucket Models for Traffic Shaping	225
15.9.4.	Scheduling Policies	226
15.9.5.	Congestion Prevention	228
15.9.6.	Service Levels Summary	228
16.	OVERLAY NETWORKS AND FILE SYSTEMS	229
16.1.	Classification of Overlay Networks	229
16.1.1.	Overlay Network Usage and Node Lifecycle	229
16.1.2.	Key Management Requirements	230
16.2.	Unstructured Overlays	230
16.2.1.	Napster (1991): Centralized Lookup	230
16.2.2.	Gnutella (2000): Flooding-Based Unstructured ON	231
16.3.	Structured Overlays: Distributed Hash Tables	232
16.3.1.	Key DHT Principles	232
16.3.2.	Chord	233
16.3.3.	Pastry	234
16.3.4.	DHT Applications in Practice	235
16.4.	Distributed File Systems	236
16.4.1.	NFS: Network File System	236
16.4.2.	AFS: Andrew File System - Quality File Systems	236
16.4.3.	Modern Global File Systems	236
16.5.	Google File System (GFS)	237
16.5.1.	GFS Design Assumptions	237
16.5.2.	GFS Design Novel Strategies	237
16.6.	HDFS: Hadoop Distributed File System	239
16.6.1.	HDFS Architecture	239
16.6.2.	HDFS Replication	240
16.7.	Summary: Properties Behind These Building Blocks	240
17.	DATA STORAGE	241
17.1.	The NoSQL Movement	241
17.1.1.	NoSQL Motivations	241
17.1.2.	Requirements of Today's Workloads	242
17.1.3.	Different NoSQL Data Models	242
17.2.	Key-Value Abstraction	242
17.2.1.	Key-Value / NoSQL Data Model vs. RDBMS	243
17.2.2.	Column-Oriented Storage	243
17.3.	Cassandra	244
17.3.1.	Cassandra Data Center Model	244
17.3.2.	Cassandra Architecture	244

17.3.3.	Key-to-Server Mapping: The Ring	244
17.3.4.	Cassandra Keyspaces	245
17.3.5.	Data Placement Strategy	245
17.3.6.	Snitches	246
17.3.7.	Write Operations	246
17.3.8.	Writes at a Replica Node	246
17.3.9.	Bloom Filter	246
17.3.10.	Compaction	247
17.3.11.	Read Operations	247
17.3.12.	Cluster Membership and Failure Detection	247
17.3.13.	Eventual Consistency and ACID vs. BASE	248
17.3.14.	Cassandra Consistency Levels	248
17.3.15.	Quorums in Detail	249
17.3.16.	Performance: Cassandra vs. MySQL	249
17.4.	MongoDB	249
17.4.1.	MongoDB Data Model	249
17.4.2.	MongoDB Queries	250
17.4.3.	MongoDB Distributed Architecture	250
17.4.4.	MongoDB Replication	250
17.4.5.	MongoDB Read Preferences and Write Concern	250
17.4.6.	MongoDB Balancing and Consistency	251
17.5.	ACID vs. BASE: A Fundamental Trade-Off	251
17.6.	Big Data Infrastructure Properties	251

18. DATA BATCHING 252

18.1.	Big Data Characteristics and Requirements	252
18.2.	Batch Data Processing in Large Clusters	252
18.2.1.	Data Parallelism in Today's Large Clusters	253
18.3.	MapReduce: Programming Model	253
18.3.1.	Functional Language Origins	254
18.3.2.	The Google MapReduce Definition	255
18.3.3.	Map Phase in Detail	255
18.3.4.	Reduce Phase in Detail	255
18.3.5.	Running a MapReduce Program	256
18.3.6.	Word Count Example (Full Code)	256
18.3.7.	Other MapReduce Applications	256
18.4.	MapReduce Implementation and Architecture	256
18.4.1.	Implementation at Google	256
18.4.2.	Architecture: Master and Workers	257
18.4.3.	Scheduling and Execution	257
18.4.4.	Favouring Data Locality	257
18.4.5.	Fault Tolerance	257
18.4.6.	Backup Tasks (Stragglers)	258
18.5.	Hadoop: The Open-Source MapReduce	258
18.5.1.	YARN Resource Manager	258
18.5.2.	YARN Architecture	259
18.5.3.	YARN Workflow	259
18.5.4.	Hadoop on OpenStack (Sahara)	259
18.6.	Apache Spark	259

18.6.1.	Why Spark?	259
18.6.2.	Spark Basics	260
18.6.3.	Resilient Distributed Datasets (RDDs)	260
18.6.4.	RDD Operations	261
18.6.5.	Persisting RDDs	261
18.6.6.	Fault Tolerance via Lineage	262
18.6.7.	Spark Performance: Why It Wins on Iterative Workloads	262
18.6.8.	Other Spark Engine Features	262
18.6.9.	Spark Architecture	263
18.6.10.	Spark Deployment Modes	263
18.6.11.	Spark Provisioning and Output	263
18.7.	The Big Data Tools Ecosystem	263
18.8.	Big Data Resource Analysis	264
18.8.1.	Resources as Unifying Concepts	264
18.8.2.	IT Properties Required	264





1. INTRODUCTION

Package: Class Starting - 1-Starting26.pdf

The course builds a deep, practical understanding of **large distributed systems**, approached from two angles:

- The **user perspective**: what services are offered and consumed.
- The **designer/implementer perspective**: what is behind the architecture, how it behaves, how it is managed.

The **focus is on runtime operations across the full system lifecycle**, not just static design or configuration. The idea is that execution time dominates the life of any system, and that is where the most interesting engineering challenges lie.

We live immersed in distributed IT infrastructure, personally, socially, and professionally. Every organization, regardless of its attitude toward ICT (*Information and Communication Technology*), has consolidated usage of **off-premises resources**.

Key features of today's environments:

- **Big data** available to anyone
- Processing distributed across many **remote** localities
- Large-scale applications running continuously

Both companies and individuals share the same tools and requirements. The IT landscape has become **homogeneous**.

1.1. The Cloud



Cloud

A **cloud** is a **coordinated network of multiple geographically distributed data centers** that work together to provide unified, high-quality services to users.

Data centers must be far apart enough to:

- Survive local disasters **independently**,
- Provide geographically **replicated** services.

A **connected multi data center cloud is far more efficient and sustainable than any isolated facility**.

Benefit	Explanation
Workload shifting	Move tasks to greener or cheaper regions. <i>e.g., wind energy peaks in Denmark at night</i>
Higher utilization	Consolidate loads, turn off idle servers.
Cooling efficiency	Route workloads to colder climates or facilities with free cooling.
Redundancy without overprovisioning	Spread failover capacity across sites instead of duplicating hardware everywhere.
AI-driven global optimization	Predictive maintenance, self-optimizing behavior.



Lifecycle & circular economy	Reuse hardware across regions, reduce e-waste.
Edge + core coordination	Process data locally, send only results to core DCs ⇒ less energy.

! Problem:

Cloud and Big Data infrastructure consumption is enormous and growing:

- Global data centers consume around **200 TWh** of **electricity** per year, that is comparable to entire industrialized countries.
- Cloud, Big Data, and AI workloads are increasing **water usage** (for cooling) and the overall **carbon footprint**.
- Exponential growth of digital services makes redesigning infrastructure a necessity

Main challenges: **Energy Consumption, Water Consumption, CO₂ Emissions.**

✓ Solution:

- **Renewable energy:** solar, wind, Power Purchase Agreements (PPAs), 24/7 carbon-free energy matching.
- **Low-impact cooling,** such as free cooling (outside air), liquid cooling, hot/cold aisle containment, AI-driven thermal management.
- **Workload optimization** shifting workloads based on renewable energy availability.
- Recover waste heat for district heating, refurbish/reuse servers (circular economy).
- More transparent metrics (like PUE and WUE).

1.2 DC Physical Architecture

- **Racks** are vertical cabinets holding servers (computers), storage arrays, network switches, and PDUs (Power Distribution Units).
- **Rows:** racks are arranged in a **hot aisle / cold aisle** configuration to optimize airflow. Cold air enters from one aisle, is consumed by servers, and exits as hot air through the opposite aisle.
- **Rooms / Pods:** rows are grouped into isolated cooling zones, each with its own power feed and network aggregation layer.

👉 Note: Physical layout directly affects cable length, airflow efficiency, and energy consumption.

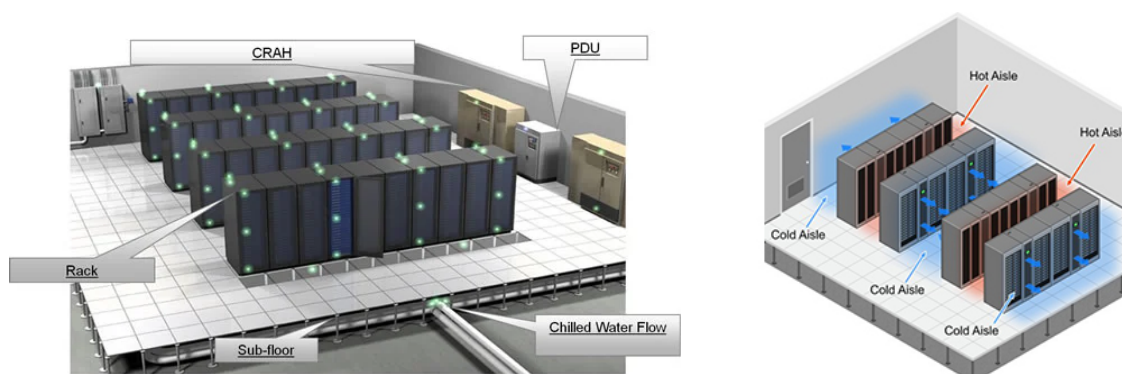


Figure 1 – Data center physical architecture.

- **UPS (Uninterruptable Power Supplies):** backup power, either centralized or at rack level.
- **PDUs:** distribute power inside racks.



- **Cooling systems:** often the largest non-IT energy cost. Technologies include CRAC, CRAH units, in-row cooling, and liquid cooling.

1.3. DC Network Architecture

Before shifting to the architecture, we must introduce the concept of east-west traffic and north-south traffic.



East-West Traffic

East-West traffic is the data flow between servers within the same data center:

- server ↔ server,
- server ↔ storage,
- GPU cluster communication.



North-South Traffic

North-South traffic is the data flow between the data center and external entities, that passes through core switches:

- server ↔ client.

Less dominant in modern architectures.

1.3.1. Traditional 3-Tier Architecture

In *Figure 2*, we see on the left the **traditional architecture** of a data center network, with 3 layers:

- **Core Layer:** high-capacity routers, connects DCs to internet/WAN (and other DCs).
- **Aggregation Layer:** connect multiple rack by providing routing, security and load balancing.
- **Access Layer:** Top-of-Rack (ToR) switches, connect directly to servers, handle local traffic inside the rack.

Pro	Cons
• Simple • Well-understood	• Inefficient for east-west traffic

1.3.2. Modern Leaf-Spine Architecture

The right side of *Figure 2* shows the modern **leaf-spine architecture**, designed to **optimize east-west traffic**, nowadays it is the **dominant model** for cloud and high-throughput workloads.

- **Leaf Switches:** connect directly to servers (like ToR), but also connect to multiple spine switches, creating a full mesh.
- **Spine Switches:** high-capacity switches that interconnect all leaf switches (intra-rack traffic).

Every server can reach every other server in exactly **2 hops** (leaf → spine → leaf), giving:

- Predictable, **uniform latency**.
- **High** and **scalable bandwidth**.
- Better energy efficiency per bit.



- Ideal behavior for microservices and distributed systems.

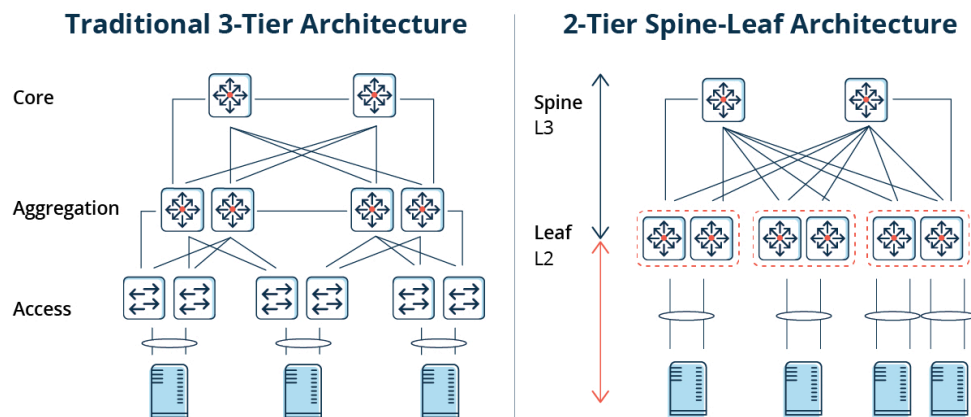


Figure 2 - Data center network architecture layers.

👉 **Note:** Above the two architectures, there is the upper layer with the routers for the Internet and WAN connectivity.

So, the switches and routers inside the two architectures handle intra-DC (east-west) traffic, while those other routers above are for north-south traffic.

In synthesis:

3-Tier	Leaf-Spine
Tree topology	Full Mesh topology
Travel time variable	Travel time fixed (2 hops)
Non uniform latency	Uniform latency
Ideal for C/S (Nord-South)	Ideal for Microservices and Distributed Systems (East-West)

Analogy: DC Networks

Think of the 3-Tier architecture like a traditional city layout with suburbs and a single main highway: if two neighbors in different suburbs want to talk, they must both drive onto the congested main highway (Core layer) to reach each other.

The Leaf-Spine architecture is like a perfect grid-city (Like Manhattan): every street (Leaf) intersects with every avenue (Spine). To get anywhere, you only ever need to make exactly one turn (2 hops), and traffic is spread evenly across all avenues.

1.4. Compute Architecture Inside a DC

Servers are grouped into **clusters** based on **workload type** :

- **Compute clusters:** general-purpose servers with support for VMs, containers, microservices.
- **Storage clusters:** SAN (block-based), NAS (file-based), distributed storage (Ceph, HDFS).
- **GPU/AI clusters:** use high-bandwidth interconnects (such as *InfiniBand*, *NVLink*, *RDMA over Converged Ethernet*).
- **Hyperconverged Infrastructure (HCI):** software-defined framework that unifies compute, storage, networking, and virtualization on commodity hardware into a single system. Think of it as collapsing the entire stack into one manageable unit.

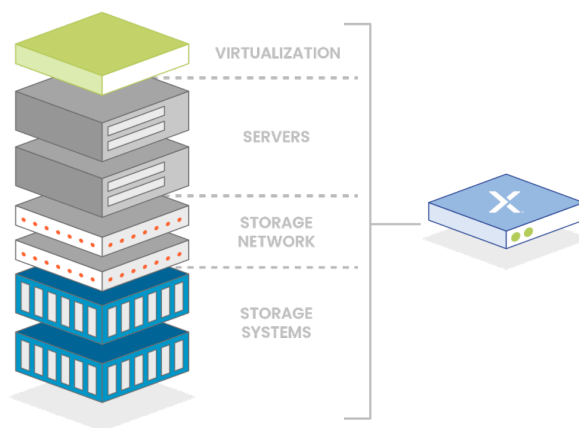


Figure 3 – Hyperconverged infrastructure (HCI)

Different cluster types have different communication patterns and energy profiles.

Analogy: HCI Analogy

Before HCI, building a DC was like buying a separate camera, a GPS navigator, an MP3 player, and a cell phone, and trying to wire them together. HCI is the “Smartphone” approach: compute, storage, and networking are all packaged into a single, standardized building block managed by one operating system. If you need more power, you just buy another smartphone (node) and add it to the cluster.

1.5. Hyperscalers and Sustainability

Hyperscalers (Google, Microsoft, AWS, Meta, IBM...) treat sustainability as a **full-stack engineering requirement**, redesigning every layer:

Layer	Hyperscaler Approach
Energy	Power Purchase Agreements, on-site renewables, 24/7 carbon-free.
Cooling	Free cooling, liquid cooling, AI-driven thermal control.
Hardware	Custom servers, TPUs, efficient power supplies.
Software	Autoscaling, serverless, workload consolidation.
Power distribution	48V DC (vs traditional AC), rack-level batteries, fewer conversion steps.
Location	Climate-optimized siting, renewable-rich regions.
Lifecycle	Recycling, refurbishing, modular construction.

1.6. Large Cloud Ecosystem

Smaller/regional data centers and large hyperscale facilities working **together** form a **hybrid cloud ecosystem** that is more sustainable than either alone.

The principle: **distribute locally when appropriate, optimize globally when beneficial.**



Benefit	Small DCs role	Large DCs role
Latency	Local processing of IoT, video, real-time analytics	Global coordination
Efficiency	Offload heavy/batch tasks	Run them at lower energy per operation
Cooling	Avoid thermal peaks	Absorb load in cool regions
Redundancy	Less overprovisioning	Provide pooled backup capacity
Sustainability	Reduce data movement	Use renewables + AI optimization

The full hierarchy spans from device-level edge (sensors, wearables) through fog computing and regional data centers up to hyperscale public cloud.

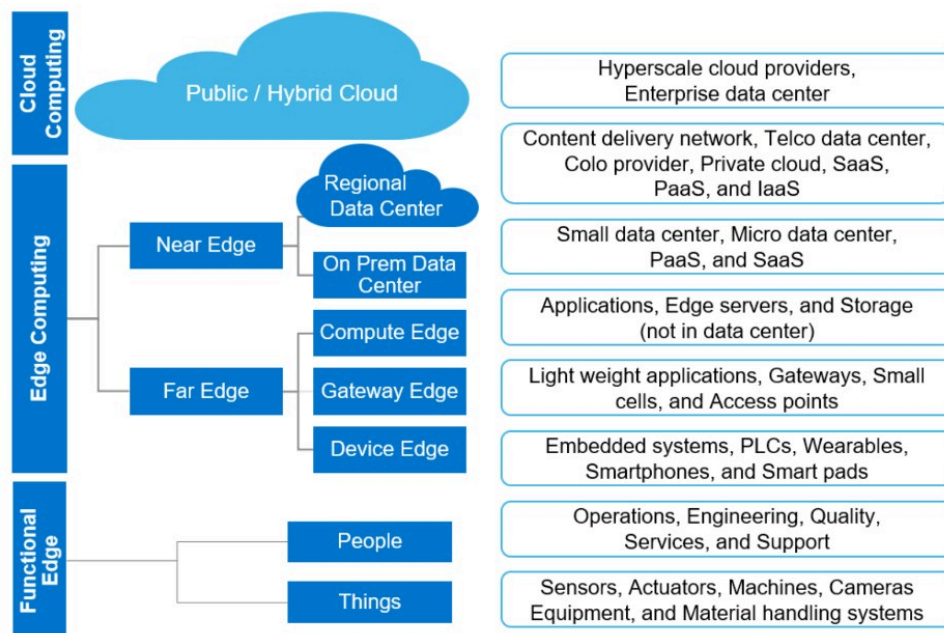


Figure 4 – Large Cloud Ecosystem.



Costs Comparison

Large data centers have overwhelming cost advantages:

Technology	Small DC (1K servers)	Large DC (50K servers)	Cloud Advantage
Network	\$95/Mbps/month	\$13/Mbps/month	7.1×
Storage	\$2.20/GB/month	\$0.40/GB/month	5.7×
Administration	140 servers/admin	>1000 servers/admin	7.1×

This is why cloud is winning: the bigger the infrastructure, the **cheaper** each unit of service.



1.7. Big Data



Market Context

The scale of investment confirms how central Big Data has become:

- **Digitalization** market: \$880B in 2023 → \$4.40T by 2030 (27.6% CAGR)
- **Big Data investments**: \$327B in 2023 → \$924B by 2030 (14.9% CAGR)
- **ICT industry overall**: ~\$6T in 2025 → \$7.86T by 2030

Drivers: mobile broadband, cloud services, social business, and analytics.

1.7.1. The 5 (or 6) Vs

Information systems need a quality-aware view of data across its full lifecycle. Big Data is characterized by:

5Vs for new data processing and novel data treatment:

- **Volume** → Massive quantities of data (size).
- **Variety** → Many formats and structures.
- **Velocity** → Speed of ingestion and processing.
- **Value** → Extracted insight (relevance).
- **Veracity** → Trustworthiness (or reliability) of data.

6Vs also for Dynamicity:

- **Variability** → Data dynamicity (meaning and structure change over time).

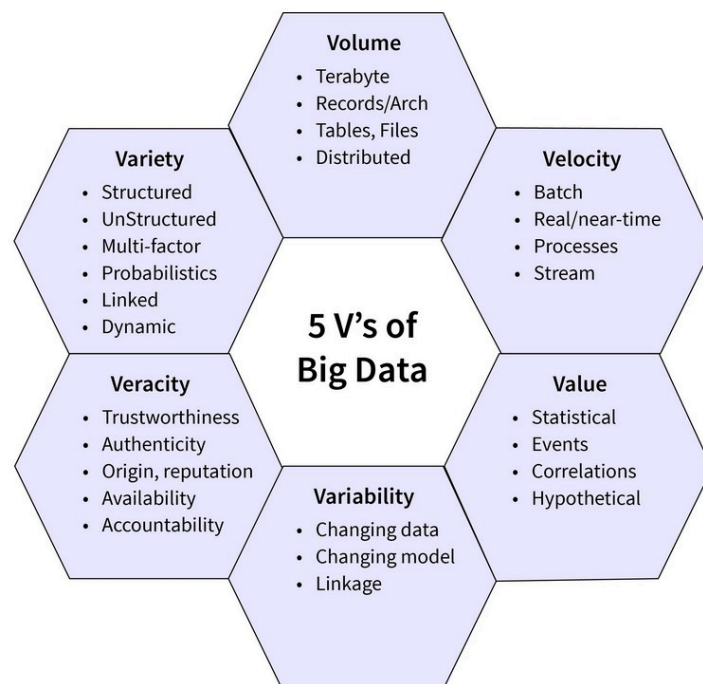


Figure 5 – The 5 (or 6) Vs of Big Data.



1.8. Application Scenarios

1.8.1. Smart Cities

Smart cities generate massive streams of heterogeneous data from diverse sensors (IoT). The smart city model is built on six dimensions: Smart Economy, Smart Mobility, Smart Governance, Smart Environment, Smart Living, Smart People.

From a systems perspective, a smart city involves:

- Groups of replicated and interacting components,
- Co-creation of content (videos, pictures),
- Collection and harvesting of big data and open data,
- Management of public services and logistics workflows.

Sensor sources include: building sensors, smart grid meters, pollution sensors, meteorological sensors, traffic cameras, medical sensors on ambulances, industrial automation sensors, and wearables.

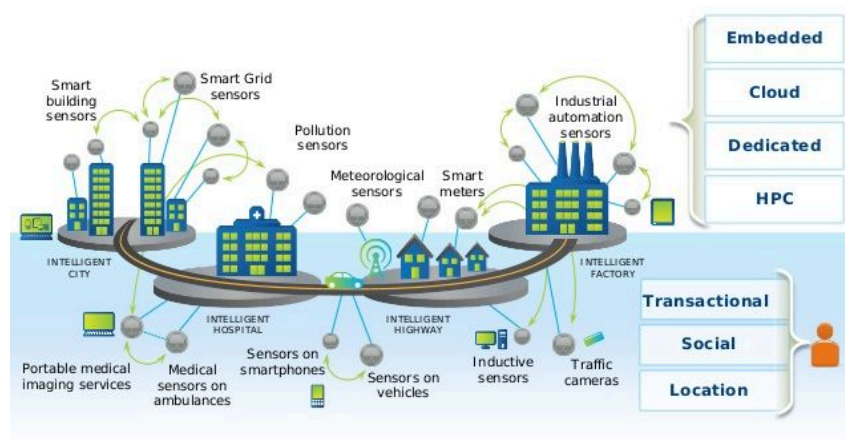


Figure 6 – Smart city data sources and applications.

1.8.2. Industry 4.0 and 5.0

Industry 4.0 connects physical production with ICT: machines communicate with machines (M2M), customer data merges with machine data, and components autonomously manage production in a flexible, efficient way. Built on IoT, cloud platforms, big data, and wireless intelligence.

Industry 5.0 (EU Commission, 2021) extends the focus beyond efficiency to societal goals:

- **Human-centric:** technology adapts to workers, not the other way around
- **Sustainable:** circular processes, reduced energy and waste
- **Resilient:** robust supply chains, adaptable production capacity, critical infrastructure that withstands crises

1.9. Quality of Service

In distributed systems, correctly delivering a service is necessary but not sufficient. The critical goal is QoS.



QoS consists in meeting parameters and requirements that reflect what users actually need.



QoS can include: response time, correctness, availability, confidence, security, user satisfaction.

1.9.1. Old World vs New World

Metric	Old World	New World
Priority	Reliability, enforced consistency (ACID)	Scalability and availability
Latency	Acceptable delays.	Milliseconds matter.

1.9.2. Supporting QoS

- **Meaning:** data must be interpretable and understood everywhere.
- **Replication:** multiple copies of resources for fault tolerance.
- **Grouping:** coordinating replicas coherently.
- **Simplified delivery:** tools to accelerate development and deployment.
- **Automated management:** infrastructure that manages itself with minimal human intervention.
- **Batch data processing:** storing and processing massive amounts of data (e.g., Google Web indexing).
- **Streaming data:** handling continuous information series from sensors, video feeds, etc..

👉 **Note:** QoS is a recurring theme throughout the course. Resource-level QoS management (SLA, allocation strategies, load balancing) is covered in the [Resource Management Models chapter](#). Network-level QoS protocols (IntServ/RSVP, DiffServ, traffic shaping) are covered in the [QoS: Quality of Service chapter](#).

1.10. Middleware

Complex, large-scale distributed applications cannot be built from scratch every time. The answer is **Middleware**.

Middleware

Middleware is a set of pre-built **tools** and **components** that provide the best system performance under user-required conditions (data and processing).

Analogy: Middleware

Think of a restaurant.

The User/App is the customer sitting at the table.

The Hardware/OS is the kitchen staff and the raw ingredients.

The Middleware is the waiter and the ordering system: the customer doesn't need to speak the chef's language or know how the oven works, they just give the order to the waiter (the uniform interface), who translates it, routes it to the correct station, and brings the finished meal back.

Middleware can:

- Make **ready-to-use applications** available when a user needs new functions, with no user intervention required.
- **Simplify development** of new applications when needed functionality doesn't yet exist.
- **Adapt the system** across its lifecycle as requirements and conditions change.



1.10.1. Middleware Families

- **Object middleware**: enable remote object interaction, examples are CORBA, COM, .NET.
- **Message exchange middleware (MOM)**: asynchronous communication between components.
- **Overlay networks, novel file systems**, NoSQL support.
- **Cloud middleware**: OpenStack and similar platforms for managing cloud resources.
- **Big Data middleware**: for storing, processing, and streaming large datasets, such as Hadoop, Spark, Kafka, Cassandra, MongoDB.

1.10.2. Major Use Cases

Private/company middleware supports an organization's internal users:

- Works on local data center servers.
- Accesses remote (cloud) resources when needed.
- Provides transparent services: replication, load balancing, naming.
- Provides non-transparent services: monitoring, decision systems.
- Manages computing, storage, and network resource delivery.

Cloud provider middleware supports external customers at scale:

- Manages large pools of heterogeneous internal resources.
- Serves many customer organizations simultaneously under agreed contracts.
- Key functions: physical infrastructure management, resource isolation (security and performance), Customer Relationship Management (CRM).
- Must honor SLAs, coordinating with other providers if necessary.

1.11. Cloud as an Evolution

Cloud is not a revolution, it is a necessary **evolutionary step**, building on decades of distributed systems experience. What changes is the **packaging**: web-accessible resources in remote data centers, offered as services with the following properties:

- **Ready-to-use**: no setup burden on the customer.
- **Easy**: low barrier to entry.
- **Pay-per-use**: cost tied to actual consumption.
- **Transparent** (or selectively non-transparent): complexity hidden behind clean interfaces.
- **Elastic**: resources scale up or down on demand.
- **Reliable**: high availability guaranteed by contract.
- **Secure**: isolation and access control enforced by the provider.

1.12. Roadmap

The course covers distributed systems lifecycle operations, focusing on execution-time aspects:

- System management and QoS
- Observation and monitoring of all resources
- Replication and partitioning of localities and resources
- Dynamic resource variation during the lifecycle
- Recovery, tuning, and disaster recovery
- Sustainability and resilience

Middleware tools covered in practice: CORBA, OpenStack, Hadoop, Spark, Kafka, Docker, Kubernetes, orchestrators, and more.



2. GOALS, BASICS, MODELS

Package: Initial Models and Issues - 2-InitialModels26.pdf

2.1. Modern Distributed Systems

Modern distributed systems are **complex** but extremely **widespread**. They are interesting precisely because many problems remain **unsolved**, *which is why studying them matters*.

A good middleware must handle several fundamental challenges at runtime:

- **Partial failure** ~> Components can fail independently, and the system must keep working.
- **Heterogeneity** ~> Different hardware, OS, languages, and protocols must coexist.
- **Integration and standards** ~> Components from different vendors must work together.
- **Changing conditions** ~> The system must adapt while staying online.

Beyond those challenges, three macro-requirements define what a large distributed system must provide:

1. **Scalability and safe answers** ~> The system must **scale** and **always respond correctly**.
2. **Predictability and performance control** ~> Behavior must be measurable and controllable.
3. **Maintainability, operability, and simplicity** ~> The system must be easy to operate and evolve.

2.2. ICCBD Approach: Resources and APIs

Understanding an IT system means understanding its **resources**: what they are, how they are defined, and how they behave internally.

Every system also exposes an **external interface** to its users, that is a set of functions called **APIs** (Application Programming Interfaces). APIs allow clients to request services without knowing internal details.

Two complementary perspectives are always needed:

- **External view (black box)**: understand the system from its visible APIs and infer what services it offers. ⇒ **Transparency**
- **Internal view (open box)**: understand the internal design, components, and mechanisms. ⇒ **Visibility**

Both perspectives are valuable and often used together in a gradual approach starting abstract, then going deeper.

2.3. Properties of Large Systems



Properties of Large Systems

Large IT systems are expected to have a **long life cycle**. This brings specific requirements:

- **Abstraction** ~> Hide complexity, focus on what matters at each level.
- **Transparency** ~> Hide implementation details from users.
- **Granularity** ~> Choose the right size for components and resources.
- **QoS** (Quality of Service) ~> Meet agreed performance and reliability targets.
- **Staticity/Dynamicity** ~> Balance between stable and changeable parts.



- **Observability/Explainability** $\leadsto$ The system must be monitorable and understandable.

2.3.1. Abstraction

Abstraction is a fundamental guideline in any complex system. It means focusing on the right level of detail, ignoring what is not relevant at that level.



Abstraction: The underground Map

A good analogy is the **London Underground map**: the geographic map is accurate but overwhelming. The simplified tube map hides distances and curves, but is perfectly useful for navigating the network. Abstraction is not about being inaccurate $\leadsto$ it is about being **appropriately** simplified for the task at hand.



Figure 7 – Abstraction example: the London Underground map.

2.3.2. Transparency vs Visibility

Transparency means hiding details so that the user does not need to know about them. It is the opposite of **visibility** (where internal details are exposed). Those two properties are the incarnation of the **black box** and **open box** perspectives (respectively, [see here](#)).

Common forms of transparency in distributed systems:

- **Access** $\leadsto$ Local and remote resources are accessed the same way.
- **Allocation** $\leadsto$ Resources are allocated independently of their physical location.
- **Name** $\leadsto$ Names do not reveal where a resource is hosted.
- **Execution** $\leadsto$ Local and remote execution feel the same.
- **Performance** $\leadsto$ No perceivable difference between using local or remote services.
- **Fault** $\leadsto$ The system continues to work even if some components fail.
- **Replication** $\leadsto$ Multiple copies of a resource improve QoS without the user noticing.

👉 **Note:** Transparency is not always the best choice. **Location-awareness** (knowing where things are) is sometimes essential.

For example, when providing services that depend on the user's physical position. Always consider both perspectives: when to hide details and when to expose them.

2.4 TINA-C

TINA-C (Telecommunications Information Networking Architecture Consortium) is a middleware framework for telecom systems.



Historically, traditional telecom systems had tightly coupled hardware and software. TINA-C was created to overcome this limitation by treating networks as distributed computing environments. It is considered an excellent reference model because it implements a strict **Separation of Concerns**, decoupling telecommunication services from the physical network infrastructure.

2.4.1. Roles and Views

TINA-C defines multiple parties to clearly separate who manages the infrastructure from who provides the service:

- **Initiating User and Responding User**: the end users.
- **Service Provider**: Manages user access, service logic and features.
- **Network Provider**: Manages the transport network (the physical routing of data).

Two complementary views are defined to understand the system from different perspectives:

- **User view**: what the user sees (access points and service interactions). It treats the underlying network as a black box.
- **Interaction view**: how providers, clients, and third-party applications connect and negotiate resources behind the scenes.

Analogy: TINA-C

Consider a modern VoIP call (e.g., Skype over a TIM connection).

- **Roles**:
 - **Initiating/Responding Users**: the callers.
 - **Service Provider**: Skype (manages the directory and call logic).
 - **Network Provider**: TIM (manages the physical fiber/cables).
- **Views**:
 - **User view**: the user merely opens the app and starts a call.
 - **Interaction view**: the system maps out how Skype's servers communicate with TIM's routing infrastructure to guarantee bandwidth and deliver the packets.

TINA provides several fundamental architectures separated and interacting and many models for computing, service, management, network architectures.

2.4.2. Architectural Layer View

Each node in the system is organized in layers (bottom to top). This stratification prevents developers from having to deal directly with hardware specifics.

- **Hardware resources** → The physical components (machines, cables, routers, ...).
- **NCCE** (*Native Computing and Communication Environment*) → The local OS and hardware abstraction layer. This layer introduces system heterogeneity.
- **DPE** (*Distributed Processing Environment*) → The middleware layer that makes nodes work together as a unified distributed system.
- **TINA applications** → The actual services and apps running on top.

The **DPE** is the key layer: it hides the heterogeneity of the underlying hardware and OS, providing a uniform programming model.

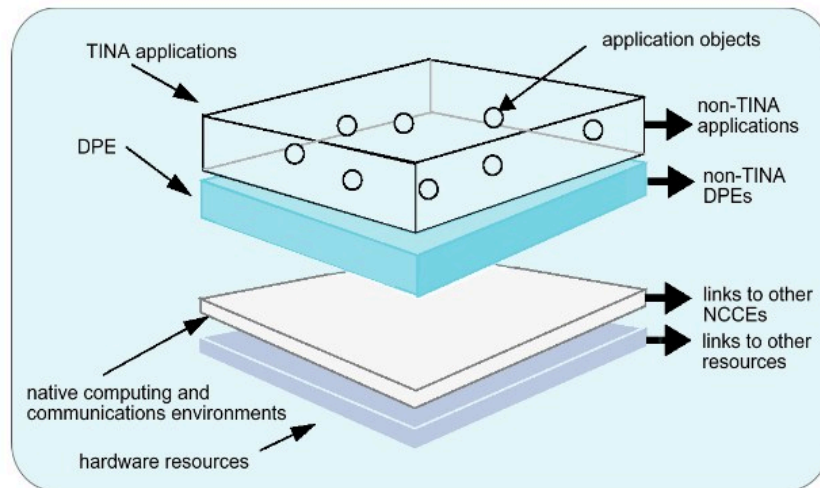


Figure 8 - TINA-C architectural layers.

👉 **Note:** Without the DPE layer, developers would need to write different versions of a telecommunication service for every specific type of router or server (e.g., one version for Cisco hardware, one for Huawei, one for Linux). The **DPE acts as an intermediate**: applications are written once for the DPE, which then translates the instructions for the specific underlying hardware, unifying the entire global network into a single virtual computing platform.

2.4.3. Transparent vs. Non-Transparent Architecture

- **Transparent view:** applications work with logical entities (**S**ervices, **R**esources, and **E**lements) without knowing on which physical machine they run.
- **Non-transparent view:** exposes DPE, Inter-DPE interfaces, and NCCE, used during **design and development** phases, when visibility of the full stack is needed.

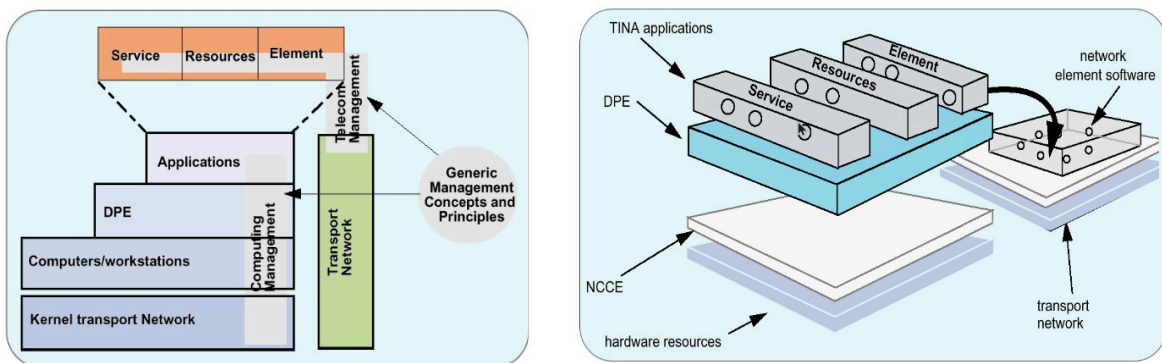


Figure 9 - TINA-C transparent vs non-transparent architecture.

2.5. Monitoring and Observability

⚠️ Monitoring vs. Observability

Modern IT systems separate concerns into three planes (User / Management / Control), the full model is covered in the [Resource Management Models chapter](#).

- **Monitoring** = **gathering** metrics (what is happening now: CPU, memory, latency, error rate).
- **Observability** = **analyzing** those metrics to **understand why** and **react** (tuning resources, rerouting traffic, scaling).



Monitoring feeds data, observability acts on it.



Monitoring

Monitoring is a critical support function that **collects** information about the **current state** of the system (processor load, resource usage, network bandwidth) to allow **control** and **adaptation**.

Method of data collection:

- **Periodic sampling** $\leadsto$ observe all values at fixed intervals.
- **Statistical/historical data** $\leadsto$ summarize past behavior.
- **Event-based** $\leadsto$ collect data only when something notable happens (discretization).



Continuity

Monitoring relies on the key assumption of **continuity** (*natura non facit saltum* $\leadsto$ nature does not make jumps). This means that the **current load** can be used to forecast **near-future behavior**, because applications tend to behave consistently over short intervals.

2.5.1. Minimal Intrusion Principle

In general-purpose systems, the monitoring infrastructure **competes for the same resources** as the application it is monitoring. This leads to a fundamental design principle:



Minimal Intrusion Principle

Any support function must **minimize** its **resource consumption**, so that it interferes as little as possible with the application.

In other words, monitoring must do its job while “stealing” as few resources as possible from the application (typically ~5-10%, not more).

2.6. Granularity



Granularity

Granularity refers to the **size** of the **units** (resources, components, services) used to build and manage the system.

Two extremes:

- **Fine-grained**: many small units $\Rightarrow$ **detailed control**, but **high management overhead** and many interactions.
- **Coarse-grained**: fewer large units $\Rightarrow$ **easier to manage**, but harder to update and **less flexible**.

👉 **Note**: Neither extreme is universally better. The right choice depends on the use case (as we will see 😊).



Analogy: Granularity

Think of granularity like building a model house.

- **Very fine-grained (Processes)** is like using thousands of grains of sand: you can mold any shape perfectly (max detail), but it's a nightmare to manage and keep together.
- **Coarse-grained (Monolithic)** is like buying a pre-molded, solid plastic house: it's incredibly easy to handle, but if you want to add a window, you can't.
- **Microservices** are like Lego bricks: they are small enough to give you flexibility, but large enough to be easily managed.



Granularity in History

The industry has historically oscillated to find the perfect balance:

Entity	Granularity	Characteristics & Slide Notes
OS Processes	Very Fine	Too many to handle, unbounded scope. Nightmare to manage at scale.
Monolithic Components	Coarse	Clean boundaries but too large. Hard to scale independently.
Early Web Services	Medium/Coarse	Good separation of concerns, but often bulky and prone to failure cascades.
Microservices	Fine/Medium	Fast adoption. Solves the "process" issue by grouping logic into manageable, bounded contexts.
Serverless (Functions)	Very Fine	Back to fine-grained, but this time, the cloud provider manages the complexity.

2.7. Service-Oriented Architecture (SOA)



Service-Oriented Architecture (SOA)

SOA is a design paradigm centered on the delivery of **services**. It enables **platform-independent** communication and **interoperability** by **decoupling** the service implementation from its interface (defined by a **contract**), allowing heterogeneous systems to integrate, **reuse** logic, and evolve independently.



Service

A **service** is an abstraction of any business process, resource, or application:

- with a **standard interface** (API),
- **published** and discovered by others,
- **reusable**,
- **black box** defined.



Properties of Services

- **Reusability** $\leadsto$ the service can be used in any context.
- **Formality** of interface $\leadsto$ Interface is unambiguous and clear.
- **Loose coupling** $\leadsto$ Platform-independent.
- **Autonomy** $\leadsto$ Do not depend on external context, they are self-managing.
- **Stateless** $\leadsto$ Minimize internal state, the **client** maintains it.
- **Discoverability** $\leadsto$ Any service can be found through naming/discovery.
- **Composability** $\leadsto$ Existing services can be combined to create new ones.

2.7.1. The 3 Actors of SOA

SOA defines three roles:

1. **Service Provider** implements and exposes the service.
2. **Service Requestor (Client)** discovers and uses services.
3. **Discovery Agency** is a registry where providers publish their services and clients look them up.

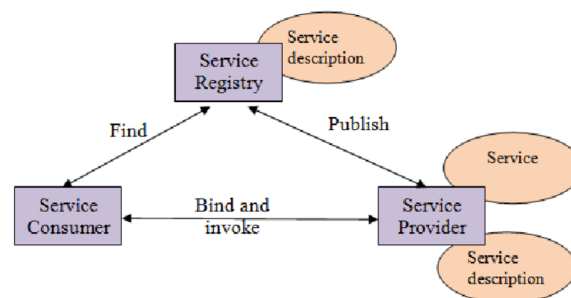


Figure 10 - SOA components and flow.

The **client-server** relationship is between Requestor and Provider. Discovery Agencies act as intermediaries for finding the right provider. The key agreement is on the **common interface**.

Analogy: SOA and the Internet

The Internet itself is a natural SOA implementation:

- **Discovery Agency** $\leadsto$ DNS (translates names to addresses).
- **Service Provider** $\leadsto$ Server (hosts and exposes the service).
- **Service Requestor** $\leadsto$ Client (looks up and consumes the service).

The classic **Client/Server (C/S)** model is an SOA implementation, but without discovery agencies. In C/S:

- The **client** **knows the server** directly.
- Interaction is **synchronous** (client waits for result) and **blocking**.
- It uses **tight coupling**: both parties must be available at the same time.

C/S has known weaknesses (rigidity, tight coupling), typically addressed by small tailored variations.

2.7.2. Granularity problem

SOA faces the same granularity trade-off seen earlier: **coarse-grained services** give higher abstraction but are harder to manage, while **fine-grained components** add flexibility at the cost of fragmentation and limited internal interaction. Historically the choice moved from coarse-grained



entities (compact, easy to move) toward fine-grained ones, driven by the need for dynamicity and runtime changes.

Some examples:

- Web started coarse grained.
- Components started middle-sized (processes were too fine grained).
- Microservices are fine grained.

2.8. Enterprise Application Integration (EAI)

Large companies run many different IT applications (CRM, ERP, SCM, Finance, HR, DMS, CMS, and more). These systems often evolved independently and are heterogeneous.

EAI (Enterprise Application Integration) aims to build a **unified integrated environment** where all business applications work together synergically, including both:

- **Legacy components**, existing systems that must be reused.
- **New components**, newly designed systems that must integrate quickly.

A key benefit of complete integration is **observability**: you can monitor the performance of any part of the business in real time, get fresh data, and react quickly to changes.

2.8.1. Traditional vs SOA-oriented Architecture

Traditional approach: each system (CRM, ERP, Financial) is isolated with its own database. Processes like Sales, Delivery, and Accounting talk to their own silo.

SOA-oriented approach: a shared **Enterprise Level Services** layer exposes fine-grained services (e.g., `GetCustId`, `CreateOrder`, `UpdateStock`, `MakeInvoice`) that all business processes can call. Data sources (Orders, Stock, Customer, Accounts) are accessed through this service layer, not directly.

2.9. Technology Life Cycles: The Gartner Hype Cycle

Every technology follows a predictable pattern of adoption, described by the **Gartner Hype Cycle**:

1. **Technology Trigger** → the technology emerges.
2. **Peak of Inflated Expectations** → excessive hype, unrealistic expectations.
3. **Trough of Disillusionment** → reality sets in, failures are publicized.
4. **Slope of Enlightenment** → real use cases are understood.
5. **Plateau of Productivity** → mainstream adoption with realistic expectations.

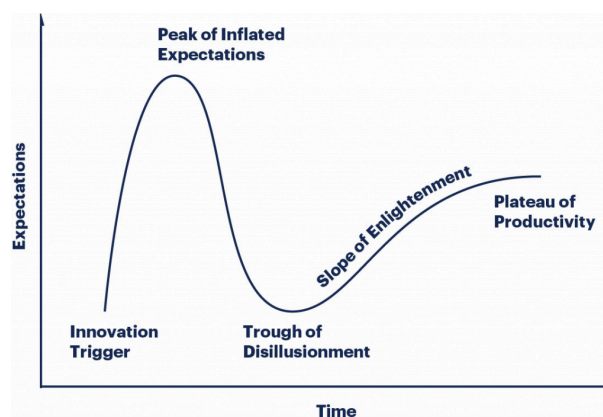


Figure 11 – Gartner (Tech) Life Cycle.



Both **Cloud** and **SOA** are on the Plateau of Productivity: they are mature, widely adopted technologies.

2.10. Resources in Distributed Systems



Resource

A **resource** is any component (hardware, software or both) needed during execution to produce any visible result in a distributed system.

2.10.1. Classification of resources

Dimension	Examples
Low-level vs. Application	CPU/RAM vs. Web server, app service
Physical vs. Logical	Real CPU vs. virtual CPU, thread
Physical vs. Virtualized	Physical NIC vs. virtual network
Static vs. Dynamic	Fixed hardware vs. auto-scaled VMs

Resources in a distributed system include:

- Physical memory (RAM)
- Disk (persistence)
- CPU (computing)
- I/O and network
- Sensors, actuators (IoT devices)
- OS services, virtual machines, virtualized networks
- Application services (web servers, databases, custom apps)

Managing all of these efficiently, especially at the application level, is one of the hardest problems in distributed systems.

2.11. Objects, Components, and the Path to Services

2.11.1. From Objects to Components

Objects (Java, C++, etc.) are the basic unit in OO programming. They encapsulate data and expose operations. But objects are:

- **Fine-grained** → there can be thousands of them.
- **Loosely bounded** → they can interact in hidden ways (e.g., via shared memory).
- **Tightly tied** to their language/runtime environment.

For distributed systems, this is problematic. A better abstraction is needed.

Components are a **coarser-grained**, more disciplined unit:



Component



A **component** is a **static abstraction** of a confined entity with a defined interface for communicating with the external world via **ports**.

Analogy:

Think of a component as "an object in a tuxedo" (Michael Feathers): software that is packaged and ready to interact with the world in a clean, predictable way.



Component Properties

- **Staticity** → has its **own lifecycle**, independent of any specific application run.
- **Abstraction** → internal structure is hidden, **only ports are visible**.
- **Port-based communication** → the only interactions with the outside world go through declared **IN ports** (incoming requests) and **OUT ports** (outgoing requests)
- **Self-containment**

Effects of the component model (vs. object):

- **Better reusability** → no hidden interactions, so components can be moved between environments easily.
- **Substitutability** → one implementation can be replaced by another without changing the container (dynamic replacement).

2.11.2. Interfaces vs. Classes

Distributed systems separate:

- **Interfaces**, the contract (what a component offers).
- **Classes/implementations**, the actual code (how it works).

Multiple implementations can satisfy the same interface (e.g., different QoS levels). The **interface is stable, implementations can change**. Middleware systems are designed around interfaces, not classes (typically).

2.12. Containers



Container

A **container** is a server-side environment that **hosts components** and provides them with many support services, so that the components themselves only need to implement business logic.



Container Properties

A container provides:

- **Lifecycle management** → activates, deactivates components as needed.
- **Resource sharing** → manages shared resources.
- **Composition** → helps combine components into new services.
- **Activity support** → manages interactions between components.
- **Control** → monitors and handles components.
- **Mobility** → can extract and move components between environments.

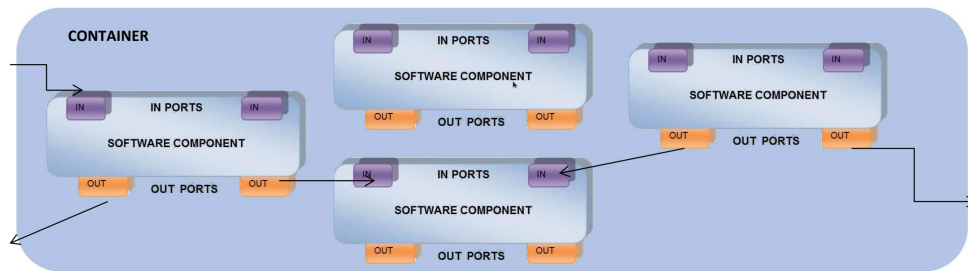


Figure 12 - Components composition $\Rightarrow$ Container.

The **only** way to access a component from outside is **through its container**. Inside the container, component interactions are **disciplined and checkable**. The container can make autonomous management decisions.

👉 **Note:** The term **container** is used with many names in the literature: engine, middleware, support environment. They all refer to the same idea: a managed runtime that supervises hosted components.

2.12.1. Container Delegation

A container relieves components of cross-cutting concerns by providing them **automatically**, so that the component(s) only implements business logic:

- **Lifecycle support:** activates/deactivates servants on demand, maintains state, handles persistence (interface with DB).
- **Name system support:** discovery of servants and services.
- **Federation** with other containers.
- **QoS support:** fault tolerance, selection among deployments, monitoring and control of negotiated QoS.

J2EE (Java 2 Enterprise Edition) and EJB (Enterprise Java Beans) are classic examples: a container hosts EJBs and automatically provides lifecycle management, naming, transactions, and persistence, the developer writes only the business logic.

2.13. DevOps

DevOps

DevOps (Development + Operations) is a methodology that couples the application development phase with the infrastructure/operations phase, enabling **continuous** and **rapid** deployment of changes without disrupting the running system.

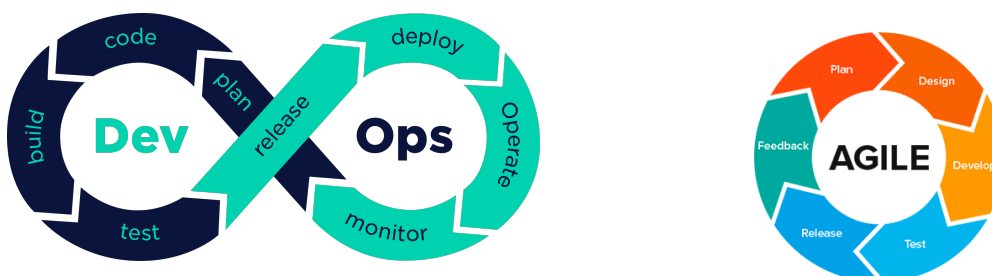


Figure 13 - DevOps vs. Agile.



The core motivation is that **large systems require frequent updates** (new releases, bug fixes, scaling adjustments) and doing so safely while the system is live is a first-class engineering requirement.



DevOps CI/CD Cycle

The cycle is **continuous** and **agile** :

- **Build** (Plan, Design, Develop) → develop the new version of the component.
- **Deploy** → package and push to the target environment.
- **Test** → validate behavior in isolation and in integration.
- **Release** → promote to production, rolling out with no downtime.

The two **key practices** are **CI** (Continuous Integration) and **CD** (Continuous Deployment) .

Stage	Description
Continuous Integration	Build → Test → Merge: developers integrate frequently; automated tests run on every merge.
Continuous Delivery	Automatically release to a repository: a deployable artifact is always ready.
Continuous Deployment	Automatically deploy to production: every passing change goes live.

Updates are **rolled out gradually** (blue/green or canary deployments), ensuring old clients are still served until the new version is validated.



Twin System Principle

DevOps enables the **coexistence of production and test systems**: the new version is deployed and tested in a shadow environment, while the current version keeps serving users. Once validated, the switch is made cleanly ⇒ **zero-downtime** upgrades become achievable.

2.14. Microservices



Microservice

A **microservice** is a **fine-grained**, independently deployable **component** that implements one specific business function and communicates with others over **web interfaces** (typically REST or gRPC).

Microservices are essentially a modern answer to the granularity problem: they are **small enough** to be independently deployable and scalable, but **bounded enough** to be manageable .



Microservices Properties

- **Fine-grained** → each service is small and focused on a single responsibility.
- **Independently deployable** → one service can be updated without touching the rest.
- **Scalable in isolation** → only the overloaded (micro)service needs more replicas.
- **Composable** → the application is assembled from many cooperating microservices.



- **Container-native** $\leadsto$ designed to run inside containers, making them portable.

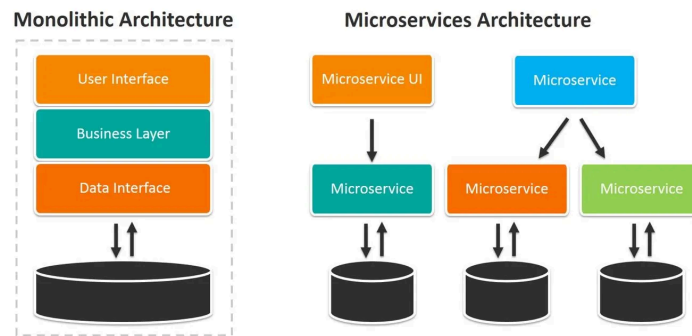


Figure 14 - Monolith vs. Microservices.

🤔 Why Microservices succeeded?

Traditional monolithic applications are **hard to update at scale**: touching one part risks breaking others. Microservices solve this by separating concerns into independently releasable units. Combined with DevOps and containers, you get an architecture where **each service can evolve at its own pace** without coordination overhead.

Analogy: Microservices vs. Monolith

A monolithic application is like a **large ship**: powerful, but any change requires dry-docking the whole vessel. Microservices are like a **fleet of small speedboats**: each can be repaired, upgraded, or replaced independently while the fleet keeps sailing.

👉 **Note**: Microservices are **not a new idea**: the underlying principles (small components, web interfaces, independent deployment) existed long before the term. What changed is the **tooling** (containers, orchestrators) that made them practical at scale.

The full microservice decomposition, trade-offs, hosting problems, and Docker are covered in the [Components, Microservices, and Containers chapter](#).

2.15. Docker and Modern Containers

The rise of microservices created the need for a lightweight, portable way to package and run components. **Docker** became the de-facto standard.

Docker

Docker is a set of tools for Linux-based containers that allows to **design, host, control, and optimize** services, both statically (at deployment time) and dynamically (at runtime). It packages an entire application together with its minimal support and dependencies into a single portable unit.

2.15.1. Docker Architecture

Docker is organized around three elements:

- **Registry** (Docker Hub or private): a **repository** of images to pull and share.



- **Client**: the CLI that issues commands to the Docker daemon.
- **Target Nodes**: the machines on which containers actually run, managed by a container engine.

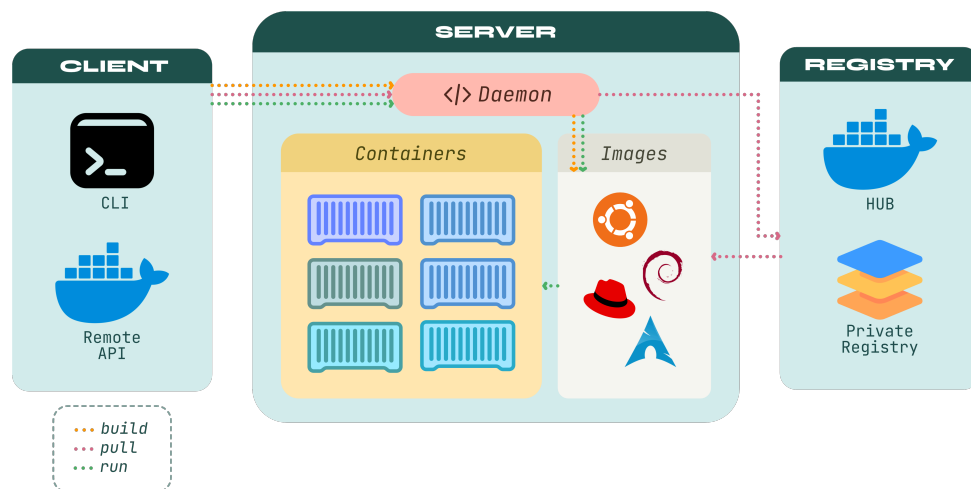


Figure 15 - Docker Architecture.

2.15.2. Docker Containers vs. Virtual Machines

Container	Virtual Machine
Shares the host OS kernel	Has its own Guest OS
Starts in milliseconds	Starts in seconds/minutes
Lightweight (MBs)	Heavy (GBs)
Process-level isolation	Full hardware-level isolation
Ideal for microservices	Ideal for strong isolation

👉 **Note:** Containers are *not weaker* or worse than VMs, they are a different trade-off. Many deployments run containers *inside* VMs to get both portability (containers) and strong security boundaries (VMs).

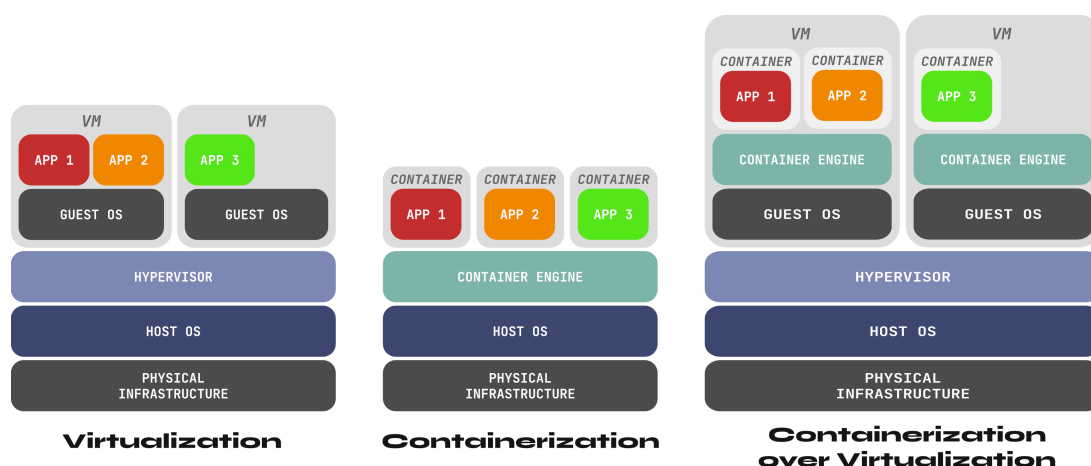


Figure 16 - Virtualization vs. Containerization vs. Hybrid.



2.15.3. Docker Container Lifecycle



Docker Container Operations

A container is **created from an image** and then managed through its lifecycle:

- **Deploy** → instantiate from an image.
- **Run** → execute the self-contained application.
- **Stop / Start** → pause and resume.
- **Move** → migrate to a different host.
- **Delete** → remove permanently.

Docker exposes an API (used by its own CLI) that allow to execute commands to manage and control containers and to exec commands inside the containers.

The **Dockerfile** describes how an image has to be (its **layers**), in a **standard, uniform** way. From **images** containers are built. The **image is static and immutable**.

👉 **Note:** The full Docker engine internals (images, layered filesystem, networks, Dockerfile instructions, Docker Compose) are covered in the [Components, Microservices, and Containers chapter](#).



Container as an Application Platform

A Docker container:

- Includes **all the details** for running a self-contained application.
- Acts as an **isolated** and **safe security boundary**. Every container is mapped as a separated process.
- Is organized like a **directory** internally.
- Is created from a **Docker image** (the static template).

Containers are the **executing components** of Docker, images are the blueprint.

2.15.4. Kubernetes

When microservices scale to hundreds or thousands of containers, manual management becomes impractical. **Kubernetes** is the dominant **container orchestrator**: it automates deployment, scaling, load balancing, and self-healing across a cluster of nodes (one master + several workers).

*Kubernetes will be covered in depth in a dedicated chapter. Here, the key idea is that Docker solves the **packaging and portability** problem, while Kubernetes solves the **large-scale orchestration and lifecycle management** problem.*



3. RESOURCE MANAGEMENT MODELS

Package: Resource Management Models - 3-Resource Mngm Models 26.pdf

Once an application is launched, it should ideally **run forever without interruption**. In practice, systems need to be **observed** and **controlled** during execution. Resource management is the discipline of:

- **Monitoring** and **observing** the system to understand resource usage and trends.
- **Controlling** and **changing** resources to correct problems and maintain performance.

This chapter covers the fundamental models and strategies behind this runtime management.

3.1. Quality of Service (QoS)

Every system must clearly define the **quality** of the services it offers. The notion of **QoS** (Quality of Service) **quantifies how well a service is delivered** and connects directly to the system lifecycle.

Quality of Service (QoS)

QoS defines the **context of system operation** and **quantifies the observable features** of service delivery, from the provider's perspective toward its requestors. It spans the **entire life cycle** of the service while running.

Telco providers historically defined service levels via measurable indicators: throughput, latency, jitter, Mean Time Between Failures (MTBF), etc. Application service providers must extend these with **user-experience** indicators (KPIs, Key Performance Indicators).

The concept of "tag qualities" (telco-oriented measurable properties attached to the service) is the origin of modern QoS indicators.

QoS must cover many dimensions:

- Performance
- Scalability
- Correctness
- Reliability
- Security

3.1.1. Functional vs. Non-Functional Properties

Type	Description	Measurability
Functional (simple & objective)	Directly measurable : avg packet delay, bandwidth, packet loss rate, throughput.	Easy to quantify.
Non-Functional (hard & subjective)	Hard to measure : long-term availability, security level, perceived video quality (Quality of Experience, QoE).	Subjective or indirect.



👉 **Note:** Both types must be considered. QoE is almost as important as QoS.

3.1.2 QoS Verification and Control

Once QoS is defined, the system must **verify** it at runtime and **act** when deviations occur. Two mechanisms are required:

1. **Monitoring support** → collects fresh information on system state (logs, online metrics) to identify trends and anticipate problems.
2. **Control actions** → allow interventions on the system to correct settings before a failure or degradation occurs.

3.1.3 The 3 Planes

To support an application execution, modern IT systems separate concerns into distinct planes:

Plane	Role
User Plane / Data Plane	Application execution and main data traffic → Business logic . The level where the application actually runs.
Management Plane	Collects monitoring data, observes performance, provides internal information out.
Control Plane / Signaling	Takes decisions on possible changes . Goal: avoid system blocks or downs. May induce system reconfiguration.

👉 **Note:** The control and management planes introduce overhead, anyway they are needed. Business logic (data plane) and control/steering are **separated**. Typically in microservices, these are interconnected by a service mesh (e.g., Istio).

👉 **Note:** Network-level QoS protocols (IntServ/RSVP, DiffServ, SIP, SNMP, router scheduling) are covered in the [Network Quality of Service chapter](#).

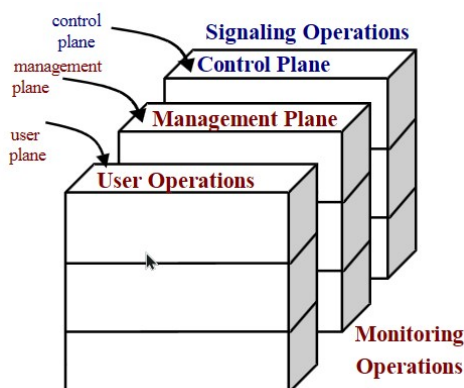


Figure 17 – The 3 planes.



3.1.4 Service Level Agreement (SLA)

SLA – Service Level Agreement

An **SLA** is a **clean and non-ambiguous contract**, a formal agreement between a service provider and its requestors that precisely defines **how the service has to be granted**: the quality targets to be met and the consequences if those targets are violated.

SLAs translate abstract QoS goals into concrete, verifiable commitments.

AWS S3 SLA

Amazon S3 defines a Monthly Uptime Percentage commitment and ties it to refunds:

Monthly Uptime	Service Credit
∈ [99.0%, 99.9%]	10%
∈]95.0%, 99.0%]	25%
< 95.0%	100%

Azure VM SLA

Azure guarantees:

- **99.99%** connectivity for VMs across ≥ 2 Availability Zones.
- **99.95%** for VMs in the same Availability Set.
- **99.9%** for a single VM with Premium/Ultra SSD.
- **99.5%** for Standard SSD.
- **95%** for Standard HDD.

3.2. General Resource Management Models

Systems are extremely varied in requirements. There is no magic recipe. The design of resource management is split into two phases:

- **Static (out-of-band)**: decisions taken **before** execution. Complex algorithms are feasible here (no runtime cost).
- **Dynamic (in-band)**: decisions taken **during** execution. **Must be fast and lightweight** to not interfere.

In-band vs Out-of-band

The terms come from signaling: control work can travel on the **same channel as the data** (in-band) or on a **separate channel** (out-of-band).

- **Out-of-band**: done **outside** the live service, on a separate path or **before** execution starts. It does not compete with the running application, so it can afford slow and complex algorithms.
- **In-band**: done **inside** the live service, sharing the **same resources as the real traffic**. It competes with the application, so it must stay fast and lightweight.

This is exactly why **static** work is out-of-band and **dynamic** work is in-band.



Concurrency among services and management actions can introduce overhead, but also allows **load balancing** that improves average performance.

3.2.1. Preventive vs. Reactive Models

	Preventive (Pessimistic)	Reactive (Optimistic)
Idea	Avoid undesired events a priori .	React after an undesired event occurs.
Cost	Fixed cost, always paid (even when nothing is happening).	Lower cost if bad events are rare. Works well if the system is far from saturation .
Applicability	When failures are too costly to tolerate.	When failures are rare or recoverable.

👉 **Note:** Both are required. The key is to have the more static decisions, so the overhead is less at runtime.

→ Define as much as possible before execution (static/preventive), and keep a reactive layer for the rest.

3.2.2. Static vs. Dynamic Models

Static Models (⇒ predefined)	Dynamic Models
User number predefined and fixed before run.	Users can be added and deleted during execution.
Process number predefined and fixed before run.	Processes can be added and deleted during execution.
Node numbers predefined before run.	Processors can be added and deleted during execution.
Client traffic predefined and limited.	Client traffic can vary during execution.
Services predefined and fixed.	Services can be added and modified during execution.

👉 **Note:** Modern systems are mostly **dynamic**. Static models are simple but inflexible, the trend is toward dynamic systems with **orchestrators** that handle the variability **automatically**.

3.3. Application Deployment

Any application must be **deployed** onto real physical resources. This requires two decisions:

1. **Partitioning**: divide the application into constituent components (P1, P2, ..., P9). These are components/partitions that work on data.
2. **Mapping**: decide on which physical node each partition runs.



Application resources include: processes, components, objects, classes.

System resources include: processors, networks, clusters, cloud nodes.

Manual deployment requires answering: *who is in charge?* The application designer? The support? The middleware? This is why automation and declarative deployment exist.

3.3.1. Process Allocation

Placing processes on processors involves a **cost model** that considers per-processor:

- Memory m_i
- Execution capacity x_i
- Bandwidth b_i

Each processor is conceptually a **bucket** to be filled. **Communication is not as easy**, the communication cost is **non-linear** (depends on reachability), making exact placement NP-complete. Linear models exist but even those can be complex.

- **Static allocation**: out-of-band, permits expensive exact algorithms or heuristics (genetic, Tabu search).
- **Dynamic allocation**: in-band, must use **simple policies** with minimal overhead.

3.3.2. Static vs. Dynamic Resources in Applications

Applications contain two kinds of resources:

- **Static resources** $\leadsto$ known before run, easy to allocate at deployment time.
- **Dynamic resources** $\leadsto$ created **during execution**, may not even exist in every run.

In dynamic systems, you can also create **non-forecasted dynamic resources** and think about **reallocating existing resources** (migration) – settings can change during execution. **Resources re-allocation** can be heavy.

3.3.3. Allocation Strategies

Strategy	Pros	Cons
Static	Allocation cost paid before execution.	Inflexible : no adaptation at runtime.
Dynamic	Adapts to current situation; allocated only when needed (on-need).	Allocation cost impacts execution time.

3.3.4. Allocation Decision Models

- **Explicit** (user-driven) $\leadsto$ User specifies the mapping for each resource before execution.
- **Implicit** (automatic) (data centers) $\leadsto$ The system handles mapping, both at deployment and at runtime.
- **Hybrid** $\leadsto$ System applies a default policy to both static and dynamic resources, including initial allocation of new resources and possible migration during run. User hints can refine decisions.

3.3.5. Deployment Support

- **Manual** $\leadsto$ User passes each object to a node with explicit commands.



- **Script-based** → User writes shell/Python/Perl scripts specifying configuration steps and dependencies by phases.
- **Model-driven / Declarative** → Automatic configuration through declarative languages.
For Cloud: Ansible, Chef, Puppet, Salt, SmartFrog, Radia, Terraform, CloudFormation.

3.3.6. Modern Deployment: Application + Configuration Together

Traditional approach: define **how** to configure for the specific environment.

Novel approach (Docker/microservices): **ship the application together with its required configuration**, so it can be ported to any support environment. The orchestrator then automates all management operations.

Docker and Kubernetes are covered in dedicated chapters. Here, the key idea is that modern deployment inverts the traditional model: instead of adapting the application to the environment, you bring a self-contained unit that describes its own needs.

3.4. Resource Services and Agent Management

A **resource** exposes its services through a **simple interface** (ideally following the SOA principle):

- **Service request model (C/S)**: the client explicitly asks the server.
Good for a small number of clients, direct and simple, but does not scale to many concurrent requestors.
- **Distributed File System (DFS) / Middleware model**: a transparent, allocation-invisible service. The middleware in the middle makes the service available and users are freed from knowing where the resource lives.

Transparency simplifies the interaction: users do not need to know which node provides the service.

3.4.1. Agent-Based Management (DFS)

In distributed deployments, **agents** coordinate among themselves to provide a unified service:

- The deployment is a **coordinate agent system** providing unique service for generally distributed actions.
- Agents **negotiate** and **coordinate** to give the best result. → Any kind of negotiation is possible among agents, including refusing a service.
- Each node hosts an agent that manages local resources and communicates with **peer** agents.

The middleware in the middle makes the service available and allocation transparent.

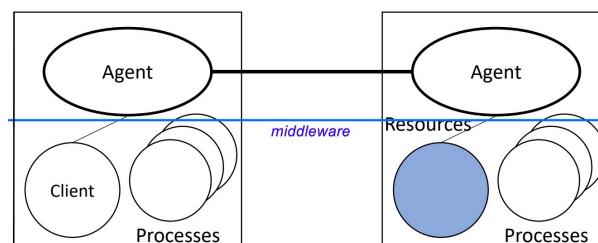


Figure 18 – Agent-based management.

3.5. Load Sharing and Load Balancing

Load sharing is formal (literature, frameworks, ...), instead load balancing is more practical. In real modern deployments, dynamic load balancing is the norm.



- **Load Sharing** (static, out-of-band) $\rightsquigarrow$ Best possible allocation plans defined before the run. Resource allocation without moving any resource once allocated. No migration after initial allocation, actuated eventually at resource creation.
- **Load Balancing** (dynamic, in-band) $\rightsquigarrow$ Migrates already-running, active resources during execution depending on load, to obtain better global efficiency (dynamic allocation).

! **Problem:** I/O is the problem when moving thing, because you can loose where things are, link and whatever.

👉 **Note:** In Unix, luckily, persistent communication elements are used
 $\Rightarrow$ VMs can be migrated without this problem 😊, but if the intermediate migrates how will we re-establish communication? 🤖

3.5.1. The Farm Pattern

A common and widely used pattern for load distribution:

Farm Pattern

A **Farm** consists of one **Master** that distributes tasks to multiple **Workers** that execute in parallel and return results. The master handles input decomposition and result aggregation.

Apache Spark uses a Farm pattern: the Spark driver is the master, distributing data partitions to executor nodes that search or process them in parallel.

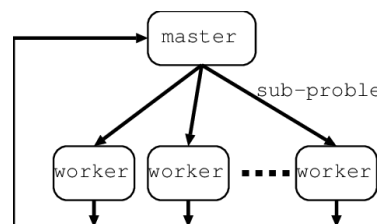


Figure 19 - Farm Pattern.

👉 **Note:** This approach is widely used, also out of this context and often the terms can vary with similar meaning words (such as Master-Slaves, Main-Workers, White-Blacks).

It is a matter of ethics/discrimination... 🤖

3.5.2. Static Load Sharing Strategies

When processes are created, the system must find a suitable processor. Three classic static strategies:

- **Logical ring** $\rightsquigarrow$ static,
- **Logical hierarchy** $\rightsquigarrow$ static,
- **worm** $\rightsquigarrow$ dynamic.

3.5.2.1. Logical Ring

In the **Logical Ring** (Token Bus) the processors form a **ring**. A **token** circulates. The token holder becomes the current strategy maker, broadcasts a load-state request, and distributes the new process. The token is passed after a maximum permanence in a node.



If a **node crashes** it is excluded and **token regeneration** is needed. If load is low the **election** strategy is activated. It is **easy to maintain** and **fast to recover** in case of fault.

👉 **Note:** 2 connections per node (economic) but communication takes time (predecessor/successor).

👉 **Note:** Token circulation is **pessimistic**, cost is always paid.

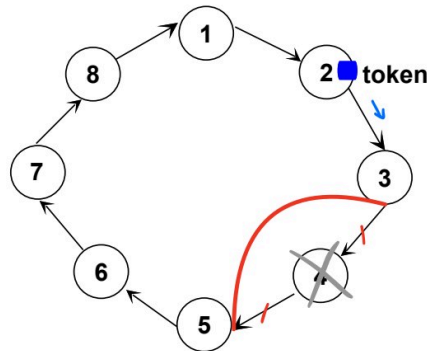


Figure 20 - Token bus. 4 crashed $\Rightarrow$ excluded.

3.5.22. Logical Hierarchy (MICORS)

In MICROS (Logical Hierarchy) nodes form a **hierarchical Farm**: Workers (computing $\leadsto$ slave) and Managers (handling/controlling). Allocation is only on workers. Level number depends on workers. Multiple managers provide fault tolerance. The hierarchy can shrink and expand dynamically.

👉 **Note:** If a manager fails, the workers (under) are lost $\Rightarrow$ duplication of connections.

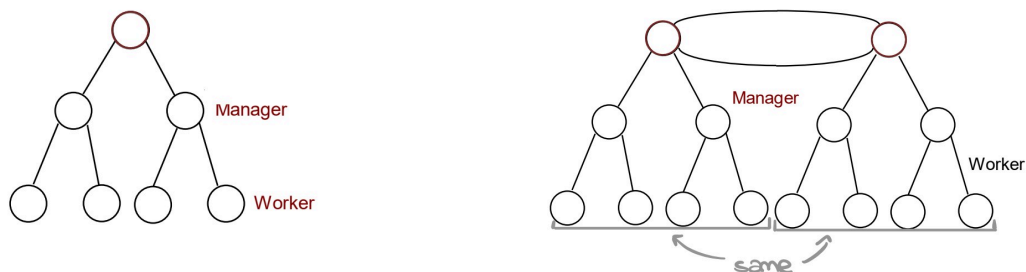


Figure 21 - Logical Hierarchy (Farm pattern).

3.5.23. Worm

Worm is used in small-size systems (<100 nodes). It is **dynamic**. A worm is a set of multiple segments, each executing a process, who can communicate for load-sharing. Segments expand by sending probes to discover free close nodes and cloning there (one copy only). There is not a predefined topology since the discovery of nodes is dynamical.

It is inspired by the informatics virus worm.

3.6. Load Balancing: Process Migration



Migration



Migration is the act of moving an already-running process (or even VM) from one node to another during execution, transparently to the user, to achieve **better resource utilization**.

Goals of transparent migration:

- Better and more efficient resource usage.
- Balancing of computational and communication load.
- Dynamic decisions and long-term policies.

Requirements:

- **Performance** $\Leftarrow$ use resources at the best.
- **Efficiency** $\Leftarrow$ limited overhead.
- **Continuous operation** $\Leftarrow$ minimal intrusion.

3.6.1. Internal Migration Problems

When a process migrates, it must prepare the **mobility phase** and manage all previously available resources. This involves an **environment change** of the mobile resource (files, sockets...).

Before and during mobility, the process state must be carefully frozen, managed, and transferred through these sequential phases:

1. **State Identification** $\rightsquigarrow$ Identify which internal resources to carry on to the new location, determining their exact runtime state (code, data, and visible resources).
2. **Process Blocking** (before mobility) $\rightsquigarrow$ Freeze the process execution. One part of the execution state may be **not transportable**, so specific cleanup is required before moving.
3. **Manage Last Wishes** $\rightsquigarrow$ Close local files or code to be managed.
4. **Resource Storing** $\rightsquigarrow$ Serialize and store all transportable resources so they can be sent and safely re-enabled on the target node.
5. **Transfer & Synchronization** $\rightsquigarrow$ Move the resources through nodes. During the transient phase, **two copies exist** (old and new) and require data synchronization.
6. **Activity Activation** $\rightsquigarrow$ Complete the activity on the old node, trigger the activation mechanisms on the new node, and de-activate the temporary movement infrastructures.

Files, sockets, resources move through nodes. So "Where is the process? The context?" this is the fundamental challenge of transparent migration.

3.6.2. External Migration Problems

After the migration, old references to the process must be updated $\Rightarrow$ change of name of mobile resources.

3 alternative approaches:

- **Message Redirection** (pessimistic/proactive) $\rightsquigarrow$ Old node keeps track and forwards all messages to the new location. Forwarding chains can grow for mobile processes.
- **Requalifying of Allocation** (pessimistic/proactive) $\rightsquigarrow$ Old node forwards during transfer only, clients receive new location reference afterward.
- **Client Recovery** (optimistic/reactive) $\rightsquigarrow$ Old node takes no action, failing clients must find the new location themselves.

3.6.3. Migration Policies

Migration involves three steps:



1. **V (Valuation)** $\leadsto$ evaluate local load vs. global load to decide if migration is needed.
2. **T (Transfer)** $\leadsto$ decide which process to transfer and when to do it. In heterogeneous systems, maybe some resources are **NOT movable to specific nodes** (such as in heterogeneous systems).
3. **L (Location)** $\leadsto$ decide where to migrate the process.

Those actions are taken at runtime, so they should be simple. T & L are often intertwined and interdependent, and must integrate with local scheduling.

Policy type	V / T / L details
Static (low cost)	- V: fixed threshold. - T: move the newest process. - L: fixed to a predefined sink node.
Semi-Dynamic (limited cost)	- V: variable threshold (probabilistic). - T: cyclic identification. - L: cyclic sink node.
Dynamic (higher cost)	- V: comparison with neighbors (dynamic avg load). - T: process state information. - L: discover sink via probing messages in the neighborhood.

Migration policies can use **probes** (messages sent to neighbors to ascertain possibility of moving) probing (T & L together), using conditioned/unconditioned acceptance, bidding strategies.

3.6.4. Sender vs. Receiver Initiative

Migration requires a sender (overloaded) and a receiver (underloaded):

- **Sender initiative:** the overloaded node seeks a receiver $\leadsto$ more suitable for **low system load**.
- **Receiver initiative:** the underloaded node seeks work $\leadsto$ more suitable for **medium-high system load**.
- **Mixed solutions** combine both.

3.6.5. Centralized vs. Decentralized Migration

Approach	Description
Centralized	Unique entity controls migration. High cost in transporting data, long time to actuate control. Only works well with small systems.
Decentralized	Coordination of many different entities. Implicit or explicit collection of distributed state info (piggybacking). Favors local movements in a neighborhood. Fresh data locally available and fast decisions, but typically local strategies and not optimal values.

3.6.6. Key Lesson from Migration



Migration Lesson

Migration has a cost... but it may be very effective.



- Even **simple policies** yield significant improvements over no-migration.
- More sophisticated policies do **not** give significantly better results in general, and cannot be generally applied apart from very specific (not so common) situations.
- Goals: **stability** (avoid thrashing), **efficiency** (simple to compute), **sub-optimality** (optimality is not a real goal), **minimal intrusion**.



Separation of Policies and Mechanisms

Mechanisms (*how do things*) are system-tailored and should be kept **immutable** (if possible). Policies (*what to do/what strategy to choose*) are general-purpose and should be kept separate, able to vary under user control.

Always keep strategies and mechanisms separated.

Examples of Separation:

- **CPU:** Mechanism = Context Switch (how to swap) | Policy = Round Robin/FIFO (who is next).
- **Migration:** Mechanism = Freeze & Network Transfer (how to move) | Policy = Load Balancing (when/where to move).
- **Storage:** Mechanism = Block Write/Caching (how to store data) | Policy = RAID Level / Replication factor (how redundant it must be).

3.7. Scalability Problems: Orchestrators

When dealing with **many containers and microservices**, manual management becomes infeasible. Two complexity problems arise:

- Possible **too large a number** of containers and microservices.
- **Dynamic changes** → crashes, addition/deletion of items at runtime.

An **orchestrator** is a manager that can automate operations on all microservices, all containers, all nodes and any management action.

Available orchestrators: Docker Swarm, Kubernetes, MESOS, Amazon ECS, Google Container Engine, Azure Container Services, Cloud Foundry's Diego, CoreOS Fleet, Mesosphere Marathon.

3.7.1. Kubernetes



Kubernetes

Kubernetes (K8s) is a Cloud Native Foundation **orchestrator** for containerized applications. It provides **application-centric management** in a loosely coupled infrastructure where any component is a separate unit.

Key features:

- **Auto-scalable infrastructure and Automated Scheduling**.
- **Horizontal Scaling and Load Balancing**.
- **Self-Healing** capabilities (to overcome crashes).
- Environment consistency across development, testing, and production.
- Automated rollouts and rollbacks.
- Provides higher density of resource utilization.

👉 **Note:** K8s manages **state outside**, in just one component (**etcd**). This is the key design decision that makes K8s scalable.



The full Kubernetes architecture (control plane, pods, services, networking, autoscaling, and RAFT consensus) is covered in the [Kubernetes](#) chapter.

3.8. Modern Microservice Support: Service Mesh

With microservices, you have many user-specified services, each self-contained in its own container. **How can you manage them?** Orchestration helps, but a **service mesh** provides additional management functions:

Service Mesh

A **service mesh** is a mesh of services that provides a dedicated infrastructure layer for handling service-to-service communication, providing functions for: **identification, discovery, interaction, monitoring, separation** (security), **composition, migrations, traffic management**, and more.

A large number of messages is needed between services. The service mesh abstracts and manages this communication transparently.

Service areas covered by a service mesh:

- **Observation of service** (metrics, tracing, logs)
- **Traffic management** (routing, load balancing, versioning)
- **Composition and extensions** (add-ons, proxies)
- **Separation** (security, isolation)

Open source: **Istio, Envoy**.

Proprietary: Mulesoft.

Cloud-native: Azure, IBM, AWS have their own tools.

3.8.1. Istio

Istio

Istio is an open-source service mesh that provides **observability**, traffic management, and security for microservices via **Envoy proxy** sidecars injected next to each service. A central control plane distributes configuration to all proxies.

👉 **Note:** The full Istio architecture (traffic management policies, mTLS security, distributed tracing, and integration with Kubernetes) is covered in the [Kubernetes chapter](#).

3.9. Continuous DevOps (CI/CD)

An application can be **continuously upgraded while in execution** without disrupting current users. New versions are rolled out gradually (blue/green, canary deployments) so old clients are served until the new version is validated.

👉 **Note:** The DevOps methodology, CI/CD pipeline stages (Integration, Delivery, Deployment), and the Twin System Principle are detailed in the [Goals, Basics, Models chapter](#).



3.10. Infrastructure as Code (IaC)

In the DevOps direction, **IaC** (Infrastructure as Code) allows managing the infrastructure with the same tools used for application code: **declaratively**, **automatically**, and **reproducibly**.



Infrastructure as Code (IaC)

IaC is the practice of **managing and provisioning computing infrastructure using configuration files**, instead of manual hardware setups or interactive tools. The core idea is to **treat infrastructure exactly like software application code**, allowing to version it, automate its deployment, and change it safely through declarative files.

The system changes configuration in order to adapt to some conditions, this is what distinguishes IaC from manual configuration management.

Tools in this space:

Tool	Description
Ansible, Chef, Puppet, Salt	Configuration management tools for cloud and on-premises systems.
AWS CloudFormation	AWS-specific IaC: JSON or YAML templates that describe all AWS resources. Integrated with CloudWatch for observability. <i>Not portable ⇒ risk of vendor lock-in. Completely integrated with AWS suite but cannot be used for multi-cloud.</i>
HashiCorp Terraform	Open, cloud-agnostic IaC: declarative language targeting any cloud provider. Supports multi-cloud and CI/CD integration. Actions automatically adapted for the target. Works across AWS, Azure, GCP, and more.

3.11. Computational Models for Parallelization

To reason about how well a problem scales across multiple processors, we use two formal indicators.

*Algorithms have intrinsic complexity in time and results are about **that** (the algorithm's inherent complexity, not just the data). Both time complexity $CT(N)$ (abbreviated $T(N)$) and space complexity $CS(N)$ matter.*

3.11.1. Speed-up and Efficiency

Let $T(1, N)$ be the sequential time for problem size N , and $T(P, N)$ the parallel time with P processors.



Speed-Up

Speed-up $S_P(N)$ is the **potential improvement** factor over sequential introduced by P using processor. It can be at most P (ideal).

$$S_P(N) = \frac{T_1(N)}{T_P(N)}$$



Efficiency

Efficiency $E_P(N)$ measures how effectively a parallel system utilizes its computational resources compared to the overhead introduced by parallelization. It can be at most 1 (ideal).

$$E_P(N) = \frac{S_P(N)}{P} = \frac{T_1(N)}{T_P(N) \cdot P}$$

3.11.2. Grosh Law and Loading Factor

Loading Factor

The **loading factor** is the problem size per processor:

$$L = \frac{N}{P}$$

Grosh's Law

The best deployment for a program is sequential execution using a unique processor (local/centralized machine is ideal).

This is never practically possible.

👉 **Note:** In reality, when $N \gg P$, the system can approach ideal speed-up and efficiency.

The Hook behind Grosh's Law:

- **Historical Context (1950s):** Computer power scaled quadratically with cost (Performance = $k \cdot \text{Cost}^2$). Economically, it was cheaper to build one giant central mainframe ($P = 1$) than multiple smaller interconnected machines.
- **Modern Trade-off:** Today, microprocessors broke Grosh's Law (commodity hardware in parallel is cheaper than an ultra-powerful monolithic chip).
- **The Architectural Link:** To beat Grosh's ideal centralized model, you must keep the loading factor high ($N \gg P$). If N is too small, processors spend more time talking over the network than computing, collapsing efficiency.

3.11.3. Amdahl's Law

Amdahl's Law

Any program can be split into a (potentially) parallel part and a sequential part. The sequential fraction sets the **hard limit** on achievable speed-up, regardless of the number of processors.

$$T_P(N) = T_{\text{CompP}} + T_{\text{CompS}} + T_{\text{Comm}}$$

where Comp = computation and Comm = communication.

Amdahl's Law

A program with 100 operations: 80 parallelizable, 20 sequential. With any number of processors, speed-up cannot exceed 5 (the 20 sequential operations always dominate). Of course, it can become worse than that due to communication overhead.



In practice, at growing P :

- Speed-up first grows linearly, then flattens (limited by the sequential part).
- Efficiency first stays near 1, then drops as overhead grows.

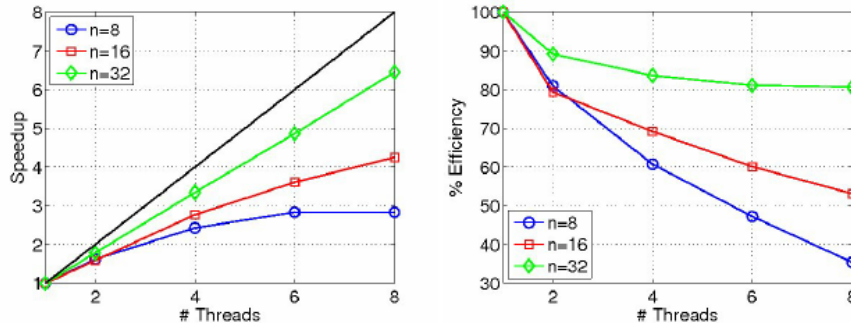


Figure 22 – Speedup and Efficiency plots (depend on P , but also N).

The **heavily-loaded limit** $T_{HL(N)} = \inf_P T_{P(N)}$ is the minimum T_P over all P – the P that yields the least complexity. It is ideally achieved when $\frac{N}{P}$ is very high: all processors are **heavily loaded**, carrying out work right up to the sequential limit.

3.11.4. Overhead Time



Total Overhead

The **total overhead** $T_0(N, P) = P \cdot T_P(N) - T_1(N)$ represents the **wasted work** (time spent on coordination, synchronization, and communication) rather than useful computation. T_0 indicates the **lost work**.

$$T_P(N) = \frac{T_0(N) + T_1(N)}{P}$$

Efficiency can be rewritten as:

$$E_{P(N)} = \frac{T_1(N)}{T_0(N) + T_1(N)} = \frac{1}{\frac{T_0(N)}{T_1(N)} + 1}$$

A higher overhead lowers efficiency. To keep efficiency constant as P grows, N must grow proportionally.

3.11.5. Case Study: Sum of N Integers

Processors arranged in a **binary tree** (leaves receive values, root produces the sum).

Identity size ($N \approx P$)

Data Flow Architecture:

- $T_{P(N)} = O(\log_2 N)$
- $S_{P(N)} = O(N / \log_2 N) = O(P / \log_2 P)$
- $E_{P(N)} = O(1 / \log_2 N) = O(1 / \log_2 P) \Rightarrow$ efficiency goes to zero $\rightarrow$ **low efficiency**: processors are idle most of the time.

Instead, a **pipe** is good if there is no idle time due to **continuous data flow**. If data keeps arriving and processors are always busy, efficiency is recovered.

Independent size ($N \gg P$, each processor sums $L = \frac{N}{P}$ values locally, then communicates):

- $T_{P(N)} \approx N/P + 2 \log_2 P$ (exact, instead of 0 notation)



- $S_{P(N)} = NP / (N + 2P \log_2 P)$
- $E_{P(N)} = N / (N + 2P \log_2 P)$
- As $N \gg P$: speed-up $\rightarrow P$ and efficiency $\rightarrow 1 \Rightarrow$ **ideal behavior**.

The lesson: **keeping processors heavily loaded** (high L) is essential for good efficiency. The T_0 overhead (for the tree case) $\approx 2P \log_2 P$ – depends mostly on the **number of engaged processors**, as coordination grows with P .

3.11.6. More Real Indicators

In practice, the real speed-up is **not exactly the reference** due to communication or other overheads. The typical behavior:

- Initial **linear zone** at P growing (speed-up growing).
- Then **constant speed-up** with **lowering efficiency** – paying the cost of synchronizing more CPUs.
- **Target stays constant** (Amdahl's limit).

Speed-up is usually *constrained because of the overhead* $\leadsto$ we cannot maintain an ideal speed-up for long. The real speed-up curve bends down from the ideal diagonal.

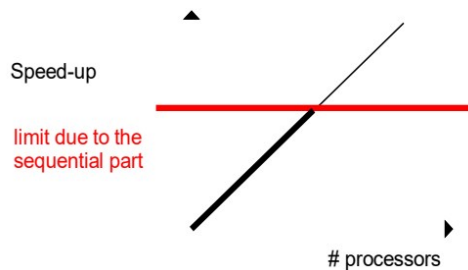


Figure 23 – Real speedup.

3.11.7. Iso-Efficiency

Iso-Efficiency Factor K

The iso-efficiency factor $K = T_0(N, P) / T_1(N)$ measures how well a parallel system maintains **same efficiency with different numbers of processors**. If K is constant and small, the system is **scalable**. If K grows with P , the system loses efficiency at scale.

$$T_0(N, P) = K \cdot T_1(N)$$

Goal: **keep efficiency constant** $E_P(N, P) = 1 / (K + 1)$ as P varies.

For the binary tree sum: $K \approx 2P \log_2 P / N$ – depends on both P and N .

- If K is small $\Rightarrow$ **high scalability possible**.
- If K is high $\Rightarrow$ **less scalable system**.
- K non-constant $\Rightarrow$ **non-scalable systems** (mostly all real systems).

For the tree case, K is $2P \log_2 \frac{P}{N}$ – so the system is *scarcely scalable*.



Scalability Conclusion



When the problem is compute-bound, it's scalable. But when we deal with communication, it is not (because of idle time).

In general, all real systems are non-scalable. The goal is to keep K as small as possible by maximizing the loading factor $L = \frac{N}{P}$, ensuring all processors are heavily loaded at all times. A heavily loaded limit is a good target.



4. UNIX FILES AND SINGLE PRIMITIVE ATOMICITY

Package: UNIX files and Single Primitive Atomicity – [unix_file_direttori.pdf](#)

UNIX has a very clean and simple semantic model: only a few **primitives** on files and directories are **guaranteed atomic** operations in the kernel. Everything above that level is the application's responsibility. Understanding this model is fundamental for reasoning about consistency and synchronization in any system built on top of UNIX-like operating systems.

4.1. UNIX File Primitives

Atomic Primitives in UNIX

- **File operations:** `open` / `read` / `write` / `close` / `remove`
- **Directory operations:** `opendir` / `readdir` / `closedir` / `rmdir`

The kernel grants **minimal atomicity** on each **single primitive**. It offers only mechanisms, no policy.

👉 **Note:** Policies are chosen by the application itself.

Atomicity in UNIX

A **UNIX primitive** is **atomic** if the kernel guarantees that its effects are applied as one **uninterruptible action**, so that no other process can observe a partial result.

👉 **Note:** Atomicity in UNIX ensures **kernel-level consistency**, not application-level isolation. That is the base for predictable behavior under concurrency. Synchronization and consistency across primitives are left entirely to the applications.

4.2. UNIX File System Architecture

The UNIX file system has a layered structure connecting user space to persistent disk storage:

File Access Layers (recalls)

- **User space:** each process holds **file descriptors (fd)** pointing into the process table for open files.
- **System space:** the kernel maintains a **Global File Table** with all active file entries. Each entry holds the current **I/O pointer**.
- **i-node:** the kernel-level data structure that tracks all disk blocks constituting a file. A file is managed and accessed via its i-node.
- **Memory cache:** block copies are loaded into a kernel cache to shorten access time (open → read/write → close operates on cached data).
- **Persistent disk:** the actual data blocks.



When a father process forks a son process, they may **share the same open file entry** (same I/O pointer).

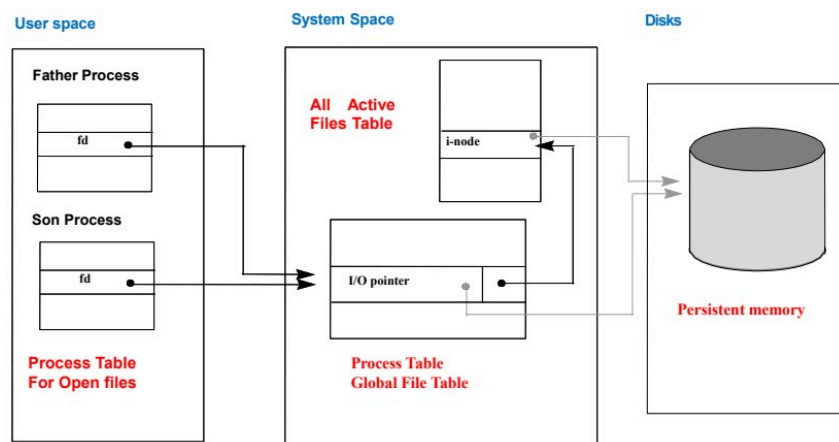


Figure 24 - Unix full picture.

4.2.1. i-nodes and Directories (recalls)

i-node

An **i-node** is the metadata structure that keeps track of all **disk blocks constituting a file**. It stores file **metadata** (permissions, timestamps, size) and **pointers** to data blocks on disk.

Directory

In UNIX, a **directory** is a **special file** (with ad-hoc primitives: `opendir`, `readdir`, `closedir`) that maps names to i-nodes of contained files and subdirectories.

Every directory contains:

- `.` $\rightsquigarrow$ a reference to itself.
- `..` $\rightsquigarrow$ a reference to its parent directory.
- Entries for all contained files (pointing to their i-nodes).

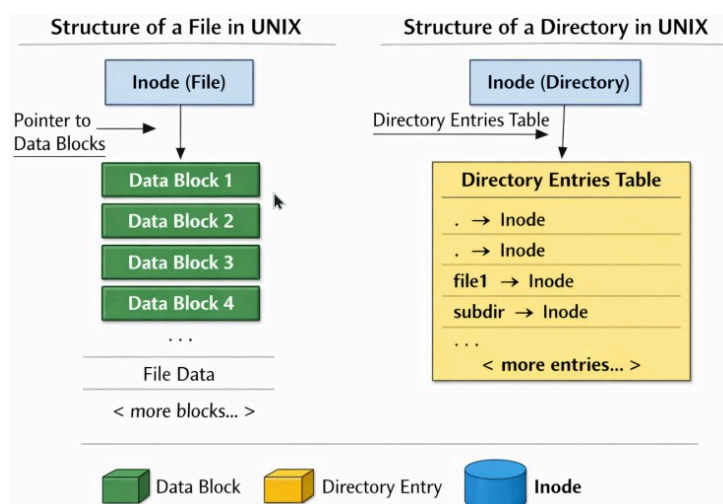


Figure 25 - Inode comparison (file vs. directory).



4.3. Consequences of Per-Primitive Atomicity

4.3.1. Non-Determinism Under Concurrent Writes



Two Processes Writing the Same File

In UNIX, two processes may open the same file with different I/O pointers (two separate `open` calls):

- P1 writes at position 0: "AAAAA"
- P2 writes at position 3: "BBBBB"

Each write is individually atomic, but the ordering between the two is not defined (non-determinism in scheduler). The final pattern depends on scheduling:

- "AAABBBBB" (P1 first, P2 second $\leadsto$ P2 overwrites from position 3)
- "AAAAABBB" (P2 first, P1 second $\leadsto$ P1 overwrites from position 0)

If both insist on the same position, either "AAAAA" or "BBBBB" wins, it is unpredictable. No application-level isolation is provided by the kernel.



No Cross-Primitive Consistency

UNIX guarantees atomicity **per primitive only**, not across sequences of primitives. Any compound operation (read-then-write, check-then-act) is subject to race conditions if multiple processes operate concurrently on the same files. Applications must implement their own locking or coordination mechanisms (e.g., `fcntl` locks, lock files, atomic rename).

4.3.2. Directory Listing Under Concurrent Modification



Recursive Directory Listing as a Moving Target

In UNIX, listing a directory recursively (e.g., `ls -R`) makes many separate `opendir()` and `readdir()` calls. The kernel guarantees each `readdir()` call is atomic, but the whole traversal is not.

If files are being created or deleted concurrently:

- Some listed files may no longer exist by the time they are printed.
- Other files created during traversal may not appear at all.

The entire listing is **inconsistent**, it is **not a snapshot** but a sequence of many pictures of a moving target. This is **eventual consistency at the primitive level**, not an atomic view.

🤔 **Why** not make the whole traversal atomic?

Making a recursive traversal atomic would require locking the entire subtree for the duration. This would be prohibitively expensive and would block all concurrent access. UNIX's design philosophy is to keep the kernel simple and minimal, providing mechanisms, not policies. Applications that need consistent snapshots must implement them at a higher level (e.g., copy-on-write filesystems, snapshots, or versioned databases).

Analogy: The Panoramic Scan Error

UNIX per-primitive atomicity is like taking a panoramic photo while moving the smartphone. Each single vertical slice captured by the sensor is crisp and accurate (atomic `readdir()`), but the entire sweep takes time. If the subject moves during the scan, the final image will be distorted and inconsistent (showing duplicates or missing



pieces of a moving target). A true atomic snapshot would require a 360 camera that freezes the entire landscape instantly, but that would mean forcing everyone in the scene to stand perfectly still during the shot.



5. COMPO- NENTS, MICROSERVICES, AND CONTAINERS

Package: Components, Microservices, Container – [a - Microservices Container.pdf](#)

Modern distributed applications are no longer deployed as single blobs of code. This chapter traces the path from **monolithic** architectures to **microservices**, then explains how **containers** solve the hosting, isolation, and reproducibility problems that microservices introduce.

5.1. From Monolith to Microservices

5.1.1. The Monolith

Software Monolith

A **monolith** is an application built as a **single deployable unit**: one codebase, one build pipeline, one technology stack, deployed all-at-once.

Monolith Benefits

- **Simple mental model**: developers work on a single, unified codebase.
- **Single deployment unit**: one artefact to build, ship, and run.
- **Simple scaling model**: run multiple identical copies behind a load balancer.

Monolith Problems

- **Codebase intimidation**: the **code base grows huge** and difficult to navigate.
- **Slow tooling**: refactoring takes minutes, builds take hours, CI test suites take days.
- **Coarse-grained scaling**: a single bottleneck forces scaling the **whole system**, even unused parts, extremely resource-intensive.
- **Infrequent deployments**: **re-deploying means halting the whole system** failed deploys increase the perceived risk of any change.
- **Technology lock-in**: the whole system is bound to one stack, even when parts would benefit from a different one.

5.1.2. Microservices

Microservice

The **microservice** architectural style is an approach to developing a single application as a **suite of small, independently deployable services**, each running in its own process and communicating via lightweight mechanisms (typically HTTP/REST). Services are built around



business capabilities, may be written in different languages, and use different data-storage technologies.



Core Properties of a Microservice

- **Runs in its own process** ~> faults are isolated ⇒ one service crashing does not bring others down.
- **Deployed independently** ~> a new version of one service does not require re-deploying anything else.
- **Scales independently** ~> identified bottlenecks can be addressed directly without scaling the whole system.
- **Owns its data** ~> no shared databases, **each service manages its own storage** (**Bounded Context**).
- **Polyglot** ~> **each service can choose the language and stack** best suited for its task.
- **Single responsibility** ~> each service does one thing well.
- **Composable** ~> services are combined to build higher-level functionality.
- **Small team** ~> a microservice must be manageable by a single development team.
- **Decentralized governance** ~> no system-wide technology mandates.
- **Culture of automation** ~> continuous integration and deployment are non-optional.

🤖 **Why** a microservice own its data??

Sharing a database between services creates hidden coupling: a schema change in one service can silently break another. Owning data forces explicit, **versioned interfaces between services** and enables each service to choose the storage model (relational, document, time-series...) best suited for its access patterns.

5.1.3. Benefits of the Microservice Decomposition

Independent codebase: each service has its own repository. Tools are fast (build/test/refactor in seconds), startup is quick, and there are no accidental cross-dependencies between codebases.

Independent technology stack: teams can select the best tool for each job and experiment with new technologies at low risk – a bad experiment affects only one small service.

Fine-grained scaling: only the bottleneck service needs to scale. **Data sharding** can be applied per service. Services that are not bottlenecks stay simple and unscaled, reducing resource waste.

Independent feature evolution: a new version of a service can be deployed without touching the rest of the system, up to and including a complete rewrite (as long as the **service interface** remains stable).



The Cost of Microservices

Microservices do not come for free:

- Many services ⇒ many deployment units ⇒ **more operational complexity**.
- **Automation** is not optional, manual deployment at microservice scale is impossible.
- Distributed system problems (network failures, partial outages, data consistency) become routine.



5.1.4 Prerequisites Before Adopting Microservices



Microservice Prerequisites

- **Rapid provisioning**: dev teams must be able to automatically provision new infrastructure.
- **Basic monitoring**: essential to detect and diagnose problems across many services.
- **Rapid application deployment**: deployments must be controlled, traceable, and rollback-friendly.

Analogy: You Build It, You Run It

In a monolith, a separate ops team manages production. In a microservice world, the team that writes a service is also responsible for running it in production. This forces engineers to think about operability from day one, but it also means the team that knows the code best is the one paged at 3 a.m.

5.2. Hosting Problems

Once services are broken apart, **where and how do you run them?** Four fundamental problems arise:

- **Process isolation**: services must not interfere with each other's memory or resources.
- **Process management**: start, stop, and monitor independent processes reliably.
- **Process packing**: fit multiple services on shared hardware efficiently.
- **Reproducible environment**: guarantee the same execution environment across development, test, and production.

Running everything directly on **bare metal** solves none of these: services cannot be isolated, dependency conflicts arise, resource allocation is unpredictable, and reproducing an exact environment on a different machine is manual and error-prone.

5.3. Process Isolation



Process

A **process** is the OS representation of a running program. It has a private **memory space** and kernel data structures tracking its execution state.

UNIX isolates processes by default through:

- **Memory space isolation**: each process has its **own virtual address space**.
- **Privilege isolation**: a process inherits the privileges of the user that created it.

But default UNIX isolation is coarse ~> processes share the same filesystem root, the same network stack, the same PID namespace, etc.

Containers extend isolation to cover all of these dimensions.

Containers are processes with their own "system" space we can say... 😊

5.3.1. A Brief History of Container Isolation

From the first **chroot** (1979) to modern container runtimes:

- **1979 – Unix V7** **chroot**: confines a process to a subtree of the filesystem.
- **2000 – FreeBSD Jails**: extends **chroot** with network and process isolation.
- **2006 – Process Containers (Google)**: resource-limit groups, later renamed **cgroups**.



- 2008 – LXC: first complete Linux container implementation using namespaces + cgroups.
- 2013 – Docker: wraps LXC (later its own runtime) with a developer-friendly image format and tooling.
- 2015 – Open Container Initiative (OCI): industry standard for container image and runtime specs.

5.3.2 Linux Namespaces

Linux Namespace

A **namespace** limits the **scope of kernel names and data structures** at process granularity
 ~> each namespace gives processes an **isolated** view of a system resource .

Available namespace types:

- **MNT** ~> mount points (filesystem view)
- **PID** ~> process IDs (a container sees only its own processes)
- **NET** ~> network devices, stacks, ports
- **IPC** ~> message queues, semaphores, shared memory
- **UTS** ~> hostname and NIS domain name
- **USR** ~> user and group IDs
- **CGRP** ~> cgroup root directory

100 Prof. Question: Can you name some namespaces? (at least 3)

MNT (filesystem), **PID** (process IDs), **NET** (network stack and ports), **IPC** (shared memory, semaphores), **UTS** (hostname), **USR** (user/group IDs), **CGRP** (view of the cgroup hierarchy).

By **combining namespaces** a container gets an **isolated view of the world** while still sharing the **same kernel** as other containers.

👉 Note: Full isolation is only the **default**. A container can also **share** a namespace with the host or with another container when that is useful. A monitoring agent that must see the host processes is given the **same PID namespace**, and the containers of a Kubernetes Pod share one **NET namespace** so they reach each other on **localhost** .

100 Prof. Question: Can two containers share one or more namespaces? Does it make sense?

Yes. A monitoring container shares the **PID namespace** to read other processes while keeping its own files (separate **MNT**). Sharing **all namespaces** makes them behave like plain host processes, as in a K8s Pod. Even then containers still pay off, because **packing, reproducibility** and **management** do not depend on isolation.

Under the hood, namespaces work by decoupling a process from the global system tables and mapping its execution to private kernel pointers tree structuring links. Every node can see only itself. This abstraction enables dynamic translations, most notably within the **PID** namespace, where the main containerized process is assigned **PID 1** inside its isolated scope—behaving like the system **init** process—while the host OS simultaneously tracks it using a standard global PID. Similarly, the network stack isolation enforced by the **NET** namespace is bridged to the physical world by creating a virtual ethernet pair (**veth**), which acts as a software patch cord routing traffic between the container's private stack and the host's network bridge.

Analogy: The Isolated Apartment

Namespaces are like separate apartments in the same building. Each tenant sees their own private space (filesystem, network, processes) but the building's structural elements (the kernel) are shared. Walls prevent tenants from seeing or interfering with each other, but they all pay rent to the same landlord.



5.3.3. Linux Control Groups (cgroups)



cgroups

cgroups (Control Groups) are a kernel mechanism to **limit**, **account for**, and **isolate** the **resource usage** of groups of processes.

Key controllers:

- **cpu** → enforce fair CPU time distribution among groups of processes.
- **memory** → set soft and hard memory limits, hierarchical, child cgroups contribute to ancestor totals.
- **net_cls** → tag network packets with a class identifier for traffic shaping.
- **cpuset** → pin processes to specific CPUs or NUMA nodes.



Note: Namespaces provide **visibility isolation** (what a process can see), while cgroups provide **resource isolation** (how much a process can consume). Together they form the foundation of every Linux container.



Prof. Question: What mechanisms enable isolation, and what is the difference between namespaces and cgroups?

Namespaces isolate **visibility** (what a process can see). **cgroups** isolate **resources** (how much it can consume). Both apply to a process that still shares the host kernel.

Conversely, control groups manage physical resource boundaries rather than visibility, exposing their interface to user-space through a virtual filesystem mounted at `/sys/fs/cgroup/`. Setting consumption ceilings simply involves writing raw configuration values into these kernel-managed text files. Memory allocation is governed by strict **Hard Limits**, meaning that if processes exceed their defined byte threshold, the kernel triggers the **OOM (Out Of Memory) Killer** to instantly terminate the offender. On the other hand, CPU allocation relies on **Soft Limits** and proportional shares, which do not artificially throttle performance when the host is idle, but instead guarantee a fair, weighted slice of CPU cycles only during resource contention.

Analogy: The Isolated Apartment (Extended)

- **Namespaces (The Walls & Meters):** Give each tenant their own apartment numbers (PID), their own private Wi-Fi router (NET), and their own front door key (MNT). They cannot see into other apartments.
- **Cgroups (The Circuit Breakers & Valves):** Are the utility caps imposed by the landlord. If a tenant uses too much water (Memory), the main valve shuts off instantly (OOM Killer). If everyone turns on the AC at the same time (CPU contention), the landlord throttles the power dynamically based on who pays a higher contract (CPU shares).

5.4 Containers



Container

A **container** is a **lightweight** form of **process virtualization** built on kernel **namespaces** and **cgroups**. It runs as a normal process but has a **restricted, isolated view** of the system (its own filesystem root, its own network stack, its own PID space) while **sharing** the host kernel with other containers.



Container vs VM

- **VMs** virtualize hardware: each VM runs a full OS on a hypervisor. Strong isolation, heavy overhead.



- **Containers** virtualize the OS user space: all containers share the host kernel. Much lighter, faster startup (seconds vs. minutes), but isolation is weaker than a VM.
- **No emulation**: because a container shares the host kernel, it **cannot present hardware the host lacks** (a different CPU architecture, an absent GPU). A VM can, because it virtualizes the hardware itself. This is the line between **virtualization** (same architecture, shared efficiently) and **emulation** (different architecture, reproduced in software).
- **Bare processes**: no isolation at all, no resource limits, no reproducible environment.

👉 **Note**: The container-vs-VM line is blurring. **MicroVMs** such as AWS **Firecracker** run each workload in a stripped-down virtual machine, giving **VM-grade isolation at almost container speed and density**. AWS Lambda and Fargate use them to run untrusted code safely.

⚠ **Why Containers for Microservices**

Containers answer all four hosting problems at once:

- **Isolation**: namespaces prevent services from interfering.
- **Management**: container runtimes provide start/stop/inspect/restart primitives.
- **Packing**: cgroups allow fitting many containers on one host with predictable resource budgets.
- **Reproducibility**: the container image bundles the application with all its dependencies. The same image runs identically on any machine.

100 **Prof. Question**: Tell me about Linux containers. What are the 4 fundamental characteristics?

A container is just a **process with a very restricted view of the world**, sharing the host kernel (it is not a VM). The four characteristics: **Isolation** (namespaces + cgroups), **Process management** (lifecycle), **Process packing** (app bundled with its dependencies), **Reproducible environment** (identical from dev to prod).

5.5. Docker

Docker is the dominant **container engine**: a platform for configuring, building, distributing, and managing the full lifecycle of containers.

5.5.1. Docker Engine Architecture

Docker follows a **client-server** architecture:

- **Docker daemon** (`dockerd`): background service that manages containers, images, networks, and volumes.
- **Docker CLI / REST API**: client that sends commands to the daemon.

The daemon handles the **container lifecycle**: create → start → pause → unpause → stop → remove.

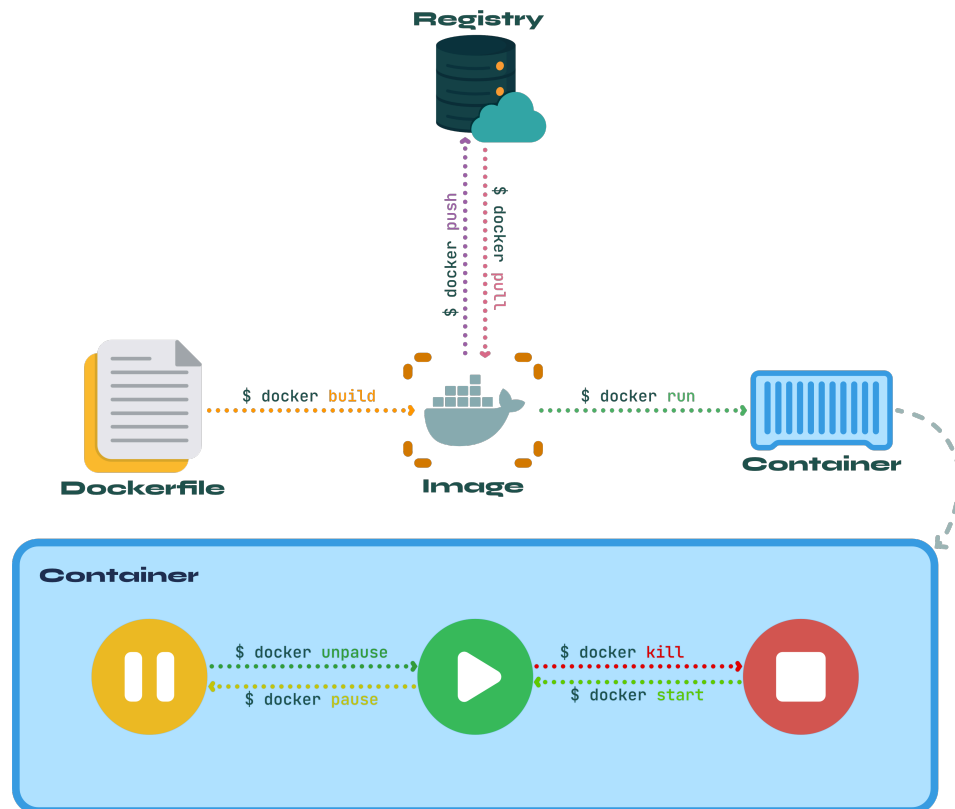


Figure 26 – Docker operation on container(s) and container life-cycle.

5.5.2. Docker Images and Layers

Docker Image

A **Docker image** is a read-only, layered filesystem snapshot that bundles the application binary with its OS libraries, runtime dependencies, and configuration. Each layer records a set of filesystem changes. Layers are identified by cryptographic content hashes (SHA-256) and shared across images to save space.

- **Image layers are read-only**. Every `RUN`, `COPY`, or `ADD` instruction in a Dockerfile creates a new immutable layer.
- **Container layer**: when a container starts, Docker adds a thin **read-write layer** on top. Writes go here, but they do not modify the image.
- **Copy-on-Write**: reading a file reads through the stack, writing copies the file into the R/W layer first.

Why layered images?

Layers are shared across images. If 10 services all base on `ubuntu:22.04`, the base layers are stored once on disk and in the registry. Only the diff layers unique to each service need to be pulled or pushed, making distribution efficient.

100 Prof. Question: Describe the lifecycle from development to container execution.

The path is **Dockerfile** → **Image** → **Container instance**. Develop the Dockerfile, `build` it into



a layered image, `push` it to a **registry**, `pull` it in another environment, `run` it (adds the R/W layer), then manage it with `stop` / `start` / `rm`.

5.5.3. Dockerfile

A `Dockerfile` is the **recipe** for building an image :

- **Starting image** (`FROM`): base layer to build on.
- **Configuration instructions** (`ENV`, `ARG`, `EXPOSE`): metadata, no new layer.
- **Filesystem instructions** (`RUN`, `COPY`, `ADD`): `modify the filesystem` $\Rightarrow$ `create a new layer`.
- **Multi-stage builds**: use intermediate images to compile code, then copy only the built artefact into the final image, keeping it small.

100 Prof. Question: *Can you write an example Dockerfile? How does it map to the images it creates?*

A Dockerfile is the **contract** for building an image. **Metadata** instructions (`FROM`, `ENV`, `EXPOSE`, `CMD`) add no layer. **Filesystem** instructions (`RUN`, `COPY`, `ADD`) each create one **immutable layer**, hashed by content (SHA-256) and shared across images. A running container adds a thin **read-write** layer on top (Copy-on-Write).

5.5.4. Docker Networks

Docker provides several network drivers:

- **Bridge** (default): software bridge $\leadsto$ containers on the same bridge can communicate, others are isolated.
- **Host**: `removes network isolation`, the container shares the host's network stack directly.
- **Overlay**: software-defined network spanning multiple Docker hosts (used in Swarm/orchestration).
- **Macvlan**: assigns a MAC address to the container, appears as a physical device on the LAN.
- **None**: disables all networking.

5.5.5. Docker Plugins

Plugins extend the Docker Engine with third-party functionality:

- **Volume plugins**: allow volumes to persist across hosts (e.g., NFS, cloud storage backends).
- **Network plugins**: advanced networking policies and configurations.
- **Authorization plugins**: access control within Docker environments.
- **Log plugins**: route container logs to external systems.

5.5.6. Configuration and Secrets

Docker provides three mechanisms to pass configuration to containers:

- **Environment variables**: simple key-value pairs injected at runtime.
- **Config files**: mounted into the container filesystem.
- **Secrets**: encrypted, stored separately, mounted only in memory, for sensitive data (passwords, tokens).



5.5.7. Docker Compose

`docker-compose` (or `docker compose`) is a tool for defining and running multi-container applications using a single YAML file.

It allows specifying services, their images, networks, volumes, and dependencies in one place, enabling the entire application stack to be brought up with a single command (`docker compose up`).

Docker Compose is the natural stepping stone between running containers manually and using a full orchestrator like Kubernetes. It works well for local development and small deployments, but does not handle cross-host scaling or self-healing (that is Kubernetes's domain).

5.6. Microservice Resilience Patterns

When services call each other over the network, failures in one service can cascade across the whole system. Two foundational patterns address this: the **Circuit Breaker** and **Retry/Timeout** policies.

5.6.1. Circuit Breaker

Circuit Breaker

The **Circuit Breaker** is a stability pattern that prevents **cascading failures** in a distributed system. It wraps calls to a downstream service and monitors their success/failure rate. When failures exceed a threshold, the breaker **trips open** and short-circuits all subsequent calls immediately, returning a fallback response without touching the broken service, giving it time to recover.

The breaker operates as a **three-state machine**:

- **CLOSED** (normal): all requests flow through. The breaker tracks outcomes in a rolling window. If failures exceed the **Failure Rate Threshold**, it trips to OPEN.
- **OPEN** (broken): all requests are immediately short-circuited, no call reaches the downstream service. A **Sleep Window** timer runs → when it expires, the breaker moves to HALF-OPEN.
- **HALF-OPEN** (probing): a limited number of trial requests are let through. If they succeed, the breaker returns to CLOSED. If any fails, it goes back to OPEN and resets the timer.

Analogy: Electrical Breaker

The pattern is named after the electrical circuit breaker.

An electrical circuit breaker monitors current. If it spikes, it trips to protect the wiring and appliances. The software circuit breaker monitors call failure rate. If it spikes, it trips to protect the upstream service's resources and let the downstream service recover.

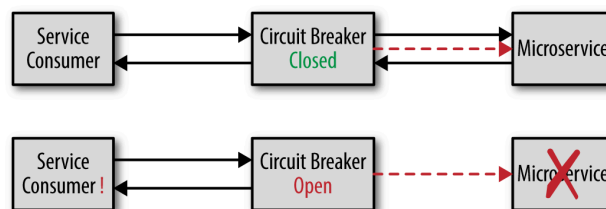


Figure 27 – Circuit Breaker pattern.

Key Configuration Parameters

- **Failure Rate Threshold** → percentage of failures (e.g., 50%) within the rolling window that trips the breaker OPEN.



- **Sliding Window Size** → sample size for computing the failure rate: count-based (last N calls) or time-based (last N seconds).
- **Minimum Number of Calls** → minimum volume required before the failure rate can be calculated (prevents premature tripping on 1-2 early failures).
- **Wait Duration in Open State** → length of the Sleep Window before the breaker tries HALF-OPEN.
- **Permitted Calls in Half-Open** → how many trial requests probe the downstream health.
- **Slow Call Rate Threshold** → calls exceeding a duration threshold are treated as failures.

⚠ **Benefits of Circuit Breaking**

- **Prevents cascading failures** : a slow downstream service cannot exhaust thread pools in upstream services.
- **Graceful degradation** : return cached data or a default response instead of a hard crash.
- **Self-healing** : downstream services get breathing room to recover without being bombarded by retries.
- **Fast failure** : instead of blocking for a 10-second timeout, the caller gets an instant response.

👉 **Note**: Circuit breakers should **not** trip on 4xx client errors (e.g., `404 Not Found`), those indicate caller mistakes, not service health. Only trip on 5xx server errors, network timeouts, and connection failures.

5.6.2 Implementation

Circuit breakers can be implemented at two levels:

- **Application level**: embed a library in each service. Common choices: **Resilience4j** (Java/Spring Boot), **Polly** (.NET), **go-resiliency** (Go). Netflix's **Hystrix** is legacy/archived.
- **Infrastructure level (Service Mesh)**: the circuit breaker logic lives in a sidecar proxy, transparent to application code. **Istio/Envoy** uses outlier detection, **Linkerd** tracks failure rates per cluster. This keeps application code free of resilience plumbing.



6. MIDDLEWARE AND CLOUD MODELS

Package: Middleware and Cloud - [Middleware and Cloud 25.pdf](#)

This chapter covers two deeply connected topics:

1. **Middleware** → the software layer that glues distributed applications to heterogeneous infrastructure.
2. **Cloud computing** → which is, in large part, what middleware has evolved into at industrial scale.

6.1. What is Middleware?



Middleware

Middleware is the **software layer** that sits **between applications and low-level support** (hardware, local OS, network technology). It provides a **uniform access API** to intrinsically heterogeneous local functions, allowing applications to be designed, deployed, and evolved independently of the underlying infrastructure.

The term goes back to **1968**, coined at a NATO school on Software Engineering. It became significant in the **1990s** when distributed systems became widespread.

Why Middleware exists?

- **Hide component and resource physical distribution:** make the split of an application across different machines transparent.
- **Hide heterogeneity:** abstract over different hardware, OSes, protocols, and data formats.
- **Provide common interfaces:** allow legacy parts to be composed and reused without modification.
- **Provide basic services:** naming, discovery, fast storage, parallel processing, security.
- **Grant availability and QoS:** manage the system's quality properties at runtime.

Analogy: Middleware as a Universal Adapter

Heterogeneous systems are like devices with different plugs from different countries. Middleware is the universal adapter: you plug in whatever device you have, and the adapter takes care of the conversion. Without it, every integration requires a custom cable, transformer,...

Another definition: middleware is a decoupling layer among all system layers. It allows a continuous simplified design of any application part, and also of the support part itself, by allowing any overcoming of intrinsic heterogeneity.

6.1.1. Middleware Support Functions



What Middleware Supports

- **Presentation Management:** print, graphics, GUI, user interaction.
- **Computation:** common procedures, character services, sorting, parallelization, fast access, internationalization.
- **Information Management:** file manager, record manager, database manager, log manager.
- **Communication:** messaging, RPC, message queues, mail, electronic data interchange.



- **Control:** thread manager, scheduler, transaction manager, activity planning.
- **System Management:** accounting, configuration, security, performance, fault management, event handling.

6.2. Middleware in a Layered View

Middleware is not a single monolithic layer but a **stack of four sub-layers**, from bottom to top:

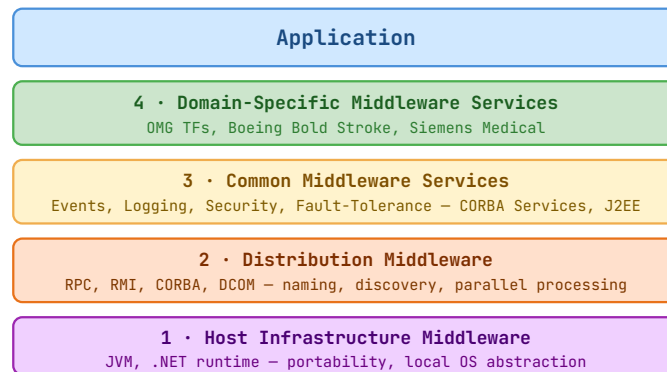


Figure 28 – The four middleware layers, from host infrastructure to domain-specific services.

6.2.1. Host Infrastructure Middleware

The lowest layer. It **encapsulates and prepares common services** to support distribution and ease communication.

It **provides portability**: some APIs are unified toward a single support across different environments.

Examples: JVM, .NET, other local runtime models.

6.2.2. Distribution Middleware

This layer provides the **programming models for distribution** and eases applications in configuring and managing distributed resources.

It allows an **easier communication and coordination** of all nodes in the system by introducing:

- A **resource model**: a conceptual model for naming and accessing distributed resources.
- **Communication APIs**: proposing and enforcing a new conceptual model.
- Other basic functions: **name support, discovery, fast storage and access, parallel processing.**

Examples: RPC, RMI, CORBA, DCOM, .NET, SOAP.

6.2.3. Common Middleware Services

Added-value services, typically higher-level, to facilitate the duties of the designer and enforce a component-oriented perspective.

Several **additional services** can be added at any time depending on needs: **events, logging, streaming, security, fault tolerance, ...**

Examples: CORBA Services, J2EE, .NET Web Services.



6.2.4 Domain-specific Middleware Service

A set of application tools and services **grouped according to specific domains**, defined by task forces within standards bodies (OMG). Always defining standards.

Examples: Electronic Commerce TF, Finance/Fintech TF, Life Science TF, Boeing Bold Stroke (flight transport), Syngo Siemens Medical Engineering.

6.3. Taxonomy of Middlewares

Taxonomy

The main middleware families are:

- **RPC / RMI**: Remote Procedure/Method Call, **synchronous client-server**.
- **MOM** (Message Oriented Middleware): **asynchronous message exchange**.
- **DOC** (Distributed Object Computing): **object-oriented** distribution.
- **DTP Monitor** (Distributed Transaction Processing): **ACID distributed transactions**.
- **DB Middleware**: Database integration and access.
- **Adaptive & Reflective**: self-adapting to the application context.
- **Self-* Middleware**: autonomic systems (self-configuring, self-optimizing, self-healing, self-protecting).

Special-purpose:

- **Mobile & QoS Middleware**
- **Agent-based Middleware**

6.3.1. RPC / RMI Middleware

Remote Procedure Call

RPC exposes **remote services** as if they were local function calls. The client uses an **IDL** (Interface Definition Language) to define the contract. A **stub** on the client side serializes the call while the server-side **skeleton** deserializes and invokes the real function.

RPC Properties

- **Synchronous**: the client blocks waiting for the result from the server.
- **Heterogeneous data handling**: stubs serialize/deserialize across different representations.
- **Binding** often static: the server endpoint must be known upfront.

RPC Limitations

The RPC model is too **rigid, not scalable, and not replicable with QoS**:

- The server design must be explicit, any provisioning must be explicitly defined.
- No easy sharing of private and public resources.
- Not flexible enough as services grow and evolve.
- **RMI evolves into "in-the-large", RPC stays "in-the-small"**.



6.3.2 MOM

Message Oriented Middleware

MOM distributes data and code via **message exchange** between **logically separated entities**. Messages can be typed or untyped, synchronous or asynchronous. A **broker** manages delivery with different strategies and QoS levels.

MOM Properties

- **Wide autonomy** between components: **sender and receiver are fully decoupled**.
- **Asynchronous and persistent actions**: messages survive sender/receiver disconnections.
- **Handler/broker** with configurable strategies and QoS.
- Easy support for **multicast, broadcast, publish/subscribe**.

 **Note**: Persistent messages are useful even when not explicitly requested: messages are not lost if the receiver is temporarily unavailable. Non-persistent messages are dropped if the receiver is absent.

Examples: MQSeries (IBM), MSMQ (Microsoft), JMS (Sun), DDS, MQTT, RabbitMQ, ActiveMQ.

6.3.3 DOC / OO Middleware

Distributed Object Computing

DOC distributes data and code via **operation requests** and **replies** between **clients and remote servers**, using object-oriented languages. A **framework** and a **broker** act as intermediaries for operation object handling.

DOC Properties

- The **object model** simplifies design.
- The **broker** provides base services and can automate some operations completely.
- **System integration** is easier and effective.
- Usually **open source**, implementations can be very **scalable and available**.

 **Note**: DOC **clients and servers block** during the call (clients wait for the server to complete the request). The broker mediates so that clients do not need to know the server's physical location.

 **Note**: Typically **server is stateless**, clients keep the state.

Examples: CORBA, COM, .NET, Java Enterprise.

6.3.4 DTP Monitor

Distributed Transaction Processing Monitor



A **DTP Monitor** is a middleware to declare and support **distributed transactions**, ensuring **ACID** (Atomicity, Consistency, Isolation, Durability) guarantees across multiple distributed data stores.



DTP Features

- **Specialized interface** for queries by lightweight clients.
- **Standardized actions** and ad-hoc languages.
- **Multi-level applications** adopting flexible RPC (beyond synchronous semantics).
- **Efficiency** in the addressed applicative area.

👉 **Note:** Cloud providers do **not** guarantee full ACID: they typically offer **eventual consistency (BASE)**. DTP monitors are more suitable for on-premises deployments where strict consistency is required.

Examples: CICS (IBM), Lotus Notes, Tuxedo (BEA).

6.3.5. DB Middleware



Database Middleware

DB Middleware enables integration and eased usage of information stored in heterogeneous and different databases, hiding implementation-specific details behind standard interfaces.

The key standard is **ODBC** (Open DataBase Connectivity):

- Works **without requiring modification** to existing DBs.
- Emphasizes **data access** rather than optimization or transactions.
- **Only synchronous and standard operations**.
- Evolves toward **data mining**.

Examples: Oracle Glue, OLE-DB (Microsoft).

6.3.6. Adaptive & Reflective Middleware

Middleware able to **self-adapt** to the specific application, also in a **dynamic, reactive, and radical** way.

- **Static variations:** typically component-dependent.

→ They adapt the middleware to the architectural structure and protocols of the application, typically frozen at deployment.

- **Dynamic variations:** typically system-dependent.

→ They react at runtime to fluctuating infrastructural conditions, such as network latency or CPU load.

Via **reflection**, action policies are expressed and visible in the middleware itself and can change as system components, obtaining **adaptation and flexibility at execution time**.

👉 **Note:** Not yet widely deployed in production, but an active research area, especially relevant for cloud-edge continuum and IoT.



6.3.7. Self-* Middleware

Inspired by modeling computing systems as **human bodies**: capable of taking care of themselves and changing accordingly to life-cycle variations.

Complex systems organize as **self-managing** and **self-administering** entities. Also termed **self-*** (related to computer agents):

- **self-configuration** $\rightsquigarrow$ autonomy
- **self-optimization** $\rightsquigarrow$ social ability and cooperation
- **self-healing** $\rightsquigarrow$ reactivity
- **self-protection** $\rightsquigarrow$ proactiveness

6.3.8. Specialized Middlewares

- **Mobility Middleware**: transparent allocation and re-allocation across layers (network to application).
- **Enterprise Middleware**: EAI (Enterprise Application Integration): rapid prototyping and integration of existing enterprise tools.
Examples: SAP (enterprise management), Websphere/Oracle (IT/resource management), SOA.
- **Real-Time Middleware**: guarantees response times and deadlines for RT service development.
- **Ad-hoc Networking Middleware**: lightweight components for environments with limited resources and consumption capacities.

6.4. Middleware Usage Scenarios

Middleware comes in three archetypes based on how it is used:

6.4.1. Scenario 1: Minimum Cost Middleware

This scenario focuses on driving the configuration of a single application according to a fixed, internal interaction model, entirely excluding dynamic runtime scenarios. The system operates in a closed, highly stable environment, prioritizing **minimal operational cost and low software intrusion**.

Disappearing Middleware

A **disappearing middleware** is a lightweight, highly optimized middleware layer whose runtime footprint “disappears” because it avoids any dynamic management or discovery overhead.

- **Static Architecture**: It defines a fixed, statically determined set of nodes and hardware resources at deployment time.
- **Rigid Interaction**: Communication channels and interfaces are default, rigid, and non-adjustable, yet highly optimized for maximum performance.
- **No Operational Overhead**: It lacks support for dynamic reconfiguration, runtime resource provisioning (turning on/off resources), or dynamic service registries.

MOM

Traditional MOMs fall into this category: they are deployed with predefined queues and static topologies to ensure predictable, low-overhead message routing.



6.4.2 Scenario 2: Middleware for Fast Applications

Targeted at highly streamlined and optimized applications that require rapid, efficient service provisioning. Instead of relying on a heavy, permanent infrastructure, applications cooperatively provide services to one another, and the middleware dynamically coordinates these currently active components to self-adapt to the real-time usage situation.

On-Demand Integration Middleware

An **On-Demand Integration Middleware** is a flexible layer designed to facilitate straightforward cooperation and interoperability among running applications at execution time.

- **On-Demand Deployment:** The middleware infrastructure is spun up strictly on demand when applications need to interact (e.g., via Distributed Object Computing - DOC).
- **Tied Lifecycle:** The middleware's lifetime is tightly coupled to the lifecycle of the applications using it; it does not persist independently. When the last application exits, the middleware shuts down.

Microsoft Component Middleware

Classic Microsoft enterprise solutions (such as COM+ or early .NET components) exemplify this approach, where infrastructure services are loaded dynamically into the application process space on demand.

6.4.3 Scenario 3: Middleware for Continuity

This paradigm targets large-scale, mature, and comprehensive enterprise environments where the fundamental requirement is to extend the lifetime of services indefinitely, moving toward an **infinite Lifecycle**.

Evolving Ecosystem Middleware

An **Evolving Ecosystem Middleware** is a permanent, heavy-duty infrastructure that serves as a continuous, shared foundation for an organization's entire software ecosystem.

- **Coarse-Grained Services:** It hosts a comprehensive catalog of high-level, coarse-grained features made readily accessible to streamline application development.
- **Incremental Enrichment:** Rather than requiring reboots or redeployments, the middleware updates and enriches itself seamlessly over time; it is continuously **populated by different applications** that introduce new services at runtime.
- **Continuous Availability:** it strictly maximizes system lifetime by exhibiting absolute zero downtime during upgrades or component modifications.

CORBA & Enterprise .NET

Architectures like CORBA or enterprise-grade .NET environments act as permanent corporate backbones, constantly running and accumulating new institutional capabilities over decades.

6.5. Middleware Design Issues

As middleware grows in function set, several critical design tensions emerge:



- **Scalability:** the increasing set of functions (objects, resources, etc.) makes scalability very hard. Middleware tends to introduce **indirect and dynamic mechanisms** (interception) to enable management, introducing **overhead** that must be minimized.
- **Management costs:** require increasingly sophisticated monitoring, accounting, security, and control tools.
- **Mobile and dynamic devices:** need continuous adaptation to the current context and situation.



Core Design Tension

Every middleware must balance **functionality** against **intrusion**. Adding more services improves expressiveness but consumes resources that compete with the application. **Minimizing overhead** is always a first-class requirement.

6.6. The Problem Space

The drivers behind cloud computing:

- **Explosion of data-intensive applications** on the Internet.
- **Fast growth of connected mobile devices.**
- **Skyrocketing costs** of power, space, and maintenance in traditional data centers.
- **Advances in multi-core computer architecture.**

6.7. A Brief History: Before the Cloud



Grid Computing

Grid computing shares **heterogeneous resources** (compute, software, data, memory) in **highly distributed environments** to create a **virtual organization**.

Interfaces are often too fine-grained, with low abstraction levels and non self-contained. Application areas are very limited and specific (parallel computation for scientific/engineering scenarios). HPC is more **research-oriented** and not so likely to offer **industrial services for free market**.



Utility Computing

Utility computing offers computational and storage capabilities as a **utility**, like energy or electricity, on a **pay-per-use** base.

Other precursors:

- **Virtualization (2005+):** VMware, Xen, and server farm virtualization.
- **Web 2.0:** AJAX-based asynchronous UIs, SaaS prototypes (Google Docs, Salesforce).
- **Microservices & Orchestrators (2015+):** Docker, Kubernetes.



Cloud Computing = Intersection



Cloud Computing sits at the **intersection** of Grid Computing, Utility Computing, and Software-not-on-Premises. It inherits resource sharing from grid, pay-per-use from utility, and remote software delivery from SaaS.

6.8. What is a Cloud?



Cloud Computing

A **cloud** is an IT service that provides IT resources as a **Service**, delivered to users that have:

- A **user interface** that makes the underlying infrastructure **transparent**.
- **Massive scalability** on demand.
- A **service-oriented management** architecture.
- **Reduced incremental management costs** as additional IT resources are added.

Services are available via **Web or REST interfaces**; user requirements may vary based on **geographical preferences, localization constraints** (e.g., low latency, local law compliance).



Cloud Keywords

- **On demand**: resources provisioned as needed, immediately.
- **Reliability**: cloud implies a reliable context with agreed service levels.
- **Virtualization**: pools of virtualized compute resources.
- **Provisioning**: rapid live provisioning while demanding.
- **Scalability**: elastic architecture that scales with load.

6.8.1. How Cloud Differs from its Predecessors

- **vs. Software not on Premises**: a cloud is more than hosted software: it **manages** the resources (provisioning, workload balancing, monitoring) and sits on top of a data center for efficiency.
- **vs. Grid**: a cloud provides a **mechanism to manage** resources, not just share them.
- **vs. Utility Computing**: cloud users want to **neglect infrastructure**; the provider is in complete control. Grid/utility users want control over each server.

6.9. The SaaS / PaaS / IaaS Model

The cloud is delivered as a **layered architecture** of service models:



SaaS - Software as a Service

In **SaaS** resources are **simple applications** available via remote Web access. The user just uses the application, no infrastructure knowledge is needed.

Examples: Gmail, Google Docs, Salesforce CRM, Microsoft 365.



PaaS - Platform as a Service



In **PaaS** resources are **whole software platforms** available for remote execution: several programs capable of interacting with each other. Users can deploy and run their own applications on the platform.

Examples: Google App Engine, Azure App Service, Heroku.

IaaS - Infrastructure as a Service

Resources are offered in a **wider and complete way**, from hardware platforms to operating systems, to support final user applications. The user may control **any resource configuration**.

Examples: AWS EC2, Google Compute Engine, Azure VMs, OpenStack.

Layered Architecture and Actors

- **SaaS** → End Users (highest value visibility).
- **PaaS** → Application Developers. ~→ The most dangerous for **vendor lock-in**, developers build directly on proprietary platform APIs.
- **IaaS** → Network Architects (most control, lowest abstraction).

The layers build on each other: IaaS provides compute/network/storage, PaaS adds components and services on top, SaaS adds the user interface.

6.9.1. Architecture Comparison: What the Customer Manages

- **On-premises (Private Cloud)**: customer manages everything: applications, data, OS, virtualization, servers, storage, networking.
- **IaaS**: provider manages servers, storage, networking. Customer manages OS upward.
- **PaaS**: provider manages up through the runtime. Customer manages applications and data only.
- **SaaS**: provider manages everything. Customer just uses the application.

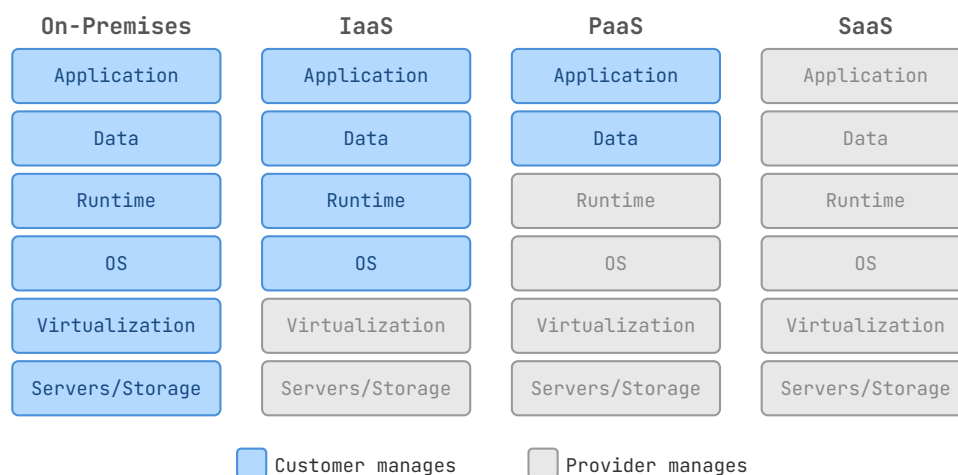


Figure 29 - Cloud service models comparison.



6.10. XaaS: Anything as a Service

XaaS

XaaS (Anything as a Service): all Cloud stakeholders provide the richest set of services for any possible user request, accompanying users toward the best choices (Storage as a Service, Container as a Service, and more).

Key extensions:

- **FaaS** (Function as a Service): the user specifies only functions. The provider activates them when triggering events occur, and can operate and support composition and results: the foundation of **serverless computing**.
- **BaaS** (Backend as a Service): the user is **not aware of all resources** needed. All services are provided as coordinated **-aaS**: storages of different kinds (block and object), configuration of all services, accessory services (intelligent tools).
- **MaaS** (Metal as a Service): gives only **native machines** to users who build on them. Adds all services for virtualization, storage, processing, backend, and interconnection design: but in the direction of **visibility** (the user sees and controls low-level details).

6.11. The NIST Cloud Definition Framework

The National Institute of Standards and Technology (NIST) provides the reference classification:

Essential Characteristics:

- On Demand Self-Service
- Broad Network Access
- Resource Pooling
- Rapid Elasticity
- Measured Service

Common Characteristics:

- Massive Scale, Geographic Distribution
- Homogeneity, Resilient Computing
- Virtualization, Service Orientation
- Low Cost Software, Advanced Security

Service Models: SaaS, PaaS, IaaS.

NIST Cloud Actors

The NIST reference architecture also names **five actors** in a cloud ecosystem:

- **Consumer**: uses the service.
- **Provider**: offers and runs the service.
- **Auditor**: an independent third party that assesses the service (security, performance, compliance).
- **Broker**: mediates between providers, and between provider and consumer, integrating or repackaging services.
- **Carrier**: provides the connectivity (network transport) that links consumer and provider.

Deployment Models:

Deployment Models

- **Private cloud**: enterprise owned or leased, the infrastructure available only to the single organization.



- **Community cloud**: shared infrastructure for a specific community (e.g., government, academic consortia).
- **Public cloud**: sold to the public, mega-scale infrastructure (AWS, Azure, GCP).
- **Hybrid cloud**: composition of two or more clouds (private + public): a company uses an internal data center coupled with one or more external clouds.
- **Multi-cloud**: an organization uses offerings from many different providers to optimize costs, scalability, efficiency, flexibility, and geographic constraints, and crucially to **reduce lock-in**. As of 2020, 93% of enterprises use multi-cloud strategies.

6.12. QoS-Related Properties

These **non-functional properties** are crucial to solution acceptance, especially on the long term:

- **Correctness**: consistency, stability, timeliness.
In the sense of giving/getting the expected result.
- **Efficiency**: common procedures, optimal usage of resources, prompt answer.
- **Scalability**: dynamic usage of resources, limited operating costs.
- **Robustness**: fault tolerance, replication, availability, reliability.
- **Security**: thread manager, scheduler, transaction manager.

A cloud provider going out of business has contracts with others that give them resources. Availability in the context of cloud means the system continues to operate even when components fail: the provider's infrastructure absorbs the failure transparently.

6.13. Cloud Architecture

In Cloud, resources must be considered in a more flexible way than traditional systems. You can define and command:

- **Logical resources**: already considered in classical distributed systems.
- **Physical resources**: already considered.
- **Virtual resources**: not only machines, but **any kind**: virtual networks, virtual storage, virtual functions.

You decide how to **map logical components over virtual resources**, and then how to **map virtual resources over physical ones**. The degree of freedom is very large, and so are the architectural choices and their impact on final behavior.

6.13.1. Data Center Organization

Cloud is typically organized in **different remote Data Centers** that host the storage and compute. They must be organized carefully to **favor local intra-DC organization** and the **inter-DC infrastructure**:

- Any family of data must be **based on replication widely localized**: several copies in different DCs and several ones in any of them.
- Any DC must optimize access to its copies and have **mechanisms to ease the access** (key-values, DHT, local ring configuration, ...).
- Some policies for **configuration** must be decided and actuated **out-of-band** (before data access) and also data **operations must be monitored and controlled during execution** (in-band monitoring, dynamic reconfiguration).



6.13.2. Data Center Network Topology

The DC does not use a flat network but typically **hierarchical interconnect machines** that can be optimized by exploiting specific dynamic connections.

Fat-Tree Topology

A **Fat-Tree** topology adds **more connections in layers**: more expensive but making traversal shorter, more fault-tolerant, and with enhanced bandwidth. Organized in **Pods**, each with Edge, Aggregation, and Core layers.

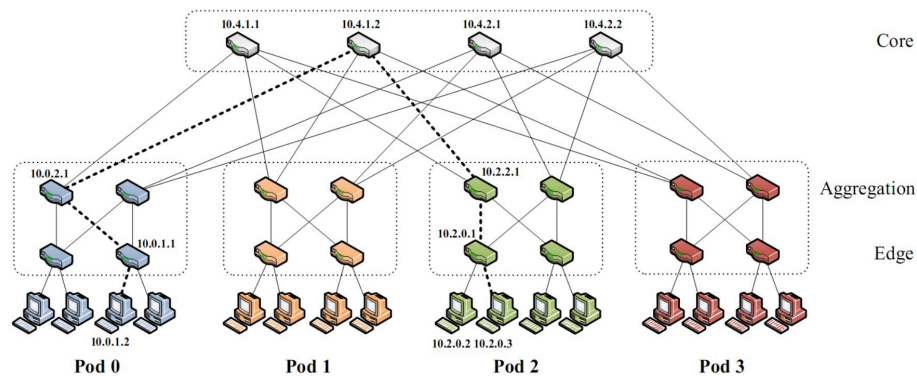


Figure 30 - Fat tree scheme.

Clos Network

A **Clos network** is a non-blocking multistage topology that achieves the **same goals** as a **Fat-Tree** (shorter paths, fault tolerance, high bandwidth through layered redundancy). It is the basis of modern DC **spine-leaf** architectures.

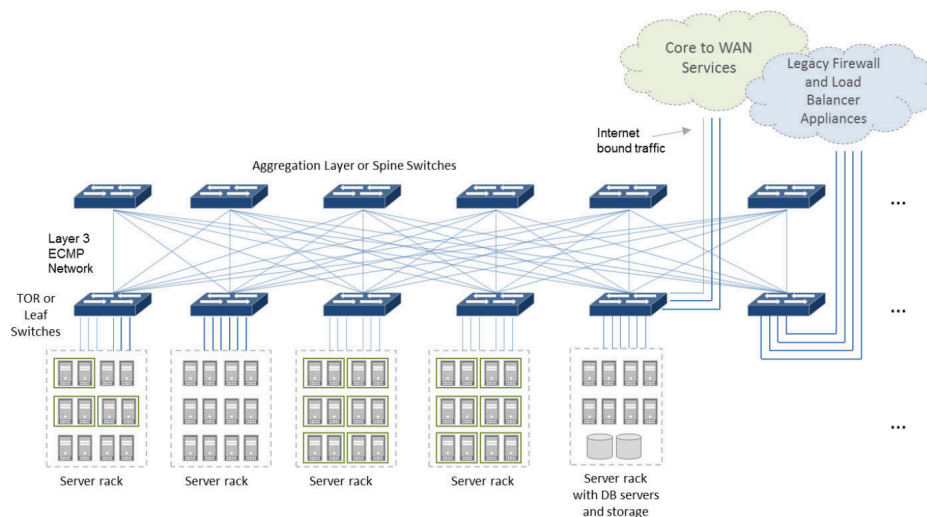


Figure 31 - Clos network scheme.

The larger the DC, the more interconnected (and expensive) it must be. The hierarchy is the standard model.



6.14. Cloud Management and Monitoring

6.14.1. Remote Management for QoS

In remote/outsourced environments, it is **compulsory to ascertain the current state** of the remote installation: not only for accounting but for actual management. A rich management interface must allow:

- **Access** to all user-related resources (processing, memory, persistent data, network, any -aaS).
- **Control** of consumption of any user-related resource (current state, history, peaks, trends, user-defined indicators).
- **Discovery** of new services and new available resources (off-the-shelf ready-to-use solutions).
- **Installation** of special user settings and environments (new service developed from composing available ones).
- **Enlargement** to **federated environments** for resource integration.

6.14.2. Areas of Cloud Management

- Provisioning and Orchestration
- Service Enablement
- Inventory and Classification
- Monitoring and Observability
- Identity, Security and Compliance
- Cloud Migration, Backup and DR
- Cost Management and Resource Optimization
- Automation
- Life Cycle management

6.14.3. Cloud Monitoring

Cloud monitoring is available at two levels:

- **Infrastructure (internal):** physical and virtual resource monitoring with many-to-many communication for **fine-grained** local monitoring. Cloud nodes publish status via a Data Distribution Service, subscribers feed schedulers and status monitors.
- **Customer-facing:** evolved monitoring functions to control customer expenditures.

Common features across all platforms:

- Build your own dashboard.
- Define your own alarms.
- Define your retained state.
- Define data collections and analytics.

Key platforms: Amazon CloudWatch, Google Cloud Monitoring.

6.15. Middleware for Cloud



Cloud from the Provider Perspective

From the **user perspective**: provisioning of virtualized resources obtained in an elastic and fast way, in any phase of user request.



From the **provider perspective**: provide services (**-aaS**) according to agreed SLA, following two principles:

- **Efficiency**: respond to all users.
- **Effectiveness**: carefully use available resources.

Every provider uses its resources and finds the best mapping of configurations and QoS for better services.

The key scenarios going forward are **many-to-many**:

- **Customers** interested not only in one provider's resources but in **balancing** across multiple providers in accordance with internal policies.
- **Federation** between Cloud providers to exchange resources and services.
- **Cloud as integrator** of software resources (**full stack or IPaaS** ~> Integration Platform as a Service).

👉 **Note**: Standardization is unavoidable: clear roles and responsibilities, open source standards and implementations, integration with existing protocols (mobile access), supports for sustainability, and global/local legal clarity.



7. CLOUD AND DC GLOBAL STRATEGIES

Package: Cloud and Data Center Global Strategies - [Cloud and DC Global Strategies.pdf](#)

A Cloud is not just a collection of servers: it is a **big data center (DC)**, federated with other ones, capable of giving **good and fast answers** to users. Two contrasting requirements must be balanced simultaneously:

- **Speed**: users expect very fast answers $\Rightarrow$ the system must never make them wait too long.
- **Safety/Correctness**: answers must be correct and data must be persistent.

In globally distributed environments, many copies of data reside in many geographical locations, making these requirements even harder to satisfy together.

7.1. Cloud Data Center Architecture

The Cloud DC serves users through a **two-level architecture**. The two levels have very different responsibilities and trade-offs.

7.1.1. Level 1: Cloud Edge

Cloud Edge (Level 1)

The **Cloud Edge** is the layer very close to the client, responsible for **fast answers** to user needs. It handles many possible client requests, including concurrent ones with user reciprocal interaction. Its first requirement is **velocity**.

- **Read operations**: replication makes **parallel reads easy**, no problem.
- **Write operations**: updates are tricky. Edge uses a **guessing model**: try to forecast the update outcome and answer fast, but **operate on the update in the background with level 2**.

🤔 **Why** a guessing model?

Waiting for all replicas to confirm a write would make the response slow. Instead, the **edge optimistically predicts** the result and sends the answer immediately, then reconciles asynchronously. If the guess is wrong, **corrections happen later**.

7.1.2. Level 2: Cloud Internal

Cloud Internal (Level 2)

The **Cloud Internal** layer is responsible for **stable, correct answers** to queries given to level 1. It is hidden from users and focused on **deep data management**, replication, and consistency. It is in charge of replicating data and keeping caches to favor user answers.

- Replication policies do **not** require replicating everything: **only significant parts** (called **shards**) are replicated, dynamically decided based on usage.



- The second level optimizes data in terms of **shards** and also supports many forms of **caching** services (memcached, Dynamo, Bigtable, ...).

Analogy: Two levels as front-office and back-office

Level 1 is the front desk: it must answer immediately, even if it does not have the most up-to-date information yet. Level 2 is the back office: it processes everything carefully, keeps the real records, and eventually reconciles the front desk's guesses.

7.1.3. Two-Level Replication

Both levels replicate, but with different goals:

- **Edge (Level 1):** resources are replicated for user demands and to answer within the negotiated SLA. Replication is tailored to user needs, transparent to the final user.
- **Internal (Level 2):** replication supports user long-term strategies and mutual replication across areas. The whole load drives decisions, not individual users. Data is split into **shards** that are small enough not to clog the system.



Dynamic Replication

Any level replicates in a dynamic way, according to current necessity. A piece of data that becomes critical gets replicated in more and more copies. A piece operated upon by several processes concurrently may be sharded to avoid bottlenecks.

7.2. Sharding



Shard

A **shard** is a smaller, well-defined piece of data that is the **unit of replication and management**. The system identifies the proper pieces to replicate (shards) to achieve **high availability** and **increased performance**.

Sharding rules:

- Shards must be **dynamically decided**: usage patterns change.
- Not too small, to avoid fragmentation management overhead.
- Not too large, to avoid making coordination too hard.
- Shards can be **local** or **coordinated with other data centers** to grant consistency.

The most requested pieces are replicated most, those operated upon by concurrent processes are sharded along their access pattern to allow parallelism.

AWS uses sharding in DynamoDB and RDS. Microsoft Azure uses it in Azure SQL Database and Cosmos DB. Google Cloud uses it in Cloud Spanner and Bigtable.

7.3. Consistency vs. Speed: The Core Tension

7.3.1. Critical Paths and Asynchronous Effects

Fast answers are **difficult** when working **synchronously** with **subservices**. If a service calls multiple slow subservices, the total response time is the **critical path**: the longest chain of



dependent calls. With replicas and parallelism, reads can be served quickly, but **write operations** involving multiple replicas introduce asynchronous effects:

- Replicas receiving operations in a different schedule → their final state can differ (inconsistency).
- If some replicas fail, the given answer may be incorrect.
- Agreements between different copies must be reached **eventually**.

All these issues contribute to **inconsistency** that clashes with **safety** and **correctness**.

7.3.2 Is Inconsistency Always Bad?

🤔 **Why** Inconsistency is not always the devil?

In small confined machines, strict consistency is natural. But in Cloud environments, ask: *do we really need strict consistency at all times?*

- YouTube video counters: is it a real issue if the view count differs by a few?
- Amazon "units available" counters: small variations are invisible to customers.

There are many cases in which you do not need a real correct answer, but only a good **approximation**: the closer the better, but perfect is not required.

7.4 ! The CAP Theorem

📖 CAP Theorem

At most 2 of the following (3) properties can be obtained:

- **Strong Consistency (C)**: all clients see the same view, even in the presence of updates.
- **High Availability (A)**: all clients can find at least one replica anytime, even in the presence of failures.
- **Partition Tolerance (P)**: system properties hold even when the system is partitioned and must keep working.

👉 **Note**: Since **availability** is paramount for fast answers, and transient faults often make it impossible to reach all copies, caches must be used even if they are stale. The CAP conclusion is to **weaken consistency for faster response**: choosing AP and neglecting C.

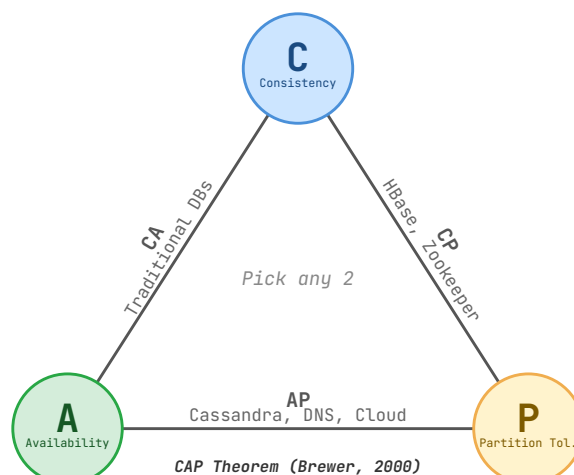


Figure 32 - CAP Theorem: pick any two. Cloud systems typically choose AP.



The three CAP combinations:

- **CA** (Consistency + Availability): single-site or clustered databases. When a partition occurs, no work can go on and reconnection must be awaited.
- **CP** (Consistency + Partition Tolerance): consistent even during partitions, but some requests may not get answers.
- **AP** (Availability + Partition Tolerance): **the Cloud choice**. Work during partitions, reconcile afterwards. DNS, web caches, and most internet-scale systems follow this.

100 Prof. Question: *I want something consistent and partition tolerant. Can I?*

Yes, that is a **CP** system. By CAP you get at most two of the three, so a CP system (like **RAFT/etcd**) keeps **consistency** during a partition by making the nodes without quorum **unavailable** (it sacrifices A). For strict consistency on top of Kubernetes: lock on etcd, or use a broker with exactly-once (Kafka transactions).

7.4.1. Consistency: Two Perspectives

Viewpoint	Position
Academic / Pessimistic (e.g. ACID)	Clean abstractions, strong semantics, formally provable. Correctness above all.
User / Optimistic (e.g. Internet)	Systems that work most of the time, that scale well. Consistency is not important per se.

7.5. ACID Properties

ACID

The **ACID** properties define **maximum consistency** for database transactions:

- **Atomicity:** either all operations commit or none (abort).
- **Consistency:** database goes from one consistent state to another.
- **Isolation:** concurrent transactions are invisible to each other (serializability via locking).
- **Durability:** once committed, updates cannot be lost or rolled back.

A “serial” ACID execution runs at most one transaction at a time. **Serializability is the illusion of serial execution**, but with increasingly heavy costs.

Two extreme cases:

- **Embarrassingly easy:** transactions that never conflict at all (Facebook updates by a single owner).
- **Conflict-prone:** transactions that interfere and can leave replicas in conflicting states: **scalability here is terrible**.

ACID Cost in the Cloud

The costs of transactional ACID on replicated global data can be surprisingly high. Solutions must involve ad-hoc mechanisms such as **sharding** and **coding ad-hoc transactions**. Brewer’s CAP theorem states: **“you cannot use transactions at large scale in the cloud”**.



Cloud systems do use transactions, but only in the **back end**, shielded away from users as much as possible to avoid overload and not create bottlenecks.

7.5.1. Nested Transactions and Two-Phase Commit

In distributed systems, one transaction may contain others at **different nesting levels**. Effects must be visible only when the whole transaction has been agreed upon and committed.

- 👉 **Note:** It is crucial to distinguish between **local logic** and the **distributed protocol**:
- **Local Nesting:** Allows a transaction to execute sub-operations in an isolated and atomic manner at the level of a single node or shard. If a sub-transaction fails, the node can perform a partial rollback without involving the global coordinator.
 - **Distributed Consistency (2PC):** Intervenes only when the main (root) transaction must make changes permanent across multiple shards. The 2PC protocol “closes” the tree of nested transactions, ensuring that the entire set of local changes is atomically accepted or rejected globally.

📖 **Two-Phase Commit (2PC)**

2PC is the standard protocol to achieve ACID quality for distributed transactions. One coordinator manages multiple participants via **4(N-1) messages**:

- **Phase 1 (Vote collection):** coordinator sends Prepare to all participants; each votes Yes or No.
 - **Phase 2 (Decision):** if all vote Yes, coordinator sends Commit; otherwise sends Abort. Participants acknowledge.
- If **anyone disagrees**: global undo (rollback).
 - If the **coordinator fails**: participants are blocked waiting, this is the fundamental weakness of 2PC.
 - **Concurrent coordinators** over the same data $\rightsquigarrow$ one of the two must abort.

👉 **Note:** 2PC guarantees atomicity but is expensive and has a **single point of failure** (the coordinator). In large-scale cloud environments with many replicas across different data centers, 2PC creates serious performance **bottlenecks**.

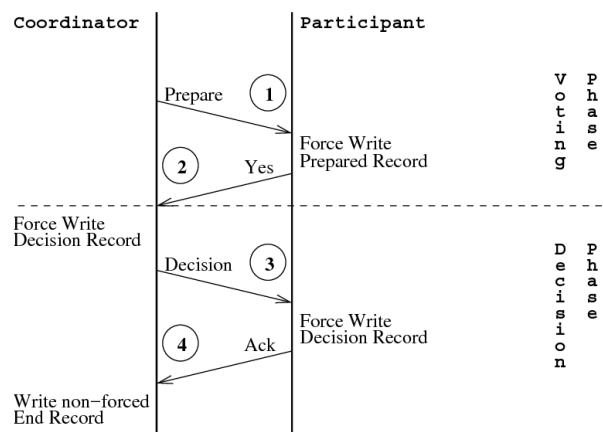


Figure 33 – 2 phases commit.



7.6. BASE Properties

BASE

BASE is the “opposite” of ACID, reflecting experience with real cloud applications:

- **Basically Available:** **fast response** even if some replicas are slow or crashed. Partitioning faults are mapped to crash failures, forcing isolated machines to reboot. but rapid responses are still needed.
- **Soft State Service:** the **first tier cannot store any permanent data** and restarts in a “clean” state after a crash. To maintain data, either replicate in memory in enough copies or pass to another service that keeps “hard state”.
- **Eventual Consistency:** send “optimistic” answers to the external client. Could use cached data without checking for staleness and/or could **guess** at the outcome of an update. It might skip locks hoping no conflicts and later correct any inconsistencies with an **offline cleanup activity** (reconciliation).

Analogy: BASE as the eBay philosophy

eBay researchers found that programmers with a transactional mindset built applications that did not scale well on their cloud. BASE was designed to guide those programmers: **embrace the reality that big distributed systems are inherently uncertain, and design around it rather than fighting it.**

7.6.1. BASE Effect in Practice

With BASE:

- Code is **more concurrent**, hence faster.
- **Locking is eliminated**, making end-user experience snappy and positive.
- **Weird behavior may occasionally appear** when looked at hard: but the achieved speed is worth the behavioral change.



eBay Auction

In a fast-running eBay auction, does every single bidder necessarily see every bid? And in the same order? Clearly everyone needs to see the **winning bid**, but slightly different bidding histories should not hurt much: and that makes eBay **10x faster**.

7.6.2. ACID vs. BASE Summary

ACID	BASE
Strong consistency: highest priority	Availability and scaling: highest priorities
Availability less important	Weak consistency
Pessimistic	Optimistic
Rigorous analysis	Best effort
Complex mechanisms	Simple and fast



7.7. Consistency Spectrum

There is no single definition of “consistency”, it is a spectrum:

- **Strict consistency**: updates happen instantly everywhere. A read must return the result of the latest write. Not realistic with instantaneous propagation.
- **Linearizable**: updates appear to happen instantaneously at some point in time, ordered by a global clock. Used for formal verification of concurrent programs.
- **Sequential**: every client sees its writes in the same order. Order of writes from different clients may differ. Equivalent to Atomicity + Consistency + Isolation.
- **Eventual consistency**: when all updating stops, eventually all replicas converge to the identical values. Equivalent to CAP’s AP choice.

7.7.1. Eventual Consistency Implementation

Write propagation in two phases:

1. **Epidemic stage**: final local propagation spreads an update quickly, tolerating incomplete coverage for reduced traffic overhead.
2. **Correcting omissions**: a final phase grants that all replicas that were not updated during the first stage get the update.

Writes are written to a **log** and applied in the same order at all replicas (timestamps and “undo-ing”).

Writes are logged with logical clocks to ensure causal ordering. The system employs a two-tier propagation: a low-latency Gossip Protocol for rapid distribution, followed by a mechanism to guarantee eventual consistency and repair transient omissions.

7.7.2. Technology Examples and CAP Position

Technology	CAP / consistency notes
Memcached	No special guarantees
Google GFS	File is current if locking is used
BigTable	Shared key-value store with many consistency properties
Dynamo	Eventual consistency (Amazon shopping cart)
Databases	Snapshot isolation with log-based mirroring (ACID-like)
MapReduce	Functional computing model with very strong guarantees
Zookeeper	Yahoo! file system with sophisticated properties
PNUTS	Yahoo! database, sharded data, spectrum of consistency options
Chubby	Locking service: very strong guarantees

7.8. ! eBay Principles: Five Commandments for Internet Scale

At internet scale, standard transactional thinking fails.



! eBay Five Commandments

1. Partition Everything
2. Use Asynchrony Everywhere
3. Automate Everything
4. Remember: Everything Fails
5. Embrace Inconsistency

Consistency is viewed as a **spectrum**, not a **specific position**: some operations (bids, purchases) require immediate consistency, others (search results, preferences) can tolerate eventual or no consistency.

7.8.1 Partition Everything

The goal is to **avoid global state and monolithic bottlenecks**. If you can scale a component independently, you can scale the entire system indefinitely.

⇒ KISS (Keep It Simple, Stupid), and shot in time.

Functional Segmentation:

Instead of a single application, **split the system into isolated domains** (e.g., Search, Bidding, Checkout). Each domain has its own services and databases (**isolation**).

Horizontal Split:

Never let a single database store “everything”. **Split data** so it fits on commodity servers.

- Use a sharding key (e.g., range, modulo of a key, lookup, ...) to route requests to the correct shard.
- Scaling is as simple as adding more servers to the pool.

Corollaries:

- **No Database Transactions**: no client-side transactions, no two-phase commit. **Auto-commit** for the vast majority of DB writes. Consistency is not always needed/possible.
- **No Session State**: user session flow moves through multiple application pools. No session state in the application tier: sharding only.

7.8.2 Asynchrony Everywhere

The core principle is to offload non-critical processing to **asynchronous flows**, **decoupling** system **components in both time and space**.

Benefits:

- **Scalability**: **scale components independently**.
- **Availability**: **decouple availability state and retry operations**.
- **Latency**: improve user-experience latency at cost of execution latency.
- **Cost**: spread peak load over time.

Patterns:

Event Queue / Streams decoupling:

Decouple components by producing **events** (e.g., **ITEM.NEW**). Consumers subscribe to topics, ensuring **at-least-once delivery** paired with **idempotent** processing to handle retries safely.



Message Multicast:

Used for high-fanout updates (e.g., Search Feeder). The primary DB broadcasts updates, nodes listen to specific subsets (shards) to update in-memory indexes in real-time.

Periodic Batch:

scheduled offline batch processes for infrequent, periodic, or non-incremental computation (such as import third-party data, generate recommendations, process end-of-auction items).

7.8.3. Automate Everything

Prefer **adaptive/automated systems** over manual systems:

- **Scalability:** scale via machines without human intervention.
- **Availability/Latency:** fast adaptation to changing environment.
- **Cost:** machines are far less expensive than humans and learn/improve over time.
- **Functionality:** explore solution space more thoroughly and quickly.

Patterns:

Adaptive Configuration:

Define SLA for a given logical consumer (e.g., 99% of events processed in 15 seconds); dynamically adjust configuration to meet the defined SLA.

Machine Learning :

Dynamically adapt search experience, determine best inventory and assemble optimal page for that user and context.

Feedback loop: collect user behavior → aggregate and analyze offline → deploy updated metadata → decide and serve appropriate experience.

7.8.4. Everything Fails

Design assuming failures happen constantly.

Patterns:

Failure Detection:

Servers log all requests (all application activity, DB and service calls on a multicast message bus), listeners automate failure detection and notification.

Rollback:

Absolutely no changes to the site that cannot be undone. The system does not take any action if irreversible actions are to be taken. Every feature has an on/off state driven by central configuration. Features can be deployed “wired-off” to unroll dependencies.

Graceful Degradation:

Application marks down an unavailable or distressed resource. Non-critical functionality is removed or ignored. Critical functionality is retried or deferred: retried until completed in case of failure.



7.3.5. Embrace Inconsistency

Choose Appropriate Consistency Guarantees: according to Brewer's CAP Theorem, prefer eventual consistency to immediate consistency.

⇒ To guarantee availability and partition-tolerance, trade off immediate consistency.

Avoid Distributed Transactions :

- No two-phase commit.
- Minimize inconsistency through **state machines and careful ordering of operations**.
- Achieve eventual consistency through **asynchronous event or reconciliation batch**.

👉 **Note:** This does not mean "be inconsistent" but rather **release consistency**: look at it as a spectrum and not a specific position. Choose the right consistency level per operation.

- *Immediate consistency: bids, purchases.*
- *Eventual consistency: search engine, billing systems.*
- *No consistency required: user preferences.*



8. CORBA, ADVANCED C/S MODELS, EVENTS AND MOM

Package: CORBA CS and Events and MOM - 6 - CORBA CS and Events and MOM 26.pdf

This chapter deepens the distributed middleware model by examining CORBA (an industrial-strength DOC middleware), advanced C/S interaction patterns, event-based systems, publish-subscribe, the tuple model, MOM (Message Oriented Middleware) and concrete tools like Kafka and MQTT, ending with group communication and multicast routing.

8.1. Remote References and the Proxy Pattern

Every distributed communication model (whether Client/Server or message-based) requires the initiator to **identify** its partner across network boundaries. A local variable or pointer cannot cross a machine boundary, instead, the middleware must provide a **non-local identifier** (a remote reference) that can be resolved at runtime.

A **name server** (or any form of directory service) is the basic building block: it maps an abstract name or specification to a concrete remote reference.

Examples: DNS for hostnames, CORBA Naming Service for object interfaces, a Trading service for attribute-based lookup.

8.1.1. Proxy

Once a remote reference is obtained, the client still needs a local object to talk to. This is where the **proxy** pattern comes in:

Proxy

A **proxy** is a **local intermediary** that stands in for a remote entity. From the client's perspective, calling the proxy looks identical to calling a local object: the proxy **transparently** handles all the communication details (marshalling arguments, sending the network request, unmarshalling the result).

In general, a communication can involve **two kinds of proxies**:

- **Client-side proxy (Proxy C)**: sometimes called **stub**. Located near the client, it translates local method calls into network messages. The client code calls the stub as if it were the real object.
- **Server-side proxy (Proxy S)**: sometimes called **skeleton** or **adapter**. Located near the server receives incoming network requests and translates them into real method calls on the servant.
- **Broker / Link Manager**: a third possible intermediary between the two proxies, responsible for **dynamic binding** at runtime (e.g., routing the request to the right server when multiple servers implement the same interface).

Analogy: Proxy as a diplomatic interpreter

Two delegations (client and server) speak different languages and are in different cities. Each sends a local translator (proxy) to the negotiation table. The translators handle the language and logistics: the delegations just



talk normally. If an intermediary (broker) is added, it routes each delegation's message to the right counterpart even when the address is not known in advance.

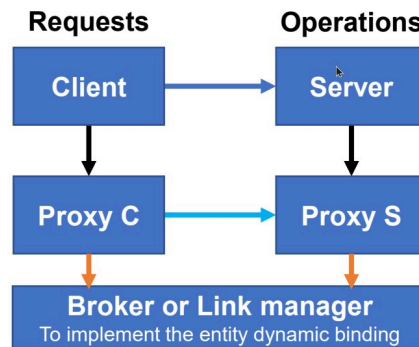


Figure 34 - Proxy stub + skel + broker.

Java RMI is a concrete realization of this pattern: it generates two proxies automatically (a stub on the client JVM and a skeleton on the server JVM) freeing the programmer from dealing with sockets. However, RMI still requires solving several practical problems: how to obtain the initial server reference (a name registry), where to find the ancillary classes, what happens if the server is not running, and whether two different references point to the same object.

8.2. CORBA



CORBA: Common Object Request Broker Architecture

CORBA is an OMG standard for **Distributed Object Computing (DOC)**: it provides an **Object Request Broker (ORB)** that mediates invocations between distributed objects written in different languages, running on different OS and hardware, without the client knowing the location or implementation of the server object.



Core CORBA Concepts

- **Object**: any CORBA entity with a defined interface. Has a **remote reference (IOR, Interoperable Object Reference)** that uniquely identifies it globally.
- **Servant**: the concrete language implementation of a CORBA object (e.g., a Java class). Created by the language environment, **not by CORBA itself**.
- **Interface**: declared in IDL. Defines the **contract**: attributes, methods, exceptions.
- **ORB**: the **middleware bus**. Routes requests from clients to the correct servant, transparently.
- **POA (Portable Object Adapter)**: on the server side, maps incoming requests to the correct servant instance.

Analogy: CORBA as an Object Phone Network

Think of objects scattered across many machines as people in different cities with different languages. CORBA is the telephone operator: you give it a reference (phone number) and it routes your call to the right person, translating protocols and formats along the way. You never need to know where the person is or what language they speak internally.

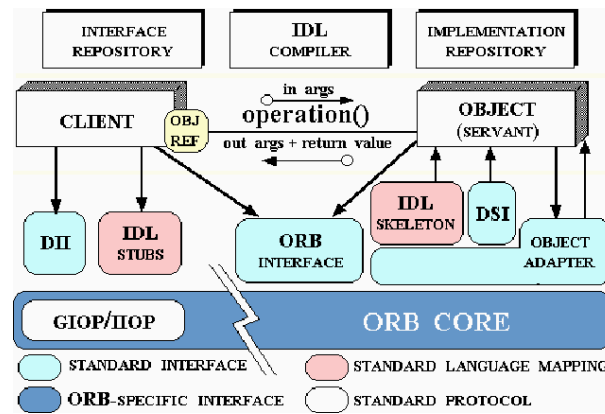


Figure 35 - CORBA scheme.

8.2.1. ORB: Object Request Broker

The ORB is the central infrastructure piece. It does **not** create or move objects: it connects (route) them, since it is a bus (not a server).

ORB Responsibilities

- **Interaction Broker:** Acts as a communication bus, enabling object interaction. By default, it uses synchronous blocking calls, but supports asynchronous patterns.
- **Language Independence:** Delegates the final execution to specific language mappings, maintaining neutrality between client and servant environments.
- **Separation of Concerns:** Specifically avoids managing object creation or object migration, these tasks are delegated to the underlying Servant implementations.
- **Reference Management:** Facilitates the discovery and resolution of remote objects (IORs), acting strictly as a routing layer between the client and the target servant.

- 👉 **Note:** The ORB does not host objects, it routes invocations. To locate a target, it uses:
- **Stringification:** Converting IORs into human-readable strings and back.
 - **Directory Services:** Utilizing Name or Trading services to resolve service names into specific remote addresses.
 - **Parameter Passing:** Dynamically passing references as arguments during method invocations.

The communication flow is:

Client → Stub → ORB → network → ORB → Skeleton → Servant.

- 👉 **Note:** The stub and skeleton are generated automatically by the IDL compiler.

8.2.2. Object Management Architecture (OMA)

The ORB is the core of a broader framework called the Object Management Architecture (OMA), defined by OMG.

OMA: Object Management Architecture

OMA organizes the full CORBA environment in layers around the ORB:



- **Object Services (CORBA Services):** fundamental services any distributed application needs (naming, trading, events, lifecycle, transactions, security, ...).
- **Common Facilities (CF):** horizontal features shared across many applications (UI, system management, information management).
- **Domain Interfaces (DI):** vertical, industry-specific interfaces standardized per sector (manufacturing, finance, healthcare, ...).
- **Application Interfaces:** non-standard, application-specific interfaces defined by individual projects.

OMA explains why CORBA comes with a Naming Service, Trading Service, Event Service and so on: these are not add-ons but integral parts of the standard.

8.2.3. Static vs Dynamic Invocation

CORBA supports two invocation paths, which can coexist in the same system:



Static vs Dynamic Invocation

- **SII (Static Invocation Interface):** the default. The IDL compiler pre-generates a **stub** on the client side and a **skeleton** on the server side. Calls go through these fixed proxies. Fast, but requires the interface to be known at compile time.
- **DII (Dynamic Invocation Interface):** the client builds and sends a request **at runtime** without a pre-generated stub. Supports blocking, oneway (fire-and-forget), and deferred-synchronous (send now, retrieve result later) invocation modes. Useful for gateways and generic tools operating on unknown interfaces.
- **DSI (Dynamic Skeleton Interface):** the server-side counterpart of DII. A server can accept requests for **any** interface without a pre-generated skeleton, inspecting the incoming request dynamically.

8.2.4. Repositories and Inter-ORB Protocols



Interface and Implementation Repositories

- **IR (Interface Repository):** a runtime catalogue of IDL interface descriptions. While stubs embed type information statically, the IR allows it to be discovered **at runtime**, enabling DII and generic introspection tools.
- **IMR (Implementation Repository):** tracks where servants are actually deployed. Enables **on-demand activation**: servants do not need to run continuously, the ORB consults the IMR to start them when needed.



GIOP and IIOP

Different ORB implementations interoperate via standard wire protocols:

- **GIOP (General Inter-ORB Protocol):** defines a **binary message format** and CDR (Common Data Representation) encoding so that ORBs from different vendors exchange data consistently.
- **IIOP (Internet Inter-ORB Protocol):** GIOP carried over TCP/IP. This is what makes CORBA a truly open standard: a client compiled against one vendor's ORB can invoke objects hosted on any other vendor's ORB.



👉 **Note:** POA, repositories, and the DII/DSI request objects are **pseudo-objects**: they have IDL-described interfaces but exist only locally and cannot be invoked remotely.

8.2.5. CORBA IDL

IDL - Interface Definition Language

CORBA IDL is a purely **declarative, object-oriented language** (derived from C++) used to specify **data and method interfaces**, completely independently of any specific programming language.

IDL Capabilities

- **Interface definition** (with multiple inheritance).
- **Attribute definition** (accessed via auto-generated `_get` / `_set` operations).
- **Exception definition** (with `completion_status`: `COMPLETED_YES`, `COMPLETED_NO`, `COMPLETED_MAYBE`).
- Automatic management of **attributes mapping** across languages.
- **Module grouping** of interfaces (for logical aggregation).

IDL - Stock Example

```
module Stock {
    exception Invalid_Stock {};
    exception Invalid_Index {};
    const long length = 100;

    interface Quoter {
        attribute float quote;
        readonly attribute float quotation;
        long get_quote(in string stock_name) raises (Invalid_Stock);
    };

    interface SpecialQuoter: Quoter {
        attribute float quotehistory[length];
        readonly int index[length];
        long get_next(in string stock_name) raises (Invalid_Index);
        long get_first(in string stock_name) raises (Invalid_Index);
    };

    interface CancelQuoter: SpecialQuoter {
        long cancelhistory(out float cancelledquote[length]);
    };
}
```

IDL - BankAccount Example

```
module BankAccount {
    struct transaction { string data; float amount; };
    exception RossoException { string message; };
    typedef sequence <actions> list_ops;

    interface Account {
        float balance(in string cc);
        list_ops bankStatement(in string cc);
    };
}
```



```
void withdrawal(in string cc, in float balance,
               out float balance) raises(RossoException);
Account accountTwin(); // returns an object (the interface itself)
};
}
```

Objects are returned by reference (the interface), not by value. Parameters use `in`, `out`, and `in out` modes.

8.2.6. Data Types in CORBA IDL

- **Object references:** references to CORBA objects or interfaces, passed by reference.
- **Exceptions:** not values but signaled conditions.
- **Basic values:** short, long, ushort, ulong, float, double, char, string, boolean, octet, enum, `Any`.
- **Constructed values:** Struct, Sequence, Union, Array.
- **`Any`:** a generic type that can hold any primitive or any CORBA interface, analyzable at runtime (used to dynamically carry the current type).



Object by Value (CORBA 3)

Objects passed **by value** (`valuetype`) are copied from one environment to another: they **cannot** be accessed remotely. Used to overcome heterogeneity when no remote reference is possible. This is the only case where objects cross the wire as data.

8.2.7. Language Mapping

CORBA defines interfaces but code must run in concrete languages. **Language mapping** is the process of translating IDL constructs into specific programming language constructs.



Language Mapping Responsibilities

- **Strategy for consistency** of concrete language types with the CORBA model.
- **Transformation functions** to manage types automatically: putting structures together simply.
- Additional support functions: **naming, trading, suggested development methodologies**.
- Each language provides a **Helper** and a **Holder** utility class.



Holder (Java)

In Java, `out` and `in out` parameters cannot be passed directly (Java is pass-by-value for primitives). A **Holder** is a **container object** whose internal value changes but whose identity remains fixed. After the invocation, the holder contains the result.

```
public final class BalanceHolder {
    public float value;
    public BalanceHolder() {}
    public float _read() { return value; }
```



```
public void _write(float value) { this.value = value; }
}
```



Helper (Java)

A **Helper** maps between the CORBA `Object` type (`org.omg.CORBA.Object`) and the specific concrete type. Functions include:

- **Narrowing**: casting from a generic CORBA Object to the specific interface type.
- **Reading and writing** a type on an object stream: treating type dynamically.

Every language must guarantee interoperability with CORBA through such helpers.

👉 **Note**: The **cost of CORBA is very high**: both in terms of developer learning curve and runtime performance overhead. This is a core limitation acknowledged even by its proponents. Despite this, it was widely used (Orbix/IONA, Visibroker/Borland, JacORB as open source).

8.3. Advanced Client/Server Interaction Models

The classic C/S model is **synchronous blocking**: the client sends a request and waits. This is too rigid for real distributed applications.



Novel C/S Variants

- **Pull** (synchronous non-blocking): the client gets the result afterwards, without waiting for it immediately.
- **Push** (synchronous non-blocking): the server gives the result to the client afterwards: the client does not wait for it.
- **Delegation**: a delegate waits for the result on behalf of the client (synchronous non-blocking for the client). The delegate holds the result and notifies.
- **Notification**: for the result: the delegate notifies the client when a result arrives.
- **Events** (typically asynchronous, non-blocking): an event is generated by a producer and advertised to consumers.
- **Provisioning**: other parties can be interested in the call chain, apart from the direct C/S pair.

8.3.1. Delegation Patterns

In a synchronous non-blocking model, an intermediate entity handles result delivery:



Poll Object

A **Poll Object** is an **intermediate** entity the client periodically queries. Used for **short operations** with bounded response time.



Flow: Sender → Request → Receiver.
Sender polls Poll Object and Poll Object returns response.

Call-Back Object

A **Call-Back Object** is an **intermediate** entity that is **notified by the server** when the result is ready and then invokes the client-specified callback. Allows even **long operations independent of client life cycle**. The code to handle the response is executed when the response arrives.

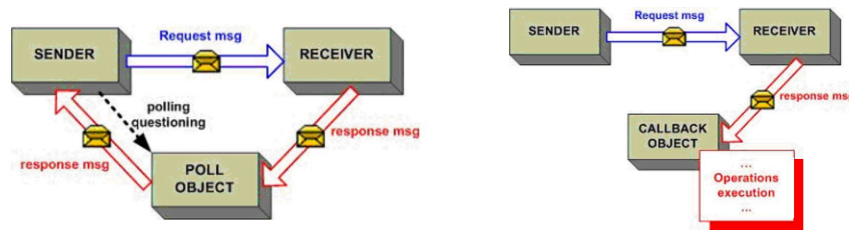


Figure 36 - Poll object (left) vs. Callback object (right).

8.4. Message Exchange

Message Exchange

Message exchange is a flexible but primitive communication model. Sometimes messages carry only **synchronization signals** (no data): the message itself is the trigger.

Message Exchange Properties

- **Synchronous / non-synchronous**: does the sender receive a reply?
- **Symmetric / asymmetric**: do both sides know each other?
- **Direct / indirect**: is there an intermediate entity?
- **Blocking / non-blocking**: is the sender blocked during transmission?
- **Buffered / unbuffered**: are messages queued?
- **Reliable / unreliable**: with or without message loss guarantee?
- Models with **multiple receivers**: group messages (multicast, MX) and broadcast (BX).

8.4.1. Modes of Message Exchange

Three Communication Modes

- **Rendez-vous**: one-to-one, synchronous, blocking, symmetric, unbuffered, **coupled** (more constrained than C/S). Both parties must be present.
- **With one intermediate entity** (channel, socket, queue): typically asynchronous, non-blocking, asymmetric, buffered, decoupled. This is **MOM**.
- **With intermediate entity and group of receivers** (events): typically asynchronous, non-blocking, asymmetric, decoupled and **many-to-many**.



8.4.2 Coupling and Decoupling

Coupling

Coupling constraints imposed by communication tools can limit flexibility in three dimensions:

- **Space:** interacting entities must know each other and be co-located.
- **Time:** interacting entities must be present at the same time.
- **Synchronization:** interacting entities must wait for each other.

Why Decoupling Matters

Decoupling enables greater flexibility and leverages the potential distribution of load.

The less coupled the components, the easier it is to scale, replace, and evolve them independently.

MOM systems decouple in all three dimensions simultaneously.

8.5. Events and Publish-Subscribe

8.5.1. From Local to Distributed Events

Local events (e.g., Windows GUI events) already reverse control: the user process registers a handler and the framework calls it back when the event occurs. Responses from the framework are called **backcall** or **upcall**.

Distributed event systems go further: designed **without any locality constraints** (no coupling). Their strength is the **non-locality** of interacting entities.

 **Note:** Constraining events to co-residence (same node, producer and consumer on the same machine) contradicts the event model's purpose. That is one of the worst misuses of the technology.

Event System Design Indicators

- **Cost** in distributing events: to limit.
- **Performance:** to optimize.
- **Scalability:** to keep high.
- **Latency:** to limit in time.
- **Pervasivity** of provided services: high.
- **Independent development and execution:** high.
- **Fault tolerance:** maximal possible.

Implementing event systems grants **viability**: all indicators keep acceptable (possibly constant) values as the system scales.

8.5.2. Evolution of Events

Generation	Properties
Primitive events	On/off signals, no content: interrupt events and low-level signals.



Generation	Properties
Events with contents	Some events carry information and filters based on interest about specific information (e.g., RSS feeds).
Events with QoS	Differentiated service for different users, persistent events, event priority .

Persistent events: users not online do not lose any event, delivered as soon as they reconnect.

8.5.3. Publish-Subscribe

Publish-Subscribe

In a **publish-subscribe** system, **producers** generate events freely (publish/PUB) without worrying about delivery. **Consumers** register their interest (subscribe/SUB) in specific events, **topics**, or types. The **event support infrastructure** handles delivery. Producers and consumers are not required to be present at the same time.

Message Filtering in PUB-SUB

- **Topic-based:** based on a predefined topic/channel (e.g., RSS on a specific feed).
- **Content-based:** based on message contents (keywords or more complex relationships).
- **Type-based:** based on message type.
- **QoS:** persistency, priority, guaranteed maintenance and duration .

PUB-SUB Operations

- **Producers** (publishers) provide events. They may declare intent via `publishIntent` .
- **Consumers** (subscribers) first `subscribe` , then receive `notify` callbacks.
- The infrastructure holds: storage and management of subscriptions, subscribe/unsubscribe/notify operations.
- 2 management operations + 2 content operations $\Rightarrow$ **symmetric model**.

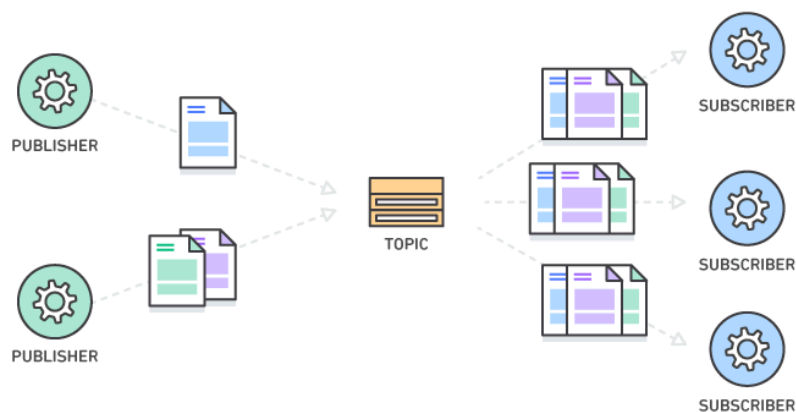


Figure 37 – PUB/SUB model.



8.6. Tuple Model: Linda (Gelernter)

Tuple Space

A **tuple space** is a **shared memory abstraction** combined with communication. It is a set of structured relationships organized as a **container for attributes and values**. Tuples can be **deposited** (Out) or **extracted** (In) as high-level information without any interference or incorrectness. No duplication is allowed in the space.

Linda is an implementation of **tuple model**. A tuple example could be `(to, from, message)`

Linda Core Operations

- `out(tuple)`: Atomically **inserts** a tuple into the tuple space, making it available for future matching. The tuple persists until it is explicitly consumed.
- `in(pattern)`: Atomically **removes** a matching tuple from the space. If no tuple matches the pattern, the operation blocks (waits) until a suitable tuple is produced.
 ↳ **Non-Deterministic Selection**: If multiple tuples satisfy the pattern, the system selects one. The choice is **non-deterministic**.

Non-Determinism

The Linda model explicitly leaves the **implementation strategy free** in choosing which tuple to extract from multiple matches. This freedom allows the system to optimize for cost, locality, or load, at the price of not guaranteeing order.

Decoupling in Tuple Spaces

- **In space**: consumers do not know producers. Only tuple contents matter: zero knowledge of other party.
- **In time**: a producer deposits and leaves, the consumer arrives later and retrieves. No co-presence required.
- **In quality (QoS)**: tuple spaces are **persistent**: tuples are safely deposited without limit (in memory and time). No preference for any specific process.

Tuple spaces are available with **high-level operations** (closures, in-transaction semantics, partition replication) to support well-formed local communication.

Analogy: Tuple Space as a Corkboard

Imagine a physical corkboard in an office. Anyone can pin notes (Out). Anyone can take a note matching their need (In). You don't need to be there when it was pinned, you don't need to know who pinned it: the corkboard is the mediator.

8.7. MOM: Message Oriented Middleware

MOM

Message Oriented Middleware (MOM) organizes data communication and distribution via **message exchange** between logically separated entities, using point-to-point or group messages. It



provides typed and untyped, asynchronous and synchronous message exchange with **wide autonomy** between components, **persistence**, and **broker** support for different strategies and QoS.

Examples: MQSeries IBM, MSMQ Microsoft, JMS Sun, OMG DDS, MQTT, RabbitMQ, ActiveMQ, Apache Kafka, ZeroMQ, NATS.



MOM's Role

MOM is a **Disappearing / Glue / Thin service layer**: it keeps together different autonomous systems and organizes their specific interconnection in a **static way**. The MOM can plan better support for communications because the configuration can be known a priori: optimize support costs to the specified need.

8.7.1 MOM Deployment

The **specific deployment and interconnection graph (routing) is always static** (no name services needed).

- **Necessity of high-level Routing** (as in ONs, but static).
- **Data treatment** while communicating between different environments.
- **Predefined and static participating entities**.



Two Extreme Deployment Models

- **Centralized model**: MOM with a central node as **hub** (hub-and-spoke) responsible for supporting and passing messages between different clients. Typically **replicated** for availability.
- **Distributed model**: MOM located on any client node forming a static overlay network, operating through **P2P communication** between nodes.

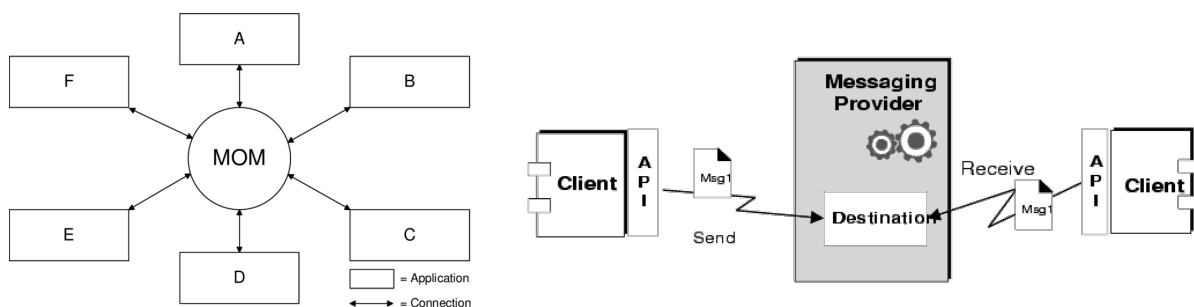


Figure 38 - MOM centralized model (left) vs. distributed (right).

8.7.2 MOM Infrastructure Components



MOM Building Blocks

- **Queue managers**: guarantee the expected operation level and message forwarding. Applications interact via API RPC to put/extract messages from local queues.
- **Inbound and outbound queues**: on interested machines, connected univocally.
- **Routing system**: connects different queues (as overlay networks do for application-level routing).



- **ReLay**: intermediate entities that allow the implementation to scale and organize high-level routing for scalability.
- **Message Broker**: entity able to support message content transfer between environments with **different representations**: can modify formats, organize routing based on contained information, work on application data to specify action sequences.

Analogy: MOM as a Postal Service

You drop a letter (message) in a mailbox (local queue). The postal service (MOM) picks it up, routes it through sorting centers (relays), possibly translates the address format for international delivery (broker), and delivers to the recipient's mailbox. Neither sender nor recipient need to be present simultaneously.

8.7.3. MQSeries IBM

MQSeries by IBM is a widely used MOM implementation.

- **Queue manager** controls the static routing via routing tables defined at configuration time.
- **Message Channel Agents (MCAs)**: handle all delivery details: different delivery policies, message type, duration, maximum allowed state, persistence, etc.
- **MCA coordination** via primitives enabling flexible coordination and QoS.
- Organization is **decentralized**: Relay, MCA, and Broker are local entities; each application location has its own entities.
- An **MQ Broker** can operate on messages by: modifying formats, organizing routing based on contained information, working on application information to specify action sequences.

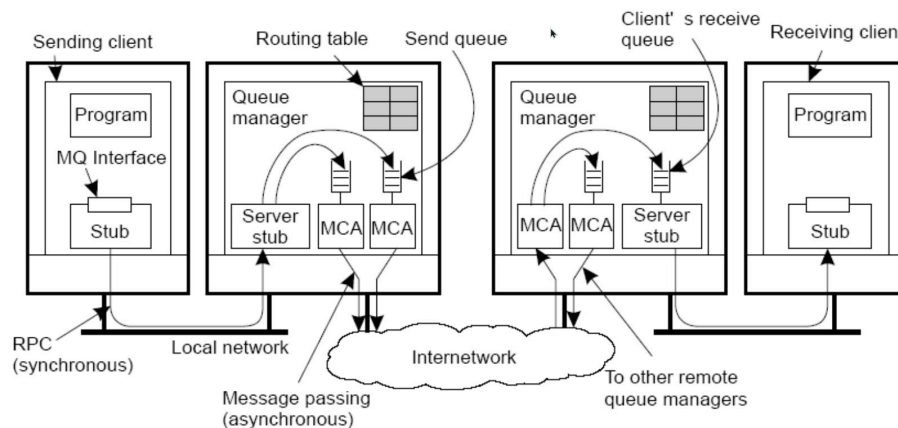


Figure 39 – MQSeries MOM IBM scheme.

8.8. Apache Kafka



Apache Kafka is a general-purpose distributed pub/sub system and streaming middleware with many additional features.

Developed at LinkedIn in 2010 (in Scala), widely adopted by big techs for **scalability** (Netflix, Uber, LinkedIn).



Kafka Requirements

- **Fault-tolerant**
- **Horizontally scalable**
- **High throughput** of data arriving and ingested (billions of messages)
- **Processing** of data, not only messaging

8.8.1. Kafka Pub/Sub Model

Kafka organizes **messages in topics**. A topic is a repository **where many producers can publish** and many consumers can subscribe.

- **Producers**: publish messages to a Kafka topic.
- **Consumers**: subscribe to topics and process the published message feed.

8.8.2. Topics, Partitions, and Brokers



Topic

A **topic** defines the set on which messages are published. The Kafka cluster maintains a **partitioned log** for each topic: an **append-only, totally-ordered** sequence of records, ordered by time.



Partition

A topic is split into a pre-defined number of **partitions**: the unit of parallelism of the topic. Each partition is an **ordered, numbered, immutable sequence of records**, similar to a log. Each record has a **monotonically increasing sequence number** called the **offset**. Partitions are replicated across brokers.



Broker

A **broker** is a Kafka cluster server. The cluster maintains a distributed log of data over many servers called brokers. Each partition has one **leader** (handles read and write requests) and may have **followers** (replicate the leader passively). Each broker is a leader for some partitions and a follower for others.

- **ZooKeeper**: hierarchical, distributed key-value store used by Kafka for coordination metadata (list of brokers, list of consumers and their offsets, list of producers).
- **Consumer Group**: a logical subscriber entity. Each partition is assigned to **exactly one consumer** within the group, parallelizing consumption without conflicts. If a consumer fails, the group rebalances automatically, assigning the orphaned partitions to remaining members. Adding more consumers than partitions leaves the extras idle.

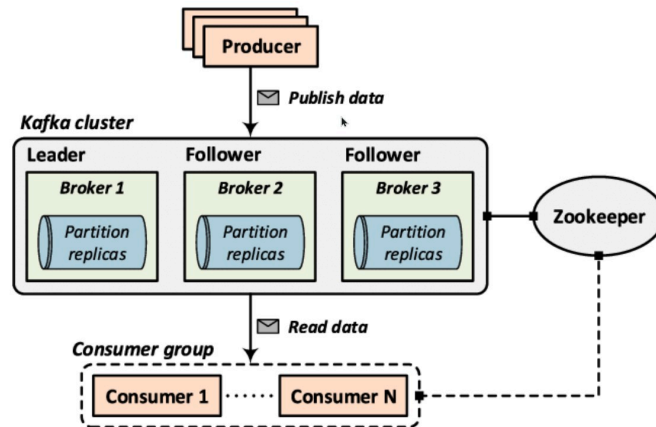


Figure 40 – Apache Kafka architecture: producers push to broker partitions, consumers pull independently per group, ZooKeeper coordinates metadata.

Analogy: Sorting Facility

To understand why each partition is assigned to exactly one consumer, imagine a large-scale sorting facility:

- **The Conveyor Belt (Topic):** A massive, high-speed stream of packages arriving simultaneously.
- **The Lanes (Partitions):** The belt is split into parallel lanes to handle the volume.
- **The Staff (Consumer Group):** You have a team of workers. If you let all workers pick from the same lane, they would collide, duplicate work, and lose the order of the packages.

The Kafka Approach: You assign specific lanes to specific workers. Each worker processes only their assigned lane(s) in complete isolation. This provides two key benefits:

1. **Zero Coordination:** No locks, no mutexes, and no communication between consumers are required.
2. **Ordered Processing:** Since a single worker processes the lane, the original sequence of packages (the offset order) is strictly preserved.

Scaling Rule: The maximum degree of parallelism is defined by the number of partitions. Adding more consumers than partitions results in idle resources, as Kafka cannot assign a partition to more than one consumer within the same group.

8.8.3. Kafka QoS



Ordering Guarantees

- Producers append to specific **topic partitions** in the order sent.
- Consumers see records in the order stored within a partition.
- **Total order within a partition**, not across partitions.
- **Per-partition ordering** combined with key-based partitioning is sufficient for most applications.



Replication and Fault Tolerance

- Partitions use **passive replication** over a predefined number of brokers: followers replicate the leader and act only as backups, ready to be **promoted** if the leader fails.
- Spreading partition leadership across brokers **distributes load evenly** across the cluster.

100 Prof. Question: What are *synchronized partitions* (in Kafka)?

The **In-Sync Replicas (ISR)**: the follower replicas that are **fully caught up** with the leader's log for a partition. A write is safely committed once the ISR have copied it, and only a replica in the ISR can be **promoted to leader** if the current leader fails.



Message Visibility Pipeline

A message written by a producer is **not immediately available** to consumers. The pipeline is:

Producer writes to leader → followers replicate → followers send ack to leader → message becomes readable by consumers

Only after **all followers have acknowledged** the write does the leader mark the message as available. This guarantees that even if the leader crashes immediately after a write, a follower promoted to leader already has the message. The cost is **latency between write and read**.

If this strong guarantee is not needed, the default can be relaxed: a producer can request ack from the leader only (or even no ack), trading consistency for lower write latency.

Retention and Replay

- Kafka **retains messages for a configurable period** (not until consumed): old messages are not deleted immediately after being read.
- Consumers track their own **offset**, and can reset it to **replay** past messages, e.g., to recover from a crash or reprocess data.

8.8.4. Kafka Producers and Consumers

Producers Responsibilities

- **Publish** data to topics of their choice, sending directly to the broker **leader** of the target partition.
- Decide **which partition** to write to, applying a random or key-based strategy.

Consumers: Pull Model

Kafka uses a **pull approach** for consumers: consumers use the offset to track which messages have been consumed. Messages can be replayed using the offset.

The **pull model** (Kafka's choice) grants better scalability (less burden on broker), more flexibility. But if broker has no data, consumers may end up busy waiting.

Instead, push model (broker pushes to consumers): broker must deal with different consumer types. ~> CORBA choice.

8.9. MQTT: Message Queue Telemetry Transport

MQTT

MQTT is a pub/sub messaging protocol designed for **sensors and edge devices** (IoT). A **broker** manages the pub/sub, **sensors** are the producers, **clients** can register (subscribe) to receive information from producers (publish).



The broker is in charge of maintaining messages and filtering for the interested receiver consumers. Used in constrained environments: low bandwidth, low power (HTTPS and/or MQTT transport).

8.10. OPC UA: Open Platform Communications Unified Architecture

OPC UA is the industrial successor to the original OPC (OLE for Process Control) standard, designed for modern manufacturing and Industry 4.0 environments. It is platform-independent (unlike the original OPC which was Windows-only) and supports both secure connectivity and rich data modelling.

OPC UA supports two communication models:

- **Client-Server**: used in demanding industrial environments where a control system needs precise, synchronous, low-latency access to sensor or actuator values. A client sends a read/write request and the server responds immediately.
- **Pub-Sub**: used for broadcast-style event notification flows where many subscribers need to react to state changes, without polling. Suitable for higher-level monitoring, dashboards, and historian systems.

The architecture is layered: vendor-specific extensions sit on top of core and companion information models (standard device and process object models), which are accessed via a rich Information Model Access layer (browse, data access, event notifications), mapped finally onto one of several protocol bindings (UA TCP, HTTPS, MQTT, AMQP).

OPC UA is the standard backbone for Industry 4.0 interoperability: it lets different machines, PLCs, sensors, and MES/ERP systems talk to each other using the same semantic model, regardless of manufacturer.

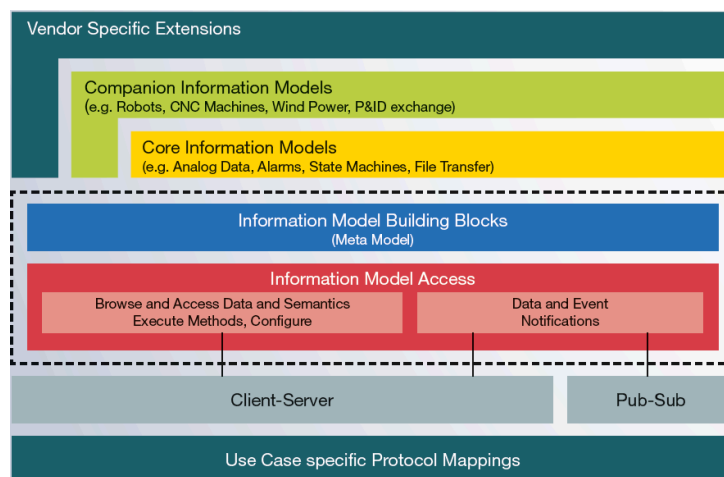


Figure 41 – OPC-UA layers architecture.

8.11. Group Communication and Multicast

8.11.1. Group Communication Semantics

Broadcasting and multicasting introduce new semantic problems:

- How to cope with **answers** (if any)?
 - **No wait**: asynchronous operations.
 - Wait for **one answer only**.
 - Wait for **some answers only** (how many? how long?).



- Wait for **all answers** (how many? how long? when to stop?).
- How to send general messages to **all currently present processes** or to a **subset** (a group)?

On a **single LAN**, broadcast is easy. On **different networks and locations**, it is hard: expressive incapacity, excess overhead, lack of QoS.

8.11.2. IP Group Communication



IP Multicast vs Broadcast

- **IP Broadcast**: limited and directed, inside local network only.
- **IP Multicast**: heavier duty with a dedicated protocol. Uses **class D addresses**. Internet uses **IGMP** (Internet Group Management Protocol, RFC 1112 & 2236) since 1989 to implement local multicast.
- The protocol often operated only on local subnetworks and was implemented in different, incompatible forms.

8.11.3. IGMP: Local Group Management



IGMP

IGMP (Internet Group Management Protocol) allows sending a unique packet to multiple receivers in the same locality, using class D names to identify a group. Every local network must have at least an **IGMP router** capable of managing local incoming and outgoing traffic and controlling the group with IGMP messages.



IGMP Versions

- **IGMPv1**: only two messages: **IGMPQUERY** (router periodically verifies existence of hosts in a specific IP D address) and **IGMPREPORT** (node signals state change: **join only, no leave**). Only one report per node per network.
- **IGMPv2**: adds support for explicit **leave**. Nodes that leave the group must notify the manager. More MX routers can be in charge (interference settled by IP numbers order).



TTL-Based Scoping for IP Groups

TTL	Scope
0	Local send only
≤ 32	Local area
≤ 64	Local region
≤ 128	Local continent
> 128	Global



👉 **Note:** IGMP implementations offer only **best-effort** semantics: we do not know whether messages were **all delivered to all recipients** and in **which order**.

8.11.4. Global Multicast Routing

For **global multicast**, routing must create and maintain a **distribution tree** from sender to receivers, minimizing overhead.

⚠️ Global Multicast Principles

- **Single sender** support.
- **Variable number of receivers** (up to N) that can be added or removed dynamically.
- Maximize **sharing**: send only 1 copy of a message instead of N different unicasts.
- Protocols identify a **central tree** from sender to current receivers with **optimal shared paths**.
- The goal: **employ the most shared hops possible from root to leaves**.
- The tree is **extremely dynamic**: must consider only currently active receivers.

8.11.5. Multicast Spanning Tree

Building the multicast routing tree requires **two control phases**:

📌 Phase 1: Root to Leaves (Forward)

- Build a **spanning tree** connecting root to known leaf nodes.
- Use **unicast routing** information to organize and aggregate paths.
- Send a **flooding message** toward every possible recipient. The main objective is to create a **bone multicast**.
- Root identifies shortest paths from replies.
- Some receiver nodes are reached through **multiple paths**: second phase must choose one.

📌 Phase 2: Leaves to Root (Upward/Backward)

- **Reverse Path Broadcast (RPB)**: leaves send broadcasts toward the root during normal routing.
- **Minimal path messages** are sent backward from leaves to root, only some paths are selected.
- Root can **reorganize the tree**, aggregating sub-paths to produce an optimal tree.
- Routing info is **soft-state** (not persistent): re-calculated dynamically since it is a dynamic scenario.

8.11.6. Pruning and Grafting

📖 Pruning

Pruning: cutting branches of the multicast tree. Networks with no members are cut: routers with no connected receivers are excluded with 'cut' messages.

📖 Grafting



Grafting: reinserting parts of the tree when new receivers join. Done without reorganizing the tree from scratch (explicit graft from the bottom).

Reverse Path Multicasting (RPM) is performed autonomously by leaves. State is kept for a limited and predefined time (SOFT-STATE): the definition of the RPM time interval is critical.

8.11.7. Multicast Protocols



Key Multicast Routing Protocols

- **DVMRP** (RFC 1075): Distance Vector Multicast Routing Protocol. Builds the distribution tree using a multicast version of RIP (distance-vector).
It uses Reverse Path Multicasting (RPM): flood first, then prune branches with no interested receivers. Historically used in the MBONE (multicast backbone), tunneling multicast traffic through unicast-only routers.
- **MOSPF** (RFC 1584): Multicast extension of OSPF (link-state). Each router has a complete map of the network topology and computes the shortest-path tree from each source on demand. More accurate than DVMRP but scales poorly to large networks because every router must store per-source state.
- **PIM** (RFC 2117): Protocol Independent Multicast. Unlike the above, PIM does not maintain its own routing tables, it reuses whatever unicast routing protocol is already in place. It has two modes:
 - **Dense mode:** suitable for networks where most routers have interested receivers. Uses flooding-and-prune like DVMRP.
 - **Sparse mode:** suitable for low-density deployments. Nodes explicitly join a shared **Rendezvous Point (RP)** tree rather than flooding the whole network.
- **CBT** (RFC 2201): Core Based Trees. Rather than building a separate tree per sender, CBT builds a single shared tree rooted at one or more **core** routers. All senders send data toward the core. Receivers join the shared tree from below. The tree is suboptimal in path length but avoids per-sender state, making it practical for large groups. Widely used by streaming providers: clients connect to the nearest core node.

👉 **Note:** All these protocols are **incompatible** with each other and were historically championed by different vendor communities. In modern practice, **PIM Sparse Mode with a Rendezvous Point** dominates in enterprise networks, while large streaming providers use CDN-style architectures (which achieve the same goal via application-level replication at edge nodes) rather than relying on IP multicast at all.



9. KUBERNETES

Package: Kubernetes - [b - Kubernetes.pdf](#)

Kubernetes is the industry-standard platform for running containerised applications at scale. Where Docker solves the problem of **packaging** a single service, Kubernetes solves the problem of **operating thousands of containers** across a fleet of machines: scheduling them, keeping them healthy, connecting them to each other, and exposing them to the outside world.

9.1. What is Kubernetes

Kubernetes (k8s)

Kubernetes (k8s) is an open-source container **orchestration system** originally designed by Google for automating application deployment, scaling, and management inside a cluster.

Its main goal is to **hide the complexity** of managing a fleet of containers by providing users a REST **API** (CRUD operations on abstract resources).

Kubernetes is **portable**: it runs on public or private clouds (AWS, Azure, OpenStack, Apache Mesos) without change.

9.1.1. From Docker to Kubernetes

Docker **communicates between its CLI, daemon, and container runtime via HTTP/JSON** messages following the OpenAPI v3 contract. Kubernetes exploits the **same principle**: it talks directly to the container runtime through the CRI (Container Runtime Interface), bypassing Docker entirely.

Why Docker Compose → Kubernetes?

Docker and Docker Compose already offer isolation, immutable infrastructure, portability, fast deployments, and versioning, but they do not handle:

- **Networking** across multiple hosts.
- **Deployment** and rolling updates at cluster scale.
- **Service Discovery**: dynamically finding where each service is running.
- **Auto Scaling**: adjusting the number of replicas under load.
- **Persisting Data** with distributed storage.
- **Logging and Access Control** at scale.

Kubernetes addresses all of these.

Kubernetes Key Features

- **Multi-container** application deployment.
- **Automatic and dynamic scaling** of application replicas.
- **Rolling updates** that preserve service availability during changes.
- **Computation, network, and storage resources** with built-in service discovery.
- **Infrastructure independence**: the same manifest runs on any conformant cluster.

Long-term payoff

Kubernetes requires significant upfront investment in configuration and operations.

The payoff comes over time: a more manageable, resilient, and self-healing application infrastructure that would otherwise require extensive custom tooling.



9.2. Cluster Architecture

Kubernetes Cluster Architecture

Kubernetes uses a **Master-Worker** (control plane/data plane) architecture. The cluster consists of:

- **Master node(s)**: run the control plane components. Recommended odd number for quorum via RAFT.
- **Worker nodes**: run actual application workloads (**Pods**).

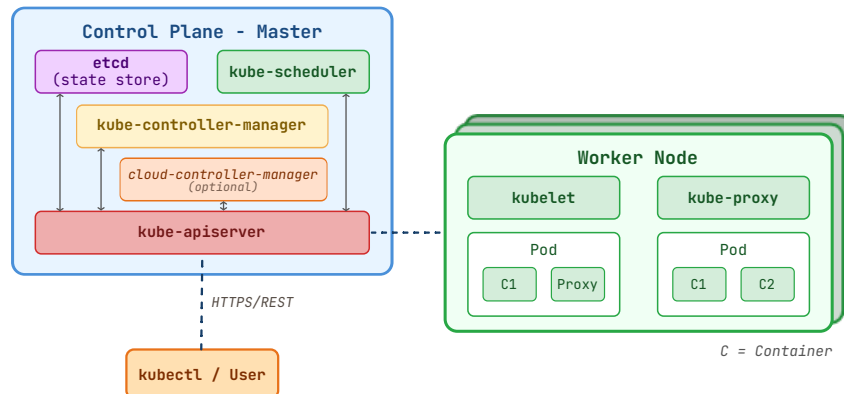


Figure 42 – Kubernetes cluster: control plane components and worker nodes with Pods and sidecar proxy.

9.2.1. Control Plane

Control Plane

The **control plane** makes **global decisions** about the cluster: scheduling, detecting and responding to cluster events (e.g., a Pod dying).

It is the “brain” of Kubernetes.

Control plane components:

- **etcd**: the **only stateful component** of the infrastructure. A distributed **key-value store** that holds the entire cluster state and configuration.
- **kube-apiserver**: the **front door** of Kubernetes. **All components communicate exclusively through the API server**.
- **kube-controller-manager**: runs the core **control loops** that keep actual state converging toward desired state.
- **kube-scheduler**: decides **which node** a newly created Pod should run on.
- **cloud-controller-manager** (optional): integrates with the cloud provider’s API (load balancers, node lifecycle, storage) at a pace independent of the main project.

9.2.2. Worker Node Components

Each worker node runs:

- **kubelet**: the **primary node agent**. It watches the API server for PodSpecs assigned to its node and ensures the described containers are running and healthy.
- **kube-proxy**: the **network proxy**. It runs on every node, reflects Services defined in the Kubernetes API, and programs the data plane (iptables/IPVS) to forward traffic.



- **Container Runtime:** the software that actually runs containers (containerd, CRI-O...).

Analogy: The Airport Analogy

The API server is the airport's central hub. Every passenger (request) must pass through it. etcd is the database of all flight bookings: the one source of truth. Controllers are ground crew teams each responsible for one type of task (gate changes, baggage, refuelling). The scheduler is the dispatcher who assigns planes to gates. Kubelets are pilots on each plane, ensuring their specific aircraft is operational.

100 Prof. Question: Tell me about Kubernetes. What are the fundamental architectural elements?

An open-source **orchestrator** using a **declarative model**: you state the desired state, it reconciles. **Master/worker** architecture. Control plane: **API server** (single gateway, everything goes through it), **scheduler** (places Pods), **controller-manager** (control loops), **etcd** (the only stateful component, holds all state). Worker: **kubelet** (node agent) and **kube-proxy** (Service networking).

9.3. The Declarative Model



Declarative Model

In Kubernetes you **declare the desired state** of your application in a **manifest** file (JSON or YAML). You post the manifest to the API server and Kubernetes is responsible for making reality match the declaration and keeping it that way.

The workflow is:

1. **Declare** the desired state of your microservices in a manifest.
2. **Post** it to the Kubernetes API Server.
3. K8s **stores** the desired state in etcd.
4. K8s **implements** the desired state on the cluster.
5. K8s **implements control loops** via the controller-manager to ensure the current state never drifts from the desired state.

👉 **Note:** The **only state** Kubernetes cares about is the **desired one**, not the current one. This is the fundamental insight behind the declarative model.

🤖 **Why desired state?**

Imperative commands ("start 3 replicas") are one-shot: if a replica crashes, nothing restarts it. Declaring desired state ("there should always be 3 replicas running") hands that responsibility to Kubernetes. The system becomes **self-healing** automatically: no operator needs to be paged at 3 a.m.

9.3.1. Kubernetes Resources

Kubernetes manages different kinds of **resources**:

- **Workloads:** applications running on Kubernetes: Deployment, ReplicaSet, StatefulSet, DaemonSet, Job, CronJob.
- **Services, Load Balancing, Networking:** networking resources that enable communication between workloads: Service, Ingress, IngressController, EndpointSlices, Gateway API, Network Policies.
- **Configurations:** dynamic configuration injected into workloads: Secrets, ConfigMaps.
- **Storage:** PersistentVolumes, PersistentVolumeClaims, StorageClasses.
- **Custom Resources (CRDs):** user-defined resource types that extend the API.



9.4. Controllers and the Control Loop

Controller

A **controller** is a **non-terminating control loop** that watches the shared state of the cluster (via the API server) and makes changes attempting to move the **current state** towards the **desired state**.

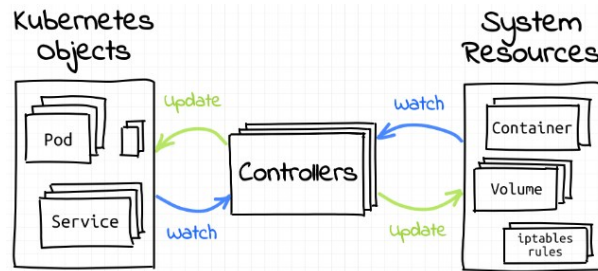


Figure 43 – Kubernetes controller Observe-Analyze-Act.

The loop follows the **Observe-Analyze-Act** cycle:

1. **Observe** (watch) the actual state of the cluster.
2. **Analyze** the difference between actual and desired state.
3. **Act** (update) by making API calls to reconcile the difference.
4. Repeat forever.

State is persisted in **etcd**, which also provides the **synchronisation** mechanism (**distributed locks**) used when multiple controller replicas run for fault tolerance.

 **Note:** Controllers adopt **microservice principles**: it is better to have many simple, independent controllers than one monolithic control block. Controllers can fail, and Kubernetes is designed to tolerate that.

9.4.1. Controller-ETCD Interaction

The interaction flow between a controller and etcd:

1. External changes to Resources are saved directly into etcd via the API server.
2. The **Controller subscribes to change notifications** for a particular resource in etcd (pub/sub pattern).
3. **On notification, the Controller fetches the new state and acts on it**.
4. The Controller also schedules **periodic reconciliation** tasks to catch any divergence that might have been missed.

Multiple controller replicas coordinate via **distributed locks** in etcd. When a controller acquires a lock, it becomes the **leader** and is the only one that actually acts. This prevents duplicate actions while still allowing fast failover when the leader dies.

 **Note:** Every Controller always pass through the API to communicate with ETCD.

 **Note:** Pub/sub is not implemented with the classical broker. ETCD is just a key-value store. The API sets a long-live connection when controllers demand to watch for updates and always the API server, as a gatekeeper, updates controllers after ETCD is updated.



9.4.2 The Operator Pattern

Kubernetes Operator

A **Kubernetes Operator** extends the controller pattern to **domain-specific operational knowledge**. An operator consists of:

- A **Custom Resource Definition (CRD)** that introduces a new resource type (e.g., `MongoDB`, `Prometheus`).
- A **custom controller** (the operator pod) that watches changes to that CRD and reconciles the actual cluster state accordingly.

The operator lifecycle:

1. The user creates or modifies a Custom Resource (e.g., a `MongoDB` object with `replicas: 3`).
2. The operator detects the change event.
3. The operator **reconciles**: creates/modifies/deletes native Kubernetes resources (Pods, PVCs, Services) to realise the desired state.
4. In case of error, the reconcile loop retries.

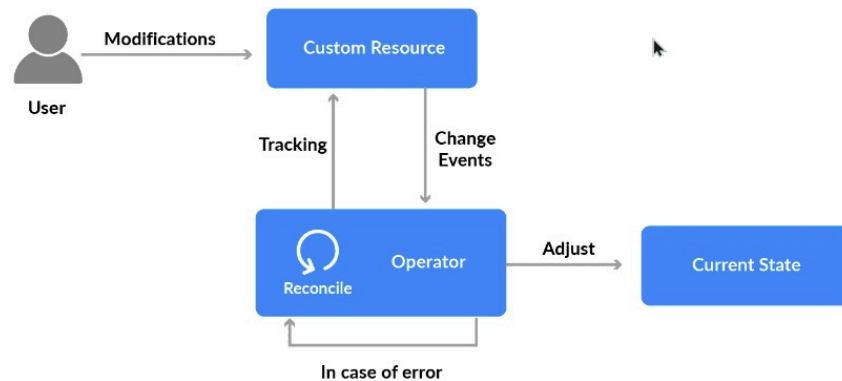


Figure 44 – Kubernetes operator pattern.

Prometheus Operator

The Prometheus Operator introduces CRDs such as `Prometheus`, `AlertManager`, `ServiceMonitor`, and `PodMonitor`. Defining a `ServiceMonitor` resource automatically configures Prometheus to scrape the matching services: the operator handles the complex Prometheus configuration file generation, reloads, and certificate management.

Operator frameworks and the OperatorHub.io registry provide a growing ecosystem of pre-built operators for databases (MongoDB, PostgreSQL, Redis), messaging (Kafka, RabbitMQ), monitoring (Prometheus), and more.

100 Prof. Question: *How is a Kubernetes Operator built? How does it work?*

An Operator is an **application-specific controller**. It adds a **CRD** (a new resource type, e.g. `MongoDB`) plus a **custom controller** that runs the reconcile loop: read the desired state from etcd, compare it with the observed state, and act to close the gap. It is how complex **stateful** apps get domain-specific management logic.



9.5. etcd and RAFT

9.5.1. etcd

etcd

etcd (distributed etc directory) is a **strongly consistent**, distributed key-value store that implements the RAFT algorithm. In Kubernetes it stores **all cluster data and configuration**: the single source of truth.

Properties:

- **Strongly consistent**: every read returns the most recently written value.
- **Fault tolerant**: replicates data across N nodes $\Rightarrow$ tolerates $\lfloor \frac{N-1}{2} \rfloor$ failures.
- Implements **locking**: controllers use etcd's distributed locking to coordinate.

100 Prof. Question: *Can RAFT work with a single node? And etcd?*

Yes. With one node the quorum is 1/1, so it is always its own leader and commits immediately. etcd works too (a log with checkpoint and compaction), but it is **not highly available**: if that node dies, the service is gone.

100 Prof. Question: *In Kubernetes, what can I do if I want strict consistency?*

etcd does not give strict consistency for **application** operations by default. Either take a **distributed lock on etcd** (compare-and-swap, so only one instance runs the critical operation), or use a **broker with exactly-once** semantics (e.g. Kafka with transactions).

9.5.2 RAFT

RAFT is the core of etcd's distributed nature.

RAFT

RAFT is a **consensus algorithm** designed to be **easy to understand**. It allows a cluster of servers to agree on a sequence of values even when some servers fail.

100 Prof. Question: *Tell me about the RAFT algorithm.*

A **consensus** algorithm, designed to be easier to understand than Paxos, that makes a group of servers agree on a **sequence of values** despite failures. It splits the problem into **leader election**, **log replication**, and **safety**. It is formally proven, tolerates a **minority** of failures, and in the common case needs one RPC to a majority.

RAFT decomposes the consensus problem into three relatively independent subproblems:

Leader Election

- One server is selected as **leader** at any time.
- If the leader crashes, a new leader is elected.
- Each server gives only **one vote per term** (term: logical clock increment on each election).
- **Majority** required to win $\leadsto$ only servers with up-to-date logs can become leader.
- Randomised timeouts prevent split votes and work well when timeout $\gg$ broadcast time.



How an election starts?

Servers know when an election is needed because the actual leader periodically broadcasts heartbeats to followers.

Prof. Question: How does leader election work in RAFT?

Nodes are **Follower**, **Candidate**, or **Leader**. A follower that misses heartbeats past a **random timeout** becomes a candidate, increments the **term**, and asks for votes. Each node votes **once per term**. The candidate that gets a **majority** ($\lfloor \frac{N}{2} \rfloor + 1$) becomes leader and starts heartbeats. Random timeouts avoid split votes, since one node usually wakes first.

Prof. Question: How does the quorum work? Can I have 4 nodes?

Quorum is the **absolute majority** needed to decide. 3 nodes tolerate 1 failure, 5 tolerate 2. **4 nodes work but are no better than 3**: quorum is 3/4, still only 1 tolerated failure, with more overhead. Always prefer an **odd number** of nodes.

Log Replication (normal operation)

- The **leader accepts commands from clients, appends them to its log**.
In kubernetes, the client is the API server.
- The leader **sends AppendEntries RPCs with the log to all followers**.
- Once a majority acknowledges, the entry is **committed**.
- The leader executes the command, returns the result, and notifies followers.
- Crashed or slow followers: the leader retries until they succeed.
- Optimal performance: **one successful RPC to any majority**.

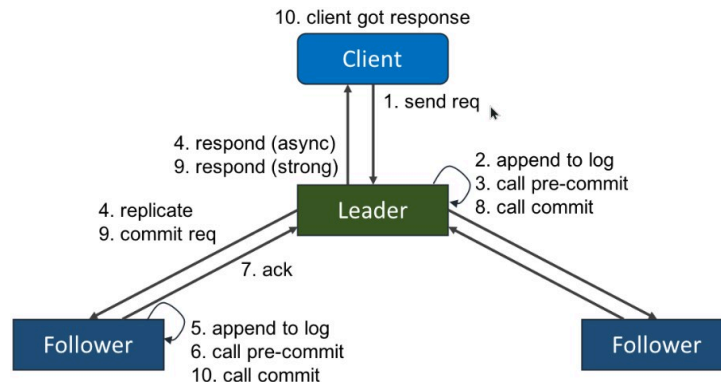


Figure 45 - RAFT log replication.

Prof. Question: How does consistency work in RAFT?

All nodes keep a copy of the **replicated log**. An entry is **committed** only after a **majority** has written it. By the safety rule, only a node with an up-to-date log can become leader, so committed entries are never lost. If the leader dies mid-replication, the new leader (the most up-to-date one) finishes it. The one thing that must survive is the **log on disk**.

Safety

- Logs are kept consistent across the **cluster**.
- Only servers with up-to-date committed logs can become leader.
- Leader is **stateless** between terms: it rebuilds state from the log.



👉 **Note:** Leader is stateless between terms (not in general) because every replica is equal, no-one has an internal state that state it is the leader or not. Obviously, since etcd is the only one stateful component, it has a state, also every replica has and that is the log.

Analogy: RAFT as a Parliament

RAFT's leader is like a Parliament's Speaker: only one can chair a session at a time. A bill (log entry) only passes when a majority of members vote for it. If the Speaker is absent, members elect a new one, but only a member who has read all previous bills can stand for election.

👤 **Prof. Question:** What happens if a node dies in RAFT?

A **follower dies**: the system keeps working while the quorum holds, and the node catches up on restart. The **leader dies**: followers stop getting heartbeats, after a random timeout one starts an election, and the node with the most up-to-date log wins. The service is briefly unavailable during the election. The **log on disk** must not be lost.

9.6. Nodes, Pods, and Workloads

9.6.1. Nodes

📖 Node

A **node** is a **computational resource** (physical or virtual machine) that runs workloads. Each node communicates with the control plane through gRPC calls.

Node components: `kube-proxy`, `kubelet`, Pods.

9.6.2. Pods

📖 Pod

A **Pod** is the **atomic unit** of deployment in Kubernetes: the smallest deployable object. A Pod represents a group of **one or more containers** that share:

- A **network namespace** (same IP address and port space).
- **Volumes** (shared storage).
- A common lifecycle (co-scheduled, co-located, co-terminated).

Key Pod properties:

- Pods are **mortal and unreliable**: if a Pod terminates, Kubernetes starts a new one (with a different IP) rather than restarting the old one.
- Pods are the minimum unit of **scaling**: never scale individual containers, scale Pods.
- When a Deployment creates Pods, each Pod is **scheduled** (tied to a Node), but if that Node fails, identical Pods are scheduled on other available nodes.

🤔 Why a group of containers?

Some services consist of tightly-coupled helper processes that must share localhost (e.g., a web server + a log collector sidecar). Grouping them in a Pod guarantees co-location and shared networking without full container merging. For most workloads, one container per Pod is the norm.



100 Prof. Question: *What happens if I place two containers in the same Pod? Is it convenient?* They share the same **IP and port space**, the **volumes**, and the **network namespace** (they talk via `localhost`). **Pros:** zero-latency local communication and shared storage. **Cons:** you cannot scale the containers independently, only the **whole Pod** scales (the atomic unit). Convenient when they are **tightly coupled**, as in the **sidecar** pattern (app + log collector).

9.6.3. Scheduler

Scheduler

The **scheduler** watches for newly created Pods that have no node assigned, and selects the **best node** to host each Pod.

Scheduling algorithm:

1. Filter all nodes down to **Feasible Nodes** by checking constraints (resource requests, node selectors, affinity/anti-affinity rules, taints/tolerations).
2. **Score** each feasible node by running a set of scoring functions.
3. Assign the Pod to the node with the highest score (**binding**).
4. If no feasible node exists, the Pod remains **unschedulable**.

Note: **Labels** are key for scheduling : they let targeting workloads to specific machine pools (e.g., GPU nodes, high-memory nodes) using node **affinity rules**.

9.6.4. Kubelet

Kubelet

The **kubelet** is the primary **node agent** running on each worker node. It takes a set of PodSpecs provided through the API server and ensures that the containers described in those PodSpecs are running and healthy.

Note: The kubelet acts like a node-level Operator, but its state is local to the worker node while the source of truth (etcd) resides on the Master plane. Because state updates (Pod statuses) and heartbeats must travel over the network to the API Server, network latency and overhead naturally create windows of **temporary inconsistency** (stale state).
Observability is key.

9.6.5. Workload Resources

9.6.5.1. Deployment and ReplicaSet

ReplicaSet



A **ReplicaSet** ensures that a **specified number of Pod replicas** are running at any time. If Pods die, the ReplicaSet creates replacements. If too many run, it deletes the excess.

Deployment

A **Deployment** wraps a ReplicaSet and adds **rolling update** and **rollback** capabilities. It is the recommended way to manage stateless applications. A single Deployment can manage only a single type of Pod.

Deployments use a second ReplicaSet during rolling updates: the new ReplicaSet scales up while the old one scales down, ensuring zero downtime .

9.6.5.2. StatefulSet

StatefulSet

A **StatefulSet** runs a group of Pods and maintains a **sticky identity** for each. Unlike Deployments, StatefulSets give each Pod:

- A stable, **unique network identifier** (predictable DNS name).
- Stable, **persistent storage** (each Pod's PVC is not destroyed on reschedule).
- **Ordered, graceful deployment and scaling**.
- **Ordered, automated rolling updates**.

StatefulSets are used for applications that require persistent storage or stable network identity, such as databases, message brokers, distributed caches.

9.6.5.3. DaemonSet

DaemonSet

A **DaemonSet** ensures that **all (or some) nodes run a copy of a Pod**. As nodes are added to the cluster, Pods are added to them; as nodes are removed, those Pods are garbage collected.

Typical uses:

- Running a **cluster storage daemon** on every node (e.g., Longhorn, Ceph).
- Running a **log collection daemon** on every node (e.g., Fluentd).
- Running a **node monitoring daemon** on every node (e.g., Prometheus Node Exporter).

9.6.5.4. Jobs and CronJobs

Job

A **Job** creates one or more Pods and ensures that a **specified number of them successfully terminate**. Jobs are for finite, batch workloads.

CronJob



A **CronJob** runs a Job on a repeating schedule (cron syntax).

9.7. Storage



Kubernetes Volume

A **volume** in Kubernetes is a **directory accessible to containers in a Pod**. It solves the problem of data sharing between containers in a Pod and data persistence across container restarts.

Two fundamental types:

- **Ephemeral volumes**: lifetime tied to the Pod. When the Pod ceases to exist, Kubernetes destroys ephemeral volumes.
- **Persistent volumes**: exist beyond the lifetime of a Pod. Data is preserved across container restarts, Pod replacements, and node failures.

Common volume uses:

- Populating configuration from a ConfigMap or Secret.
- Providing temporary scratch space for a Pod.
- Sharing a filesystem between two containers in the same Pod.
- Sharing a filesystem between two different Pods (even on different nodes, via distributed storage).
- Durably persisting data across Pod restarts/replacements.

👉 **Note**: The declarative approach allows to claim a volume of a certain class without caring about the underlying implementation.

9.8. Networking

9.8.1. The Kubernetes Network Model



Kubernetes Network Fundamentals

- Every Pod gets its own **unique cluster-wide IP address**.
- A Pod has its own **private network namespace** shared by all its containers: containers within a Pod communicate via `localhost`.
- The **pod network** (cluster network) handles communication between Pods.
- **All Pods can communicate with all other Pods** on any node, without NAT or proxies, whether they are on the same node or different nodes.

🤔 **Why flat networking without NAT?**

NAT creates hidden address translations that complicate service discovery and debugging. A flat model where every Pod has a real, routable IP simplifies the programming model enormously: you can treat any Pod like a local process.

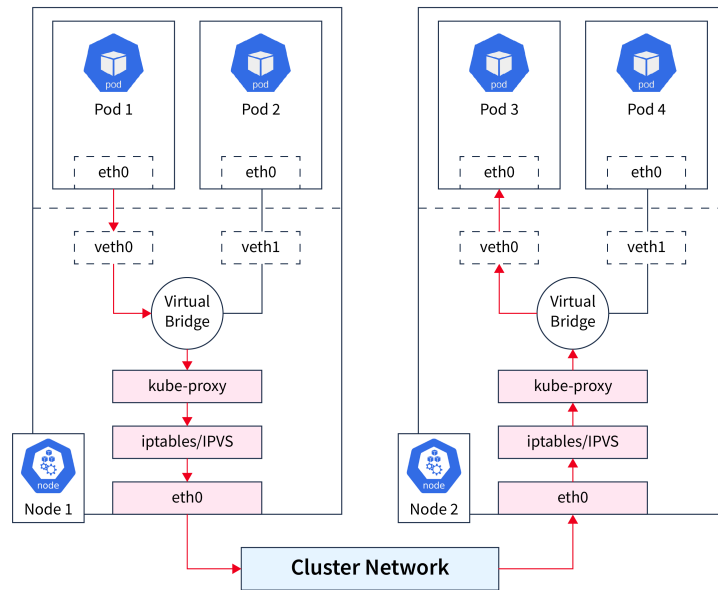


Figure 46 - Kubernetes network scheme.

100 Prof. Question: *How does networking work in Kubernetes (high-level)?*

Every Pod gets its own **cluster-wide IP**, and every node its own IP. The core rule: **any Pod can talk to any other Pod directly**, on the same or a different node, **without NAT or proxies**. This flat model makes a remote Pod look like a local process.

9.8.2 Kube-Proxy



Kube-Proxy

kube-proxy is a network proxy that runs on each node. It reflects Services defined in the Kubernetes API and implements forwarding (via iptables, IPVS, or eBPF) so that traffic sent to a Service IP/port is correctly forwarded to one of the backend Pods.

9.8.3 CNI Plugins

The **Container Network Interface (CNI)** is a standard for how Kubernetes communicates with the network plugin. Popular plugins:

- **Flannel**: simple overlay network using VXLAN tunnelling between nodes. Each node gets a subnet (e.g., `10.244.1.0/24`); Flannel tunnels inter-node Pod traffic through the underlay.
- **Canal**: extends Flannel with **Calico** network policy enforcement (BGP, IPIP, VXLAN, UDP options; policy enforcement via Calico).
- **Cilium**: modern approach using **eBPF** in the kernel, bypassing iptables entirely. Provides higher throughput, lower latency, and deep observability.

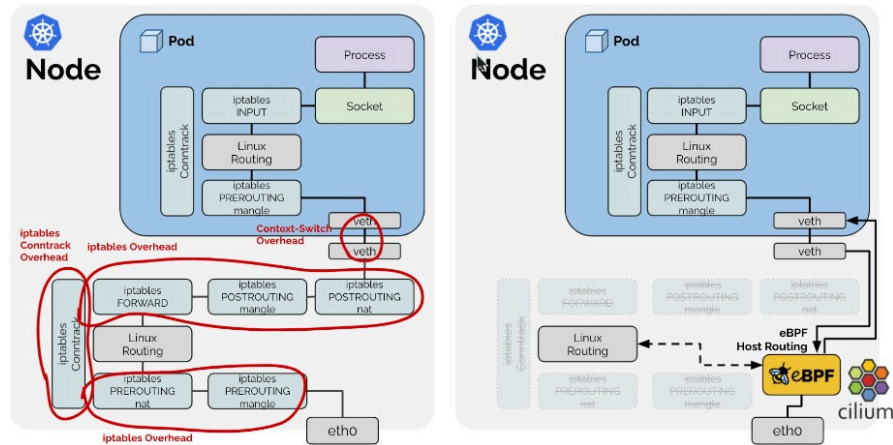


Figure 47 - eBPF vs. standard k8s.

9.8.4 Services

Kubernetes Service

A **Service** is an **abstraction** that defines a logical set of Pods and a policy by which to access them. It provides a stable, long-lived **IP address** or **hostname** for a set of backend Pods that may come and go.

Kubernetes automatically manages **EndpointSlice** objects to track the set of Pods currently backing a Service. A service proxy implementation monitors these objects and programs the data plane accordingly.

Note: A Service is **domain-specific deployment**: it decouples the consumer from the producer. Consumers address the Service, Kubernetes handles load balancing across the Pods.

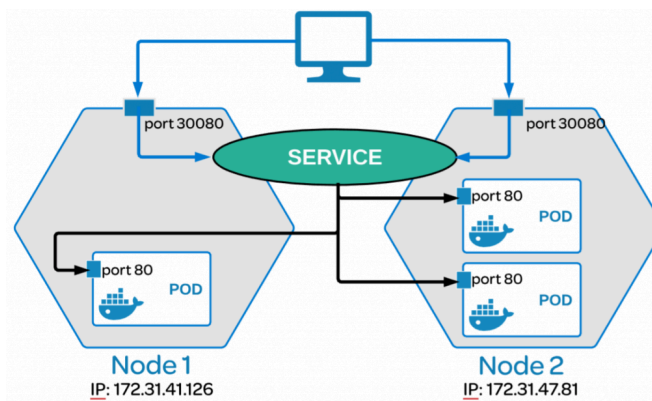


Figure 48 - Service in Kubernetes.

9.8.5 Ingress

Ingress

An **Ingress** exposes **HTTP/HTTPS routes** from **outside the cluster** to Services within the cluster. Traffic routing is controlled by rules defined on the Ingress resource (host-based and path-based routing).



- A **cluster** can host multiple Ingress implementations sharing the same interface .
- Each implementation has a custom **Ingress Controller** that manages the lifecycle of the ingress and translates rules into configuration for the underlying load balancer (NGINX, Traefik, HAProxy, cloud LBs...).

Analogy: Ingress as Hotel Reception

An Ingress is like a hotel receptionist: external guests (clients) arrive at the hotel front door (Ingress). The receptionist inspects the guest's name or room number (hostname/path) and directs them to the correct floor and room (Service and Pod).

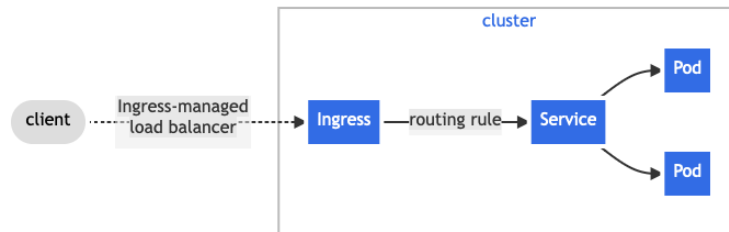


Figure 49 - Ingress in Kubernetes.

100 Prof. Question: What is a Service in Kubernetes? And an Ingress?

A **Service** gives a **stable endpoint** (DNS name + fixed IP + port) for a changing set of Pods, load-balancing across them via label selectors. Modes: **ClusterIP** (internal only), **NodePort** (a port on every node), **LoadBalancer** (external cloud LB). An **Ingress** maps external HTTP/HTTPS hosts and paths to internal Services (e.g. `unibo.it/students` → Service "students"). It needs an **Ingress Controller** to do the routing.

9.9. Load Balancing and Autoscaling

9.9.1. Horizontal Pod Autoscaler (HPA)

Horizontal Pod Autoscaler

The **HPA** automatically updates a workload resource (Deployment, StatefulSet) by **scaling the number of Pod replicas** to match observed demand.

It is implemented as a Kubernetes API resource and a controller.

The HPA controller **runs in the control plane** and periodically queries metrics. It computes the desired replica count using:

$$\text{desiredReplicas} = \left\lceil \text{currentReplicas} \times \frac{\text{currentMetricValue}}{\text{desiredMetricValue}} \right\rceil$$

Configurable parameters:

- **Min/Max replicas**: hard bounds on the replica count.
- **Percent of replicas** that can be concurrently scaled up or down.
- **Stabilisation windows**: prevents flapping by requiring the metric to stay above/below a threshold for a configurable duration before scaling.

Observed metrics can be:

- Per-pod resource metrics (CPU, memory).



- Per-pod custom metrics.
- External metrics (e.g., queue depth from a message broker).

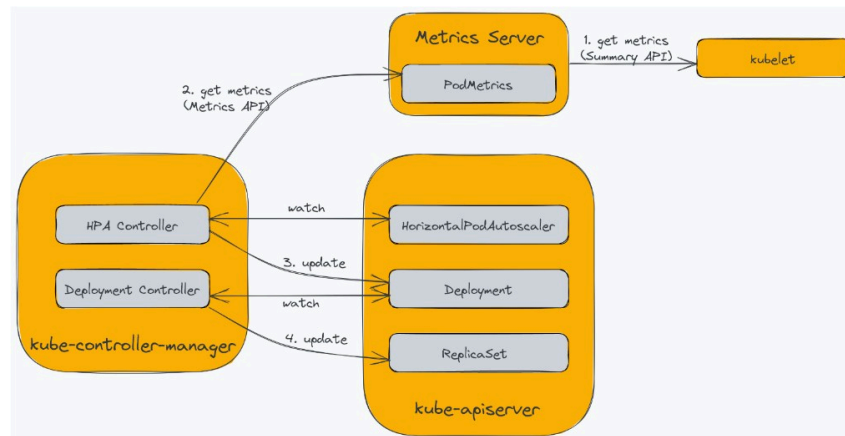


Figure 50 - HPA.

9.9.2. KEDA: Kubernetes Event-Driven Autoscaling

KEDA

KEDA (Kubernetes Event-Driven Autoscaling) extends the HPA mechanism to support **event-driven scaling** based on the number of events needing to be processed. Key properties:

- **Lightweight and flexible:** runs as a single deployment.
- **Built-in scalers** for 50+ platforms (databases, cloud queues, messaging systems).
- **Scale to zero:** can optionally reduce a workload to 0 replicas when idle.

Custom or third-party schedulers can also be deployed alongside the default scheduler if the default scoring functions are insufficient for your workload.

Prof. Question: How do I host a microservice web service on Kubernetes?

Use a **microservices** architecture on K8s (portability, autoscaling, self-healing, declarative). (1) one **Deployment** per microservice, with its image. (2) one **Service** per Deployment to expose it (ClusterIP internal, LoadBalancer external). (3) if the frontend is fast and the backend slow, add an **HPA** to scale the backend Pods, with a Service in front to load-balance them. Alternative: put a **MOM** (Kafka) between frontend and backend to decouple them and absorb spikes.

9.10. Service Mesh

9.10.1. What is a Service Mesh?

Service Mesh

A **service mesh** is an **overlay infrastructure layer** that manages communication between microservices in a distributed application.

It simplifies complex service-to-service communication by providing managed and configurable:

- **Traffic Management**



- **Observability**
- **Security**
- **Dev/Ops** capabilities (fault injection, canary releases...)

100 Prof. Question: *Why does Kubernetes offer two models? What is a Service Mesh?*

K8s has two abstraction levels: the **base primitives** (Deployment, Service, Ingress) for standard workloads, and the **Operator pattern** (CRD + custom controller) for complex stateful apps with domain logic. A **Service Mesh** is a dedicated layer for service-to-service communication, usually via **sidecar proxies**, giving **observability**, **mTLS security**, and **traffic control** (circuit breaking, retry, timeout, canary). Examples: Istio, Linkerd.

Why a service mesh?

In a modern microservice deployment with dozens or hundreds of services, each service needs to handle retries, timeouts, circuit breaking, mTLS, load balancing, and distributed tracing. Implementing this logic in every service is error-prone and creates duplication. A service mesh moves all of this to the **infrastructure layer**, keeping business logic clean.

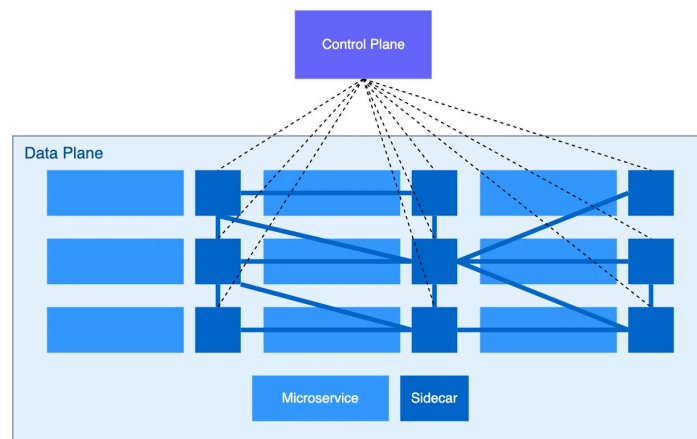


Figure 51 - Service mesh.

9.10.2. Service Mesh Architecture

A Service Mesh has two macro layers:

- **Data Plane:** realises secure, monitored, and reliable execution and communication. The main component is the **Service Mesh Proxy** installed as a **sidecar** in the same Pod as each hosted service. The sidecar acts as the unique point of access for all communication and implements monitoring and security strategies inside the Pod.
- **Control Plane:** a Kubernetes Controller that acts as registry for configuration of proxies: realises observability, traffic management, and security features.



Istio

In Istio, the control plane component is called **Istiod**. It provides:

- **Traffic Management:** converts high-level routing rules into proxy-specific configuration and propagates them to the sidecars at runtime.
- **Secure Communication:** acts as a Certificate Authority (CA) and generates certificates to allow secure mTLS communication in the data plane.

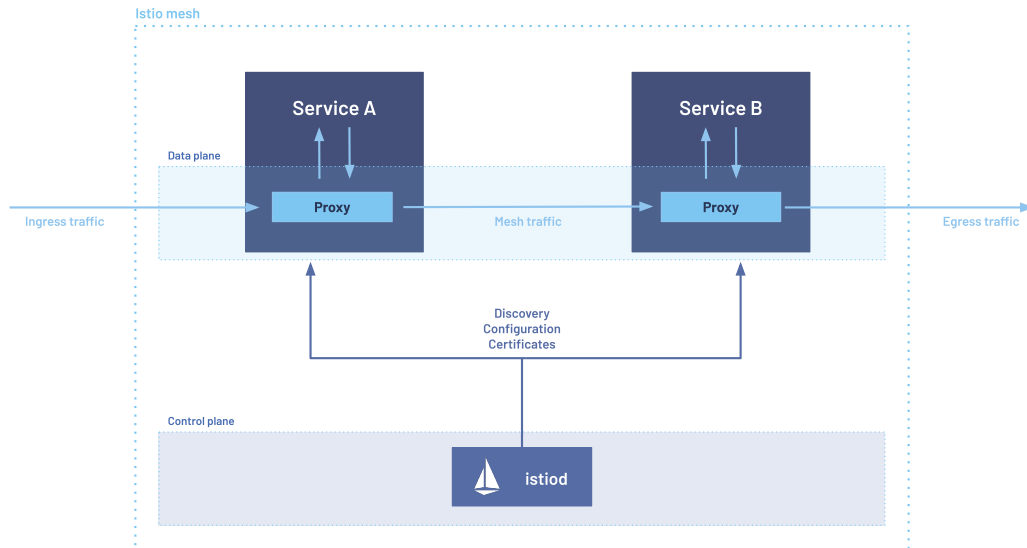


Figure 52 - Istio service mesh: Envoy sidecars in the data plane, Istiod in the control plane.

9.10.3. Traffic Management

Service Mesh manages network requests between services. Because traffic flows through the **local sidecar proxy**, the mesh can implement at infrastructure level:

- **Request Routing**: path-based and header-based routing rules.
- **Load Balancing**: round-robin, least connections, consistent hashing.
- **Fault Mitigation**: retries, timeouts, circuit breaking.
- **Traffic Shifting**: canary releases (e.g., 5% to new version, 95% to stable).
- **Traffic Mirroring**: shadow traffic to test new versions without user impact.
- **Content-based traffic steering**: route by **User-Agent**, cookies, or request headers.
- **Locality-aware load balancing**.
- **Ingress/Egress abstractions**.

Note: Traffic splitting is decoupled from infrastructure scaling: the proportion of traffic routed to a version is independent of the number of instances supporting that version.

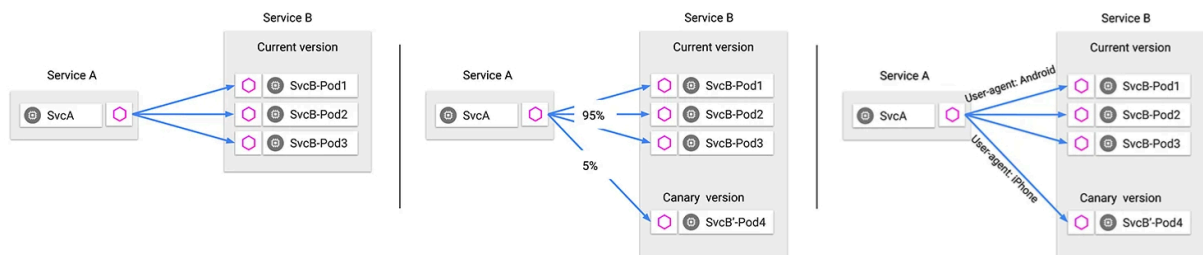


Figure 53 - Traffic management examples.

9.10.4. Observability

Service Mesh enables deep observability of the entire fleet:

- **Logs**: agents (proxies) installed in each Pod and node collect service and infrastructure logs.
- **Monitoring**: agents actively collect metrics on running services (request rate, latency, error rate).



- **Traces:** communication among services is tracked to generate dependency graphs, monitor performance, and find bottlenecks or root causes of failures.

9.10.5. Security

Service Mesh supports security at the infrastructure level:

- **Automatic Service Certificate Management:** get, verify, and renew certificates.
- **Authentication and Authorization.**
- **Secure Communication establishment** (mTLS between all services).
- **D-DOS protection and rate limiting.**

9.10.6. Dev/Ops Capabilities

- **Fault Injection:** deliberately inject failures to test resilience.
- **Circuit Breaking:** stop sending traffic to a failing service automatically.
- **Canary Release:** gradually shift traffic to a new version.
- **A/B Testing.**
- **Test Performance Collection.**
- **Connection Replication.**
- **Production/Test Environment Setup.**

9.10.7. Service Mesh at Meta (Case Study)



Meta's Hyperscale Service Mesh

Meta operates at hyperscale with hundreds of millions of concurrent requests and many distributed services across geo-distributed regions. They evaluated multiple L7 router implementations and chose different approaches depending on context:

- **Sidecar-Proxy** load balancing (standard).
- **Dedicated Load Balancers** (centralized).
- **Lookaside Load Balancers:** a centralized lookup service that clients query.
- **Embedded Routing Libraries:** routing logic compiled into the client itself.

Their architecture relies on three components:

- **RIB (Routing Information Base):** a strongly consistent key-value store replicated across five geographic regions for high availability. Uses thousands of RAFT learner replicas to ensure read throughput and availability even when disconnected.
- **RIBDaemon:** runs on each machine; maintains a `miniRIB` that caches only the service metadata needed by the RPCs on that machine.
- **SRLib:** a language-specific library that acts as interface to other services, providing the same functionality as a sidecar proxy without a separate process.

The key lesson from Meta's work: there is **no single best architecture** for a service mesh. The trade-off between sidecar, lookaside, dedicated, and embedded approaches depends on latency requirements, operational complexity, and the consistency vs. availability needs of the specific service.



10. OPENSTACK

Package: OpenStack - c - Openstack 26.pdf

OpenStack is an open-source **cloud operating system** that pools physical hardware resources into on-demand compute, storage, and network services exposed through standardized APIs. It is the canonical example of an open-source **IaaS** platform, and understanding its architecture reveals the fundamental design patterns behind any large cloud infrastructure.

10.1. Cloud Data Centres and IaaS

10.1.1. The Cloud Data Centre

A **cloud data centre** is a large-scale facility, often federated with other sites, that must simultaneously satisfy two contrasting requirements:

- **Real-time answering:** users must not wait; latency is a first-class metric.
- **Safety of data:** correctness and durability must not be sacrificed for speed.

In global deployments, data and computation are replicated across many geographical locations to satisfy both goals.

10.1.2. Cloud Resource Virtualization

Building a cloud requires virtualizing resources in two successive steps:

Step 1 - Server virtualization: hypervisors (VMware ESX, KVM, Xen) partition a physical host into multiple **virtual machines** (VMs/instances), providing a hardware-abstraction layer and dramatically improving resource utilization.

Step 2 - Network and storage virtualization: once servers are virtualized, the network and storage are also pooled:

- **Compute pool:** virtualized servers.
- **Network pool:** virtualized networks.
- **Storage pool:** virtualized storage.

The result is a unified resource pool that offers flexibility and efficiency for many simultaneous applications.

Analogy: Cloud as a Utility

Just as the electrical grid abstracts the physical generators from the consumer, cloud resource virtualization abstracts physical hardware from application developers. Plugging in does not require knowing which power plant you are connected to.

10.2. OpenStack Overview

10.2.1. History and Positioning



OpenStack



OpenStack is an open-source **cloud operating system** (IaaS platform). It is distributed under the Apache 2.0 licence with a six-month release cycle.

Key positioning facts:

- Open-source alternative to Amazon AWS, Microsoft Azure, VMware.
- Supports multiple hypervisors: KVM, Xen, XenServer, Hyper-V.
- Exposes Amazon-compatible and Rackspace-compatible APIs.
- Designed for both public and private cloud deployments.

10.2.2. High-Level Architecture

OpenStack is a **cloud operating system** that sits between applications and physical hardware:

- **Upward:** it **exposes RESTful APIs** consumed by applications, admins, and users.
- **Downward:** it creates resource pools from physical servers and automates the network.
- **Internally:** it **acts as a control plane**: scheduling, orchestration, image management, identity, and metering.

10.2.3. Logical Layers

A cloud must handle three planes:

- **Resources:** raw compute (VMs), block volumes, and network.
- **Logic/Control:** orchestration, scheduling, policy, image registry, logging.
- **Presentation:** APIs for developers and operators, user dashboard, customer portal.

Cross-cutting concerns:

- **Integration:** billing and identity services.
- **Management:** monitoring and admin APIs.

In OpenStack these map directly to named services (see next section).

10.2.4. Design Guidelines

All OpenStack services share a common architectural philosophy:



OpenStack Design Principles

- **Horizontal scalability:** **scale out by adding nodes**, not scaling up.
- **Minimal dependencies:** services are designed to be loosely coupled and each replicates its core components to avoid single points of failure.
- **Shared-nothing:** each service stores all needed information internally ⇒ **no global shared state between services**.
- **Asynchronous, pub/sub communication:** **services communicate through a message queue** (AMQP/RabbitMQ) rather than synchronous RPC, making the system resilient and decoupled.

🤔 *Why asynchronous messaging?*

In a cloud, a single user request may trigger actions across many services (compute, networking, storage, image). If each call were synchronous, one slow service would block the entire chain. Message queues decouple the sender from the receiver: requests are queued and processed when the service is ready, improving resilience and enabling horizontal scaling of any individual service.



100 Prof. Question: *What is the structure of an OpenStack service?*

Every service follows the same design: **horizontal scalability** (scale out), **shared-nothing** (each service has its own DB, no shared state), **loosely coupled** via **asynchronous pub/sub** over a MOM (some operations like VM creation are slow), **stateless where possible** (token auth), and **reliable** (replicated core components, minimal inter-service dependencies).

10.2.5. Common Service Architecture

Every OpenStack service is built from the same four internal building blocks:

- **pub/sub messaging service**: AMQP-based (RabbitMQ by default, Qpid supported).
- **One or more core components**: the actual service logic.
- **RESTful API component**: the external face of the service, interoperable with clients.
- **Local database component**: stores internal service state (MySQL, MongoDB, SQLAlchemy, HBase depending on requirements).

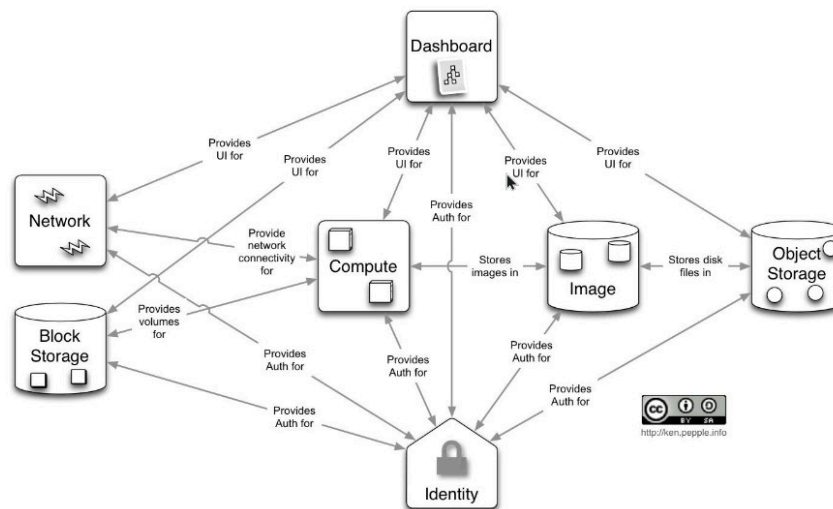


Figure 54 - Openstack.

10.3. OpenStack Core Services

OpenStack Main Services

The seven canonical OpenStack services are:

- **Horizon**: Dashboard (web UI)
- **Keystone**: Identity and authentication
- **Nova**: Compute (VM lifecycle)
- **Neutron** (formerly Quantum): Networking
- **Glance**: Image service
- **Swift**: Object storage
- **Cinder**: Block storage

Additional services: **Ceilometer** (telemetry/metering), **Heat** (orchestration).

All services communicate through Keystone for authentication and through the message queue for internal coordination. The Dashboard (Horizon) provides a unified graphical interface across all services.



100 Prof. Question: *What are the core services of OpenStack? (IaaS levels)*

OpenStack is an open-source **IaaS**. Core services: **Horizon** (dashboard), **Keystone** (identity/auth), **Nova** (compute/VMs), **Neutron** (networking), **Glance** (VM images), **Swift** (object storage), **Cinder** (block storage). Plus **Ceilometer** (telemetry) and **Heat** (orchestration).

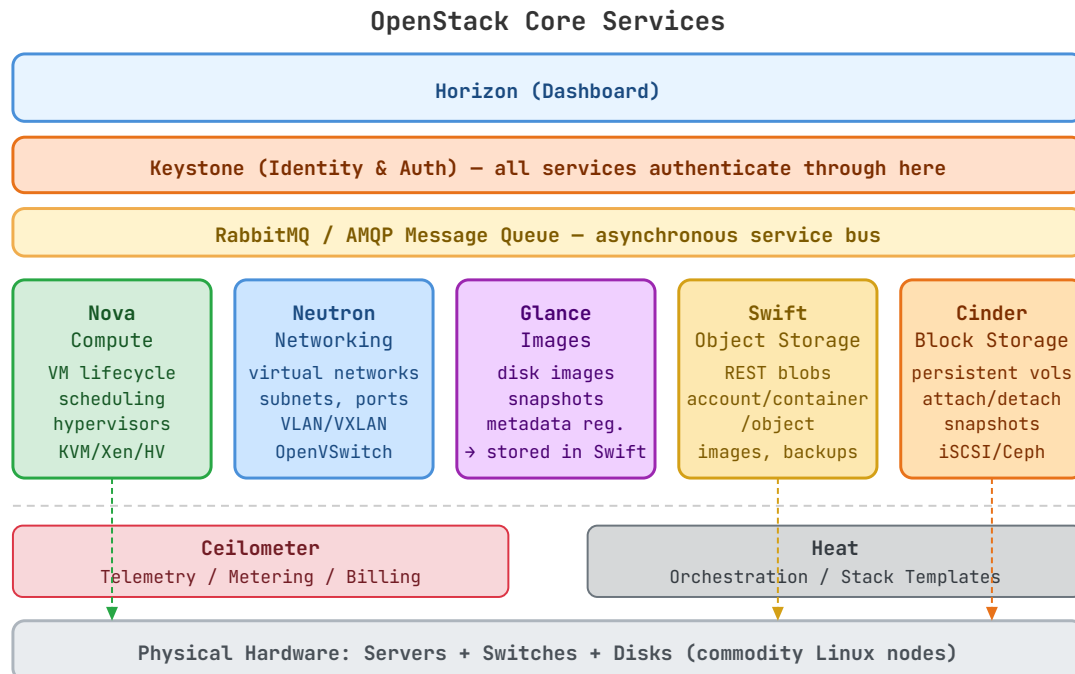


Figure 55 - OpenStack core services.

10.4 Nova - Compute Service

10.4.1. What Nova Does

Nova

Nova is OpenStack's **compute** service. It provisions and manages large pools of virtual machine instances across a cluster of hypervisor nodes.

- Provides **on-demand virtual servers (instances)**.
- Manages large networks of VMs.
- Designed to **horizontally scale** on standard commodity hardware.
- Supports multiple hypervisors: KVM, XenServer, VMware, Hyper-V.
- Exposes **compute resources via REST APIs**, web interface, and CLI.

10.4.2. Nova Internal Components

Nova is itself a distributed system, with dedicated daemons for each function:

- **nova-api**: RESTful API gateway (supports OpenStack, EC2, and admin APIs). All client commands arrive here.



- **nova-compute**: communicates with the underlying hypervisor (libvirt, XenAPI, etc.) to start/stop/manage VM instances.
- **nova-scheduler**: coordinates all services and determines placement of new VM requests: which physical host gets the new VM.
- **nova-conductor**: mediates database access from nova-compute to avoid direct DB connections from untrusted compute nodes.
- **nova database**: stores build-time and run-time state of the cloud infrastructure (typically MySQL).
- **Queue (RabbitMQ)**: the message bus that connects all Nova services. Requests are enqueued, enabling async processing and decoupling.
- **nova-console / nova-novncproxy / nova-consoleauth**: provide proxied console access to VM instance and authentication.

Note: `nova-network` (network configuration) and `nova-volume` (persistent storage) are legacy components. Their responsibilities have been moved to **Neutron** and **Cinder** respectively in modern deployments.

10.4.3. Nova General Interaction Pattern

The request flow for any Nova operation follows this pattern:

1. Client sends a request to **Nova API**.
2. Nova API validates credentials with **Keystone** and writes initial state to **MySQL**.
3. Nova API forwards the request to **Nova Scheduler** via **RabbitMQ**.
4. Nova Scheduler confirms resource availability, selects a host and publishes back.
5. The target **Nova Service** (nova-compute) polls RabbitMQ, receives the message, and executes the action.
6. Both Scheduler and Service update **MySQL** with the final state.

Analogy: Nova as a Dispatcher

Nova behaves like a taxi dispatch center: the API is the phone line (receives requests), the scheduler is the dispatcher (decides which driver handles the job), and nova-compute nodes are the individual drivers. RabbitMQ is the radio network connecting dispatch to drivers.

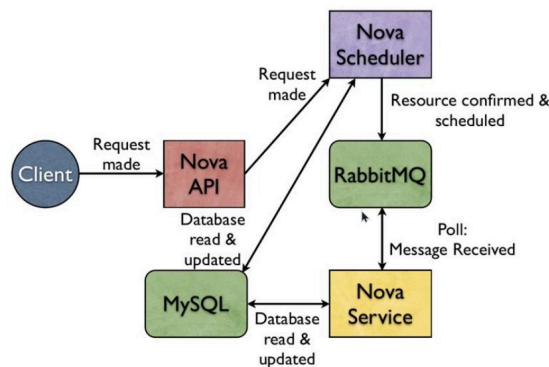


Figure 56 - Nova general interaction scheme.

10.4.4. VM Provisioning Workflow (Multi-Service)

Launching a VM involves coordinating Nova, Keystone, Glance, and Swift:

1. Client asks Nova API to launch a VM.
2. Nova API authenticates with **Keystone**.
3. Nova Scheduler picks a compute host and schedules the VM.
4. Nova API requests the **boot image** from **Glance API**.
5. Glance Registry looks up image metadata ⇒ Glance API fetches the actual image from **Swift**.



6. Nova Compute receives the image and starts the VM on the hypervisor.

10.5. Swift - Object Storage

10.5.1. What Swift Does

Swift

Swift is OpenStack's **distributed object storage** service. It stores and retrieves unstructured data objects (files, images, backups, archives) via HTTP/REST, without requiring a traditional filesystem hierarchy.

Swift is not a filesystem

Swift is not a POSIX filesystem or a block device. It stores **objects** (arbitrary byte sequences + metadata) identified by a hierarchical name (account / container / object). **Mutations create new objects rather than modifying existing ones**. This is what enables concurrent reads and efficient replication (only changed chunks need to be transferred).

- Provides **scalability, redundancy, and durability** through replication.
- No central point of control, inherently distributed.
- **Stores static data**: VM images, photo storage, email storage, backups.
- Accessed via APIs or integrated directly inside applications.

10.5.2. Object Storage Data Model

Objects in Swift are organized in a **three-level** hierarchy:

- **Account**: top-level namespace (like a user or project).
- **Container**: a named bucket within an account (like a directory, but flat).
- **Object**: the actual data blob + metadata (name, size, content-type, custom headers).

 **Note**: Objects are structurally **immutable**. If the data content changes, a new object is written entirely (or per-chunk), and the old one is asynchronously garbage-collected. If only metadata changes, just the metadata storage is updated without copying or duplicating the underlying heavy data blob.

10.5.3. Swift Components

- **swift-proxy**: the entry point. It handles all incoming requests (uploads, metadata modifications, container creation) and routes them internally.
- **Account server**: manages account-level metadata defined through the storage service.
- **Container server**: maps containers (buckets) within the object storage service.
- **Object server**: manages the actual file data on storage nodes.

The proxy is the only publicly exposed component, while the storage servers form an internal ring accessed only through the proxy.

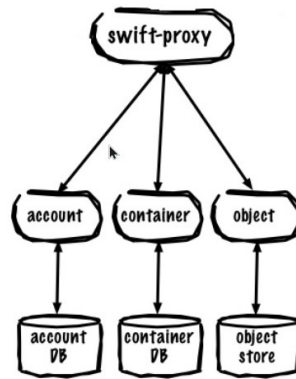


Figure 57 - Swift.

10.6. Cinder - Block Storage

10.6.1. What Cinder Does

Cinder

Cinder is OpenStack's **block storage** service. It manages persistent storage **volumes** that can be attached to VM instances, analogous to a cloud version of a SAN or network-attached disk.

Analogy: Cinder as a USB drive

A Cinder volume is like a USB drive in the cloud: it can be formatted and mounted into a running VM, and later detached and re-attached to a different VM. The data persists independently of the VM lifecycle.

- Creates, attaches, and detaches **volumes** to/from VM instances.
- Supports protocols: iSCSI, NFS, FC, RBD (Ceph), GlusterFS.
- Supports backends: Ceph, NetApp, Nexenta, SolidFire, Zadara, and more.
- Allows creating **snapshots** of volumes for backup or cloning.

10.6.2. Cinder Components

- **cinder-api**: accepts user requests and dispatches them to cinder-volume.
- **cinder-volume**: executes read/write operations; interacts with the database and message queue to maintain consistency.
- **cinder-scheduler**: selects the best storage backend/node to create a new volume.
- **cinder database**: maintains the state of all volumes (available, in-use, error, etc.).

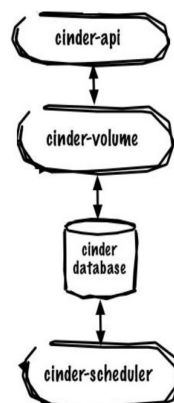


Figure 58 - Cinder.



100 Prof. Question: *Difference between Object and Block storage? Scalability and consistency?*
Object storage (Swift) keeps whole **immutable** objects with metadata, accessed by **HTTP REST**, very **scalable** on commodity hardware, **eventually consistent** (AP). **Block storage** (Cinder) exposes **raw mutable blocks** like a disk (POSIX), **strongly consistent**, but less scalable and pricier. Example: to change 4 KB in a 4 MB file, Swift must **rewrite the whole 4 MB object**, while Cinder edits only the 4 KB **block**.

10.7. Glance - Image Service

10.7.1. What Glance Does

Glance

Glance is OpenStack's **image service**. It handles discovery, registration, and delivery of disk images and virtual server snapshots.

- Allows storing images on **different storage backends** (filesystem, Swift, Amazon S3, HTTP).
- Supports multiple disk formats: Raw, qcow2, VMDK, VHD, ISO, etc.
- Images are typically stored in **Swift**, Glance acts as a **metadata catalog** and API layer on top of the actual storage.

10.7.2. Glance Components

- **glance-api**: handles API requests to discover, store, and retrieve images.
- **glance-registry**: stores, processes, and retrieves image **metadata** (dimensions, format, owner, etc.).
- **glance database**: the metadata store.

Note: Glance itself does not store image bits, it delegates that to an external repository (Swift, S3, filesystem). It is a metadata registry + delivery proxy.

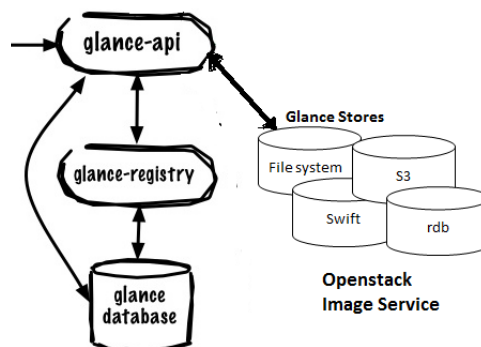


Figure 59 - Glance.



10.8. Horizon - Dashboard

Horizon

Horizon is OpenStack's **web-based dashboard**. It provides a modular, graphical interface for administrators and users to access and manage all other OpenStack services.

Actions available through Horizon:

- Launch and terminate instances.
- Assign floating IP addresses.
- Upload VM images.
- Define security groups and access policies.
- Manage volumes, networks, and object storage.

Horizon is purely a **presentation layer**: it calls the same REST APIs that any other client would use. It adds no new functionality but greatly lowers the barrier to using OpenStack.

10.9. Keystone - Identity Service

10.9.1. What Keystone Does

Keystone

Keystone is OpenStack's **identity service**. It provides centralized **authentication** and **authorization** for every other OpenStack service.

- Creates **users**, **groups** (also called **tenants/projects**).
- Manages **permissions** and role-based access control (RBAC).
- Issues **tokens** that replace password authentication after initial login.
- Maintains an **endpoint catalog** so services can discover each other's URLs.

10.9.2. Keystone's Four Sub-Services

- **Identity**: user information and authentication.
- **Token**: after login, replaces the password with a time-limited token for subsequent API calls.
- **Catalog**: endpoint registry → maps service type (compute, image, etc.) to URLs.
- **Policy**: rule-based authorization engine.

10.9.3. Keystone Request Flow

1. Client sends a request with credentials to Keystone.
2. Keystone authenticates and returns a **token** (+ tenant/role information).
3. Client forwards the token with its request to the target OpenStack service.
4. The **target service validates the token with Keystone** before processing the request.
5. Unauthorized requests are rejected at step 2 or step 4.

👉 **Note**: Since Keystone is therefore the single, centralized trust anchor for the entire cloud, if it is unavailable, no authenticated operation in the cloud can proceed.

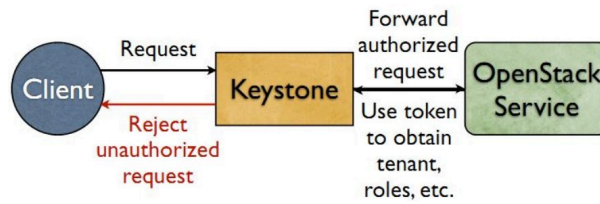


Figure 60 – Keystone.

10.10. Neutron - Networking Service

10.10.1. What Neutron Does

Neutron

Neutron is OpenStack's **networking service** (formerly called Quantum). It provides **Network as a Service (NaaS)**: pluggable, scalable, API-driven management of networks and IP addresses.

Key properties:

- **Multi-tenancy**: full isolation and abstraction per tenant $\Rightarrow$ each tenant has its own virtual networks.
- **Technology-agnostic**: APIs define the **service**, vendor plug-ins provide the **implementation**. Vendor-specific extensions are supported.
- **Loose coupling**: Neutron is a standalone service, not exclusive to OpenStack.

10.10.2. Logical vs. Physical View

Neutron Abstraction

Neutron **decouples** the logical view of the network from the physical view.

Tenants see isolated virtual networks $\Rightarrow$ the physical data-centre network is hidden behind the abstraction.

Logical concepts:

- **Network**: an isolated virtual Layer-2 domain (logical switch).
- **Subnet**: an IPv4 or IPv6 address block assignable to VMs or routers.
- **Port**: a logical switch port that defines the MAC and IP addresses of a virtual interface (VIF).

10.10.3. Neutron Components

- **neutron-server**: accepts API requests and forwards them to the appropriate plugin.
- **Plugins and Agents**: perform **real actions** $\rightsquigarrow$ connecting/disconnecting ports, creating networks and subnets, managing routing rules (iptables, Open vSwitch, etc.).
- **Message queue**: decouples neutron-server from the agents running on hypervisor nodes.
- **neutron database**: persists network state for plugins that require it.

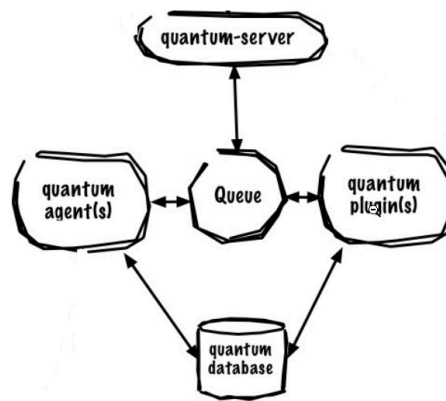


Figure 61 – Neutron (quantum).

10.10.4. Neutron Agents

- **DHCP agent:** provides DHCP services to virtual networks.
- **Plugin agent:** runs on each compute node and performs local vSwitch configuration (OpenVSwitch, Cisco, Brocade, etc.).
- **L3 agent:** provides L3/NAT forwarding for external network access.

👉 **Note:** OpenStack created **OpenFlow** support so VMs can send packets to each other without double IP/MAC translation. This avoids the performance overhead of NAT traversal and enables near-native packet forwarding rates.

10.10.5. Tenant Networks

Tenant networks are isolated, user-created networks. Neutron supports several types:

- **Flat:** all instances on one shared network, no VLAN tagging.
- **Local:** instances confined to a single compute node, fully isolated from external networks.
- **VLAN:** each tenant network maps to an 802.1Q VLAN ID in the physical network. Requires 802.1Q-capable switches and supports up to 4096 tenant networks.
- **VXLAN / GRE:** overlay tunnels that carry tenant traffic over the physical network. Allow many more isolated networks than VLAN (no 4096 limit) and avoid reconfiguring physical switches.

🤖 Why VXLAN over VLAN?

VLAN is limited to 4096 IDs. A large public cloud may have thousands of tenants each needing multiple isolated networks. VXLAN uses a 24-bit segment ID (16 million possible networks) and encapsulates traffic in UDP, allowing tenant isolation to be implemented entirely in software without modifying the physical switching fabric.

A **Neutron router** is required to route traffic between tenant networks or to reach external networks (including the Internet).

💡 Prof. Question: Does data locality make sense for VMs with Neutron?

Yes. Neutron separates the **logical** network from the **physical** one. Putting two VMs on the **same hypervisor** lets them talk locally, with near-unlimited bandwidth and minimal latency, with no physical-network cost. On different hypervisors, traffic crosses the physical network (limited bandwidth, more latency). It is the same idea as **Pod affinity** in K8s. But co-locating everything on one node is a **single point of failure**, so locality must be balanced with resilience.



10.11. Ceilometer and Heat

10.11.1. Ceilometer - Telemetry

Ceilometer

Ceilometer is OpenStack's **telemetry service**. It collects usage statistics from all other OpenStack services for metering, monitoring, and billing.

Ceilometer integrates with every core service (compute, network, storage, image) through the message bus, collecting counters without modifying those services. The data drives both operational monitoring and customer billing.

10.11.2. Heat - Orchestration

Heat

Heat is OpenStack's **orchestration service**. It allows defining an entire cloud application stack: compute, networking, storage, and their inter-dependencies, in a single declarative template.

Heat templates describe **stacks**: collections of OpenStack resources that are created, updated, and deleted as a single unit. Heat integrates with all other services and provides a CloudFormation-compatible API, easing migration from AWS.

10.12. Putting It All Together

The full OpenStack architecture has multiple layers of interaction:

- **Command-line interfaces** (nova, swift, neutron, ...) and **GUI tools** (Dashboard, Cyberbook, iPhone client) speak to the same REST APIs.
- **Cloud management tools** (Rightscale, Enstratus) sit above the APIs.
- Each service cluster (Identity, Compute, Image, Block Storage, Networking, Object Storage) is internally composed of API frontends, schedulers, workers, databases, and message queues, all following the same shared-nothing, async-messaging pattern.
- **Keystone** (Identity API) is the backbone: every inter-service call traverses it for authorization.

OpenStack as a Cloud OS

OpenStack is best understood as an **operating system for the data centre**: just as an OS abstracts hardware resources from applications, OpenStack abstracts physical servers, switches, and disks from cloud tenants: scheduling, isolating, and metering their usage through a unified set of open APIs.



11. SERVERLESS COMPUTING

Package: Serverless Computing - ICCBD Serverless.pdf

Serverless computing represents the next step in the abstraction ladder of cloud delivery models. Rather than managing servers, runtimes, or even containers, the developer delivers only **business logic**: functions, and the platform handles everything else: scheduling, scaling, isolation, and billing.

11.1. Cloud Computing Service Models

11.1.1. Serverless: Definitions

Serverless Computing

«**Serverless computing** is a method of providing backend services on an as-used basis. A company that gets backend services from a serverless vendor is charged based on usage, not a fixed amount of bandwidth or number of servers.» – Cloudflare

«A serverless architecture is a way to build and run applications and services without having to manage infrastructure. Your application still runs on servers, but all the server management is done by AWS. You no longer have to provision, scale, and maintain servers to run your applications, databases, and storage systems.» – Amazon AWS team

«A Serverless solution is one that costs you nothing to run if nobody is using it.» – Paul Johnston, AWS

Serverless Key Concepts

- **Absence of control** on scheduling and scaling logic → the platform decides.
- **No control on resource deployments** → provisioning is invisible to the developer.
- **Zero-scaling** → cost is based on number of activations; idle functions cost nothing.
- **Developer focuses only on business logic** → infrastructure is fully abstracted away.

 **Prof. Question:** *What is serverless? General definition.*

A cloud model where the **provider manages all the infrastructure**, resources are **allocated dynamically** by load, and you **pay per execution** (not for allocated resources). “It costs nothing if nobody uses it” (scale-to-zero). It differs from IaaS/PaaS in **granularity**: you pay per single function execution, not per VM or container.

11.1.2. Serverless as BaaS + FaaS

In the serverless model (sometimes called **BaaS+FaaS**), everything below the application layer is managed by the provider. In **BaaS+FaaS**, only the **Application logic** (the function code) is customer-managed, while the entire stack beneath: Data, Runtime, Middleware, OS, Virtualization, Servers, Storage, Networking, is provider-managed.



11.2. Backend as a Service (BaaS)



BaaS: Backend as a Service

BaaS is the outsourcing of backend infrastructure to the cloud provider through **plug-and-play APIs**. Developers consume ready-to-use services rather than building and operating them.

To support the development of FaaS (and also PaaS/IaaS), cloud providers offer ready-to-use BaaS components:

- Database as a Service (DBaaS)
- Storage as a Service
- CDN as a Service
- Cognitive as a Service (AI/ML APIs)
- Authentication, Alerting, Push Notifications, ...



Why BaaS for serverless?

Functions are **stateless** and **ephemeral**: they cannot hold long-lived connections or manage persistent state. BaaS services (managed databases, object storage, queues) fill this gap, giving functions a reliable backend without forcing the developer to manage servers.

11.3. Function as a Service (FaaS)

11.3.1. Definition and Properties



FaaS: Function as a Service

FaaS is an **event-centric** computing model where user-defined business logic (**functions**) is triggered and dynamically instantiated by incoming events. Each function is a small, **stateless**, independently deployable unit of logic.



FaaS Core Properties

- **Stateless**: functions carry no state between invocations, all persistent state lives in external BaaS services.
- **Event-driven**: execution is always triggered by an **event** (HTTP request, queue message, timer, file upload, ...).
- **Ephemeral execution environments**: the execution container is created on demand and destroyed after the invocation.
- **Fine-grained autoscaling with zero-scaling**: the platform scales from zero to thousands of concurrent instances automatically, and scales back to zero when idle.



Note: FaaS needs:

- **Orchestration**.
- **Choreography**.



Cold Start



When a FaaS platform has scaled a function to **zero instances** (no traffic), the next incoming request must trigger provisioning of a new container from scratch. This initialization phase (pulling the image, starting the runtime, loading the function code) is called a **cold start** and introduces **additional latency** on the first request after a period of inactivity.

Once the container is running and serving requests, subsequent invocations are **warm starts**, with no overhead.

⚠ **Cold Start Mitigation**

- **Use slim/alpine base images:** smaller images start faster (less to pull and unpack).
- **Minimize dependencies:** fewer libraries to initialize at startup.
- **Keep functions warm:** some platforms allow scheduling periodic pings or “warm pools” of pre-initialized containers (at additional cost).
- **HTTP-mode watchdogs:** persistent child processes (as in OpenFaaS HTTP mode) amortize cold start across many requests.

FaaS is a poor fit for **latency-critical applications** where every cold start is unacceptable.

👉 **Note:** Functions can be composed into **pipelines** and **workflows**. Reducing the scope of each function enables **easy parallelism** and **higher throughput**. A pipeline of functions is also called a **workflow**.

11.3.2. FaaS as Finer Decomposition

FaaS pushes decomposition one level beyond microservices:

- A **monolith** is decomposed into **microservices**.
- Each **microservice** is itself a composition of **functions**.

⇒ a microservice in FaaS is just a group of cooperating functions sharing a trigger and a lifecycle.

11.3.3. FaaS Advantages

📌 **FaaS Benefits**

- **Ease of development process:** write only the business logic.
- **Ready-to-use services:** connect to BaaS for storage, auth, messaging.
- **Reduced management costs:** no servers to patch, monitor, or scale.
- **Fine-grained autoscaling** → **pay only for what you use**.
- **Many provider-side optimization possibilities:** the provider can bin-pack, pre-warm, and cache containers transparently.

👉 **Note:** An event is treated as a state (more complex coordination), while an **orchestrator** executes programs. FaaS fits naturally into event-sourcing architectures.

11.3.4. Positioning FaaS in the Cloud Stack

In FaaS the customer manages:

- **Application layer only:** the logic, the interconnection pattern, and the lifecycle configuration.



Two coordination capabilities exist in functional services:

- **Orchestration:** a central entity manages all function executions (e.g., AWS Step Functions).
- **Choreography:** a function triggers a sequence of events, with no central manager.

11.4 FaaS Architecture

11.4.1 FaaS Workflow

FaaS Workflow

The association between a **specific event** and the **execution** of one (or more) specific functions is called a **FaaS Workflow**.

A typical workflow: `HTTP request` → `FaaS platform` → `function executes` → `returns JSON response`.

11.4.2 Essential Components of a FaaS Platform

Every FaaS platform must include at least three components:

FaaS Essential Components

- **Trigger:** bridges external **events** with ones manageable by the FaaS infrastructure.
- **Controller:** **orchestrates** event delivery and function lifecycle.
- **Function Executor:** runs the **function code in an isolated environment**.

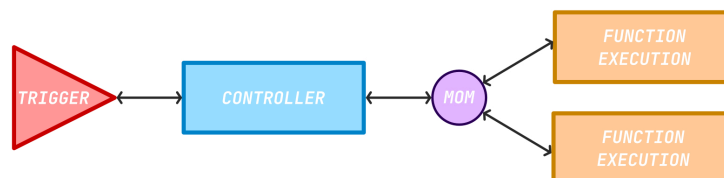


Figure 62 – FaaS components.

100 Prof. Question: *What are the three elements of a serverless/FaaS architecture?*

Trigger / Event Source: the event that starts execution (HTTP, timer, queue message, DB event).

Controller: receives the trigger and decides **where and how** to run the function (location transparency, scaling, lifecycle, async delivery).

Function Executor: runs the function in an **isolated, reproducible** environment, like a container optimized for fast startup.

11.4.3 The Trigger

Trigger Responsibilities

- Bridges external events with ones manageable by the FaaS infrastructure.
- Can convert non-event-based information into events (e.g., polling a database).
- Lifecycle and scaling managed by the infrastructure.
- Handles different protocols and formats: HTTP, TCP, RabbitMQ, Kafka, ...



11.4.4. The Controller



Controller Responsibilities

- Interacts with external controllers to provision resources.
- Handles the delivery of events from trigger to functions.
- Manages lifecycle of functions and other FaaS infrastructural components.
- Receives configurations from customers and activates workflows.



Controller Advanced Features

- **Scalability** of components.
- **Location transparency**: clients need not know where a function runs.
- Delivery of events with different **QoS** levels.
- **Load balancing** across function instances.
- **Asynchronous delivery** via a MOM.

100 Prof. Question: *Does it make sense to add a MOM to a FaaS architecture? What does it enable?*

Yes. A MOM (NATS, Kafka, RabbitMQ) decouples in **time and space** (the producer need not know the consumer's location or be active at the same time). Placed between event source and controller it adds: **spike absorption** (buffer), **delivery semantics** (at-least-once, exactly-once), **automatic retries** on executor failure, and **event persistence** for replay and debugging.

11.4.5. The Function Executor



Function Executor Structure

The Function Executor contains:

- **Invoker** (or watchdog): receives events from the controller, runs the designated function by passing the event, and manages the result.
- **Execution environment**: provides isolation (Container, VM, or WASM), dependency management (OS Libs), and reproducibility.
- **Logic**: the function code itself.

👉 Note: The execution environment must guarantee:

- **Isolation** $\leadsto$ one function cannot interfere with another.
- **Dependencies** $\leadsto$ the correct OS, libraries, and runtime are bundled.
- **Reproducibility** $\leadsto$ the same image runs identically on every invocation.

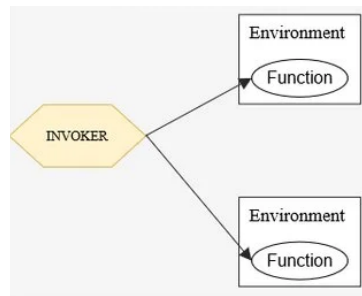


Figure 63 – Function executor.

11.4.6. Platform Architecture (detailed)

A more complete FaaS platform view adds an **API Gateway** and an **Event Queue** in front of a **Dispatcher**:

- **API Gateway**: accepts HTTP requests from clients.
- **Event sources** (object storage, messaging, mail, cloud services): emit events directly.
- **Event Queue**: buffers incoming events.
- **Dispatcher**: routes events to the appropriate **Worker**.
- **Workers**: hold the function code and execute it in isolated slots.

11.5. FaaS Workflows and Function Composition

11.5.1. Workflows of Functions

FaaS Workflow (extended)

A **FaaS workflow** is a series of FaaS functions orchestrated to complete a complex task.

Common use cases:

- *Data pipelines.*
- *API processing.*
- *Event stream handling.*

Benefits of FaaS Workflows

- **Scalability**: automatically scales with workload.
- **Cost-efficiency**: pay-per-use model reduces expenses.
- **Modularity**: functions can be developed, deployed, and updated independently.

11.5.2. Core Workflow Components

Workflow building blocks:

- **Event Sources**: events that trigger workflows (APIs, IoT events, file uploads).
- **Functions**: individual, stateless units of logic.
- **State Management**: handled externally through services like DynamoDB or Redis.
- **Orchestrators**: tools like AWS Step Functions, Apache Airflow, or Tekton.



11.5.3. Composition Patterns

Functions can be composed in three main patterns:



Function Merging Pattern

In the **function merging pattern** two or more functions are merged into one, which also defines the composition logic.

Execution is very **efficient** because everything runs locally in a single container.

The merged function acts as a monolithic coordinator.

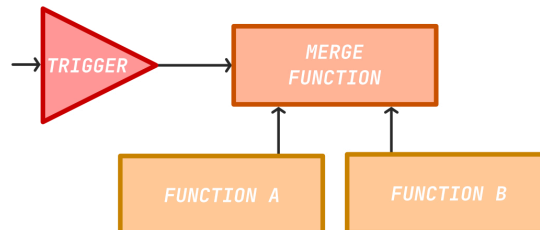


Figure 64 – Function merging pattern.



Reflective Invocation Pattern

In the **reflective invocation pattern** two or more functions are invoked by an additional **coordinating function** provided by the customer. The **coordinating function can hold state**.

Coordinated functions are **independent** and run in separate workflows (Workflow A, Workflow B), each with their own trigger. This is more expensive in resources because every trigger must pass through all network nodes.

Note: Since functions run into separate workflows, if one (such as A) is slower than the other (B), it can be replicated more (more instances).

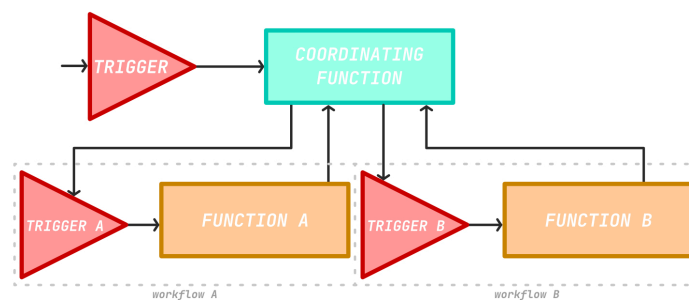


Figure 65 – Function reflective invocation pattern.



Continuous Passing Pattern (Function Chaining)

In the **continuous passing pattern** (function chaining) the **result of the execution of Function A is passed directly to Function B as its trigger** (instead of returning to the caller). The next function is hard-coded, making the chain static: changing the successor requires a new function definition.

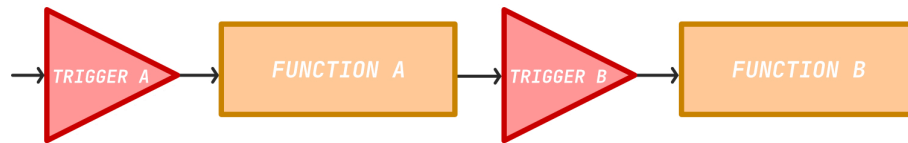


Figure 66 – Function continuous passing pattern.

Other composition examples:

- **Facade pattern:** a single function aggregates multiple downstream APIs.
- **Conditional chaining:** branches based on function output.
- **Map-reduce:** parallel map functions feed a reduce stage.

100 Prof. Question: What is function chaining and function composition?

Function chaining is **sequential**: each function's output is the next one's input, and if one fails the chain stops. **Function composition** is **parallel**: several functions run on different parts of the input and a final function aggregates the results. Chaining = dependent steps, composition = independent branches.

11.5.4. Example: E-Commerce Order Workflow



E-Commerce Order Processing

1. **Trigger:** customer places an order.
2. **Function A:** validate payment.
3. **Function B:** check inventory.
4. **Function C:** process shipment.
5. **Function D:** send order confirmation.

Key benefits: flexibility (replace individual steps), high availability (built-in fault tolerance), no servers to manage.

11.6. FaaS Platforms

11.6.1. Hosted vs. Installable Platforms

FaaS platforms fall into two categories:

- **Hosted platforms:**

AWS Lambda, Azure Functions, Google Cloud Functions, Cloudflare Workers, Vercel, Netlify Functions, IBM Cloud Functions, Twilio Functions, Koyeb, ...

- **Installable/open-source platforms:**

Knative (CNCF Incubating), KEDA (CNCF Incubating), Apache OpenWhisk, OpenFaaS, Fission, Kubeless, Kyma, Nuclio, OpenFAAS, PipelineAI, Virtual Kubelet, riff, ...

Why Open Source?

- Serverless advantages available on **on-premises** nodes, not just public clouds.
- **Total control** over components.
- **Transparency** of the architecture, no vendor black-box.

Open-source platforms (Fission, OpenFaaS, Apache OpenWhisk, Knative) bring FaaS benefits to private and hybrid clouds.



11.7. OpenFaaS

11.7.1. Architecture Overview

OpenFaaS

OpenFaaS is an open-source FaaS platform built on top of Kubernetes and Docker. It provides a gateway, auto-scaling, a monitoring stack, and a messaging layer.

OpenFaaS is organized in three layers:

- **GitOps/IaaS Layer:** OpenFaaS Cloud, GitHub.com, GitLab Self-hosted.
- **Application Layer:** OpenFaaS Gateway, Prometheus (monitoring), NATS Streaming (messaging).
- **Infrastructure Layer:** Kubernetes or Docker, Container Registry.

11.7.2. OpenFaaS Internal Flow

The flow of an invocation:

1. Client (UI/CLI/REST) sends a request to the **OpenFaaS Gateway**.
2. Gateway performs CRUD or Invoke via the **faas-provider** (abstraction over Kubernetes/Docker/containerd/AWS).
3. Provider routes to the appropriate **function container** (`fn`), identified by name and image.
4. Functions are stored in a **Registry**.

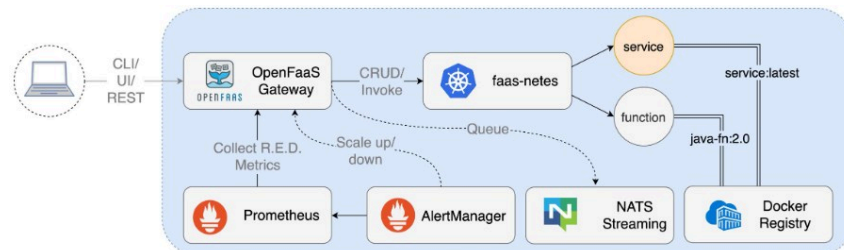


Figure 67 - OpenFaaS internal flow.

11.7.3. OpenFaaS: Gateway

The OpenFaaS Gateway is simultaneously the trigger and controller:

Gateway Capabilities

- **HTTP Trigger:** accepts incoming HTTP requests.
- **Controller:** distributes invocations over available functions and manages resources through Providers.
- **Built-in UI Portal.**
- **Function management API.**
- **Monitoring, Tracing, Logging** exposure.
- **Auto-scaling** based on Prometheus metrics.
- **Self-documented REST API** (Swagger).



11.7.4 OpenFaaS: Prometheus (Monitoring)

Prometheus is the monitoring backbone of OpenFaaS:

- Widely-spread cloud-oriented monitoring service.
- Embeds a multi-dimensional data model with time-series.
- Uses PromQL for querying.
- Each server has its own storage $\Rightarrow$ single server nodes are autonomous.
- Pull-model over HTTP: metrics are collected by Prometheus from targets.
- Targets discovered via service discovery or static configuration.
- Multiple graphing and dashboarding modes.

11.7.5 OpenFaaS: NATS (Messaging)

NATS is the message-oriented middleware used by OpenFaaS for asynchronous delivery:



NATS Properties

- Native Pub/Sub and Request/Reply support .
- Open source, lightweight, high-performance.
- Cloud-native infrastructure.
- Hybrid deployment support.
- Delivery guarantees: **at-least-once**, **at-most-once**, **exactly-once** .

The flow is: Trigger $\rightarrow$ Controller $\rightarrow$ NATS $\rightarrow$ Function(s).



OpenFaaS Asynchronous Invocation

A "Request Statement" event triggers the Gateway. The Gateway enqueues via NATS Streaming. A `queue-worker` function dequeues, calls a `Generate Statement` function that queries Postgres and uploads a PDF to S3. Finally a callback invokes `Email Customer` with the result.

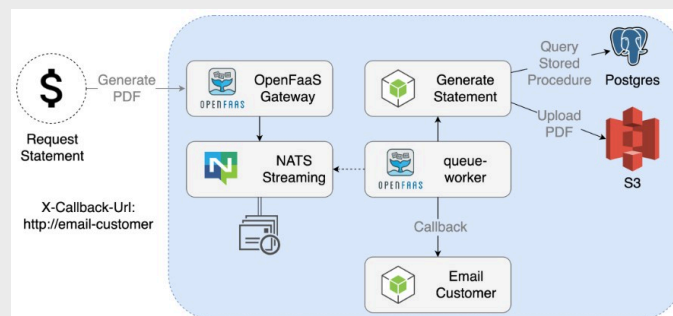


Figure 68 – Example of asynchronous invocation on OpenFaaS.

11.7.6 OpenFaaS: Providers

Providers are an abstraction layer for provisioning Function Executors in different environments (Kubernetes, containerd, AWS, ...). They provide:

- CRUD for functions (or microservices).
- Invocation of functions via a proxy.
- Scaling of functions.
- CRUD for Secrets (optional).



- Log streaming (optional).

Note: Using containers (Kubernetes pods) for functions is **expensive** in terms of overhead. Fine-grained scalability is **sacrificed** in order to gain stronger isolation.

11.7.7. OpenFaaS: Watchdog (Invoker)

Every function executes inside a container that exposes a proxy, the **Watchdog**, used to pass the function code to the forked process.



Watchdog Execution Steps

1. Receive an event from the controller.
2. Create a **child process** to execute the function code.
3. Run the function code in the child.
4. Pass the event as argument to the function.
5. Wait for the end of execution.
6. **Terminate the child.**

Two watchdog modes exist:

- **Forking mode:** for each request, a new Linux process (**heavy**) is forked. The function is executed on a dedicated process, and **once it finishes it is destroyed**. A child process is created to accomplish the function.

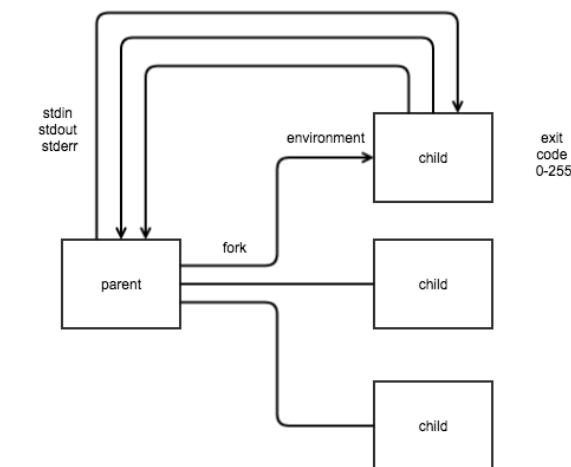


Figure 69 - Watchdog forking.

- **HTTP mode:** a **persistent child process** exposes an HTTP server at **localhost:3000** and the parent watchdog proxies requests to it, passing also the code.

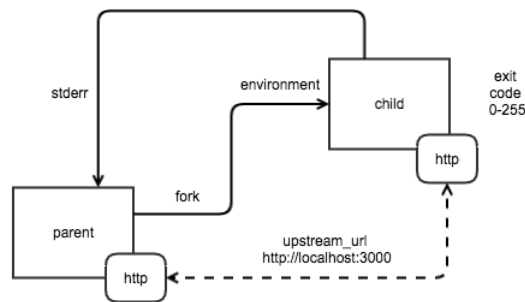


Figure 70 - Watchdog HTTP mode.

- 👉 **Note:** Keeping the child process alive breaks the strict FaaS definition but offers:
- **No-zero scaling** (the container is warm).
 - **Thread concurrency** in the child.
 - **State** can be maintained in the child.
 - **Reduced response latency**.

11.7.8. OpenFaaS: faasd

faasd is a lightweight alternative to the full Kubernetes-based OpenFaaS. It runs OpenFaaS on **containerd** directly, controlled by **systemd**, without Kubernetes overhead. Suitable for edge, IoT, and single-node deployments.

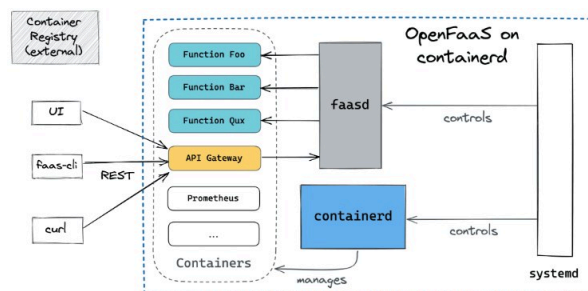


Figure 71 - faasd scheme.

11.8. Apache OpenWhisk

Apache OpenWhisk

Apache OpenWhisk is an open-source, distributed FaaS platform with an action-oriented model. It uses an event-driven architecture built on Kafka and CouchDB.

OpenWhisk Architecture

- **NGINX**: single point of access (API Gateway).
- **Controller**: manages every function call, load balances across nodes, and serves as the authentication server. Can become a bottleneck (not replicated like ETCD).
- **Apache Kafka**: enables event-based and asynchronous communications.
- **Invoker**: executes functions (actions) in Docker containers.



- **Apache CouchDB**: stores functions, parameters, configurations, and results.

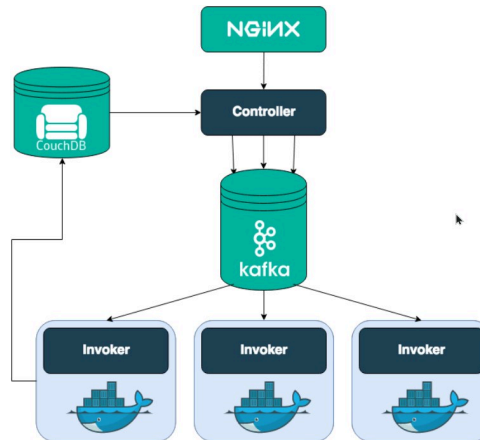


Figure 72 – OpenWhisk architecture.

11.9. Knative

11.9.1. Overview



Knative

Knative is a **platform-agnostic** solution for running **serverless** applications on Kubernetes. It simplifies deployment, scaling, and event-driven architectures.

Knative is not a FaaS: it abstracts Kubernetes resources, exposing them as serverless workloads (microservices, not functions).

100 Prof. Question: What is Knative? How does it map onto Kubernetes? Trade-offs?

Knative runs serverless workloads **on Kubernetes**. It is **not a pure FaaS**: it exposes K8s resources as serverless **Pods**, not ephemeral per-event functions. It uses **Serving** (autoscaler with scale-to-zero, controller via the Operator pattern, queue-proxy sidecar) and **Eventing**. Trade-offs vs pure FaaS: **slower cold start** (Pod startup), but **maximum control** (full K8s, any runtime). Scale-to-zero works, but with latency.

Roles in the Knative ecosystem:

- **Developers**: build and deploy apps via Knative API.
- **Operators**: deploy and manage Knative instances using Kubernetes API.
- **Users/IoT systems**: consume applications deployed by developers.
- **Contributors**: develop and contribute to the OSS project via GitHub.



Knative Benefits

- **Scalability**: automatically scales from zero to millions of requests.
- **Flexibility**: supports any runtime that runs on Kubernetes (Go, Node.js, Python, ...).
- **Cost efficiency**: pay for resources only when the service is running.
- **Integration**: seamlessly integrates with existing Kubernetes tools.



11.9.2 Knative Serving Architecture

Knative has two main subsystems: **Serving** and **Eventing**.



Knative Serving: Key Components

- **Ingress Gateway**: entry point for all external traffic.
- **Activator**: queues incoming requests when a service is scaled-to-zero and communicates with the Autoscaler to bring the service back up. It can act as a request buffer for traffic bursts.
- **Autoscaler**: scales Knative Services based on configurations, metrics, and incoming requests.
- **Queue-Proxy**: a sidecar container in the Knative Service Pod that collects metrics (useful for batching multiple events together), enforces the desired concurrency, and can act as a queue/buffer.
- **Controller**: manages the state of Knative resources within the cluster, implements the Operator Pattern (Observed State → Desired State), can become a bottleneck. Watches several objects, manages lifecycle of dependent resources.
- **Webhooks**: validate and mutate Knative resources.
- **DomainMapping**: maps custom domains to Knative Services.

Note: All Knative Serving components (Activator, Autoscaler, Controller, etc.) are **microservices**, not functions. Knative abstracts Kubernetes, providing a serverless **developer experience** without the FaaS per-invocation billing model.

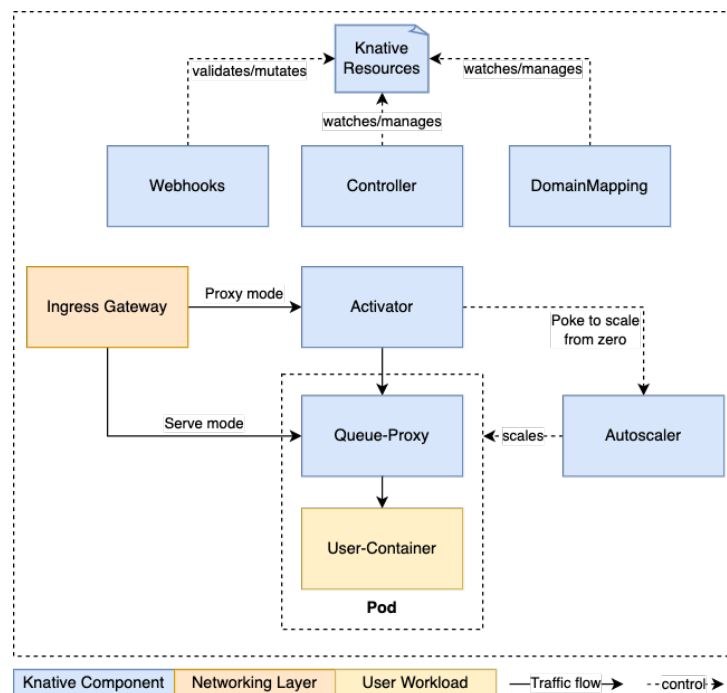


Figure 73 - Knative serving architecture.

11.9.3 Knative Serving: Resource Model

Knative Serving follows a **golden path standardization** using four Kubernetes CRDs:



Knative Serving CRDs



- **Service** (`service.serving.knative.dev`): automatically manages the whole lifecycle of a workload. Controls creation of Route, Configuration, and Revisions. Ensures your app always has a route, a configuration, and a new revision for each update.
- **Route** (`route.serving.knative.dev`): maps a network endpoint to one or more Revisions. Enables specific-path and per-revision traffic splitting.
- **Configuration** (`configuration.serving.knative.dev`): maintains the desired state for the deployment. Separates code from configuration. **Modifying a configuration creates a new Revision.**
- **Revision** (`revision.serving.knative.dev`): a point-in-time snapshot of code and configuration. Immutable: each modification creates a new Revision. Revisions can be retained as long as useful and independently scaled up/down.

11.9.4. Knative Serving: Orchestration via Revisions

Revisions are created indirectly when a Configuration is created or updated.



Revision-Based Deployment Benefits

- **Automated Rollouts**: Route performs rollouts using a single, referenceable Revision.
- **History tracking**: Revisions offer a historical record of changes.
- **Rollback**: revert to a different Revision (previous known good Configuration).

Update scenarios:

- Push image, keep config → update image, retain config.
- Update config, keep image → modify config (e.g., env vars), keep image.
- **Controlled Rollout**: test revisions gradually by adjusting traffic specs.

11.9.5. Knative Request Flow

The request flow through Knative Serving:

1. **Ingress** receives the external request.
2. If the service is **scaled-to-zero**, Ingress goes through **Proxy mode** → **Activator**.
3. Activator queues the request and signals the Autoscaler.
4. **Autoscaler** fetches metrics via **Decider** → recommends a scale target to the Pod Autoscaler (PA).
5. PA scales the **Revision** (Deployment), creating/deleting **User Pods**.
6. Each User Pod contains a **Queue-Proxy** sidecar + the **Application** container.
7. Once pods are ready, traffic switches to **Serve mode** (direct path: Ingress → Queue-Proxy → Application).
8. Queue-Proxy reports metrics back to Autoscaler (scrapes).
9. **ServerlessService (SKS)** keeps track of pod endpoints: **Private Service** (actual pods) informs SKS; SKS updates the **Public Service** (external endpoint).

11.9.6. Knative Serving vs. Eventing



Serving vs. Eventing

- **Serving** (C/S model, *default*): manages HTTP-triggered workloads. Handles revision history, traffic splitting, scale-to-zero. Very **DevOps-oriented**.



- **Eventing** (event-based model): collection of APIs enabling event-driven architectures. Supports event sources (HTTP, CloudEvents), event brokers and channels for distribution, and event-driven function triggers.

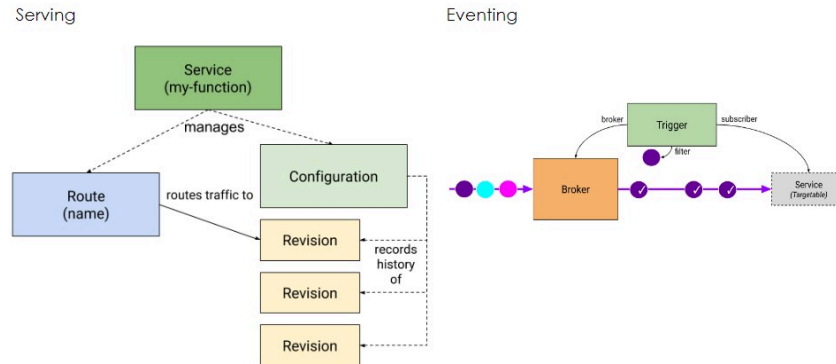


Figure 74 - Knative serving (left) vs. eventing (right) approach.

11.10. Knative Eventing

11.10.1. Motivation



Knative Eventing Goals

- **Loosely Coupled Services**: services can be developed and deployed independently on Kubernetes, VMs, SaaS, FaaS, enabling reusability across languages and tools.
- **Independent Event Production and Consumption**: producers generate events before consumers are ready, consumers express interest in events not yet being produced.
- **Flexible Service Connections**: no need to modify producer or consumer when connecting, select specific subsets of events from a producer.

Key principles:

- **CloudEvents Specification**: utilizes and extends CloudEvents for event data exchange.
- **At-least-once delivery**: prioritizes reliable delivery using HTTP POST (push) transport.

Knative Eventing can drive both Client/Server and event-driven architectures simultaneously.

11.10.2. Eventing Components



Knative Eventing Components

- **Event Sources**: support various sources: Kafka, Google Cloud Pub/Sub, custom events.
- **Flow**: events are ingested and routed to different services.
- **Triggers**: define what happens when an event occurs (e.g., invoke a function). A Trigger sends a request to a Knative serving pod computing the function.
- **Subscriptions**: define how events are consumed by a service.

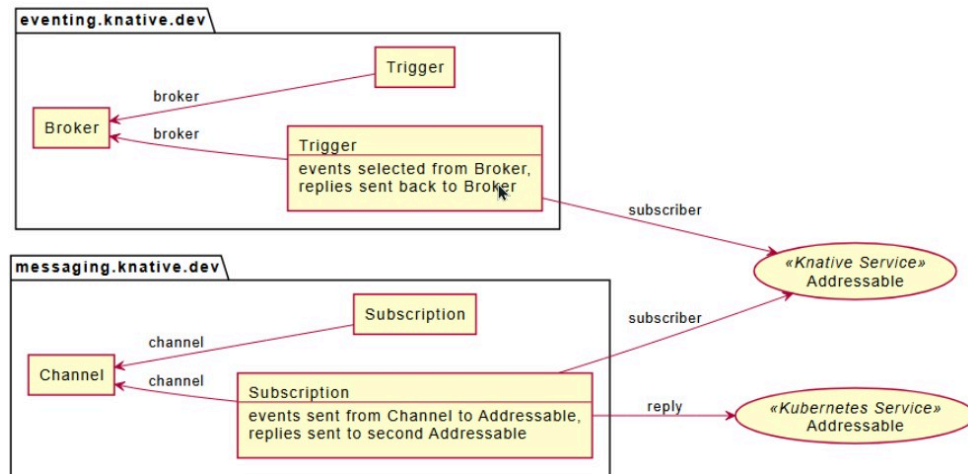


Figure 75 - Knative eventing components.

Two event-processing patterns:

- **Topology-based Event Routing** (`messaging.knative.dev`): events routed based on connections (e.g., Channel to Subscription): resembles event plumbing where flow is like water through pipes; supports routing “reply” events back through the object topology.
- **Content-based Event Routing** (`eventing.knative.dev`): events routed based on event **attributes** (not just connections), via Brokers and Triggers: “picking parts off a conveyor belt” where each event is processed separately. it handles reply events by re-enqueueing them in the originating Broker.

👉 **Note:** Knative Eventing does **not** specify models for multi-stage workflows, correlated request-reply, or sequential event processing, but these can be built using its primitives or by integrating with external systems.

11.10.3. Eventing Interface Contracts

Knative Eventing defines **3 interface contracts** to connect Kubernetes objects as event senders and recipients:

📌 Knative Eventing Interface Contracts

1. **Addressable:** resources exposing a URL to receive events (e.g., Broker, Channel).
2. **Destination:** references the event delivery destination via an Addressable object or URL.
3. **Event Source:** resources that generate events and send them to a Destination.

11.10.4. Eventing: Broker and Trigger

📌 Broker

- Central **event-routing hub** that exposes a URL for event senders.
- Implemented using different event-forwarding mechanisms (NATS, RabbitMQ, Kafka, in-memory interval).
- Supports basic event-delivery options with customizable configurations.



Trigger

- Filters events from a Broker based on **CloudEvents attributes**.
- Routes events to a Destination.
- Supports both cluster-local and external resource event delivery.

11.10.5. Eventing: Messaging Components



Channel

- Abstract interface for **asynchronous fan-out queues**.
- Allows parallel or chained event processing.
- Enables flexible messaging technology replacement across environments.



Subscription (association of function and channel)

- Defines a **destination** for events sent to a Channel.
- Delivers events independently to each subscription.
- Manages undelivered events and retries.
- Uses the Destination interface for delivering events to various destination types.

11.10.6. Knative Eventing: Summary

Knative Eventing provides primitives for building flexible, event-driven workflows by connecting Producers, Consumers, and Event Routers:

- **Trigger**: filter and route events based on CloudEvents attributes.
- **Broker**: central hub for event routing, forwarding to appropriate Triggers.
- **Channel**: provides asynchronous fan-out with parallel or chained processing.
- **Subscription**: delivers events to specific destinations with independent retries.

Advanced capabilities (multi-stage, correlated request-reply, sequential event processing) can be built using these primitives or by integrating with external systems.

11.10.7. Knative Perspectives



Knative Future Directions

- **Serverless platform integrations** with Knative.
- **Portability to hybrid clouds**.
- General-purpose platform for heterogeneous environments and use cases: Stream processing, SaaS events processing, Multimedia, Machine Learning, IoT.



11.11. Choosing the Right Model



Is FaaS the Answer to All Problems?

All Cloud Computing models are evolving to support a wider plethora of customer needs. Tailoring a model to support a use case it was not designed for has consequences. **Choosing the right Cloud Computing model depends on many factors and requires an expert engineer.**

FaaS is excellent for event-driven, bursty, stateless workloads. It is a poor fit for long-running processes, stateful computations, or latency-sensitive applications (due to cold-start times).



12. STREAM PROCESSING

Package: Stream Processing - [streaming.pdf](#)

Modern data-intensive applications must react to data **as it arrives**, not after collecting it all. **Stream processing** is the paradigm of continuously processing data of **unbounded** and **undefined** dimension, with contracts of high throughput and low latency: and it must **automate everything** to achieve special-purpose behavior at scale.



Stream Processing

Stream processing is the continuous processing of data that arrives in an **unbounded**, potentially infinite sequence of records, where results must be produced with **real-time constraints** (typically latencies of a few seconds), without waiting for a finite dataset to accumulate.

12.1. Motivation and Challenges

12.1.1. Why Streaming?

Large amounts of data arrive continuously and demand **real-time views**:

- **Social network trends**: e.g., Twitter real-time search
- **Website analytics**: e.g., Google Analytics dashboards
- **Intrusion detection systems**: e.g., in most data centers

These use cases share a common requirement: **process contents of data with some time constraints**: with latencies of a few seconds and high throughput; and there is **no viable way to use databases for storing** the stream before processing it.

12.1.2. Why Not MapReduce / Batch?



Batch is Not Suitable for Streaming

The standard batch (MapReduce) workflow is **out-of-line**: it requires waiting for the entire computation on a large dataset to complete before producing results. **Batch approaches are not suitable for long-running, unbounded stream-processing**: the stream never ends, so the batch job never starts.

Analogy: The Assembly Line

*Batch processing is like a factory that waits until a full truck of parts arrives before starting the assembly line. Stream processing is like a conveyor belt: parts are processed **one by one as they arrive**, and finished products flow out the other end continuously. There is no "wait for everything."*

Instead, stream processing uses **message-passing dataflows**: data flows between processing components via asynchronous, MOM-like (Message-Oriented Middleware) channels.

12.1.3. Problems with Message-Passing Dataflows

Switching to asynchronous dataflows introduces new problems:



- **Velocity mismatch** between output and input rates → **buffering in a queue** (cost / possible overflow)
- **Node crashes** during processing → **use replication and multiple copies** for fault tolerance
- **Dynamic intervention** while online → **change the graph while provisioning is live**

12.2. The Stream Processing Model

Stream Processing Model

A stream processing system manages three concerns: **allocation** (where operators run), **synchronization** (how operators coordinate on shared state), and **communication** (how data flows between operators). It is organized as a **dataflow graph**: a directed graph of processing operators (kernels) connected by data channels.

The dataflow graph receives inputs from an external source (e.g., sensor feeds, message queues) and passes them through a network of **kernels** (operators) that filter, transform, aggregate, and route records.

Applications that benefit most from the streaming model are those with:

- **High computation resource intensity**
- **Data parallelization** potential
- **Data-time locality** requirements (results must be fresh)

12.2.1. Support Functions

A stream processing framework must provide:

- **Resource allocation**: place operators onto compute nodes
- **Data classification**: route records to the correct operator
- **Information routing in flows**: manage the dataflow topology
- **Management of execution/processing status**: track progress, state, and failures

Apache Storm was one of the early influential systems: scalable, fault-tolerant, processing over a million data tuples per second per node, and respecting SLAs over data to be processed.

12.3. Spark Streaming

Spark Streaming

Spark Streaming is an extension of Apache Spark for large-scale stream processing. It scales to hundreds of nodes, achieves second-scale latencies, and integrates batch and stream processing in a single unified system by reusing Spark's batch engine via **micro-batching**.

It can absorb live data streams from Kafka, Flume, HDFS/S3, Kinesis, Twitter, ZeroMQ, and others; and outputs to HDFS, databases, or dashboards.

12.3.1. Discretized Stream Processing

The core idea behind Spark Streaming is **discretization**: instead of processing each record individually, **run a streaming computation as a series of very small, deterministic batch jobs**.



Discretized Stream Processing

- Chop the live stream into **batches of X seconds**
- Spark treats each batch as an **RDD** and processes it using standard Spark batch operations
- Processed results of the RDD operations are returned in batches
- Batch sizes can be as small as $\frac{1}{2}$ **second**, yielding latency of 1 second
- This enables combining batch and streaming processing in the **same system**

Analogy: Film Frames

A cinema projects 24 still frames per second: your eye perceives smooth motion. Spark Streaming takes a continuous stream and "samples" it into mini-batches at fixed intervals, giving the illusion of real-time processing while reusing the well-understood batch machinery underneath.

100 Prof. Question: What happens if Spark's window is too empty or too large?

The window sets the interval over which data is aggregated. **Too small:** too few data points (statistically weak), high per-batch overhead, and small fluctuations look like spikes. **Too large:** high latency (less real-time), high memory, **back-pressure** if data arrives faster than it is processed, and more data lost on a crash. It is a trade-off between **latency** (small window) and **statistical accuracy** (large window).

12.3.2. DStream: The Core Abstraction



DStream (Discretized Stream)

A **DStream** is a **sequence of RDDs** representing a stream of data. Each RDD in the DStream contains records from a time interval. DStreams are the primary abstraction in Spark Streaming: sources, transformations, and output operations all operate on DStreams.

Sources include: Twitter, HDFS, Kafka, Flume, ZeroMQ, Akka Actor, TCP sockets.

12.3.3. DStream Operations

There are three kinds of operations on DStreams:

- **Transformations:** modify data in one DStream to create another DStream:
 - Standard RDD operations: `map`, `countByValue`, `reduce`, `join`, ...
 - Stateful operations: `window`, `countByValueAndWindow`, ...
- **Output operations:** send data to an external entity:
 - `saveAsHadoopFiles`: saves every batch to HDFS
 - `foreach`: do anything with each batch of results



Get Hashtags from Twitter

```
val tweets = ssc.twitterStream(<username>, <password>)
val hashTags = tweets.flatMap(status => getTags(status))
hashTags.saveAsHadoopFiles("hdfs://...")
```



Each time interval produces a new batch from the Twitter API. The `flatMap` transformation applies to each batch's RDD in turn, creating a new `hashTags` DStream. The output operation saves every batch to HDFS.

12.3.4. Fault Tolerance in Spark Streaming



Spark Streaming Fault Tolerance

RDDs remember the sequence of operations (lineage) that created them from the original fault-tolerant input data.

- Batches of input data are **replicated in memory** across multiple worker nodes
- Data lost due to worker failure can be **recomputed from input data** using the lineage
- Lost partitions are recomputed on other workers automatically

This is exactly the same RDD fault-tolerance mechanism as in batch Spark: micro-batching means the fault recovery model is inherited for free.

12.3.5. Requirements Summary

For a streaming system to be production-grade:

- **Scalable** to large clusters
- **Second-scale** latencies
- **Simple** programming model
- **Integrated** with batch and interactive processing
- **Efficient fault-tolerance in stateful computations**: the hardest requirement

The last point is critical: traditional streaming systems use an **event-driven record-at-a-time** model where each node has mutable state. If a node dies, that state is lost, and making stateful stream processing fault-tolerant is fundamentally challenging.

12.4. Apache Flink



Apache Flink

Apache Flink is an open-source framework for **stateful stream and batch processing**, built for low latency and high throughput. Unlike Spark Streaming, Flink performs **true stream processing**: it processes events one at a time as they arrive, without micro-batching.

12.4.1. Core Features

- **True stream processing**: not micro-batching; each event is processed immediately
- **Stateful computations** with fault tolerance via **exactly-once guarantees**
- **Event-time processing** and **windowing**
- High scalability and performance



🤔 Why True Streaming vs. Micro-batching?

Spark Streaming achieves 1 second latency (bounded by batch interval). Flink targets **sub-second latencies** because it does not wait for a batch window to close: operators push data forward as soon as it arrives. The trade-off: Flink's stateful fault tolerance is more complex to implement, since there is no natural "checkpoint boundary" at batch edges.

12.4.2. Flink Architecture

The Flink runtime consists of two types of processes:

- **JobManager**: the master process (at least one is always present)
- **TaskManagers**: the worker processes (one or more)

12.4.2.1. JobManager

The **JobManager** coordinates distributed execution. It has three sub-components:

- **ResourceManager**: manages resource allocation/deallocation and controls **task slots** (the unit of scheduling). Supports YARN, Kubernetes, and Standalone deployments. In standalone mode, cannot start new TaskManagers; it only distributes existing slots.
- **Dispatcher**: provides a REST interface for job submission, starts a new JobMaster for each job, and hosts the Flink Web UI for monitoring.
- **JobMaster**: manages execution of a single **JobGraph**. One JobMaster per job; multiple jobs can run concurrently in a cluster.

For **high availability**: multiple JobManagers can exist; one is the leader, the rest are standby.

12.4.2.2. TaskManager



TaskManager

A **TaskManager** (also called a worker) executes the tasks of a dataflow and buffers/exchanges data streams between tasks. Each TaskManager exposes a set of **task slots**: fixed subsets of its managed resources (memory).

A TaskManager with three slots, for example, dedicates 1/3 of its managed memory to each slot. Tasks running in different slots share the JVM but are isolated in terms of memory.

12.4.2.3. Task Slots and Chains

Flink **chains** operator subtasks together into **tasks**. Each task is executed by one thread. This chaining reduces:

- Thread-to-thread handover overhead
- Buffering between operators

Chaining increases overall throughput while decreasing latency: operators in the same chain communicate via method calls rather than network buffers.



12.4.3. Pipelining



Pipelining in Flink

Pipelining is the basic building block to “keep the data moving.” Operators push data forward immediately; data is shipped as **buffers**, not tuple-by-tuple. The entire pipeline runs **online and concurrently**, giving low latency and natural handling of **back-pressure** (if a downstream operator is slow, upstream buffers fill and the producer slows down automatically).

Analogy: Factory Production Line

A pipelined factory does not wait for the entire car to be painted before sending it to the next station. Each station works on whatever arrives, and the finished products flow out continuously. Back-pressure is natural: if the painting station is slow, the queue in front of it fills up and the welding station slows down.

12.5. Streaming Semantics and Fault Tolerance

12.5.1. Processing Guarantees

A critical design decision for any streaming system is **what happens to records on failure**:

Guarantee	Meaning	How achieved
At-least-once	All operators see all events (but may see some events more than once).	Storm: replay the stream on failure. Simple but can cause duplicate state updates.
Exactly-once	Operators do not perform duplicate updates to their state: every event is processed exactly once even under failures.	Flink: Distributed Snapshots. Spark: micro-batches on batch runtime (inherits batch atomicity).



Exactly-once is the Gold Standard

Exactly-once semantics is the hardest guarantee to achieve in streaming. It is important especially for stateful computations such as counting, summing, or updating a database: duplicate processing would corrupt the result. Both Flink and Spark Streaming offer exactly-once, but through fundamentally different mechanisms.

12.5.2. Flink’s Distributed Snapshots

Flink achieves exactly-once via a **lightweight, non-blocking snapshot algorithm**: the state of all operators is captured periodically **without pausing execution**, targeting high throughput and low latency.



Distributed Snapshot (Flink)



A **distributed snapshot** in Flink captures a globally consistent state of the entire streaming topology by injecting special **barrier** markers into the data streams. Barriers flow through the topology alongside regular data records, separating the stream into “before snapshot” and “after snapshot” portions.

The mechanism uses four steps:

1. **Master initiates a checkpoint:** the JobManager emits stream barriers from all sources, recording the current source positions (e.g., Kafka offset = 162).
2. **Barriers flow downstream:** when an operator receives a barrier on all inputs, it **writes a snapshot of its state** to persistent storage (state backend) and forwards the barrier.
3. **State is acknowledged:** operators report completion to the Master, which records the checkpoint data (source positions + operator states).
4. **Sink acknowledges:** when the sink receives barriers on all inputs, the checkpoint is complete: the system has a consistent global snapshot.



Properties of Flink Snapshots

- **Non-blocking:** regular data records continue to flow through operators while snapshot is in progress
- **Incremental:** only changed state needs to be written
- **Recovery:** on failure, the entire topology is reset to the last successful snapshot, and sources replay from recorded positions
- **Back-pressure aware:** barriers naturally align with data flow rates



Barrier Alignment

A Kafka consumer is at offset 162. A counter operator has value 152. A barrier is injected:

- Records **before** the barrier → part of this snapshot (counted)
- Records **after** the barrier → belong to the **next** snapshot (backed up until the next checkpoint completes)
- The snapshot stores: `{Kafka offset: 162, counter: 152}`
- On crash and recovery, processing resumes from offset 162, counter reset to 152



12.6. Comparison: Spark Streaming vs. Flink

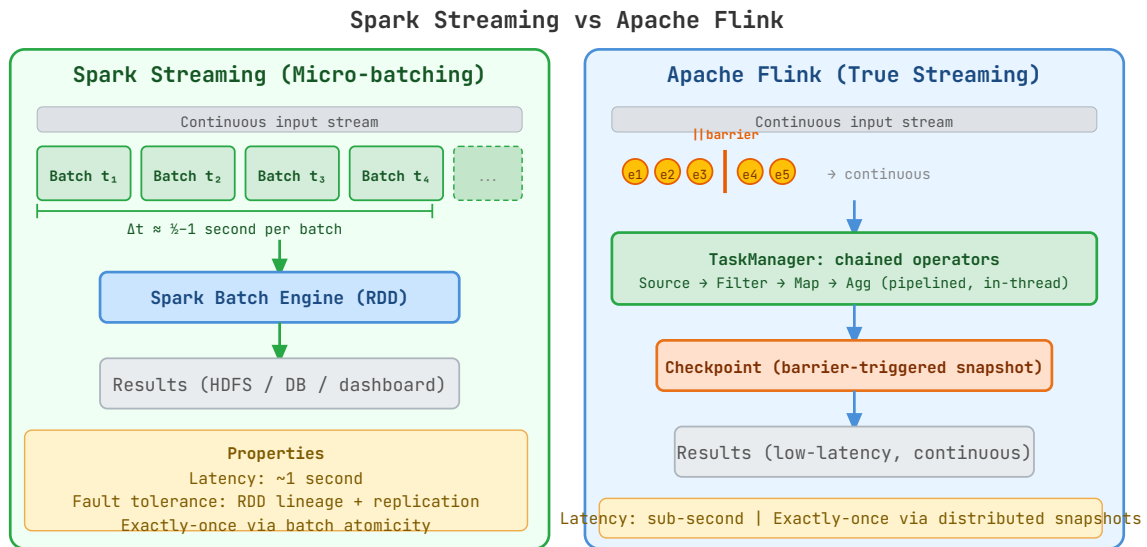


Figure 76 – Spark Streaming (micro-batch: stream chopped into RDD batches every ~1 s) vs Apache Flink (true event-at-a-time pipeline with barrier-based checkpointing for exactly-once semantics).

Aspect	Spark Streaming	Apache Flink
Processing model	Micro-batching (discretized streams)	True event-at-a-time streaming
Latency	1 second (batch interval)	Sub-second (milliseconds)
Fault tolerance	RDD lineage + replication	Distributed snapshots (barriers)
Exactly-once	Yes: inherits from batch RDD model	Yes: via distributed snapshots
Batch integration	Natural: same engine	Supported but separate path
State management	Limited (stateful ops on DStreams)	Rich, first-class operator state

👉 **Note:** The fundamental trade-off: Spark Streaming is **simpler** to reason about (batch semantics) and integrates naturally with the Spark ecosystem. Flink achieves **lower latency** and richer stateful processing but at the cost of more complex internals. For latency requirements of seconds, both work; for sub-second requirements, Flink is the right choice.

100 **Prof. Question:** *Difference between Spark and Flink? Why is Flink more used for streaming?*
 Spark Streaming uses **micro-batching**: records are grouped into batches of X seconds, so the minimum latency is the batch window (seconds). **Flink is true streaming**: it processes each record as it arrives (millisecond latency), with **permanent, always-active** functions waiting for data. Flink uses **distributed snapshots** (Chandy-Lamport) for exactly-once and keeps state explicitly. That is why Flink is preferred for real streaming.

100 **Prof. Question:** *Difference between Flink and a serverless engine? Trade-offs?*
Flink: permanent, always-active functions, no cold start, predictable millisecond latency,



explicit persistent state, but **fixed cost** (resources always allocated). **Serverless/FaaS**: ephemeral functions, **cold starts**, variable latency, **stateless** by default, but **pay-per-use** (can be zero when idle). Flink wins for **continuous high-frequency streams**, FaaS for **intermittent, bursty** workloads.



13. REPLICATION FOR DEPENDABILITY

Package: Replication for Dependability - 7 - Replication for Dependability 26.pdf

Dependable systems must continue to operate correctly even in the presence of faults. This chapter introduces the core concepts of **fault tolerance**, the models behind system replication, and the architectural strategies used in practice to achieve high availability and reliability.

13.1. Core Concepts: Dependability and Faults

13.1.1. Some Definitions

Dependability / Fault Tolerance

Dependability is the umbrella property that makes a system **trustworthy**: users and operators can rely on it to deliver correct service continuously, even under hardware failures, software bugs, and adverse events. It encompasses availability, reliability, and recoverability, all of which must hold at every level: hardware, software, and protocol.

Availability (continuity of service)

Availability is the fraction of time the system is **operational and accessible**. It is formally $A = \text{MTBF} / (\text{MTBF} + \text{MTTR})$, where MTBF is the mean time between failures and MTTR is the mean time to repair. High availability requires both infrequent failures **and** fast recovery. In-memory replication on multiple nodes is a typical technique to reduce MTTR.

Reliability (correctness of results)

Reliability is the ability to deliver **only correct results**, without any erroneous output, over an extended period. Unlike availability, it does not promise when the result arrives, only that what it returns is right. Reliable storage on disk (stable memory) is a classic mechanism for preserving correct state across crashes.

Recoverability (state restoration after failure)

Recoverability is the ability to **restore correct service state** after a fault, by returning to a previously saved consistent state (a checkpoint). A recoverable system does not lose committed work. Safety, consistency, security, and privacy are all aspects that the recovered state must still satisfy.



13.1.2. Faults, Errors, and Failures



Fault, Error, Failure

- **Failure**: any behavior not conforming with the requirements.
- **Error**: any problem that can generate an incorrect behavior or a failure (unsafety).
- **Fault**: set of events in a system that can cause errors.

An application can fail and it can cause a wrong update on a database.

- **Fault** is the *concrete causing occurrence* (several processes entering at the same time).
- **Error** is the *sequence of events* (mutual exclusion has not been enforced) that can generate the visible effect of **Failures** (to be prevented).

⇒ fault tolerance.

13.1.3. Types of Faults

Faults can be **transient**, **intermittent**, or **permanent**:

- **Bohrbug**: a fault that is **deterministic and repeatable**: it appears every time the same conditions occur, so it is relatively easy to reproduce, isolate, and fix. Named after Niels Bohr's solid atomic model (you know exactly where the electron is).
- **Eisenbug**: a fault that **disappears or changes behavior when you try to observe it**: for example, a race condition that only shows up under specific timing or load. Named after Heisenberg's uncertainty principle. Eisenbugs are tied to particular execution contexts and often vanish when debugging instrumentation is added, making them very hard to diagnose.

13.1.4. Services Unavailability

Any system can crash and may become unavailable for some time. **Planned downtime** (maintenance windows, software upgrades) can be scheduled and communicated in advance. **Unplanned downtime** is the real problem: it hits users without warning and is harder to recover from quickly.

Regardless of the cause, restoring normal operation requires two sequential phases: **fault/error identification** (detecting that something went wrong and localizing the cause) followed by **recovery** (returning the system to a correct, operational state).

Common causes of downtime by business impact: hardware drive or server failures (55%), human error (22%), software bugs (18%), and natural disasters (5%). By raw incident frequency, human error and unexpected configuration changes are actually the most common triggers.

13.1.5. Service Unavailability Indicators



Number of 9s

The standard metric for measuring availability. It expresses not only the **frequency of crashes** and the **percentage of uptime**, but also the **capacity of fast recovery**, because the uptime depends not only from fatal failure occurrences but also from the capacity of recovering.

👉 **Note**: The indicators are averaged over one year.



Uptime (%)	Downtime (%)	Downtime
98%	2%	7.3 days
99%	1%	3.65 days
99.8%	0.2%	17h, 30'
99.9%	0.1%	8h, 45'
99.99%	0.01%	52.5'
99.999%	0.001%	5.25'
99.9999%	0.0001%	31.5"

13.1.6. Failure Costs

Downtime costs vary greatly by industry due to the different impact on society or on customers:

Industrial Area	Loss/h
Financial (broker)	\$5.6M
Financial (credit)	\$2.6M
Manufacturing	\$0.8–1.5M
Retail	\$0.4M
Avionic	\$2M
Media	\$

👉 **Note:** A true and precise evaluation is very difficult. Business consequences of outages include: Transaction Loss, Idle Resources, Lost Opportunity, Penalties, Lost Customers, Lost Reputation, Litigation, and Loss of Life.

13.2. Fault Identification and Recovery

13.2.1. Fault Identification in Client/Server Systems

In a client/server interaction, both sides monitor each other :

- The **client** sends a request and waits with a timeout. If no reply arrives in time, it assumes the server has failed or is unreachable.
- The **server** sends its reply and may wait for an acknowledgment (unacknowledged messages are resent up to a configured limit).

The difficulty is that neither side can directly distinguish a crashed server from a very slow one, or a lost message from a delayed one. This ambiguity is why **fault assumptions** matter: by agreeing in advance on what kinds of faults can occur and how many simultaneously, designers can keep detection and recovery protocols tractable.

The number of message retransmissions also sets how many faults can be masked: more retransmissions mean more fault coverage, but also higher latency and overhead in the normal case.

13.2.2. Single Point of Failure (SPoF)



Single Point of Failure (SPoF)



In an architecture, a **single point of failure** is an unique points that must be available at any time.

- With **2 copies** you can identify 1 failure but not correct it.

→ That's because you can understand there is an error/inconsistency/problem comparing the 2 copies, but you cannot tell which is the wrong one.

- With **3 copies** you can identify 1 failure and can correct it.

→ With 3 copies you can individuate which one is the problematic one comparing the 3 (hopefully). In this case 2 should be equal and one problematic/different.

In general terms, with **3t copies** we can tolerate **t faults** for a replicated resource (without any fault assumption).

13.2.3. Single Fault Assumption



Single Fault Assumption

Fault assumptions simplify management and system design. **One fault at a time.**

- The identification and recovery must be less than **TTR** (Time To Repair) and **MTTR** (Mean TTR).
- The interval between two faults (TBF Time Between Failure and **MTBF** Mean TBF).
- During recovery we assume that **no fault occurs**, and the system is safe.

- With **2 copies**, we can **identify one fault** (via some invariant property), and even if fault caused the block, we could continue with the residual correct copy (in a degraded service) with single fault assumption.
- With **3 copies**, we can **tolerate one fault**, and we can **identify two faults**.

13.2.4. Fault Assumptions for Communicating Processors

Model	Description
Fail-Stop	One processor fails by stopping (halt), and all other processors can verify its failure state.
Fail-SAFE (CRASH or HALT assumption)	One processor fails by stopping (halt), and all other processors cannot verify its failure state.
Byzantine Failures	One processor can fail, exhibiting any kind of behavior, with passive and active malicious actions (see Byzantine generals). <i>Limit feasible.</i>

Typically, Fail-Stop and Fail-SAFE (especially) are used, while Byzantine is the hardest case.

13.2.5. Fault Assumptions and Number of Processors

How to implement Fail-Stop and Fail-Safe, and how many processors are needed:

- **Fail-Stop**: You need **3 processors** and **single fault assumption**. One processor fails by stopping and all others can verify its failure state.
- **Fail-SAFE (HALT)**: For HALT, you can adopt **single-fault assumption** and **2 processors**. Both stop when any different result is observed.



13.2.6. Distributed Systems Fault Assumptions

More advanced fault assumptions on communication:

Model	Description
Send & Receive Omission	One processor fails by receiving/sending only some of the messages it should have worked on correctly.
General Omission	One processor fails by receiving/sending only some messages, and it may work on them correctly or by halting .
Network Failure	The whole interconnection network does not always grant correct behavior .
Network Partition	The whole interconnection network does not work by partitioning the systems in two parts that cannot communicate with each other.

👉 **Note:** Replication is a fundamental strategy to build dependable components in all these models.

13.3. High-Level Goals and Formal Properties

13.3.1. Availability and Reliability

Availability can differ between read and write operations: with multiple copies, reads can be served even when only one copy is available, since they do not modify state.

Reliability is expressed as a time-dependent probability over an interval Δt :

- $R(\Delta t)$ = probability of reliable service over Δt .
- $R(0) = A$ (collapses to availability at $t = 0$).

13.3.2. Correctness and Vitality

Two formal properties capture the essential quality requirements of a dependable system:

- **Safety (Correctness):** nothing bad ever happens . All system invariants are always satisfied: the system never returns a wrong answer, never enters a corrupt state. If uncertain, it does nothing rather than risking incorrect output.
- **Liveness (Vitality / Availability):** something good eventually happens . Every valid request eventually receives a response; the system makes progress and never stays blocked forever.

👉 **Note:** These two properties trade off against each other under failures:

- A system with **only safety** (no liveness) will always give correct answers, but may simply stop responding when uncertain. It is correct but potentially frozen.
- A system with **only liveness** (no safety) will always respond, but may return wrong or stale data. It is responsive but potentially incorrect.
- A fully dependable system requires both . To achieve both under faults, **space replication** (multiple simultaneous copies) or **time replication** (retries, checkpoints, replaying from log) is necessary.



13.4 Fault-Tolerance Architectures

13.4.1. Replicated Components

Fault-tolerant architectures use **replicated components**: multiple copies of the same logical resource placed on different machines, so that if one fails, the others can take over. Replication applies at every level (hardware disks with RAIDs, processors, services, and data stores).

The key design question is *how active each replica is*:

1. **Passive replication (master-slave)**: only one copy, the master, actually executes requests and produces results. The other copies are backups that receive periodic state updates but do no work. Simple to reason about, but the master is a bottleneck and its failure requires explicit failover.
2. **Active replication**: all copies execute the same request independently and then coordinate to agree on a common answer. Faults are masked without failover delay, but the coordination overhead is higher.
3. **Load-sharing clusters**: all copies are equal in role but execute different requests simultaneously, splitting the workload. This maximizes throughput without requiring coordination among requests, but provides no single-fault masking on a per-request basis.

👉 **Note**: These architectures introduce a **metalevel**: parts that control the rest of the system (managing replication, detecting failures, coordinating state). The **metalevel** itself must be dependable, or the whole scheme collapses.

13.4.2. Stable Memory

Stable Memory

Uses replication strategies (**persistence on disk**) to guarantee not losing any information. It is based on the **limiting fault assumption**: a low and negligible probability of multiple faults over related memory components (single fault over connected blocks of memory).

Any error is converted into an **omission** (a control code is associated to the block and the block is considered correct or faulty). Blocks are organized in **two different copies** over different disks, with a really low probability of simultaneous faults (conjunct faults): the **two copies contain the same information**. **Replication degree is two**.

👉 **Note**: High cost of implementation, especially in terms of timing (how to limit the recovery time?)

13.4.3. Stable Memory: Support Protocols

Any operation (either read or write) operates on both copies, if one is incorrect, a recovery protocol starts:

- Any **action from a correct block**: proceeds starting from one copy and then to the other.
- Any **action from an incorrect block**: considered an omission fault and starts a recovery protocol.

The recovery protocol has the goal of recovering both copies to a safe state, even by working for a long time:



- If **both copies are equal and consistent**: no action.
- If **only one copy is correct**: the protocol copies the correct value over the wrong copy.
- If **both copies are correct but inconsistent**: the consistency is established (one content is enforced).

If copies have a **time/version indicator**, this is used to choose the correct copy.

13.4.4 TANDEM Systems



TANDEM

TANDEM (later acquired by Compaq then HP, marketed as “NonStop”) is a **special-purpose fault-tolerant system** designed for continuous online operation. It keeps data in memory (not solely on disk) and replicates every hardware component: two processors, two buses, two disks, and so on. All copies operate in **perfectly synchronous lockstep**.

The goal is a **fail-safe** system that tolerates any single hardware fault:

- Every component is mirrored, so if one fails, its twin continues without interruption.
- Stable memory is implemented by writing to both buses, which each write to a mirrored pair of disks.

👉 **Note**: The cost is very high: double hardware, strict synchrony overhead. This makes TANDEM a special-purpose solution for mission-critical environments like banking and financial transaction processing.

13.4.5 RAID: Redundant Array of Inexpensive Disks



RAID

RAID is a general-purpose technique that coordinates a set of commodity disks to improve either **performance** (via striping) or **fault tolerance** (via redundancy), or both, at a much lower cost than special-purpose hardware like TANDEM.

The original goal was to improve **throughput** via **data striping**: split a file across multiple disks so they can be read or written in parallel. Later RAID levels added **parity** or **mirroring** to recover from disk failures. Each level trades off cost, capacity, and protection differently:

- **RAID 0 - striping only**: data split across N disks for maximum parallel I/O throughput. **No redundancy**: any single disk failure loses all data. Suitable for performance-critical scratch storage where loss is acceptable.
- **RAID 1 - mirroring**: every disk has an exact copy. Maximum redundancy: survives failure of any one disk. Reads can be load-balanced across both, while writes must hit both. Cost: 50% capacity overhead.
- **RAID 3 & 4 - striping with a dedicated parity disk**: data striped byte-by-byte (RAID 3) or block-by-block (RAID 4), with a single dedicated parity disk. The **parity disk becomes a bottleneck**: only one write I/O at a time can proceed.
- **RAID 5 & 6 - distributed parity**: parity blocks are spread across all disks, eliminating the parity-disk bottleneck. RAID 5 tolerates one disk failure; RAID 6 adds a second parity block to tolerate two simultaneous failures. Best balance of performance, capacity, and fault tolerance for general use.

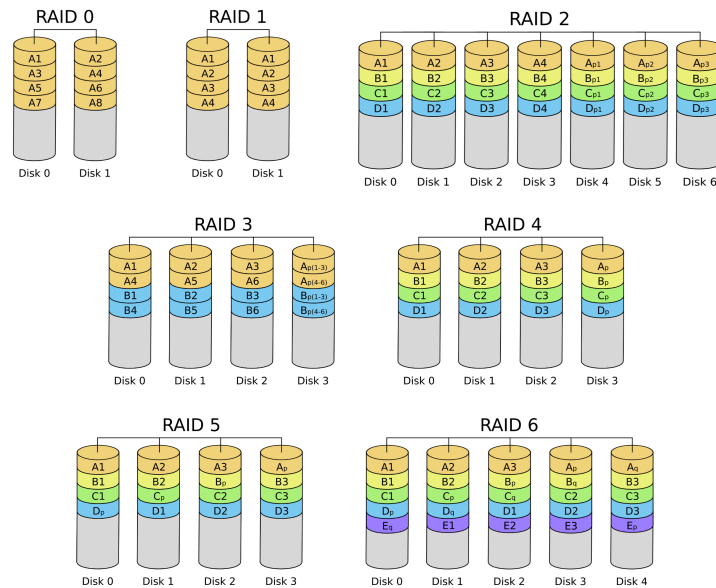


Figure 77 - RAIDs comparison.

13.5. Fault Tolerant Support: Costs and Principles

13.5.1. Fault Tolerant Support Overview

Fault tolerance requires support, resources, protocols. Protocols are expensive in terms of required resources:

- Complexity and length of the algorithms.
- Implementation of the algorithm (and their correctness).

There is no **unique strategy** for always accepted solutions: dependability is a non-functional property with many facets. In general terms, the recovery protocol must be more reliable than the application itself.

- **Special-purpose systems:** achieve dependability via an added ad-hoc architecture completely separated from the application one. Costs are high and the design is complex (formal proofs?).
- **General-purpose systems:** user resources are the only one available. The fault tolerance support **must economize on its design** so not to get too much from the resources for the application levels.

13.5.2. High Replication Costs

Dependability costs are generally high in two senses and dimensions:

- **Space:** in terms of required **resource** available (multiple copies).
- **Time:** in terms of **time, answer and service timing**.

Often fault assumptions can make the system more or less complex and viable the cost of the solutions.

Cost may depend on many different factors: memory and persistency costs, communication overhead, implementation complexity, what to replicate, how many copies, where to keep them, how to coordinate, etc.



👉 **Note:** The general trend is in the sense of **optimizing protocols, supports, infrastructures**.

13.6. Resource Management and Replication Architectures

13.6.1. Resource Management with Replicated Resources

In distributed systems, we can consider **replicated resources** with an obvious need of coordination toward a common goal (also software fault-tolerance):

- **Replicated resources:** multiple resource copies on different nodes with **several replication degrees**.
- **Partitioned resources:** multiple resource copies on different nodes (without any replication degree) to work **independently**.

Redundancy can suggest architectures to get a better QoS: **replication of processes and data**.

13.6.2. Abstract Unique Resource Model

The **replication degree** is the number of copies ([# copies]) of the entity to replicate. More copies mean more redundancy, so better reliability and availability, but also **more cost and overhead**.

The two extreme architectures are **master-slave** (one copy executes) and **active** (all copies execute as peers), with variations in between.

13.6.3. Replication Architectures: Passive Model (Master-Slave)

Passive Model (Master-Slave)

Only one copy executes, the others are back-ups. This is the first replication model, widely used in industrial plants. The **master** is externally visible and manages the whole resource. The slaves watch the master for errors and faults.

One **active process only** (the master) works on the data. The other copies become operational only when the master **fails**. **Only one copy is fresh**, the others may be stale (**cold** or **hot** copies), so on a failure with cold copies recovery must replay from the last saved state to rebuild the current one.

Structure: MASTER → CHECKPOINTING → CONTROL (slaves observe).

13.6.4. Active Replication Model

Active Model

All copies execute all operations in a more or less synchronous way and with some forms of coordination among copies.



In **TMR** (Triple Modular Redundancy) **three copies** are used: we can tolerate on faults and can identify up to two faults. In software FT, different copies can use **different algorithms** toward the goal.

Replication Models

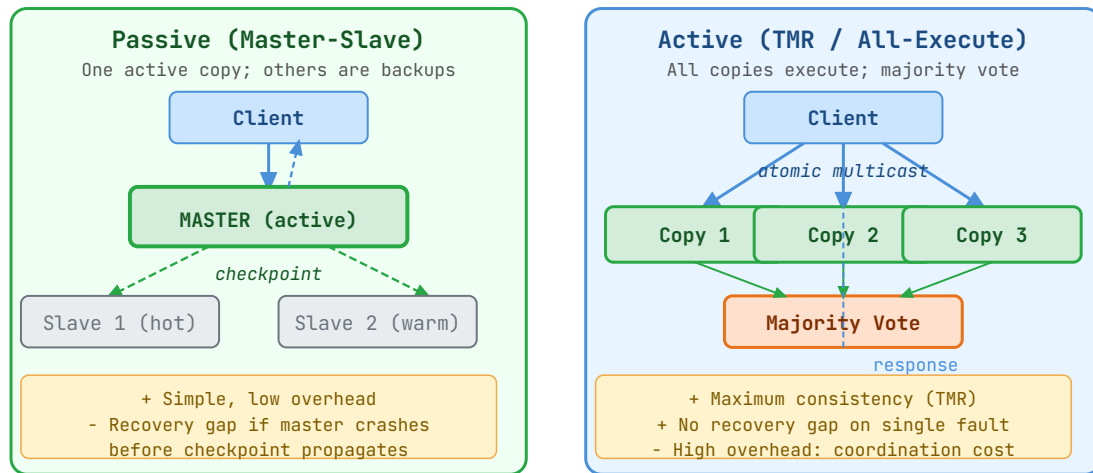


Figure 78 – Passive (master-slave) vs. active (TMR) replication: trade-off between simplicity and fault masking strength.

13.6.5. Master-Slave: Architecture

Master and Slaves are an internal architecture.

Fault Recovery: who identifies the fault and when?

Secondary copies (slaves) must identify the fault of the master **by observing its activity**: by using application messages coming from the master and by keeping the timing into account. Even ad-hoc management messages can be used and exchanged.

The organization can use:

- One slave for the control protocol (if single fault).
- A hierarchy of slaves and more complex protocols (for multiple faults).

The entire **resource**, from an external perspective, can tolerate a different number of errors depending on internal strategies and can still provide correct services in case of errors (**fault transparency**).

13.6.6. Checkpoint: Slave Checkpoint

In general, the master updates the slave states via **checkpointing** → the updating action also made **in a chain**: the master updates the first slave that updates the second, ...

The management policies can distinguish:

- The required actions to grant a correct response: **update of the primary copy** (first slave).
- From successive actions (less crucial): **update of the secondary copies** (other slaves).

Those strategies can achieve different policies and different state updating costs and quality:

- The client gets the answer with **less delay** if the master answers before the state has been updated in all copies (but only a part) or even in **no slave copy** at all (*prompt but not safe*).
- In the other case, the **delay is more**, but we grant more **consistency on the internal resource state** (*safe less prompt*).



13.6.7. Master-Slave: Checkpoint Timing

The **update of the state** and its establishment over the slaves:

- **Periodic action** (time-driven).
- **Event action** (event-driven).

In case of a **sequential resource**, the state is clearer and easier to identify and establish. In case of a **parallel resource**, all the parallel actions should be taken into account and considered toward the state saving. The state subjected to more **concurrent actions is less easy to isolate** and the **state is harder to identify** and distinguish.

👉 **Note:** The checkpoint of a resource with several operations going on at the same time is more complex to deal with and to complete correctly, because of the sharing of data between concurrent activities.

Checkpoint at **entrance/exit** and in **specific decision points**.

13.7. Active Replication: Models and Coordination

13.7.1. Active Replication Model Details

An activity executes the operation for any private data copy. Client external requests to the server can have an either **explicit** or **implicit** approach related to replication:

- If the **client has an explicit vision of FT** → **no abstraction**. This organization lacks abstraction because all clients have too much visibility of internal FT details of servers.
- If the **client has an implicit FT** → **FT transparency**. Need of a support capable of getting the request and distributing copies to server copies and vice versa for results.

13.7.2. Active Copies Replication: Manager Strategy

Fault tolerance is usually an **invisible internal detail** of the replicated resource: the client sees a single logical service and is unaware of how many copies exist. To coordinate requests across copies, a **manager** role is needed:

- **Single manager (static):** one designated copy acts as manager for all operations. It receives every request, distributes it to other copies, collects their results, and returns the agreed answer to the client. Simple to implement and easy to reason about, but the manager is both a performance bottleneck and a potential single point of failure.
- **Rotating managers (dynamic):** each operation gets a different manager, chosen by **locality** (the copy closest to the client) or by **rotation** (round-robin across copies). This distributes the coordination load evenly and removes the single point of failure, but requires careful handling when multiple operations are in flight simultaneously to avoid conflicting decisions by different concurrent managers.

👉 **Note:** When several operations are handled by different managers at the same time, the managers must avoid interfering with each other: for example, by acquiring per-object locks or using atomic multicast to impose a consistent execution order.



13.7.3. Active Copies Coordination

Active models can decide different coordination models:

- **Perfect synchrony** (full consistency or strict consistency): all copies should agree and produce a completely synchronized view, with the same internal copy scheduling for all copies (difficult for nested actions or external actions).
- **Different approaches to synchrony (less consistency)**: even if some minimal threshold can be considered, actions can complete before all copies agree on the final outcome, and the final agreement can take place later (also it does not apply even eventually).

👉 **Note:** Less synchronous strategies cost less in time, mainly client service time, and makes protocols easier and more viable but grant less in operation ordering and release some *semantic properties*. Some modern Cloud systems decide of abandoning **perfect synchrony** in favor of an **eventual synchrony**.

13.7.4. Copies Coordination: Read/Write Actions

Different actions on active copies have different requirements and management:

- **Read actions**: typically actions that can occur easily in parallel and accessing a limited number of copies.
- **Write actions**: those intrinsically require **coordination among copies**.

Any action that can change the state implies more coordination to propagate such a change:

- In case of a **clean state partitioning**, where any change applies to different partitions, those actions can proceed independently in parallel without any coordination.
- Eventually, some actions can require a **copy reconciliation** of actions that could have been interfering.

There are also actions with very specific intrinsic semantics. For instance, the **actions on a directory** can proceed with some more parallelism: add/delete of a file, read/write of a file, directory listing. Even semantic properties can distinguish operations and can make possible more efficient behavior and greater parallelism.

13.7.5. Active Copies Updating

When dealing with active copies updating, any action that requires to update the state of any copy: The **update action must occur before delivering the answer** to grant a complete consistency but that impacts on response time (more delay in case of failures) (**eager policies vs. lazy**).

If the component employs **different managers for any operation**, it is a manager duty to command the internal actions. If the component defines **parallel operations**, all managers must negotiate and conciliate their decisions, causing some conflict to be solved and some actions in incorrect order to be **undone or redone**.

👉 **Note:** In case of failure during one operation and before its correct completion, there should exist the feature of giving an answer anyway, because of the excess of accumulated delay in finishing internal agreement protocols.

13.7.6. Active Copy Agreement

Copies can reach an agreement before giving the answer:

- All copies should agree on the specific action (**full agreement**).



- **Majority voting** (not all copies must agree) **with a quorum** (also weighted): correct copies can go on freely; other copies must agree on it and then reinserted in the group (recovery).

Failure detection: *who is in charge? When?* **Reinsertion detection:** *who is in charge? When?*

There is a **strict need of monitoring** and execution control.

Group semantics: in a group, depending on agreed semantics of actions, there may be also less expensive and less coordinated actions on execution orders. **The less the coordination, the less is the cost.**

13.8. The Five Phases of Replication Operation

13.8.1. Overview

To make clear the needs of different steps and one general workflow, we can model the group operation as a sequence of **five phases**:

1. Request Arrival
2. Copy Coordination
3. Execution
4. Copy Agreement
5. Response Delivery

13.8.2. Phase 1: Client Request

The client sends its request to the replicated resource in one of two ways: to **one copy only** (a designated manager that then forwards it to the other replicas), or to **all copies at once**. Sending to one copy is simpler but loads that copy with the coordination. Sending to all is more robust to a single-copy failure, but the client must know the full replica set. The manager can be fixed at deployment time or elected per operation.

13.8.3. Phase 2: Copy Coordination

Before executing, copies must agree on **when and in what order to run** the operation. This matters when several operations are in flight at once: without coordination, different replicas could apply them in different orders and diverge. **One copy acts as manager and proposes an execution schedule.** Copies may carry different weights (as in weighted quorum) and play different roles. This is the **first coordination phase**.

13.8.4. Phase 3: Copy Execution

Now the copies run the operation. In an **active model** all replicas execute independently and produce their own result. In a **passive model** only the master executes while backups wait. A copy may have some local freedom, for example skipping execution if it already received the committed result from the manager. Conflicts from concurrent overlapping operations are resolved here or corrected in the next phase.

13.8.5. Phase 4: Copy Agreement

After executing, all copies must agree on the final result before it is delivered to the client. Some copies may have produced different outputs due to faults or to concurrent operations from other



managers. The group votes: if a quorum agrees on the same result, that becomes the committed answer. Divergent copies are dropped from the group and must be recovered and re-synchronized before they rejoin. If no agreement is reached (too many copies failed), the operation is rolled back. This is the second coordination phase.

13.8.6. Phase 5: Result Delivery

Finally, the agreed result is returned to the client. The preferred form is a **single unified answer**, so the client never needs to know replication exists. If the client sent its request to all copies in Phase 1, it may receive several identical responses and should accept the first valid one and discard the rest. Keeping the client side simple is a key design goal.

13.8.7. Observations on Active Copies Operations

The coordination overhead means the replication degree must be kept low and replication policies are to be kept simple.

👉 **Note:** Eager consistency (strong guarantees) vs. Lazy availability (fast, optimistic). The trade-off is central to replication system design.

13.8.8. Active Copy Operations Classification

To classify some FT resources replication, we can use two significant directions:

- **Who decides the updating:** only the primary copy or all copies.
- **When to propagate and take the updates:** eager (immediate and before the answer) pessimistic, or lazy (delayed after the propagation) optimistic.

(We can reverse the terms for the client perspective.)

For the updating we can distinguish:

- Eager primary copy vs. Lazy primary copy.
- Eager updating for all copies vs. Lazy updating for all copies.

13.8.9. Eager Primary Copy

Sticking to one primary, that copy executes and gives back the answer only after having updated the state of all copies in a pessimistic approach (one operation at a time with faults).

In that case, the manager is in charge of the whole coordination, but the client receives a deferred answer (for correctness sake). If more operations are active over the replicated object that does not change, they can proceed in parallel.

Flow: Phase 1 (Client Request) → Phase 2 (Server Coordination) → Phase 3 (Execution + Update) → Phase 4 (Two Phase Commit Agreement) → Phase 5 (Client Response).

13.8.10. Lazy Primary Copy

On the opposite, the manager can first answer to the client and afterwards it updates the copies with an **optimistic approach** (also several operations can go on at the same time).

In this case, the manager must also be able to control the possible reconciliation of the state of the copies ... and some problems may occur if there is a manager crash.

Flow: Phase 1 → Phase 3 (Execution + Update primary) → Phase 4 (Client Response) → then Reconciliation with other copies.



13.8.11. Update of Active Copies

Eager policies favor consistency and correctness of the operations, instead of the promptness of the answer to the client:

- The goal is **not very fast precocious answers**, because that can lead to undo actions, that are not easy to be done, and, in some cases, impossible to backtrack.

Copy coordination is **two phases toward consistency granting** (specially in case of concurrent actions):

- Those two phases are **not always needed**, but they can obtain the necessary coordination among copies and operations.
- **A-posteriori coordination to verify consistency**: if it is not verified, some **undo** must be considered (two phase protocol and roll back).
- **A-priori coordination** can ensure that all correct copies receive all correct messages and the right schedule is automatically enforced (e.g., **atomic multicast**).

13.8.12. Optimistic Eager Update

All copies are updated with some enforcing policies in an **optimistic approach** (**two-phase commit**), only afterwards the answer is provided to the client.

After copy independent executions, the final coordination ensures an agreement, otherwise some backtracking is commanded (**possible undo**).

Flow: Phase 1 (Client) → Phase 2 (Server Coordination) → Phase 3 (Execution + Update all copies) → Phase 4 (Two Phase Commit Agreement) → Phase 5 (Client Response).

13.8.13. Pessimistic Eager Update

A different approach for eager update implies coordination but tends to save the final phase. The agreement of results is granted via a delivery protocol **in a pessimistic approach**.

An **atomic multicast** can ensure that any message is correctly sent to all copies in the same order, so that there is no need for a final check (**no undo**).

Flow: Phase 1 (Client) → Phase 2 (Atomic Broadcast in Server Coordination) → Phase 3 (Execution + Update all copies, skipping Phase 4) → Phase 5 (Client Response).

13.9. Replication Forms and Widespread Models

13.9.1. Replication Forms Summary

Category	Model	Variants
Hardware Replication	-	Disks, Processors, Batteries, Energy, ...
Software Replication	Passive Model	Hot Copies, Warm Copies, Cold Copies
Software Replication	Active Model	Coordination required

- **Hot copies**: continuous updating.
- **Cold copies**: no update actions.
- **Warm copies**: some update actions, but not continuous.



13.9.2. Widespread Replication Models

In any model, the cost is influenced by the **group replication degree**: the number of copies, either working or not. In practice, the **replication degree is typically very limited** (no more than a few copies).

There are also **intermediate replication models**, non-FT oriented, with a set of resources able to work independently on the same kind of operations: they operate on **different services at the same time**, and they can share the **responsibility of being a back-up of each other** (throughput driven and load balancing).

13.10. Industrial Operations and Evolution

13.10.1. Industrial Safety Evolution

Modern industrial fault-tolerant systems are classified by how they **respond to failures**, progressing from simple detection toward autonomous prediction and prevention:

- **Fail Safe**: on fault detection, the system transitions to a known safe, inert state. The priority is preventing harm, even at the cost of stopping all operation. Classic example: a railway signal defaulting to red when the control system loses power.
- **Fail Silent**: instead of entering a safe state, the system simply **stops producing output** on fault. Other components detect the silence and take over. More flexible than fail-safe because no pre-defined safe state is required.
- **Fail Operational**: the system continues functioning, possibly in a degraded mode, by automatically reconfiguring around the failed component. This is required in autonomous vehicles: a self-driving car cannot simply stop in the middle of a motorway when a sensor fails.
- **High Dependability**: the system uses continuous monitoring, telemetry, and analytics to **predict** impending failures and act proactively before the fault occurs, avoiding any disruption.

👉 **Note**: The progression reflects the increasing autonomy of safety systems: from “stop on failure” (fail safe) to “predict and prevent” (high dependability). Automotive functional safety standards (ISO 26262) and industrial standards (IEC 61508) formalize these levels.

13.11. Case Study: ALMA Web Service Architecture

13.11.1. Small-Scale Example

ALMA ICT has a small problem in answering many requests in a short time for a specific Web Service. A better solution than a **simple server** has to be devised to grant **limited answer times** with **no errors** and some **fault tolerance to single fault occurrence**.

Users are interested in getting Web server answers after invoking a Web service that interacts asking to a backend database. Requests arrive both from final portal users and also from external programs and other internal UNIBO applications. **Correct answers are very crucial**.

13.11.2. ALMA Web Service Architecture

The devised minimal cost solution:



- **Load balancing via a hardware balancer** as a front end of the two main servers.
- **INDEX01 and INDEX02**: two Web Servers in a cluster, two Tomcat instances managed by an Apache proxy.
- **Reliability** granted by a module of **High Availability Linux** master-slave with a heartbeat.

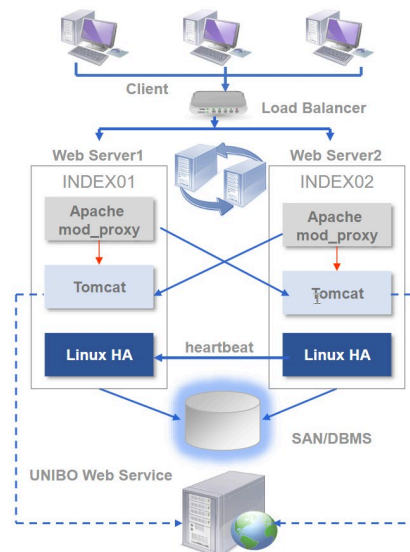


Figure 79 - ALMA architecture.

13.12. High Availability Clusters

13.12.1. High Availability Cluster

High Availability Cluster

A cluster for **high availability** consists of a set of **independent nodes** that cooperate to provide a **dependable service**, always on 24/7.

The **clusters** are a good off-the-shelf solution for high availability:

- **Robust and reliable**.
- **Cost-effective** (easy to buy off-the-shelf hardware and support).
- **Typically one Front-end**.

Clusters have different motivations: high availability, high performance, load balancing.

13.12.2. Cluster Support Operations

The cluster support must provide:

- **Service monitoring**: to dynamically ascertain the current QoS (*final and perceived*).
- **Failover (service migration)**: the failover is a hot migration of a service immediately after the crash, whichever the cause. The failover must take place very fast to limit service unavailability. *Typically, should be automatic, fast, and transparent (the sooner the better)*.
- **Heartbeat (node state monitoring)**: the heartbeat is the protocol to check node state to monitor and ascertain any copy failure. **Exchange of are-you-alive messages with low intrusion**. Some clusters can also work in case of partitioning and allow to go on and support reconciliation when reconnected.



13.12.3. Cluster: Failover and Heartbeat

In case of failover, the data must be available to the new node of the cluster via a **shared component** over the cluster. The detection of problem is via a **lightweight heartbeat protocol**: messages exchanged over both IP and non-IP networks for redundancy.

13.12.4. Storage Area Network (SAN)



SAN: Storage Area Network

A set of interconnected resources with several QoS to grant the storage service with the best suitability for different users. Users can employ SAN to get the storage resource they need without any interference and ideally without any capacity limit and with minimal delay.

In Cloud, the SAN can offer **Storage as-a-Service**.

13.12.5. Red Hat Cluster

Red Hat Cluster suite (open source): a typical two-node cluster with shared disk storage. Its main components are:

- **Cluster Infrastructure**: CMAN (Cluster Manager, handles node membership and quorum), DLM (Distributed Lock Manager, coordinates shared access to resources), Fencing (isolates a failed node by cutting off its access to shared storage, preventing data corruption), CCS (Cluster Configuration System).
- **HA Service Management**: rgmanager, the resource group manager that monitors services and triggers failover when a node fails.
- **Shared Storage**: GFS (Global Filesystem, a shared-disk filesystem all nodes can mount simultaneously), CLVM (Clustered LVM, manages logical volumes across the cluster).
- **GNBD** (Global Network Block Device): exports a local disk over the network so other nodes can use it as a block device, enabling shared storage without dedicated SAN hardware.

Red Hat Cluster suite evolved a lot and is off-the-shelf. Red Hat Cluster can coexist with most widespread architectures (e.g., OpenStack: Nova, Glance, Swift, Quantum, Cinder, Keystone).

13.13. Optimistic Lazy Policies and Eventual Consistency

13.13.1. Optimistic Lazy Policies

We use **lazy update** when one copy can answer with a little (no) coordination with other copies in an **optimistic policy that can deliver the answer very fast**: as in the case of **Amazon S3** (Amazon Simple Storage Service).

Amazon memory and persistence support **renounces to any strict consistency** and provides both **consistent** and **eventually consistent** operations.



13.13.2. Eventual Consistency

Eventual Consistency

Strong consistency has the eager update but slow answer. **Eventual consistency** (called final or tending to infinity) is a lazy update in the direction of **released consistency**: updates are commanded but not waited for.

So concurrent operations over other copies can see different values. On a long term, copy values are **reconciliated** and a consistent view is achieved. The **inconsistency window** may depend on many factors: communication delays, workload of the system, copy replication degree, ...

 **Note:** The ACID vs. BASE framing (CAP theorem, 2PC, BASE properties) is covered in the [ACID](#) and [BASE sections of the Cloud and Data Center Global Strategies chapter](#). This section focuses on eventual consistency as a replication policy.

13.13.3. Amazon S3: Optimistic Lazy Policies

In the case of **Amazon S3** you can also control both the allocation for your copies and the timing of checkpointing (**SLA control**):

- You can define your data **replicated in different buckets**: in one **local bucket** and in others (better than on the same machine), so you can have either a copy **Same-Region Replication (SRR)** (close to you) or in a distant bucket **Cross-Region Replication (CRR)**.
- The user can control the location of the copies, **either close in distance or very far regions**.
- The **distant copy CRR** can in some cases overcome big crashes of an entire region, but it takes time to propagate.
- The **neighbor bucket SRR** can be fast but may be subjected to common crashes.

S3 Replication Time Control (RTC): Amazon S3 lets you also control not only the location but also the timing of the operations via **S3 Replication time Control**. S3 RTC replicates most objects that you upload to Amazon S3 in seconds, and **99.99 percent of those objects within 15 minutes**.

 **Note:** S3 RTC by default includes S3 replication metrics and S3 event notifications, so to monitor the total number of S3 API operations that are pending replication, the total size of objects pending replication, and the maximum replication time, and also events that notify the bucket owner if object replication exceeds or replicates after the 15-minute threshold.

13.14. Docker Swarm and Modern Replication

13.14.1. Docker Swarm

Docker Swarm

Docker Swarm is Docker's native container orchestration mode. A Swarm cluster consists of **manager nodes** and **worker nodes**: managers schedule and coordinate containers (called **services**),



while workers execute them. A single manager console can distribute a containerized application across many nodes dynamically.

Docker Swarm provides **built-in fault tolerance**: if a worker node fails, the manager detects it via heartbeats and **reschedules the affected containers onto healthy nodes**, allowing the system to continue running (possibly in a degraded state until the new containers are ready).

For the manager itself, Swarm supports **multiple manager replicas** using the **Raft** consensus algorithm, removing the single point of failure. As long as a majority of manager nodes are alive, the cluster continues to schedule and coordinate services without interruption.

13.15. Apache ZooKeeper

13.15.1. ZooKeeper Overview



Apache ZooKeeper

ZooKeeper is a service for storing some **limited client data**, by using a **distributed replicated cluster of nodes** with excellent QoS, both **available** and **reliable**.

ZooKeeper is designed around four core principles:

1. **Simple**: the data model is a small hierarchical namespace of **znodes** (similar to a filesystem tree), each holding a small amount of data. No complex queries: just read, write, and watch.
2. **Replicated**: ZooKeeper runs as a **cluster** (an ensemble) of servers, **all holding a copy** of the state. Clients can connect to any server for reads, that are local and fast, while writes go through the leader.
3. **Ordered**: every write is assigned a globally unique, monotonically increasing **transaction ID**. Clients can use these IDs to impose their own ordering invariants on top of ZooKeeper's operations.
4. **Fast**: optimized for read-heavy workloads. Reads are served directly from any server's in-memory state without contacting the leader. The typical workload is far more reads than writes.

Data are kept by ZooKeeper servers as **znodes**, organized in a UNIX-like **hierarchical namespace** and replicated using **leader-follower (master-slave) replication**.

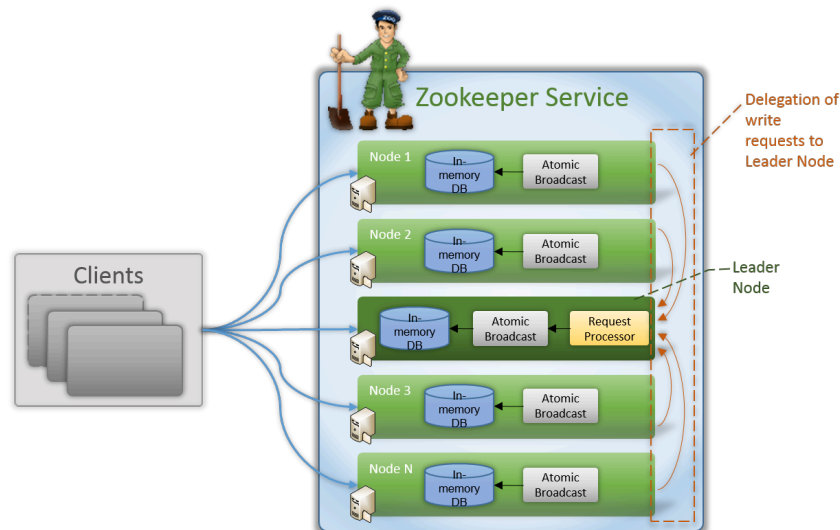


Figure 80 – Zookeeper architecture.

13.15.2. ZooKeeper Architecture

Each node in the tree is called a **znode** and can hold a small amount of data (typically metadata, not large blobs).

Two types of znodes exist:

- **Persistent znodes**: survive client disconnections. Used for configuration data and service registrations.
- **Ephemeral znodes**: exist only as long as the client session that created them is alive. When the client disconnects or crashes, its ephemeral znodes are automatically deleted. This property is the basis for distributed **presence detection** and **lock release on failure**.

The **leader** is elected among server nodes using **majority voting**. A simple API (available in Java and C) lets clients manage the namespace:

- `Create(path, data, flags)` , `Delete(path, version)`
- `getData(path, watch)` , `setData(path, data, version)`
- `getChildren(path, watch)` , `exists(path, watch)`

Clients can attach a **watch** flag to any read call: they receive a one-time notification the next time that znode changes. Watches are reset after firing, so a client that wants continuous notification must re-register after each event. Transaction IDs on every write let servers serve clients fresh, ordered data.

13.15.3. ZooKeeper Reads and Writes

ZooKeeper keeps all data **in memory** across the ensemble. This makes reads extremely fast but limits the total data size to available RAM.

- **Reads** are served **locally** by any server in the ensemble. They are fast but may return slightly stale data (since followers may lag behind the leader momentarily).
- **Writes** always go through the **leader**, which uses a **two-phase broadcast** (ZAB protocol) to commit the write to a majority of followers before acknowledging the client. This ensures linearizable writes.

The watch mechanism is the foundation for implementing distributed locks, configuration distribution, and leader election on top of ZooKeeper.



13.15.4. ZooKeeper Leader Election

Replication is passive with a leader elected. In case of leader crash, the election is based on the most recent data change among znodes (transactionID) via majority voting.

Election is also possible in case of Data Centers partitioning so to work disconnected independently.

ZooKeeper is widely used as coordination service in distributed systems, e.g., Apache Kafka uses it for broker metadata and leader election (prior to KRaft mode).



14. GROUP ISSUES AND POLICIES

Package: Group Issues and Policies - 8 - Group issues and policies 26.pdf

14.1. Partitioning and Groups



Partitioning and Replication

- **Partitioning**: several entities must be in charge of the whole function to grant **scalability** of services and support.
- **Replication**: several entities must be in charge of the whole function to grant **availability** of services and support.

The two aspects are strictly coupled and have both static and dynamic dimensions.

14.2. Group Communication



Communication Semantics

Semantics deeply depends on choices made about:

- **Global solicitation vs. Selective solicitation** → whether the message is sent to all group members or a chosen subset.
- **Positive confirmation vs. Negative confirmation** → whether the system acknowledges successful delivery or only signals loss.



Why multicast matters?

Multicast is essential whenever the same information must reach a group of processes:

- **Fault tolerance**: replicated servers must all receive the same requests in the same order so they stay consistent.
- **Object location**: locating a resource that may reside on any node in the group.
- **Data replication and streaming**: pushing updates to all replicas efficiently with one send instead of N unicasts.
- **Coordinated updates**: changing the state of multiple group entities atomically.
- **Multiple concurrent senders**: several producers pushing events to the same consumer group.

The **multicast** action could be made **atomic**, but implementations can associate different and more suitable meanings. The two aspects of multicast semantics can be untangled:

- **Reliability**: concerns whether individual group members receive a message :
 - **Reliable** → guaranteed delivery
 - **Unreliable** → only 1 attempt (Chorus model)
- **Atomicity**: concerns whether all group members receive the message, possibly with consistent ordering across multiple actions.



14.3. Reliable Multicast



Reliable Multicast

Reliability can be achieved by tolerating: sender crash, receiver crash, or message omission - through fault **identification** and **recovery** via monitoring. Recovery requires checking every ongoing communication, performing retransmissions, removing failed components, and re-admitting recovered ones.

14.3.1. Implementation Decisions

Key implementation choices for reliable multicast:

- **Dispatch** all messages to group members support and **delay** before passing them to the application, introducing **timeout** and **retransmission** (*who checks the protocol?*).
- *How long to wait?* → Problems with efficiency/latency.
- *If controller fails?* → “Quis custodiet ipsos custodes?” (Juvenal/Giovenale)



Hold-Back

In the **hold-back**, the support holds a message until it is sure that all previous others reached the destination in order.

👉 **Note:** In **dense numbering**, a message is delayed until all previous ones appeared.
⇒ message 3 must appear after message 2 (the order is kept but latency increases).



Negative Acknowledgment (NAK)

In **negative acknowledgment**, the support sends a **negative ack** only in case of losses, to identify those events in a selective way, avoiding unnecessary positive acks for every message.

14.4. Multicast Ordering

Ordering policies for group multicast form a spectrum from cheapest (no ordering) to most expensive (atomic/total ordering).

14.4.1. No Ordering



No Ordering

No ordering is exactly what states: multicast messages coming from any sending process to all receivers can present a different ordering in any copy. It is **very easy to support** and copies do not have to be synchronized in any way (they are free to operate on their own).



14.4.2. FIFO Ordering

FIFO Ordering

With **FIFO ordering** from the same sending process to all receivers, a sequence of successive multicast messages is received in the same order. Two multicast messages from the same sender reach any group member in the same order.

- (m1 and m2 from S1) and (m3 and m4 from S2) each reach everyone
- Respecting sending order of the two senders, many sequences are compatible: (m1 m2 m3 m4), (m1 m3 m2 m4), (m1 m3 m4 m2), ...

This means that single sender messages have to be ordered between them, but every other message from other sender can be in the middle of the sequence in any position/order. The important is to keep a relative logical order (similar concept of lamport clocks).

- An easy way to achieve FIFO is **message numbering** for any specific sender.

👉 **Note:** FIFO only constrains ordering from the **same sender**. With multiple senders: A sends Na; B reads it and sends Nb; C may receive Nb before Na. We need to also consider **cause/effect relationships across different senders** - which FIFO alone cannot capture.

14.4.3. Causal Ordering

Causal Ordering

CAUSAL ordering: events that are correlated with a cause-effect relationship outside the group must be delivered to everyone in the right order. **First the cause, then the effect** (Cause before Effect).

In case of causal ordering, two multicast messages in the **causal relationship** must be considered in the right order from everyone: (m1 and m2 from S1), (m3 and m4 from S2, m1 causes m3). They must reach copies respecting both FIFO and CAUSAL ordering. Many sequences are compatible: (m1 m2 m3 m4), (m1 m3 m2 m4), but **NOT** (m3 m1 m4 m2).

👉 **Note: Causal ordering limitations:** Compliance with causal ordering does not catch real-world Internet (USENET) situations. Example: A requests Na; B requests Nb - these actions are unrelated. C receives first Nb then Na; D receives first Na then Nb: so copies have different internal decisions of scheduling. Causal ordering still does not impose a **total** order among unrelated messages.

14.4.4. Atomic Ordering

Atomic Ordering

No external relations impose a scheduling, but **the group should act in a coordinated and reasonable way**, where all group members operate in the same order. Atomic ordering guarantees that **all messages are received in the same order by all group members** (so related actions can occur in the same order in all copies).



Often **no predetermined order** exists, so there is no need for previous agreement - it is only necessary to **dynamically agree on one**: and that order must be the same for all.

👉 **Note:** If a copy C decides to receive first Nb then Na, then **all copies must follow that decision**.

Example: Nb may ask to compute on a bank account; Na intends to make a withdrawal. Obviously many different atomic orderings exist that we can consider with group operations.

14.4.5. Cost of Orderings

Ordering	Description	Cost
No Ordering	Each member works freely; no synchronization needed.	Minimum (free)
FIFO	Messages from same sender arrive in send order. Easy via message numbering.	Low (partial)
Causal	Cause-effect relationships from different senders respected.	Medium (partial)
Atomic	All messages received in the same order by all group members. Total/global ordering.	High (total)

👉 **Note:** FIFO and CAUSAL are **partial orderings** - enforced only for some events. ATOMIC is a **total ordering** - enforced on every event. Costs for atomic orderings vary widely: some atomic orderings also satisfy causal and FIFO, others satisfy only one or neither.

14.5. Synchronization



Synchronization

Synchronization means to impose **orderings on events**: typically constraints on temporal ordering of some events inside a distributed system, to provide a **consistent view** to the entire set of communicating processes.

14.5.1. Clock Synchronization

The classical approach uses **physical time** and **physical clock**, but in distributed environments a unique global clock is neither feasible nor reliable. High accuracy implies high overhead and is prone to clock drift.



UTC: Universal Coordinated Time

UTC is the global reference time standard, maintained by atomic clocks. In a distributed system, nodes synchronize to UTC by receiving time broadcasts (radio, GPS, or internet servers). However, network delay introduces uncertainty: by the time the time value arrives, it is already slightly stale. The **Berkeley algorithm** handles this locally: a coordinator node polls the



clocks of all group members, computes the average, and distributes correction offsets to each node, avoiding the need for any node to have an external time source.

NTP: Network Time Protocol

NTP is the standard internet protocol for clock synchronization, designed to keep machines within a few milliseconds of UTC. It organizes servers in a **stratum hierarchy**:

- **Stratum 0**: primary reference clocks (atomic clocks, GPS) - not on the network directly.
- **Stratum 1**: servers directly wired to stratum-0 devices (most accurate).
- **Stratum N**: servers synchronized from stratum N-1; accuracy degrades with each hop.

The core problem is that a time value is already stale by the time it arrives over the network. NTP compensates by exchanging **4 timestamps**: **T1** (client sends request), **T2** (server receives it), **T3** (server sends reply), **T4** (client receives reply). From these it computes:

- **Round-trip delay**: $(T4 - T1) - (T3 - T2)$
- **Clock offset**: $(T2 - T1 + T3 - T4)/2$

NTP queries **multiple servers simultaneously** and applies statistical filtering to select the most reliable ones (discarding outliers with high jitter or inconsistent offsets). Clock corrections are applied **gradually (slewing)** - never as sudden jumps - to avoid disrupting running applications.

 **Note**: When clocks are not perfectly in sync, an event that actually happened later may receive a lower timestamp, violating cause-effect ordering. This is precisely why **logical clocks** (Lamport, vector clocks) are preferred over physical clocks in distributed systems: they depend on message passing, not clock accuracy.

Distributed synchronization is therefore **not based on physical clock agreement**, but on **different strategies** focusing only on a **subset of global events** - limiting overhead and coordination cost.

14.5.2. Synchronization Strategies

Several distributed synchronization methods:

- **Ordering of logical time of Lamport**: use logical timestamps to label relevant events and to order them → logical clocks and “happened before” relationship.
- **Token passing LeLann ring strategies**: use authorizations and the token can pass in a logical ring to order events.
- **Events based on priority**: use process priority to order correlated events. Used in real-time systems and unfair (special-purpose systems).

14.6. Lamport Relationship

In distributed systems, Lamport aims at **ordering some events (not all of them)** by **excluding physical time**. Only **some events** are considered - local events and **remote** interprocess events (send / receive) - with the goal of creating a simple ordering policy at adequate cost.



14.6.1. Happened-Before Relationship

Happened-Before ($\Rightarrow$)

Happened before ($\Rightarrow$) is based on cause-effect relationship introduced by process actions:

1. If a and b are events of the same process and a occurs before b , then $a \Rightarrow b$ (**local order**)
2. If a is the sending of a message of one process and b the receiving event within another process, then $a \Rightarrow b$ (**communication interprocess order**)
3. If $a \Rightarrow b$ and $b \Rightarrow c$, then $a \Rightarrow c$ (**transitivity**)

The relation $\Rightarrow$ introduces a **partial ordering** in system events - it exists only among some events, not all. **Two events are concurrent** iff **not** $a \Rightarrow b$ and **not** $b \Rightarrow a$.

Happened-Before Examples

With processes P_a, P_b, P_c and events $a_1, a_2, a_3, b_1, b_2, b_3, c_1, c_2$:

- $a_1 \Rightarrow a_2, a_1 \Rightarrow a_3$
- $a_1 \Rightarrow b_1, a_1 \Rightarrow b_2, a_1 \Rightarrow b_3$
- $c_1 \Rightarrow c_2$
- $c_1 \Rightarrow b_2, c_1 \Rightarrow b_3, c_1 \Rightarrow a_3$
- **Concurrent events:** $a_1 \parallel c_1, a_1 \parallel c_2, a_2 \parallel b_2, a_2 \parallel b_3, \dots$

We work in an **asynchronous environment** where transmission delays are **variable and unlimited** (but messages are not lost). We organize a **logical time system** built on the $\Rightarrow$ relationship, based on **logical clocks** rather than physical clocks.

14.6.2. Logical Clock and Timestamp

Clock Condition (Logical Clock LC)

Each process P_i has a logical clock LC_i (an integer counter). The **clock condition** states: if $a \Rightarrow b$, then $LC(a) < LC(b)$.

Note: the converse is NOT true - $LC(a) < LC(b)$ does not imply $a \Rightarrow b$.

Conditions:

- **C1:** if $a \Rightarrow b$ inside the same process P_i , then $LC_i(a) < LC_i(b)$
- **C2:** if a is the sending of a message in P_i and b the reception in P_j , then $LC_i(a) < LC_j(b)$

Implementation rules:

- **I1:** every process P_i increments LC_i between any two events
- **I2:** for any sending of a message in P_i , the message contains a clock as timestamp $TS = LC_i(a)$
- **I3:** for any reception of a message in P_j : $LC_j = \max(TS_{received}, LC_{current}) + 1$

 **Note:** These rules introduce a **partial order**: many concurrent events $a \parallel b$ can share equal timestamps. **Who doesn't receive, doesn't update:** the sender forces a logical clock update on the receiver, not on itself - it is the receiver that must advance its clock upon receiving a message.

Limitations of Lamport clocks: the $\Rightarrow$ relationship is **loosely connected with the real world**. Processes that never receive messages may maintain arbitrarily low timestamps. **Causality**



problem: two events in a Lamport causal relationship may not be causally related at all.
Hidden channel problem: an out-of-band channel can violate cause/effect ordering - the effect may carry a lower timestamp than the cause.

14.6.3. Total Ordering

Total Order Relationship #so

Given a is an event in process P_i and b an event in process P_j , then $a \Rightarrow b$ iff:

- R1: $LC_i(a) < LC_j(b)$, or
- R2: $LC_i(a) = LC_j(b)$ and $P_i < P_j$

The total ordering means that in case of events of the same logical clock, **there is an order between all process events**. It is possible to use $\Rightarrow$ to define a univocal and simple ordering to create synchronization upon.

👉 **Note:** The $\Rightarrow$ relationship **orders any pair of events** - even concurrent ones in the real world.

*Example: c2 and b2 are managed as in sequence, by considering first process P_b , then P_c . However, **Happened-before** is only one way and not **bidirectional**: $LC(a) < LC(b)$ does not imply $a \Rightarrow b$. Sometimes we need a closer relationship between the Lamport model and reality - toward a **bidirectional relationship via vector clocks**.*

14.6.4. Vector Clocks

Vector Clock

Processes must maintain a **vector of all known clocks of processes** and use that in communication. Every process P_i keeps a vector $V_i[k]$ of integers - one per process - representing **what P_i knows about the clocks of all others**.

Protocol:

1. For every process P_i , increments $V_i[i]$ between two events
2. For any sending of a message in P_i : $V_i[i] = V_i[i] + 1$, then attach the **whole vector clock** to the message
3. For any reception of a message in P_j : update $V_j[k] = \max(V_j[k], V_i[k])$ for all k , then $V_j[j] = V_j[j] + 1$

👉 **Note:** With **vector clocks**, the $\Rightarrow$ relationship becomes bidirectional: events in the $\Rightarrow$ relationship are recognized as having a cause-effect dependency; events **not** in that relationship (i.e., concurrent $\parallel$) are recognized as truly concurrent. This is what Lamport clocks cannot detect - concurrent events in the real world (c1 and b1, a1 and c2, ...) are correctly identified as concurrent rather than forced into a sequence. The cost is propagating the **entire vector** with every message.

14.7. Mutual Exclusion and Synchronization

The simplest synchronization case is a set of processes accessing a **resource in a mutually exclusive way**. Objectives: **Safety** (only one process at a time), **Liveness** (every requester eventually gets access), **Fairness** (no starvation - fixed priorities excluded).



14.7.1. Centralized Coordinator

Resource Coordinator Protocol

A unique coordinator is known to all processes:

1. A process sends a **request** to the coordinator
2. The coordinator queues requests (typically FIFO) and sends a **reply** when the resource is free
3. The process uses the resource and sends a **release** to the coordinator

3 messages per critical section access. Disadvantages: single point of failure, potential unfairness due to differentiated delays in reaching the coordinator.

14.7.2. Lamport Synchronization

Lamport Synchronization

Lamport proposes a **decentralized solution without single failure points**. N processes access a single resource in mutual exclusion, with no centralized role. Each process maintains a **local queue** of requests sorted by Lamport timestamp (T_m, P_i). Assumptions: complete mesh, FIFO channels, no message loss, static group.

Protocol:

1. P_i sends **request** $T_m : P_i$ to every process (and enqueues it locally)
2. On receiving $T_m : P_i$, P_j replies with its current timestamp
3. P_i enters the critical section when: (a) its request is **first in its local queue** ($\Rightarrow$ before all others), and (b) it has received **at least one message from every other process** with a timestamp later than T_m
4. On exit, P_i sends a **release** to all; each P_j removes P_i 's request from its queue

Cost: $3 \times (N - 1)$ messages per critical section (or $N - 1 + 2$ broadcasts). Fully distributed and fair, but heavy assumptions on static group and no faults.

14.7.3. Ricart-Agrawala Protocol

Ricart-Agrawala (R.A.) Protocol

Optimizes Lamport by eliminating the release message. On receiving request $T_m : P_i$, process P_j :

- **Immediately replies** if it does not need the resource, or if P_i has higher priority (lower timestamp, or equal timestamp but lower pid)
- **Delays its reply** if it is currently using the resource, or if its own pending request has higher priority - the reply is sent when P_j releases

P_i enters the critical section only after receiving $N - 1$ approvals. On exit, sends all queued approvals.

Cost: $2 \times (N - 1)$ messages per critical section. Same assumptions (no faults, static group, FIFO channels).



👉 **Note:** Both algorithms are **completely distributed**, **fair**, **deadlock-free** and **starvation-free**. High coordination cost and heavy assumptions (no message loss, static group) are the main limitations.

14.8. Atomic Multicast

Distributed implementation of **atomic multicast** can be less centralized than using a unique coordinator.



CATOCs

CATOCs (CAusal and Totally Ordered Communication operations Support) coordinates a **set of managers** that dynamically decide the internal request order. The group has no unique central manager but coordinates on demand, with one manager selected per request that negotiates the ordering with others.

Realization is not highly scalable and is efficient only in specific cases. Low-level broadcast support and reliable message delivery (no loss, full connectivity) significantly enhance efficiency.

14.8.1. ISIS: Atomic Multicast

*ISIS appeared in the 90s for CATOCs in UNIX. ISIS is system based on groups with **active replication** and with necessity of a vision with **different degrees of coordination** of group components. The system obtains coordination with **many different forms of group multicast** (called **broadcast**) for the same group.*

Many different multicast forms are available (BCast):

- **FBCast** (FIFO BCast)
- **CBCast** (Causal BCast)
- **ABCast** (Atomic BCast)
- **GBCast** (Group BCast)

*Providing also support to the case of no copy coordination. Any operation need a manager, typically **dynamically chosen** according to any kind of policy (vicinity, rotation, ...).*

14.8.2. ISIS ABCast



ISIS ABCast (Atomic BCast)

ABCast cost: $3 \times (N-1)$. CATOCs uses a queue for every corresponding component of the group and Lamport relationship. Messages are tagged with an **initial arriving timestamp** and are only considered (and processed) if labeled as **final** in the **right order for Lamport** relationship.

Every arrived message requests a **coordination phase of the manager** (and hold-back) to determine the final timestamp to be used by all copies to execute in the correct order. A group should be capable of **operating with all the ordering policies for any request**.

The coordinator receives the message:

- **Labels it and sends it to all others** (with its timestamp). Anyone else labels the answer with its timestamp based on its time (clock) and sends the answer back with its timestamp.
- **Labels it as final** with the received highest timestamp
- **Resends the message with the final timestamp** to all others to communicate the final decision



All members have finalized messages in the same order in their queues and execute in that order. Group members cannot operate on a request until it is sure that the message has been seen by everyone and has been ordered with respect to all others. Problems: delay and overhead, cost in messages of $3 * (N-1)$.

14.8.3. ISIS CBCast

ISIS CBCast (Causal BCast)

CBCast is a **partial ordering**. It considers only some external events to be ordered with one another; all other events can be ordered differently by group components (limiting costs and coordination).

ABCast imposes an order based on timestamping decided **inside the receiver group** (internal event ordering strategy). CBCast requests a behavior **decided outside the receiver group** that must detect **cause-effect relationships** by inferring it from **timestamps arriving from outside** (external event ordering).

The Causal Broadcast assumes a coordination between senders that must update their “logical clock” and send information to receivers (requests queued by **sender timestamps**). Group members must respect that external ordering.

If a cause would not reach the group before processing the effect? Either undo or error are necessary (!!).

14.8.4. ISIS GBCast

ISIS GBCast (Group BCast)

The group of processes can **dynamically change in cardinality**, so it is possible to join or to leave the group for different reasons (possible group inconsistencies and problems). For every concurrent multicast, the message arrives in two states:

- To every member **before** group changing
- To every member **after** a group changing

For a consistent ordering of any BCast, **either before or after** we need to define a new operation for tracking the dynamic behavior of the group. GBCast makes possible to order all BCasts: any **GBCast message must be either received after every previous BCasts** in the processes (or before, in a consistent way).

The GBCast was introduced to design a correct **dynamic grouping**, with no need to stop and reconfigure and work no stop with no problems. GBCast requires an **automatic monitoring support** for **group variation events** (any insertion and extraction trigger one GBCast). When anyone detects a failure of a copy (or a new copy to be inserted), the **GBCast is issued to all copies** to make them aware of the reconfiguration. The **group support** is in charge of invoking it. Every group member uses a **table** for other members: that table is updated by any GBCast (so all other BCasts can be aware of it and consistently ordered).

14.8.5. JGROUPS: Reliable Multicast in Java

*JGROUPS: Java Support for reliable multicast and for group concept (Designed in Java and with user defined proprieties). JGROUPS starts with a **transport level**, either not connected or connected, and it is also possible to work with JMS (Java Message Service) for message specifications. The goal of JGROUPS is **group and message delivery ordering**: it proposes a **reliable** implementation, intended as delivery with message **retransmission**, with most common different ordering: **Atomic**,*



FIFO, Causal, etc. For the group property, groups are dynamic and managed in membership: every group element benefits from group messages, both from outside that from inside the group. Possibility of security, like encryption and other secure support protocols.

14.9. Apache ZooKeeper

ZooKeeper is a **Distributed Coordination Service**: it provides group services (synchronization, configuration, naming) leveraging **replication over several znodes** with ordering semantics (**FIFO, Atomic, Causal**). This makes it a natural building block for the CATOCS-style coordination described in this chapter.

👉 **Note:** The full ZooKeeper architecture (znodes, leader election via majority voting, passive replication, reads/writes model, and how it is used as a coordination backbone such as in Kafka) is covered in the [Replication for Dependability chapter](#).

14.10. Token-Based Synchronization



Token Ring

The nodes are organized in a **logical ring** where every node knows its successor and predecessor. The node owning the **token** acts as the group manager for Mutual Exclusion: it holds the token for a bounded time period, uses it to access resources, then passes it to the next node. **N** messages are needed for a complete token turn.

Protocol: the token holder verifies it is the expected recipient, uses the resource for a maximum detention period, then forwards the token. Since only one process holds the token at a time: no conflicts, no starvation. **The scheme is proactive: the token circulates even when no requests occur.** The main risk is token loss on node failure.

14.10.1. Recovery in a Ring

The ring architecture allows **very simple recovery** for **single faults without token loss**: any node detecting a neighbor failure shortcuts the ring and re-establishes the local topology. Each node must know its further predecessor/successor for this.

14.10.2. Token Recovery

If the token is **lost** (failure of the holding node) or **duplicated**, it must be regenerated. Every node maintains a **timeout** reset on each token pass. On timeout expiry, a **recovery procedure** starts. Multiple nodes may start simultaneously, **node priority** (index) resolves the conflict.

14.11. Election Protocols



Election Protocols

The **election protocols** are used any time an **agreement among participants must be found without a predefined policy**. They are typically necessary in case of **fault** and **recovery** in a group to



obtain distributed and easy agreement on a decision. In many cases, it is based on a **potential static order of participants**.

14.11.1. Bully Algorithm

Three message types: **Election** (announcing candidacy), **Answer** (blocking a lower-priority candidate), **IAmCoordinator** (winner announcement).



Bully Protocol

Any process P_i can start an election at any time. It sends **Election** to all processes with higher priority. If no reply arrives within timeout, P_i declares itself coordinator and broadcasts **IAmCoordinator** to all lower-priority processes. If a higher-priority process replies with **Answer**, it blocks P_i and starts its own election. Several rounds can run concurrently.

Example: 4 starts, blocked by 5 and 6 answering; 5 then blocked by 6; 6 wins.

14.11.2. Election in a Ring

Election protocol to decide who must become the manager (with a unique new token) based on **static priority of processes**:

- At timeout, the process creates an **election token (ET)** with his name and enters an **election state** until the token returns
- If the process receives the normal token before the generated ET is back, the election is considered useless and terminated (**ET destroyed at return**)
- If the process receives an ET from another process, it is registered on an **election list** together with **identity of process** that generated it, and the ET is passed forward in the ring
- If it has already generated an ET token, it verifies the **static priority** and decides who has **highest priority** in the election
- If the process receives its ET, it removes it and verifies the registration list. The process generates a new token, only if it is the node with **minimum index (top priority)** inside the registration list

14.12. Global State

In a distributed system it is sometimes necessary to record a **global state** associated with the current situation, to use it as a **checkpoint for recovery** or for distributed garbage collection. The main challenge is composing a **consistent view** from locally-recorded partial states, without stopping the system.

The global state stems from the **private states of participant processes** plus the **messages currently in transit** between them. We assume an **asynchronous model**: processes on different nodes exchange messages via **one-way channels** ($\Rightarrow$ bidirectional channel are separated into two). Any node must be reachable from any other (**no partitions**). Nodes have In and Out queues.



14.12.1. Global States Consistency



Consistent Cuts

A **cut** is a partition of all events into "before" and "after". A cut is **consistent** if: whenever an event e is in the cut, every event that **happened-before** e is also in the cut, or, equivalently, no message is received without also having been sent.

- **Consistent cuts** represent a safe, replayable global state.
- **Inconsistent cuts** produce unreasonable states $\leadsto$ a replay would either lose or duplicate messages.



Consistent vs Inconsistent Cut

Consistent Cut - Message m_3 from P_1 to P_2 : the sending state is included in P_1 's snapshot, so the arrival must be **recorded within P_3 's state**: input messages must be saved.

Inconsistent Cut - Message m_2 from P_2 to P_3 : the arrival is recorded in P_3 's state, but the sending was not yet recorded in P_2 's state. In case of replay, the sender would resend the message causing a **double reception** in the receiver - an unsafe behavior to avoid.

14.12.2. Global State via Snapshot



Distributed Global Snapshot (Chandy-Lamport)

One node initiates a snapshot. Each node records its **local state** (checkpoint) and the **state of its input channels** (in-transit messages). Nodes continue normal execution during the snapshot.

Marker propagation algorithm (requires FIFO channels):

1. A process decides to snapshot (or receives the first marker): saves its local state, sends one **marker** on every OUT channel, starts recording all messages arriving on input channels
2. On receiving a marker on input channel C : stop recording C (save buffered messages as C 's state); if not yet snapshotted, do so and forward markers
3. When markers received on **all** input channels: **local snapshot complete** (process state + all channel states)

Global snapshot = union of all local process states + all channel states.

Messages in a channel can be classified by the color of sender and receiver at snapshot time:

- **bb** (white sender, white receiver): sent and received before snapshot $\leadsto$ already captured in local states.
- **rr** (red sender, red receiver): sent and received after snapshot $\leadsto$ irrelevant to this snapshot.
- **br** (white sender, red receiver): sent before snapshot but received after $\leadsto$ these **must be recorded** in the channel state (the receiver records them while its channel is open).
- **rb** (red sender, white receiver): sent after snapshot but received before $\leadsto$ a potential **inconsistency**. The protocol avoids this by construction: the marker always precedes any post-snapshot message (FIFO channels), so the receiver turns red before any red-sender message arrives.

👉 **Note**: Each process that completes its local snapshot can send the state to the initiator or to a designated collection node for eventual replay. Snapshots are rare events due to cost. Open questions: how to run concurrent snapshots? How to distinguish and compose them?



15. QOS: QUALITY OF SERVICE

Package: QoS basis and protocols - 9 - QoS 26.pdf

Quality of Service (QoS) is the ability of a network or system to provide **differentiated, negotiated, and guaranteed levels of service** to applications. This chapter covers the fundamentals of QoS indicators, management models, protocols (IntServ, DiffServ, RSVP, RTP/RTCP, SIP), router scheduling policies, and congestion prevention.

15.1. QoS in Different Environments



TCP/IP vs OSI

The key difference is **whether resources are reserved before data is sent** - which is exactly what decides if QoS can be guaranteed:

- **TCP/IP (best-effort)**: **nothing is reserved in advance**. Each packet is forwarded using whatever capacity happens to be free at each hop, with no commitment. So the IP layer gives **no guarantee** on delay, bandwidth, or loss, it just "does its best".
- **OSI (connection-oriented)**: **every node along the path reserves the resources** it will need before data flows. This up-front commitment is what lets the network honor a **negotiated SLA end-to-end**, intermediate nodes included.
- **The challenge**: modern applications (streaming, VoIP, real-time) need **real guarantees**, yet the Internet runs on best-effort TCP/IP → the rest of this chapter is about **how to add QoS on top of a best-effort network** (IntServ, DiffServ, RSVP, ...).

15.1.1. Application Classification

Applications are classified by their tolerance to QoS violations:



Elastic Applications

Elastic applications are the traditional: they do **not present quality constraints** but have requirements independent from delays:

- Work better with low delays, worst during congestions.
- **Interactive apps require delays less than 200ms.**
Examples: telnet, X-windows (interactive), FTP, HTTP (bulk interactive), e-mail, voice (asynchronous).



Non-Elastic: Real-Time Applications

Have **constraints to be respected in time**, less tolerant:

- Can **not work** outside their allowed admissibility space (failure).
- The service can be **adaptive** in two ways:
 - **Delay adaptive** → audio drops packets when delay is exceeded.
 - **Bandwidth adaptive** → video adapts quality to available bandwidth.
- Further split into **Tolerant** (adaptive to delay or bandwidth) and **Not tolerant** (not-adaptive).



15.2 QoS Indicators

QoS Indicators

Many parameters and indicators qualify a **stream of information** and its functional properties. They are both **functional** (easily measurable) and **non-functional** (dependent on external factors, observable and judged by the final user only).

Key QoS Metrics

- **Promptness in reply:** delay, response time, **jitter** (variation in deliver delay).
- **Bandwidth (throughput):** quantity of data transmitted by a channel with success per time unit (bit/byte per second).
- **Throughput:** number of operations per second (transactions).
- **Reliability:** percentage of successes/failures (MTBF: Mean Time Between Failures, MTTR: Mean Time To Repair).

15.2.1. Latency

Latency Time (RTT)

Latency is the delay for data to travel from sender to receiver. Measured there and back, it is the **Round Trip Time (RTT)**. It is the sum of three delays:

$$T_L = T_{\text{prop}} + T_{\text{tx}} + T_q$$

- T_{prop} (**propagation**): time for a bit to physically cross the distance, = distance / speed (the signal moves near the speed of light in the medium). It depends on **how far** the nodes are, not on how much data is sent.
- T_{tx} (**transmission**): time to push all the bits of the message onto the link, = message size / bandwidth. It depends on **how much data** there is and on the link speed.
- T_q (**queuing**): time the data waits in buffers at intermediate nodes. This is the **most variable** part, since it grows with congestion, and usually the hardest to predict and control.

Bandwidth-Delay Product

A good service requires to identify bottlenecks and consider **resource management**:

- Send/receive of 1 byte → latency dominates RTT.
- Send/receive of many megabytes → bandwidth dominates.
- **Resource data channel** = Latency × Bandwidth.

With 40 ms one-way latency and 10 Mbps bandwidth:

One way → 10 Mbps × 40 ms = 10 000 000 bps × 0.04 s = 400 Kb = 50 KB. By the time the first bit reaches the receiver, the sender has already put 50 KB onto the wire (the data "in flight").

Round trip (80 ms) → 10 Mbps × 80 ms = 800 Kb = 100 KB. So the sender can push 100 KB before any reply can come back, and it should keep about that much unacknowledged data in flight to keep the link busy.

- Infrastructures tend to keep **pipes full** to guarantee response time.
- A **buffering time inside applications** is typically automatically considered.



15.2.2. Jitter and Skew

Jitter

Jitter is defined as the **variance of latency** in a stream.

Optimal situation: latency stable. $\Rightarrow$ High jitter breaks real-time playback.

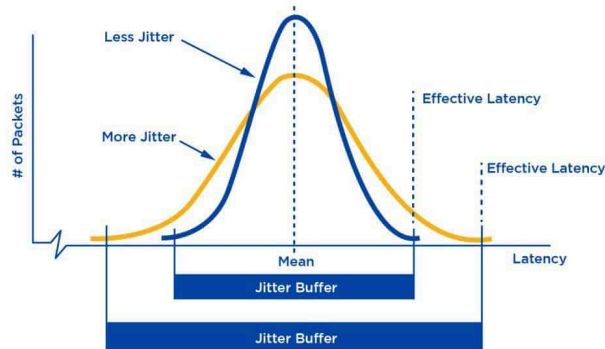


Figure 81 - Jitter graphical representation.

Skew

Skew is the possible **offset between multiple flows** (types) composing a unique stream (e.g., audio and video flows of the same session). Skew must be minimized to maintain synchronization.

Note: Skew vs Jitter:

- Skew is the **spatial/cross-flow** delta: $T_{\text{audio}} - T_{\text{video}}$ at a given frame (e.g., lip-sync errors).
- Jitter is the **temporal/intra-flow** delta: the packet-to-packet arrival time variance within a single flow over time ($T_{i+1} - T_i$).

15.2.3. User-Level QoS Indicators

QoE: Quality of Experience

The typical non-functional properties requested by a final user:

- **Relevance** (priority)
- **QoS perceived** (details, accuracy, synchronization, audio/video quality)
- **Cost** (per access, per service)
- **Security** (integrity, confidentiality, authentication, non-repudiation)

QoS Requires Negotiation

QoS can be guaranteed only through a **negotiated and controlled contract and after provisioning**. The system must be observed during execution to adjust the service dynamically (obeying user requests). This requires **monitoring and feedback loops**. The **negotiated SLA must be verified during execution** to undertake quickly corrective actions.



15.3. QoS Management

QoS Management

Good QoS management requires actions that must be **active for the whole service time**. Actions must be both **proactive** (before content distribution) and **reactive** (during deployment). Requires defining precise **Service Level Agreement (SLA)** models.

SLA Lifecycle (Static + Dynamic)

Static actions (before distribution):

1. **Requirement definition:** precise specification of QoS levels and allowed variations.
2. **SLA definition:** the static agreement describing what is expected.
3. **Negotiation:** agreement between all entities and levels interested in granting QoS.
4. **Admission control:** comparison between requested QoS and offered resources.
5. **Reservation and commitment** of required resources: needed resource definition for allocation.

Dynamic actions (during distribution):

1. **Monitoring:** continuous measurements of QoS levels and SLA parameters.
2. **Respect control and synchronization:** verify fulfillment and potential need for synchronization of different resources (video/audio).
3. **Renegotiation** of necessary resources: new contract to respect QoS and grant SLA.
4. **Change of resources** to maintain QoS and adjustment to new situations.

After renegotiation, the new SLA fulfillment must be ascertained and regularly checked.

 **Note:** Local actions (reservations, conditioning) are not directly provided by the protocols: they must be implemented at lower levels. This is a fundamental limitation of IntServ.

15.3.1. Active Path

For streaming services, the management must find and maintain an **active path** between emitter and receiver:

- The best active path is found (even by flooding).
- Among several paths, one is **chosen as the active path**.
- The active path **must change during provisioning** in case of severe problems (failure recovery).

15.3.2. SLA Negotiation Example

Example: SLA Negotiation

The SLA specification is **always a coordinated effort among emitters and receivers**:

- Emitter: "I can send that specific streaming with bandwidth B and accuracy A ."
- Receiver: "I can accept streaming with bandwidth $B1$ and accuracy A , latency L ."
- **Agreement succeeds** when receivers accept a setting for their streaming.
- The final agreement can define a **private setting** for the entire routing.
- Some sharing of part of the active path can be convenient.
- Difficult cases: receiver coordinates more emitters (multiple emitters sharing a path).
- **Impossible agreement** → look for other settings.



15.4 Video Streaming Services

Effective video streaming requires thinking about: actors (senders and receivers), streaming model, protocols, and strategies.

Streaming Topologies

- **One-to-one:** one sender, one receiver, many intermediaries. Requires simple protocols and point-to-point strategies.
- **Many-to-one:** many senders, one receiver, more intermediaries. Typical in CDN (Content Delivery Networks) with edge servers and media servers.
- **Many-to-many:** many senders, many receivers, much more intermediaries. Requires multicast or overlay networks with distributors and relays.

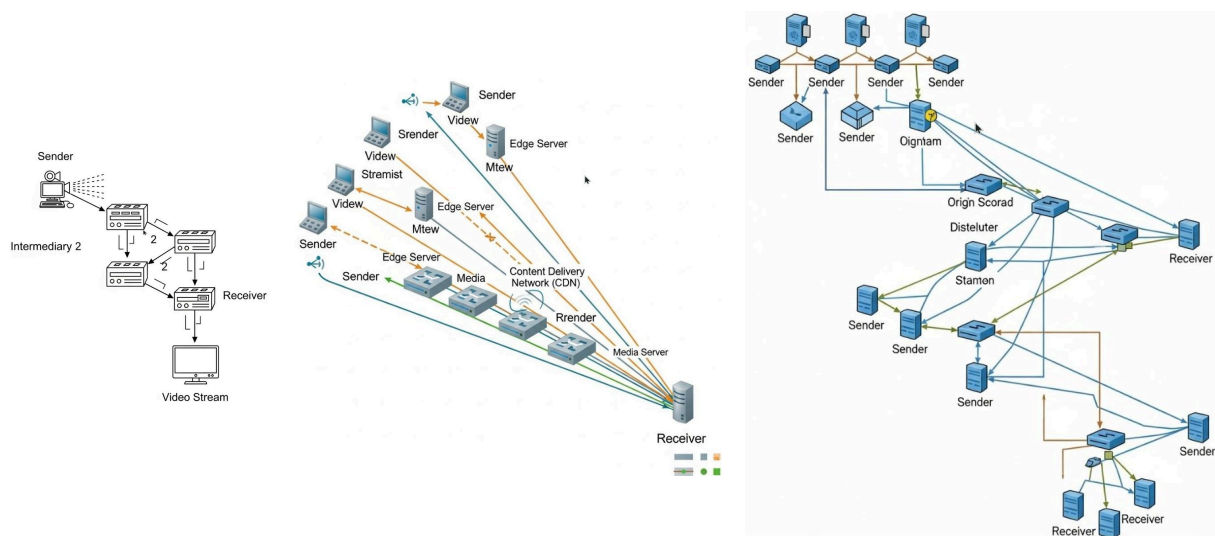


Figure 82 - Streaming topologies: one-to-one (left), many-to-one (center), many-to-many (right).

15.4.1. Three Operational Planes

Management and Monitoring Planes

QoS management requires matching the application plane with efficiency control strategies:

- **User Plane:** for defining user protocols (e.g., voice in telephony).
- **Management Plane:** for service management and monitoring (e.g., QoS handling in telephony).
- **Control Plane / Signaling:** to establish connections, negotiate and signal between levels (not necessarily in-band, in telco this level establishes the call before it starts).

15.4.2. RTSP: Real-Time Streaming Protocol

RTSP (RFC 2326)

RTSP integrates web-based streaming transported to a final client. Starts after downloading the file specification from the server.



RTSP Operation

- Player communicates with the server via UDP or TCP, trying to obtain better provisioning.
- Exploits a **local receiver buffer strategy**: receiver does not wait for the entire file.
 - UDP: wait 2-5 seconds before starting to show
 - TCP: a larger buffer is used.
- **Pull and push policies** on the server with **watermark techniques** to synchronize: if below threshold, starts pulling requests.
- **Interleaving** used to deal with packet loss.

15.5. IntServ: Integrated Services (RFC 2210)



IntServ

Integrated Services (IntServ) works at the **application level** at a **single flow level**. The goal is to produce one **active path** connecting sender and receiver for the whole flow. Must work during both static and dynamic phases to grant QoS. Requires coordination of protocols in the suite to not disturb the QoE of the final user.



IntServ Principle

For every flow, IntServ considers not only the endpoints but the **whole path**: works **hop-by-hop**, involving all intermediate nodes in the active path. The service is enabled by one active initiator (receiver/client), one service provider, and many intermediate nodes connected in the active path. Local actions that grant SLA respect must be obtained at **lower network levels** (not at protocol control level).

The IntServ suite has three protocols:

- **RSVP** (out-of-band, static): resource reservation setup.
- **RTP** (in-band): real-time data transport.
- **RTCP** (in-band): flow control and QoS monitoring.

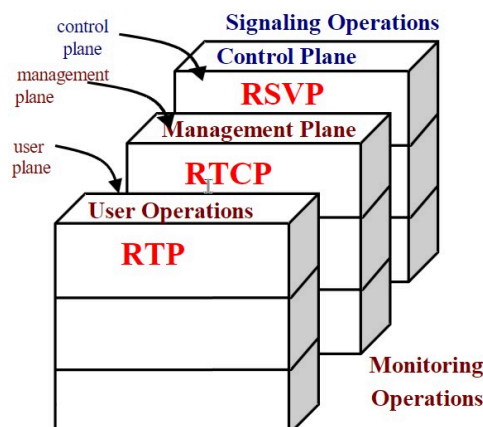


Figure 83 - Contron planes in IntServ.



15.5.1. RSVP: Resource Reservation Setup Protocol (RFC 2205)

RSVP

RSVP (Resource ReSerVation Protocol) specifies how to communicate between neighbor nodes to enable the **reservation of needed resources** to guarantee an agreed SLA, in a completely separate way from current Internet traffic. RSVP is a **static (out-of-band) two-phase protocol** with **soft-state**, where the receiver requests to enable resource reservation for the whole service duration.

- **Phase 1 - Path (sender to receivers):** provider (sender) propagates announce messages (**Path**) with offers toward potential receivers. Identifies the active path.
- **Phase 2 - Resv (receivers to sender):** receivers propagate inversely their intention of creating an active path, requiring reservations (**Resv** with TSpec + optional RSpec).
- **Soft-state:** the admitted state is maintained for a limited time and must be refreshed. Paths and resources are reserved locally either in a private or shared way.
- **Teardown:** **PathTear** from sender or **ResvTear** from receiver, or timeout.

 **Note:** The **Path** message is not routed by the sender, so it use this message to discover the route in order to reserve resources (with **Resv**).

RSVP Key Properties

- Oriented to **receiver initiative** (receivers request resource reservation).
- Produces **state on every node** of the path established from sender to receiver (during phase 2).
- Allows **sharing of active paths** (shared reservations for multicast groups).
- Compatible with any routing protocol (unicast or multicast, IPv4 and IPv6).
- **Not a routing protocol:** works before provisioning, impacts less on execution.
- One reservation can block another, producing a **ResvErr** .
- Recommended only for **limited local networks**, not global environments (legacy application compatibility issues).
- On router failure, QoS falls back to best-effort; RSVP must renegotiate the path reservation during provisioning.

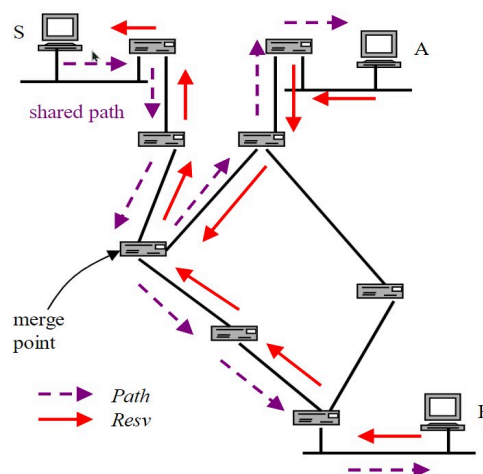


Figure 84 - RSVP **Path** and **Resv** flow.



15.5.2. RTP: Real-Time Transport Protocol (RFC 1889)

RTP

RTP is the protocol that carries the real-time media stream itself (audio, video) from sender to receiver, **in-band** with the data. It runs over **UDP**, so there is no retransmission, which fits real-time traffic where a late packet is useless.

Every RTP packet carries two key fields:

- a **sequence number**, so the receiver can put packets back in **order** and **detect losses**.
- a **timestamp**, so the receiver can play the media at the **right time**, smooth out **jitter**, and keep flows **synchronized** (e.g. audio with video).

RTP uses an **even UDP port**, and its control companion **RTCP** uses the **next odd port**. By itself RTP gives **no delivery or QoS guarantee**, it only gives the receiver what it needs to cope with loss and jitter.

RTP Features

- The **sequence number** lets the receiver reorder packets and notice gaps (lost packets).
- The **timestamp** lets the receiver reconstruct timing, measure **jitter**, and align playback.
- On a lost packet the receiver does **not retransmit but interpolates** from earlier packets, since a resent packet would arrive too late to use.
- **Intermediate nodes** can process the stream:
 - a **mixer** combines several incoming streams into one and can add timestamps useful for monitoring.
 - a **translator** forwards or transcodes one stream without merging it.
- Each source is tagged, so one path can carry **several sources**:
 - **SSRC** (Synchronization Source): the identifier of a single stream source.
 - **CSRC** (Contributing Source): the list of original sources that a **mixer** combined into one stream.

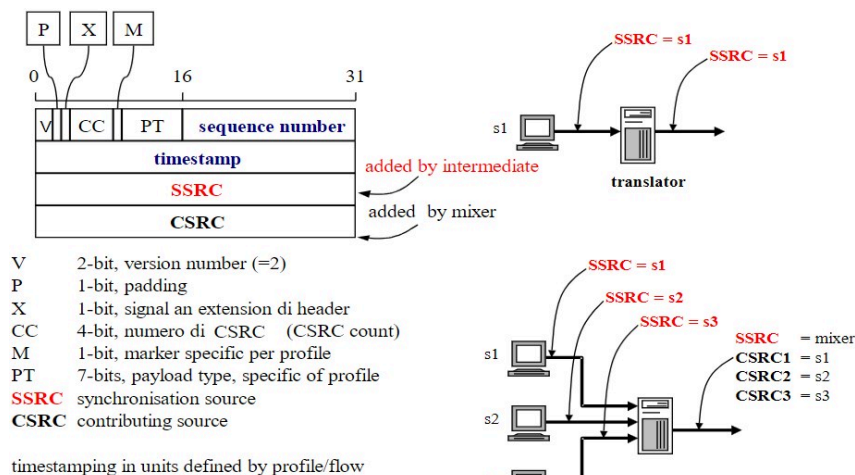


Figure 85 – RTP packet and flow scheme.



15.5.3. RTCP: Real-Time Transport Control Protocol

RTCP

RTCP is the **bidirectional control companion** to RTP. Provides global and concise information of flow control at the application level, propagating knowledge about the current situation so anyone can intervene. Travels in **both directions** and uses the **same resources as RTP** (in-band, competing with application). Limited to **5-10% of RTP bandwidth**.

RTCP Message Types

- **RR/SR** (Receiver/Sender Report): QoS per flow $\leadsto$ loss, delays, jitter, end system info, application specification.
- **SDES** (Source DEScription): ASCII strings $\leadsto$ CNAME (canonical identifier, mandatory), NAME, EMAIL, PHONE, LOC, TOOL, NOTE, PRIV.
- **BYE**: specifies abandoning of an RTP session.
- **APP**: application-specific packets.

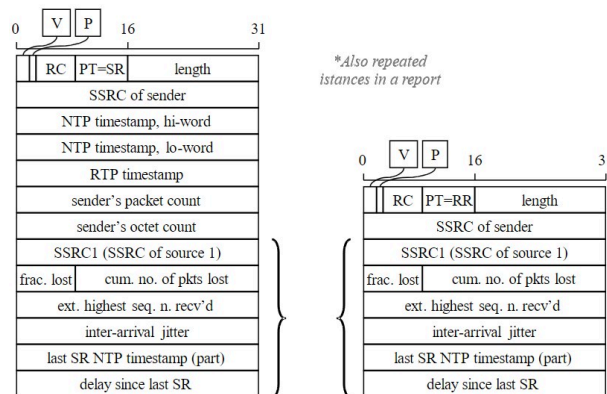


Figure 86 – RTCP messages of RR (right) and SR (Left) type.

IntServ flow summary: RSVP prepares the path and enables resource reservations (static phase). In provisioning, frames are associated with RTP and RTCP. In case of problems, a new path negotiation can occur locally (via RSVP).

15.6. DiffServ: Differentiated Services (RFC 2474/2475)

DiffServ

DiffServ differentiates flows in **classes** handled together: greater scalability than IntServ by supporting **low-level differentiation** (work at network layer, not application layer). Do not work for each information flow separately, but **aggregate network-level classes of flows**. Suitable for legacy applications (less user involvement than IntServ).

DiffServ Classes

Example class structure:

- **Gold**: 70% bandwidth



- **Silver:** 20% bandwidth
- **Bronze:** 10% bandwidth

Or alternatively:

- **Premium:** low delay
- **Assured:** high speed, low packet loss



DiffServ Mechanism

- Classification when packet **enters** based on packet content.
- **SLA** based on classification: policy arranged between user and server.
- A flow is classified at input and inserted in the right queue (**Per-Hop Behavior, PHB**); subsequent support is automatic.
- Packet marking inside **DS byte** (Differentiated Service):
 - IPv4 ToS byte
 - IPv6 traffic-class byte
- **Traffic classifiers:** Multi-Field (MF: DS byte + other fields) or Behavior Aggregation (BA: only DS byte).



DiffServ Service Classes (RFC 3246, RFC 2579)

- **Expedited Forwarding (EF):** routers keep at least two differentiated queues, guarantee delivery of expedited packets in every hop (PHB: low loss, low delay, low jitter). Creates point-to-point connection like a shared line between endpoints.
- **Assured Forwarding (AF):** four priority classes with three service levels in case of congestion (low, medium, high). Different packets labeled and processed with differentiated strategies.

15.6.1. QoS Traffic Conditioning



Traffic Conditioning Components

- **Packet Classifier:** identifies flows and assigns to appropriate class.
- **Meter:** measures traffic profile (in-profile vs out-of-profile).
- **Marker:** re-marks packets with new DS codepoint (reconditioning).
- **Dropper:** discards out-of-profile packets.
- **Shaper:** delays out-of-profile packets to smooth bursts.

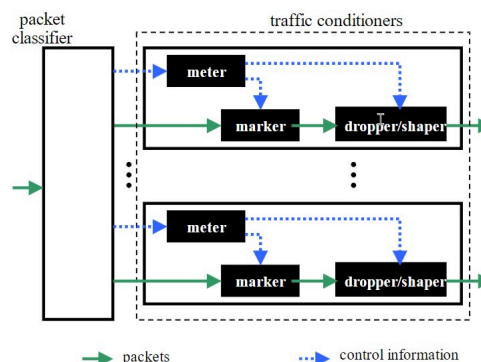


Figure 87 – QoS Meter, Marker, Dropper/Shaper.



15.6.2. IntServ vs. DiffServ

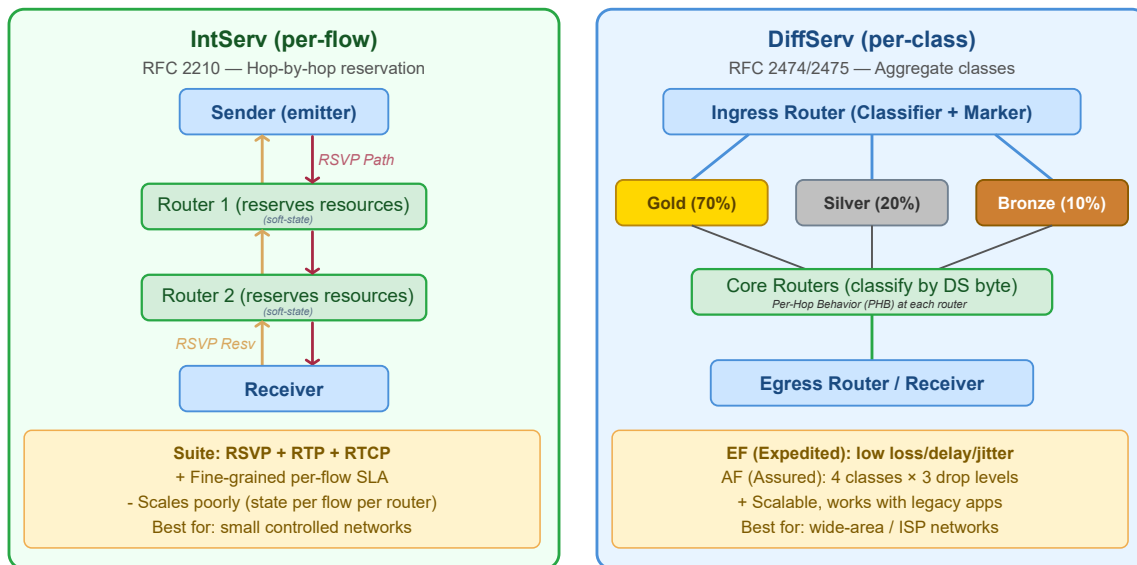


Figure 88 - IntServ (per-flow RSVP reservation, hop-by-hop) vs. DiffServ (aggregate class queues, DS byte marking).

15.6.3. IntServ + DiffServ Together

Both approaches cooperate.

- DiffServ is more scalable and supports legacy services.
- IntServ can grant specific QoS to specific flows.

Often joined in connected areas: **IntServ** used in smaller, controlled domains, while **DiffServ** in the wider Internet between domains.

15.7. SIP: Session Initiation Protocol (RFC 2543/3261)



SIP

SIP defines and manages **sessions** to support multimedia services. Has signaling capability for **establishing, modifying, and closing** multimedia sessions. Based on HTTP-compatible content: a **text-based, purely client/server** protocol. SIP itself does not carry media (other protocols do like RTP).



SIP Entities

- **User Agent**: endpoints that act as User Agent Client (REQUEST) or Server (RESPONSE).
- **Proxy Server**: application-level routers, can keep session transaction state (stateful) or be stateless.
- **Redirect Server**: sends a client to a new alternative server.
- **Registrar Service**: user registration to infrastructure.
- **Location Service**: links interested users to their location.



SIP Messages



REQUEST messages: INVITE , ACK , CANCEL , BYE , REGISTER , OPTIONS , PRACK , SUBSCRIBE , NOTIFY , PUBLISH , INFO , REFER , MESSAGE , UPDATE .

RESPONSE messages:

- 1xx : Provisional
- 2xx : Success
- 3xx : Redirection
- 4xx-6xx : Failure

SIP message structure: start-line (request-line or status-line), headers, optional message body. The body can contain SDP (Session Description Protocol) for audio/video format negotiation.



Figure 89 – SIP flow.

15.8. Network Management

We need dynamic data collection mechanisms and policies that do **not require too many resources** (minimum intrusion principle). Any correct management must reserve as few resources as possible. Performance area (monitor and data management) must define tools and policies that are **the least intrusive as possible**.

Management functional areas (FCAPS):

- Fault Management
- Configuration Management
- Accounting Management
- Performance Management
- Security Management

Standards: OSI/ISO (CMIB, CMISE), SNMP/IETF, TINA/CCITT.

15.8.1. SNMP: Simple Network Management Protocol

SNMP

SNMP uses one **manager** (only one) and some predefined **agents** that control variables representing objects, identified by unique names (**OID** in hierarchical directories). Manager requests actions (**get** , **set** , **getNext**) and receives responses. Agents wait for requests and can also send **traps**. Uses UDP (Port 161 for messages, Port 162 for traps).

SNMP Standards

- **SMI** (Structure of Management Information): defines rules for object names (ASN.1 and BER).



- **MIB** (Management Information Base): objects, types, and relationship collections (according to OSI X.500).



SNMP Versions

- **SNMPv1**: extremely simple, limited expressivity, only configuration/fault management, limited traps.
- **SNMPv2**: overcomes C/S manager-agent hierarchy with **proxy agents** (act as both agent and manager), solving the **micro-management** congestion problem. Manager orders operations; proxies actuate them locally and send aggregated results.
- **SNMPv3**: adds security (S-SNMP): integrity, masquerading prevention, privacy (prevent disclosure). Denial of service and traffic analysis not dealt with.

15.8.2. RMON: Remote Monitor



RMON

RMON controls the support parts of the communication and allows access to related statistics: oriented toward **traffic and bandwidth**, not toward devices. Introduces **probe** entities capable of monitoring packets on the network autonomously (even disconnected from the manager), tracking subsystems and reporting filtered information to the manager.

- RMON1: multiple and grafted operations.
- RMON2: guaranteed security.

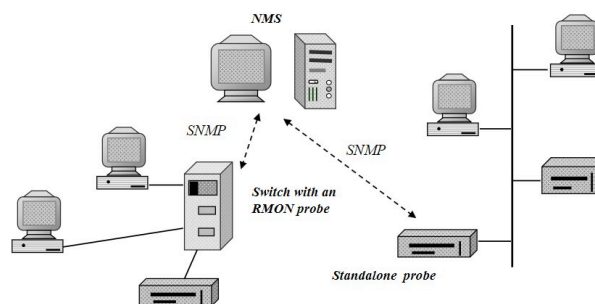


Figure 90 - RMON probe.

15.8.3. OSI Advanced Network Management

OSI Management provides a more sophisticated model with three roles:

- **Manager**: active entity issuing management policies.
- **Agents**: intermediate entities acting on manager requests (can themselves be managers, flexible hierarchy).
- **Managed Objects**: abstract representations of resources (simple or complex, created dynamically).

Protocol **CMISE/P** provides remote operations: **Set-Modify**, **Get/Cancel Get**, **Action**, **Create/Delete**, **Event Report**. Supports **dynamic addition** of attributes, actions, agents, and events during execution (also deletion).



15.9. Router QoS Policies

15.9.1. Best-Effort Router (FIFO)

The standard Internet router (best-effort) executes for every packet:

1. Verification of the destination.
2. Access to routing tables to find output path.
3. Select the best output path (maximum match length).
4. Forward the packet to the selected interface.

Simple FIFO policy: unique queue for every flow, excludes any service differentiation. Cannot reserve resources for flows with different SLAs.

15.9.2. Kleinrock Conservation Law



Kleinrock Conservation Law

For **work-conservative routers** (cannot be idle if there are packets on any port and cannot postpone arriving messages): given n flows with λ_n traffic $\forall$ flow, if flow n has service mean time μ_n , usage $\rho_n = \lambda_n \mu_n$, and mean waiting time q_n :

$$\sum_n \rho_n q_n = \text{Constant}$$

$\Rightarrow$ to give a lower delay or higher bandwidth to one flow, you **must** increase the delay or reduce the bandwidth of another.



Work Conservation

A router in Internet **must** work according to work conservation: it cannot decide to postpone any message arriving. Any QoS router must break this by **conditioning traffic**: introducing monitoring and making actions to decide more sophisticated service policies (delay some packets, discard some packets).

15.9.3. Bucket Models for Traffic Shaping



Leaky Bucket

Leaky Bucket: the router has limited memory (capacity C) and limited output flow (R). Models a router **actively shaping** services by limiting output flows:

- If data arrive too quickly beyond admissible output flow $R \Rightarrow$ **delayed**.
- If data arrive beyond capacity $C \Rightarrow$ **lost**.

Aims at **switching off packet bursts**: smooths traffic to an admissible level.



Token Bucket

Token Bucket: tokens are generated uniformly by time ticking (r tokens/sec, capacity C). Each packet needs a token to pass, stored in the bucket:



- If bucket empty $\Rightarrow$ packet waits.
- If bucket full $\Rightarrow$ tokens available for packets.
- Unlike leaky bucket: data beyond capacity are **not lost but only delayed**.
- Allows **packet bursts**: if tokens have accumulated, a burst can pass immediately.

Models flows **history** via tokens as authorization for passing.

Analogy: Leaky vs Token Bucket

Leaky bucket: a bucket with a hole; water (packets) drips out at a constant rate regardless of input. If too much comes in, it overflows (drops).

Token bucket: a ticket machine where you need a ticket (token) to send a packet. Tokens accumulate when traffic is low, allowing bursts when traffic spikes.

Note: Often leaky bucket and token bucket are used in **serial chain**. The token bucket allows bursts, while the leaky bucket smooths the output.

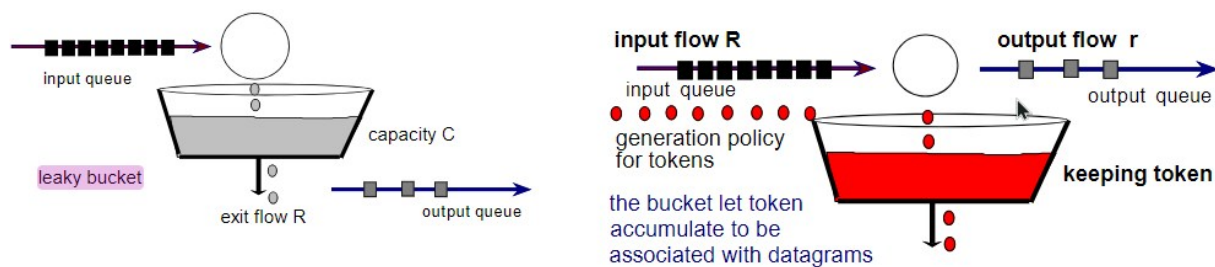


Figure 91 - Leaky bucket (left) vs. token bucket (right).

15.9.4 Scheduling Policies



Properties Required of Scheduling Policies

- **Implementation facility:** easy router design toward real feasibility.
- **Fairness and Protection:** any flow penalized same as others in same operational situation.
- **Performance limits:** constraints on correct flow operation.
- **Admission Control:** decision on admission before distribution.

15.9.4.1. Max-Min Fairness and GPS



Max-Min Fairness

General strategy: requests of different resources by different flows considered **in order of growing request** (first the ones that require less). Allocates to flow n :

$$m_n = \min(X_n, M_n) \quad M_n = \frac{C - \sum_{i=1}^{n-1} m_i}{N - n + 1},$$

where:

- C is the global max capacity of resources,
- X_n are resources request by flow n ($X_1 < X_2 < \dots < X_N$),
- m_n are previously allocated resources with success to flow n ,
- M_n are available resources for flow n



Scaling down done only in lack of resources.

👉 **Note:** It is also possible to consider different weights for different flows.



GPS: Generalized Processor Scheduling Fluid Model

Fluid traffic model: answers service requests one at a time in a very fair Round Robin order: serves only **one bit per flow** per round. Theoretically optimal for service scheduling, but **not implementable** in reality (can only serve packets, not bits). All practical policies are approximations of GPS.

👉 **Note:** GPS is not implemented in reality.

15.9.4.2 Round Robin Variants



Round Robin Policies

- **Round Robin (RR):** flows served cyclically, one packet per flow per round. Fair but does not consider packet size or flow demands.
- **Weighted Round Robin (WRR):** flows served by round-robin in proportion to assigned weight. Every queue visited a number of times per round equal to the weight.
- **Deficit Round Robin (DRR):** each flow has a deficit counter. Packet extracted if less than threshold length; otherwise waits a number of rounds proportional to its length (augmenting deficit by a specific amount per visit). Works well with limited flows and small packets on average.

15.9.4.3 Fair Queuing



Fair Queuing

Based on GPS principle applied per-packet: messages assigned **tags** for message end in every queue. Packet selected for output is the one that would **complete service first** (if it were per-bit service). A packet of size N in a flow can output only after visiting all other queues N times.

- More suitable and simple to implement: available on all routers including low-cost.
- **Weighted Fair Queuing (WFQ):** different weights associated to different flows.

- **GPS (Theoretical Fluid Model):** Serves queues simultaneously **per-bit** (like water splitting into parallel pipes). It provides perfect, instantaneous fairness but is physically impossible on real packet-switched hardware.
- **FQ (Practical Packet Model):** Emulates GPS by transmitting **one full packet at a time**. It runs a virtual GPS clock in the background and tags each incoming packet with its **virtual finishing time** (when it would finish service under GPS). The router then outputs packets sorted by the lowest tag first.

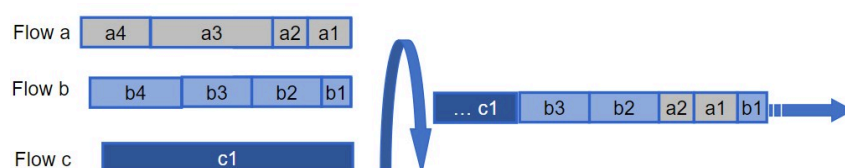


Figure 92 – Fair queuing example.



15.9.5. Congestion Prevention



RED: Random Early Detection

RED is a **proactive** congestion prevention policy: a queue for every flow, queues with equal priority. Randomly discards packets **before** congestion occurs, based on queue length:

- Queue < minimum threshold: no action.
- Queue > maximum threshold: all new packets discarded.
- Otherwise: discard with **probability proportional to queue length**.

Success: packets are randomly discarded more and more as queues grow, preventing congestion before it becomes severe.



Reactive vs Proactive

Traditional best-effort Internet only allows **reactive** actions: discard excess packets (silently) or send choke packets. QoS-aware Internet enables **preventive (proactive)** actions such as RED, transmission windows on channels, or other strategies that prevent dangerous congestion situations.

15.9.6. Service Levels Summary

Service Level	Characteristics	Use Cases
Best-effort	No guaranteed throughput, possible delays, no duplication control	Elastic Internet services
Controlled load	Similar to best-effort with low load, some delay limits	Elastic services, tolerant real-time
Guaranteed load	Tight delay limits, maximum guarantees on flows	Non-tolerant real-time services



16. OVERLAY NETWORKS AND FILE SYSTEMS

Package: Overlay Networks and File Systems - 10 - ON File systems 26 (2).pdf

Many situations require a **logical connection among different, geographically distributed entities** spread over different locations and networks. The solution is an **Overlay Network (ON)**: a network built **at the application level** that connects all those entities and makes them behave as a group.

Overlay networks may have **very different goals** but share core requirements:

- **Efficiency**: minimize overhead
- **DynamiCity**: handle nodes joining and leaving at any time
- **Scalability**: work at large scale
- **QoS**: grant agreed service levels



Overlay Network (ON)

An **Overlay Network** is a **logical network built at the application level** on top of an existing physical/IP network. It groups a set of distributed entities so that they can **freely communicate as if they had a real network connection**, using an **application neighborhood** defined by overlay edges, not physical links.

Analogy: VPN as Overlay

A Virtual Private Network (VPN) is the classic ON example: geographically dispersed branch offices, traveling employees, and home users all appear to be on the same corporate LAN, even though the underlying connections go through the public internet. The **overlay edge** is the IPSec/SSL tunnel, invisible to the IP layer.

16.1. Classification of Overlay Networks

There are two fundamentally different kinds of overlay networks, distinguished by how new nodes are admitted:

- **Unstructured overLays**: new nodes choose the neighbor to use **randomly**. The network topology is arbitrary and grows freely. Worst cases and bottlenecks can appear. Examples: Napster, Gnutella, Kazaa, BitTorrent.
- **Structured overLays**: there is a **precise strategy** to let nodes join, to **organize the architecture**, and to react to failures and discontinuities. Examples: Chord, Pastry, CAN.

👉 **Note**: ONs are used not only for P2P applications but also for Message Oriented Middlewares (MOMs), social networks, and cloud infrastructure coordination, wherever scalable, self-organizing discovery of distributed resources is needed.

16.1.1. Overlay Network Usage and Node Lifecycle

A good overlay network makes **operations among the group of current participants efficient** while answering specific requirements. All participants share a **common goal of exchanging information**: files in P2P, messages in social nets, etc.

Every node in an ON follows a lifecycle:

1. **Get in**: join the network



2. **Make its actions:** participate, serve queries
3. **Help actions of others:** route, replicate, forward
4. **Get out:** leave (gracefully or by failure)

16.1.2. Key Management Requirements

Running an overlay network is harder than running a static network because the membership changes constantly. Every practical ON must address:

- **Maintaining edge links:** each node keeps a routing table of neighbor IP addresses. These must be kept fresh as nodes move or change address.
- **Favoring insertion:** when a new node joins, it must find and connect to appropriate neighbors quickly, without disrupting the existing structure.
- **Checking liveness:** nodes periodically ping their neighbors (heartbeats) to detect silent failures before they affect routing.
- **Identifying and recovering from faults:** when a neighbor fails, its edges must be detected as stale and replaced with alternative paths.
- **Handling churn:** nodes join and leave (or crash) continuously. A well-designed ON restructures itself after each event in $O(\log N)$ operations, not $O(N)$.
- **Maintaining structure under failures:** even when multiple nodes are absent simultaneously, the overlay must remain connected and correctly routed.
- **Robustness to omissions:** message losses should be tolerable; the overlay should use redundant paths so that the failure of one route does not silence a node.



Two Fundamental ON Properties

- **DynamiCity** of supporting nodes: nodes can get in and out at any time, even by crashes
- **Replication** of resources: data must be available independently of any event pattern on the ON

16.2. Unstructured Overlays

16.2.1. Napster (1991): Centralized Lookup

Napster was the first large-scale P2P file-sharing system. It used a **non-structured** approach with a **centralized directory**:

- Any node connects to a Napster server and uploads its file list
- Search is handled by the centralized server (lookup in the central index)
- File exchange is done **peer-to-peer**, only lookup is centralized
- The “best” correct answers are selected and announced via ping messages



Napster Bottleneck

Centralized lookup = **single point of failure** and congestion. The central directory becomes the performance bottleneck, leading to **low scalability**. This was the fundamental weakness that led to fully decentralized designs.



16.2.2. Gnutella (2000): Flooding-Based Unstructured ON

Gnutella is the main representative of **unstructured ONs**: providing a **fully distributed** approach with no central coordinator.

- Any node entering Gnutella tries to connect to some others locally available
- **Fully decentralized** organization and lookup
- Nodes have **different degrees** of connections (high-degree vs. low-degree nodes)
- High-degree nodes may receive and control more links

16.2.2.1. Gnutella Join and Search Protocol

Step 0: Join the network: contact any known node.

Step 1: Determine who is on the network:

- A "Ping" packet announces your presence
- Other peers respond with a "Pong" packet containing: IP address, port number, amount of data shared
- Pong packets return via the same route as Ping

Step 2: Search:

- Gnutella "Query" asks other peers ($N = 7$ typically) for desired files
- A Query packet asks: "Do you have any matching content with the string 'Volare'?"
- Peers check for matches and respond (send "QueryHit" if matched), forwarding to connected peers if not ($N = 7$ typically)
- The TTL (Time-To-Live) field limits the number of hops a packet can traverse (typically 10)

Step 3: Downloading:

- Peers respond with a "QueryHit" containing contact information
- File transfer uses direct HTTP GET connection



Gnutella Scalability Problem

Flooding-based search is extremely **wasteful in bandwidth**:

- Enormous number of **redundant messages**
- A large (linear) part of the network is covered irrespective of hits found, **without taking into account actual needs**
- All users search in parallel: local load grows **linearly with system size**

16.2.2.2. Gnutella: Scale-Free Networks and Topology Control

The Gnutella network exhibits a **scale-free** topology following a **power law** (or exponential law) degree distribution: a **few nodes are highly connected** and many nodes have low degree. This is very different from a random network.



Scale-Free Network

A **scale-free graph** is a graph whose degree distribution follows a **power law** $P(k) \sim k^{-\tau}$ with $\tau \approx 2.07$ in real Gnutella deployments. A few **hub nodes** have a very large neighborhood of low-degree ones. Hub nodes can store an index for a large portion of the network.

Degree-biased random walk:

- Select **highest degree (hub) nodes** that have not been visited



- Walk first climbs to highest degree nodes, then climbs down into the vicinity
- Optimal coverage can be **formally proved**

16.2.2.3. Gnutella Replication

To improve search hit rates, objects are replicated across multiple nodes. Three strategies exist, each with different trade-offs:

- **Owner replication:** the original owner creates a number of replicas proportional to q_i (the query frequency for object i). Popular objects get more copies; rare objects stay rare. Simple but requires the owner to know global demand.
- **Path replication:** every node along the query path that returns a hit stores a copy. The number of replicas grows proportionally to $\sqrt{q_i}$, naturally placing copies near where demand is coming from. More balanced than owner replication.
- **Random replication:** same replication factor as path replication ($\sqrt{q_i}$), but replicas are placed on randomly chosen nodes rather than the query path. Easier to implement but less locality-aware.

👉 **Note:** Replication helps popular objects be found more easily, since more copies are spread across the network. However, **rare objects remain difficult to find** regardless of which strategy is used: in an unstructured overlay, a query for an object with very few copies must traverse many nodes before hitting one. This is the fundamental scalability limitation of Gnutella-like systems.

16.3. Structured Overlays: Distributed Hash Tables

Unstructured P2P networks allow resources to be placed at any node spontaneously but suffer from bottlenecks and unpredictable behavior. **Structured P2P networks** simplify resource location and load balancing by defining a **topology** and **rules for resource placement**, enabling efficient search even for rare objects.

The key technology: **Distributed Hash Table (DHT)**.

Distributed Hash Table (DHT)

A **DHT** uses hash principles to enable **efficient retrieval of data content and values in a distributed setting**. The hash function maps keys to nodes in the overlay:

- `put(key, value)` : store a key-value pair
- `value = get(key)` : retrieve by key

The key insight: **partitioning the whole key space** over available nodes in a ring-like structure, where each node is responsible for a contiguous range of keys.

Analogy: DHT as a Distributed Dictionary

A regular hash table assigns array slots via `h(key)`. A DHT does the same but the “array” is **spread across thousands of machines**. Instead of an array slot, `h(key)` tells you **which node** is responsible for storing that key-value pair. Nodes can join and leave, so the mapping is dynamic: that’s the hard part.

16.3.1. Key DHT Principles

- **Partitioning:** the whole key space is divided into ranges, each assigned to a node
- **Replication:** entries are replicated on multiple neighbor nodes to increase availability



- **Load balancing:** when nodes change, re-map keys (only $O(1/N)$ fraction of keys move)
- **Self-organization:** nodes cooperate to maintain the structure without any central coordinator

16.3.2. Chord



Chord

Chord (2001) is a DHT based on a **consistent hashing ring-based structure**. Both **keys** and **nodes** are hashed to **160-bit IDs** using SHA-1 (from 0 to $2^{160} - 1$). Keys are assigned to nodes by using **consistent hashing**: the key is stored in the **successor node** available in the ID space (the first node with ID $\geq$ key, going clockwise).

N nodes - K keys:

- Both keys and nodes hashed to 160-bit IDs
- Key is stored at the **successor** in the ring (clockwise first live node with ID $\geq$ key ID)

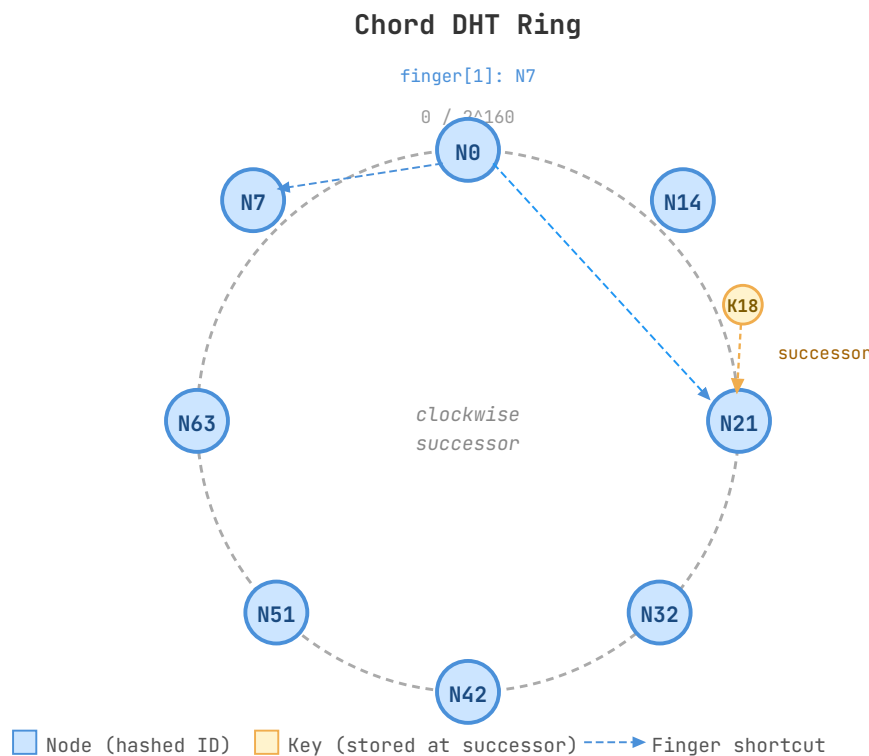


Figure 93 – Chord DHT ring: nodes placed on a 160-bit ID circle; each key is stored at its clockwise successor. Finger table shortcuts give $O(\log N)$ lookup.

16.3.2.1. Chord Primitive Lookup

The simplest lookup just **forwards the query around the ring in one direction**:

- Lookup query arrives from any node, forwarded to successor in one direction
- In the worst case, $O(N)$ forwarding is required (full traversal of ring)
- By using both directions: $O(N/2)$, still linear



16.3.2.2 Chord Efficient Lookup: Finger Tables

To achieve scalable lookup, each node maintains a **finger table**: a set of shortcuts to nodes at exponentially increasing distances in the ring.



Finger Table

The i -th entry of a node's **finger table** points to the successor of $(\text{nodeID} + 2^i)$ in the ID space. A finger table has $O(\log N)$ entries, and lookup is **bounded to $O(\log N)$ message complexity**.

- When a new node joins, it must create its own finger table **and also every neighbor node must update its own finger table** (same for predecessor and successor)
- This coordination cost is $O(\log N)$ per join

16.3.2.3 Chord Consistent Hashing Properties

- **Randomized**: all nodes receive roughly an equal share of load
- **Local**: adding or removing a node involves an $O(1/N)$ fraction of keys getting new locations, minimal disruption
- Cost of lookup via finger tables: Chord needs to know only $O(\log N)$ nodes in addition to successor and predecessor to achieve **$O(\log N)$ message complexity** for lookup

16.3.2.4 Chord Node Join and Stabilization

Node join:

- A new node finds its **successor**, then stabilizes
- The node joins immediately (lookup already works), then modifies the structure lazily
- Stabilization: **each node periodically runs the stabilization routine and refreshes all fingers** by calling `find_successor(n+2i-1)` for a random i
- Periodic cost: **$O(\log N)$ per node** due to finger refresh

Failure handling:

- Instead of one successor, each node keeps **R successors** (replication)
- More robust to node failure (can find new successor if old one failed)
- In robust DHTs, keys **replicate on the R successor nodes** of any node
- **Alternate paths while routing**: if a finger does not respond, take the previous finger or replicas if close enough

16.3.3 Pastry



Pastry

Pastry (2001) is a DHT overlay network, like Chord, but **differently organized for efficient access**. Based on a sorted ring in an ID space where nodes and objects are assigned **128-bit identifiers** (NodeIDs and objIDs are uniform random). The node responsible for a key is the one **numerically closest** in hex, not just the successor as in Chord. Pastry exploits **nested groups** for neighborhood and replication.

NodeID is interpreted as a **sequence of digits in base 2^b** (not bits). With $b = 4$, names are viewed as **successions of hex digits** (base 16).



16.3.3.1. Pastry Routing Tables

Each Pastry node maintains two data structures:

- **Routing tables** (numeric neighborhood): identify nodes that are **numerically close** in ID space. $\log_{\{16\}} N$ rows, each row containing nodes that share a progressively longer common prefix with the current nodeID. Entries in the m -th column have m -th digit as next digit; n -th row shares first n digits with current node.
- **Leaf sets** (vicinity): maintain IP addresses of nodes with **closest larger and smaller nodeIDs** in the close neighborhood. Used as replication boundary and stop condition for lookup.

Pastry properties:

- Lookup/insert in $< \log_{\{16\}} N$ routing steps (expected)
- **$O(\log N)$ per-node state**
- Network proximity routing: routing table rows prefer topologically close nodes

16.3.3.2. Pastry Insert / Lookup

A message with key X is routed to the live node with nodeID **numerically closest** to X . A complete routing table is not feasible, so Pastry uses its routing table to **progressively match more digits** of the target key at each hop: each hop brings you one digit closer in the hex ID space.

👉 **Note:** Routing in Pastry uses **two spaces simultaneously**: the **nodeID space** (for correctness) and the **proximity space** (for efficiency). Each routing step makes a **smaller and smaller numerical jump** but a **bigger and bigger topological jump** toward the target in the actual network.

16.3.3.3. Pastry Join and Failures

Join:

- Uses routing to find **numerically closest nodes** already in the network
- Asks state from all nodes on the route and initializes its own state
- Operation is **efficient and smooth**

Failures:

- Leaf set members exchange **keep-alive messages**
- **Leaf set repair (eager)**: contact a leaf node on the side of the failed node and add an appropriate new neighbor
- **Routing table repair (lazy)**: contact a live entry with same prefix as the failed entry; if none found, try longer prefix entries; get table from peers in the same row, then higher rows

16.3.4. DHT Applications in Practice

DHTs are used widely:

- **P2P file sharing**: Napster, Gnutella, Kazaa, BitTorrent
- **Social networks**: MSN, Skype, Social Networking Support (fast user discovery)
- **Cloud infrastructure**: for internal and federated discovery - *Cassandra* and *ZooKeeper* are Chord-based (*Cassandra*) or DHT-inspired (*ZooKeeper* uses ZAB, inspired by Paxos but with ring-based partitioning)
- **MOMs and middleware**: any system needing scalable, self-organizing discovery



16.4. Distributed File Systems

16.4.1. NFS: Network File System

Network File System (NFS)

NFS is the pioneer **client/server** file system and the most widespread network file system. It is based on the idea of **client** machines that **interact** with **server** machines where files **reside**. After mounting, the implementation is **transparent**: any client can access server files as if they were local.

NFS Design Trade-off

NFS was designed for **efficiency** and **cost reduction** at the expense of consistency and global view. The client is **stateful**: it must keep track of operations on files. If the server goes down, nothing is notified and the client cannot manage single file state and operations. **No replication, no QoS, no global shared view.**

NFS architecture:

- **Stateless and efficient**: there is **no heavy weight** on **server** machines; the load is on the client
- Uses **UDP** connections (and many TCP variations)
- Based on **RPC** for the entire communication support: efficient, low overhead

NFS limitation: a client can mount from **several** servers, giving each client a **different global view** (no uniform namespace), on top of the no-replication, no-QoS constraints already noted.

16.4.2. AFS: Andrew File System - Quality File Systems

Other file systems introduced **global quality** in the sense of **replication** and **uniformity in view** of the file system.

Andrew File System (AFS)

AFS provides:

- **Same view** of the file system independently of where the user is accessing from
- **Files are replicated**, so even if some nodes are not available, the contents are always available
- Replication is **dynamically managed**: the file is **cached at the client site** and more copies can be added if several clients are asking for the same contents (and deleted)
- The client uses a **call-back** to signal to the server any possible change actions (typically one per writing)
- Clients can access from any possible OS support, with many **additional services** provided
- Designed for **expected more reads than writes**

16.4.3. Modern Global File Systems

Modern **global** systems need new tools for data storage with **global scalability** and **quality**. Starting from traditional C/S file systems (similar to NFS), they move to **replication** and **dynamic management of data in all their parts** to achieve better QoS.



Major global distributed file systems:

- **Google File System (GFS)**: for Google data
- **Hadoop Distributed File System (HDFS)**: open source, analogue of GFS

16.5. Google File System (GFS)

GFS exploits **Google hardware, data, and application properties** to improve performance of **storage and search at scale**.

16.5.1. GFS Design Assumptions

GFS is designed around the specific workload and failure profile of Google's infrastructure:

- **Large scale, large files**: thousands of machines and disks; a "limited" number (a few millions) of **huge files** (100 MB to multi-GB), few small files
- **Read/append workload (almost no random write)**: mostly large sequential streaming reads (multi-MB) and large sequential appends; random overwrites are practically non-existent (new data is appended, not overwritten)
- **Component failures are the norm**: with hundreds of thousands of machines/disks (MTBF 3 years/disk $\rightarrow$ 100 disk failures/day), plus network, memory, and power failures, the system must **detect, tolerate, and recover automatically**
- **Atomic consistency for concurrent appends** at low overhead: every write covers a full block, so one block = one atomic write report even under parallel append
- **Sustained throughput** matters far more than **low latency**

16.5.2. GFS Design Novel Strategies



GFS Architecture

- **One master server** (backups replicate its state) and **many chunk servers** (100s-1000s) over Linux
- **Chunk**: 64 MB portion of a file, identified by a 64-bit globally unique ID. Chunks spread across racks for better throughput and fault tolerance (5 copies max)
- **Single master** coordinates access and keeps **metadata** (file/chunk namespaces, file-to-chunk mappings, location of replicas)
- **Files stored as chunks kept with their descriptions** (metadata) and stored as local files on Linux file system
- Reliability through **replication** (at least 3+ replicas)

Key design decisions:

- **Simple centralized design** (one master per GFS cluster)
- Global knowledge to optimize **chunk placement and replication decisions** using **no caching** (large data set/streaming reads render caching useless)
- Clients cache **metadata** (e.g., chunk locations)
- Linux buffer cache allows keeping interesting data in memory for fast access

16.5.2.1. GFS Metadata

All metadata is kept **in memory** (< 64 bytes per chunk), limiting total GFS capacity but enabling fast lookups.

Large chunks have many advantages:

- Fewer **client-master interactions** and reduced size of metadata



- Enable persistent **TCP connections** between clients and chunk servers

16.5.2.2 GFS Operations

- **Normal operations:** stream reads, going on in parallel and reading any copy
- **Rare operations:** writes, stream append write (can go in parallel if different space)
- **Even rarer:** random overwriting writes, done with some cautions
- **Control operations:** metadata changes, must be done **sequentially** (write operations called **mutations**)

16.5.2.3. GFS Step-by-Step Write (Mutation) Protocol

The write protocol carefully separates **control flow** from **data flow**:

1. Client asks master for **identities of primary chunk server holding lease** and secondaries holding other replicas
2. Master **replies** with chunk locations
3. Client **pushes data to all replicas** for consistency (pipelined)
4. Client sends **mutation request to primary**, which assigns it a **serial number**
5. Primary **forwards mutation request to all secondaries**, which apply it according to the serial number
6. Secondaries **Ack completion**
7. **Reply to client:** an error in any replica results in an error code and a client retry

Analogy: GFS Write as a Chain of Responsibility

*Think of the write protocol as a **relay race**: the client hands the data to the closest replica, which passes it to the next closest (in network topology), which passes it onward. Meanwhile, the **primary acts as the sergeant**: it assigns a serial number to ensure all replicas apply mutations **in the same order**. Control and data travel different paths for efficiency.*

16.5.2.4. Data Flow for Pipe Write

- Client can push data to **any** replica
- Data is pushed **linearly along a carefully picked chain** of chunk servers
- Each machine forwards data to the “closest” machine in network topology that has not yet received it
- **Pipelining:** servers receive and send data at the same time
- Method introduces delay but offers good **bandwidth utilization**

16.5.2.5. Mutations, Leases, and Version Numbers

- **Mutation:** write operation that changes either the **contents** (write, append) or **metadata** (create, delete) of a chunk
- **Lease:** mechanism to maintain **consistent mutation order across replicas**
 - Master grants a **chunk lease** to one replica (Primary chunk server)
 - Primary picks a **serial order** to all mutations to the chunk
 - All replicas follow this order when applying mutations
- **Version numbers:** chunks have version numbers to distinguish **up-to-date and stale replicas**; each time master grants new lease, increments version and informs all replicas



16.5.2.6. GFS Consistency Model



GFS Consistency States

- **Consistent:** all clients see the same data regardless of which replica they read
- **Defined:** consistent AND client sees the mutation in its entirety
- **Consistent but undefined:** example: initial record = AAAA, concurrent writes _B and CC_C; result = CBAC (none of the clients sees the expected result)
- **Inconsistent:** due to a failed mutation, clients see different data from different replicas

File namespace mutations (create/delete) are **atomic**. For file regions: the state depends on success/failure of mutations and existence of concurrent mutations.

In case of inconsistent data or undefined, a data reconciliation must run.

16.5.2.7. Record Append Semantics and Undefined State Avoidance

Traditional random writes would require expensive synchronization (e.g., lock manager): serializing writes does not help because of undefined interleaving.

Atomic record append: allows multiple clients to append data to the same file concurrently:

- Serializing append operations **at the primary** solves the problem
- The result of successful operations is well defined: **data is written at the same offset by all replicas with “at least once” semantics**
- If one operation fails at any replica, the client retries: replicas may contain duplicates or fragments
- If not enough space in chunk, add padding and return error: client retries

Applications using record append should include **checksums** in writing records. The reader can identify padding/record fragments using checksums. If the application cannot tolerate duplicated records, it should include a **unique ID** in records; readers use unique IDs to filter duplicates.

16.6. HDFS: Hadoop Distributed File System



HDFS

HDFS (Hadoop Distributed File System) is **inspired by GFS** but **open source**. It follows the same master/slave architecture and is based on **low cost hardware** with **high fault tolerance** and **high availability**. Applications access with a **write-once-and-read-many** model, the consistency model is similar to GFS, and **computation is moved close to the related data**.

16.6.1. HDFS Architecture

Master/slave architecture:

- **NameNode** is the master: executes file system **Namespace operations** (open, close, directories...) and decides on **mapping** of blocks to DataNodes. Manages metadata: keeps FSImage and EditLog in memory.
- **DataNodes** are slaves: one per node in the cluster. Execute **read/write operations** requested from clients and operate on blocks of data. DataNodes store blocks as files on the local Linux file system.



Files:

- Stored in **blocks** in several DataNodes
- Each file decides its **block size** and its own **replication degree**
- HDFS is **written in Java** and must work on normal hardware to store very large files on different machines

16.6.2 HDFS Replication

- Applications can **dynamically decide the replication factor**
- NameNode receives **heartbeats** and **block reports** from DataNodes:
 - **Heartbeats**: grant the operation state of DataNodes
 - **Block reports**: give the current block situations of DataNodes
- The NameNode stores: `(Filename, numReplicas, block-ids, ...)` - e.g., `/users/sameerp/data/part-0, r:2, {1,3},...`

👉 **Note**: HDFS uses a rack-aware replication strategy: one replica on the local rack, one on a different rack, balancing fault tolerance (rack failure) against write bandwidth (fewer cross-rack writes).

16.7. Summary: Properties Behind These Building Blocks

Overlay Networks and Distributed File Systems are **global tools to support new global systems**. They are **black boxes** used by any global application and **mechanisms (bricks)** to build other tools.

These systems are designed to favor strategic choices in tension with traditional approaches:

Favored	Trade-off with
Dynamicity	Staticity
Availability	Reliability (strict)
Eventual Consistency	Strict consistency (ACID)
Fast answers	Safe delayed answers

They should favor **life cycle widening**, **scalability**, **dependability**, enabling global distributed applications that must work at internet scale across thousands of nodes that join, leave, and fail continuously.



17. DATA STORAGE

Package: Data Storage - 11 - Data_storage 26.pdf

Modern global systems need **new tools for data storage of Big Data** with a renovated level of quality. We have seen distributed file systems (GFS, HDFS), but here we go further: **NoSQL** distributed storage systems built on **new principles**. These tools are within the **NoSQL (Not only SQL)** trend: not only a query language, but modern **data support** serving as the entire inspired infrastructure.

17.1. The NoSQL Movement



NoSQL

NoSQL (Not only SQL) is a general trend in addressing modern very large-dimension and big data systems. NoSQL systems can provide almost-SQL languages extended to accommodate queries with modified data definitions (JSON) and specific optimized actions. The name does not mean “no SQL at all”: it means **not only SQL**.

The movement emerged around 2010, building on earlier foundations (DHTs, P2P systems, MapReduce) and responding to the explosion of web-scale applications. All major proposals (Cassandra, MongoDB, DynamoDB, BigTable, HBase) started around that era.

17.1.1. NoSQL Motivations

Big data stores and infrastructures face a set of **novel requirements** that traditional RDBMS cannot efficiently serve:

- **Scalability**: handling billions of records across thousands of nodes
- **Efficiency of services on large volumes of data**
- **High availability and fault tolerance**: no Single Point of Failure (SPoF)
- **New data consistency strategies**: eventual consistency instead of strict ACID
- **New way of data tagging**: no schema required

To meet these, NoSQL systems ask for:

- More flexible, **schema-less data models**
- **Weak consistency** toward high availability and correct configuration
- **High replication in close storage** to avoid moving data around data centers
- Clever use of **distributed indices, hashing, and caching**
- Datacenter-friendly **partitioning across local and remote servers**
- A **web-friendly access** through a simple client interface



Why Why Not Just Use RDBMS??

Relational databases (MySQL, PostgreSQL) are great, but they are mismatched with today's workloads:

- Data is extremely **large and unstructured**
- Lots of **random reads and writes**
- Sometimes **write-heavy** operations
- **Foreign keys** rarely needed
- **Joins** are rare



Crucially, data and queries are often foreseeable: **you can prepare your data for the usage you want to optimize**: out-of-band preparation of data links. SQL forces you into a general join model even when you never need it.

17.1.2. Requirements of Today's Workloads

Beyond the motivations above, real deployments also demand **operational** efficiency:

- **Low TCO** (Total Cost of Operation).
- **Fewer system administrators** needed.
- **Incremental scalability**: scale out, not up.



Scale Out vs. Scale Up

- **Scale up** (vertical scalability): grow cluster capacity by replacing more powerful machines. Traditional approach; not cost-effective above the price curve sweet spot; requires replacing machines often.
- **Scale out** (horizontal scalability): incrementally grow capacity by adding more **COTS machines** (Components Off The Shelf). Cheaper and more effective over time; used by most companies running datacenters and clouds today.

17.1.3. Different NoSQL Data Models

NoSQL encompasses several distinct data models:

- **Key-Value Stores**: data managed as (key, value) pairs stored in efficient and scalable ways (typically as in DHT). Examples: Redis, Oracle NoSQL, DynamoDB, Cosmos DB, **Cassandra**.
- **Document Stores**: extended key-value stores where the value is a document encoded in standard formats (XML, JSON, BSON). Examples: **MongoDB**, CouchDB, CosmosDB, Firebase.
- **Wide-Column Stores**: data in a tabular format of rows and column families stored per-column-family dynamically and flexibly. Examples: **Cassandra**, BigTable, HyperTable, HBase.
- **Graph Stores**: graphs for storing data efficiently and providing more effective operations on connected data. Examples: Neo4j, Giraph, ArangoDB, Titan, AllegroGraph.

👉 **Note**: Cassandra appears in both Key-Value and Wide-Column categories: it is a column-based key-value store. The line between these categories is blurry.

17.2. Key-Value Abstraction



Key-Value Store

A **key-value store** is a dictionary data structure organized for easing operations by **key I/O**. Given the key, you get the content fast via `insert`, `lookup`, and `delete`. The main property is the requirement of being **distributed in deployment** and **scalable**.

Key insight: elements are immutable (there is not mostly modification; changing also tags key-insert). This is why key-value stores reuse many techniques from **Distributed Hash Tables (DHT)** in P2P systems and from tuple spaces.



Key-Value in the Real World

- **twitter.com**: Tweet id → information about tweet
- **amazon.com**: Item number → information about it
- **kayak.com**: Flight number → information about flight availability
- **yourbank.com**: Account number → information about account

17.2.1. Key-Value / NoSQL Data Model vs. RDBMS

Feature	RDBMS	NoSQL Key-Value
Schema	Strict schema, all rows complete	Unstructured: columns may be missing from some rows
Relationships	Foreign keys, joins supported	Do not always support joins or foreign keys
Tables	Flat tables	More data models, including nested tables
Queries	SQL (Structured Query Language)	API: <code>get(key)</code> / <code>put(key, value)</code> + CQL extensions
Index tables	Supported	Supported (column families in Cassandra, collections in MongoDB)
Consistency	Strong ACID	Eventual consistency (BASE)

17.2.2. Column-Oriented Storage

Traditional RDBMS store an **entire row together** on disk. NoSQL systems typically **store one column (or group of columns) together**.

🤔 **Why** Why Column-Oriented Storage??

Range searches **within a column** are fast: you do not need to fetch the entire database. For example: "Get all `blog_ids` from the `blog` table updated in the past month": search the `last_updated` column, fetch corresponding `blog_id` column, without touching other columns. This dramatically reduces I/O for analytical workloads.

👨‍🏫 **Prof. Question:** *Difference between a column-oriented and a document-oriented database?*

Column-oriented (Cassandra) stores columns together, so reading one column touches only that column, ideal for **column range queries** and **write-heavy** loads. **Document-oriented** (MongoDB) stores a whole entity in one BSON/JSON document, so a **single read** returns it with no joins, ideal for **retrieving complete documents**. Both avoid joins.



17.3. Cassandra



Apache Cassandra

Cassandra is a distributed **column-based key-value store** intended to run in a datacenter (and across multiple DCs). Originally designed at Facebook, open-sourced later, and today an Apache project. It offers an **easy-to-use setting** for Cloud support over several datacenters.

Used in production by IBM, Adobe, HP, eBay, Ericsson, Symantec, Twitter, Spotify, PBS Kids, Netflix (to track viewing position), and many others.

17.3.1. Cassandra Data Center Model

Cassandra organizes infrastructure in a core hierarchy:

- **Cluster**: the set of all possible servers in all data centers
- **DataCenter (DC)**: the set of all servers in one DC, organized as a ring and the base for replication
- **Rack**: the set of local servers in all DCs; at least one rack must be present as the configuration unit
- **Server**: the instance present on one physical server, which can contain several virtual entities
- **Virtual Node (VNode)**: a VNODE normally controlled automatically by Cassandra with a load factor $C > 1.2$; the goal is better distribution of elements inside partitions (more → better partition balance)

The configuration is **automatic and dynamic**. When there is a re-partition, a new node is created: the update changes structure so it is lighter.

17.3.2. Cassandra Architecture

Cassandra's layered architecture:

- **Cassandra API** (top): to retrieve data; **Tools**: for data managing across the infrastructure
- **Storage Layer**: creation and storage of data, with in-memory caching and R/W operation support
- **Partitioner**: creates multiple partitions (sub-tables) by multiple points of concurrence but also higher time for closure (more techniques). **Replicator**: handles replication, membership, consistency level
- **Failure Detector**: detects failed nodes. **Cluster Membership**: tracks which nodes are alive
- **Messaging Layer** (bottom): for all message exchange across the cluster through managers

17.3.3. Key-to-Server Mapping: The Ring

Cassandra uses a **Ring-based DHT** without finger tables or routing: a simple, elegant design.



Cassandra Partitioner

The **Partitioner** decides the mapping of keys to server nodes. The ring assigns a range of hash values to each node. Each key is hashed and placed on the node owning that range. The **primary replica** is the first node clockwise on the ring; **backup replicas** follow clockwise.

Analogy: The Ring

Think of a circular racetrack where each runner "owns" a section. A ball (key) thrown to any point on the track belongs to the runner whose section it lands in. Backups are just the next few runners clockwise.



Say the ring has $m = 7$ bits (128 positions). Key K13 maps to the first node clockwise with position ≥ 13 . The Partitioner walks the ring until it finds that node, making it the **primary replica**. Subsequent replicas are placed on the next nodes clockwise.

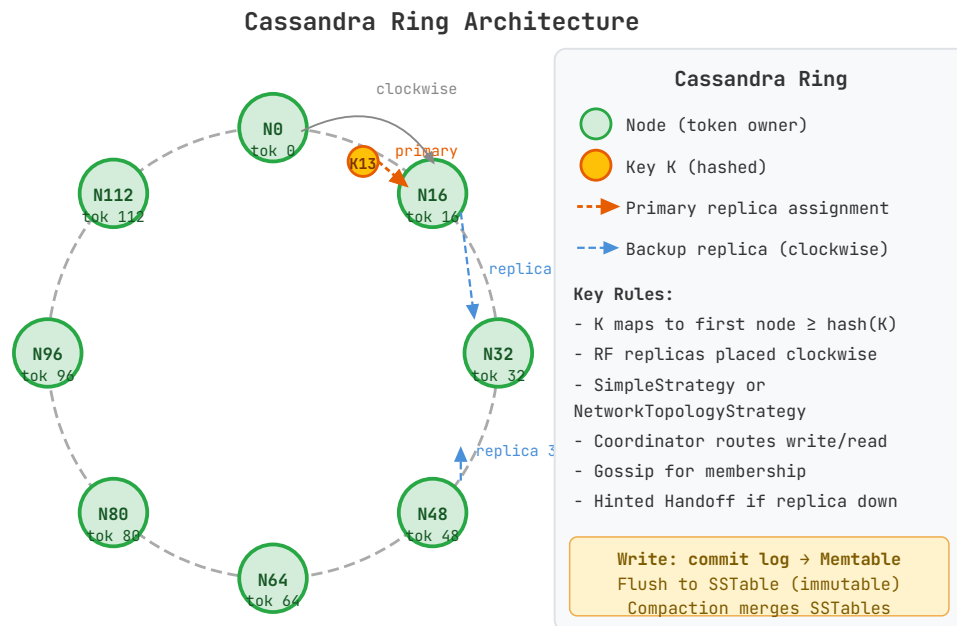


Figure 94 – Cassandra ring: nodes own token ranges; a key is routed to the first node clockwise whose token $\geq \text{hash}(\text{key})$. Replicas follow clockwise. Writes go to commit log then Memtable, flushed as immutable SSTables.

17.3.4. Cassandra Keyspaces

KeySpace (KS)

A **KeySpace** is a namespace container that defines the **data replication on nodes** and how they contain tables, the **number of replicas**, and their **replica placement strategy**.

Key properties:

- **Replication Factor (RF):** number of replicas per datacenter.
 $\text{max}(\text{RF}) = \text{max}(\text{number of nodes})$ in only one datacenter
- Heavy replicas of partitions enable more concurrent readings
- There is an overhead of updating copies to new updates: RF is a trade-off
- When there is a re-partition, a new node is created: compaction and re-partition are the most expensive operations in a distributed DB

17.3.5. Data Placement Strategy

Two replication strategies:

- **SimpleStrategy:** for single DC, with two partitioning options:
 - **RandomPartitioner:** Chord-like hash partitioning
 - **ByteOrderedPartitioner:** Assigns ranges of keys to servers (easier for range queries, e.g., "Get all Twitter users starting with [a-b]")
- **NetworkTopologyStrategy:** for multi-DC deployments:
 - Supports two or three replicas per DC



- First replica placed according to Partitioner, then go clockwise around the ring until hitting a **different rack**

17.3.6. Snitches

Snitches map IPs to racks and DCs (configured in `cassandra.yaml`):

- **SimpleSnitch**: unaware of topology (rack-unaware)
- **RackInferring**: assumes topology from IP address octets:
`101.201.202.203 = x.<DC octet>.<rack octet>.<node octet>`
- **PropertyFileSnitch**: uses a configuration file
- **EC2Snitch**: EC2 region = DC, Availability Zone = rack

17.3.7. Write Operations

Writes must be **lock-free and fast** (no reads or disk seeks).

The write path:

1. **Client** sends write to one coordinator node (per-key, per-client, or per-query)
2. **Coordinator** uses **Partitioner** to send write to all replica nodes responsible for the key
3. **When X replicas respond**, coordinator returns acknowledgement to client (**X is the majority: quorum**)



Hinted Handoff: Always Writable

If one replica is down, the coordinator writes to all other replicas and keeps the write locally until the crashed replica comes back. When **all replicas are down**, the coordinator (front end) **buffers writes** for up to a few hours. This makes Cassandra **always writable**.

For multi-DC coordination, a per-DC coordinator is elected using **Zookeeper**, which implements distributed synchronization and group services (similar to JGroups reliable multicast).

17.3.8. Writes at a Replica Node

On receiving a write, each replica:

1. **Log it in a commit log** on disk (for failure recovery)
2. **Make changes to the appropriate Memtable**
 - **Memtable**: in-memory representation of multiple key-value pairs; append-only datastructure (fast); cache searchable by key
 - **Write-back cache** (as opposed to write-through)

When the Memtable is full or old, it is **flushed to disk**:

- **Data File**: an SSTable (Sorted String Table): list of key-value pairs
- **SSTables are immutable** (once created, they never change)
- **Index file**: an SSTable of pairs (key, position in data SSTable)
- **Bloom filter**: for efficient existence checks

17.3.9. Bloom Filter



Bloom Filter



A **Bloom filter** is a compact bit-table that hints for location. It compacts the way of representing a set of items so that **checking for existence in the filter is cheaper** than searching directly. There may be some probability of **false positives** but **very low probability of false negatives**: an item not in the set may check true, but there are no false negatives.

How it works: on insert, all hashed bits are set. On “check-if-present”, return true if all hashed bits are set. With $m=4$ hash functions, 3200 bits, 100 items: false positive rate $\approx 0.02\%$.

Analogy: Bloom Filter

*Think of a bouncer with a list of VIPs. The bouncer occasionally lets in impostors (false positive), but **never** turns away a real VIP (no false negative). It is a probabilistic early rejection layer.*

100 Prof. Question: *How does data storing work in Cassandra?*

A distributed **column-oriented** store, **in-memory first**, with **no single point of failure**. Keys map to nodes on a **ring-based DHT** (consistent hashing). A write goes to the **commit log**, then the in-memory **Memtable**, later flushed to an **immutable SSTable** on disk (sorted by key). A **Bloom filter** avoids disk reads for absent keys. Consistency is **BASE** (eventually consistent), not ACID.

17.3.10. Compaction

As data updates accumulate over time, SSTables and logs need to be compacted:

- **Compaction merges SSTables** by merge-sorting updates for a key
- **Runs periodically and locally** at each server
- Old logs are kept until the update is entirely done (in case of failure, can restart); once done, tables are deleted
- The merged result is a new SSTable with a fresh Index file and Bloom filter

Deletes are handled specially: Cassandra does not delete items right away. Instead, it adds a **tombstone** to the log. Eventually, when compaction encounters a tombstone, it deletes the item (eventually after compaction).

17.3.11. Read Operations

Reads are **similar to writes**, except:

- **Coordinator contacts X replicas** (e.g., in the same rack)
- Coordinator sends read to replicas that responded **quickest** in the past
- When X replicas respond, coordinator returns the **latest-timestamped value** from those X
- Coordinator also fetches value from **other replicas** in the background, checking consistency: initiating a **read repair** if any two values differ
- This mechanism seeks to **eventually bring all replicas up to date**

At a replica: reads look at **Memtables first**, then SSTables. A row may be split across multiple SSTables, so **reads touch multiple SSTables** → reads are slower than writes (but still fast).

17.3.12. Cluster Membership and Failure Detection

Any server in the cluster could be the coordinator, so **every server maintains a list of all other servers** currently in the cluster.



Gossip-based membership:

- Nodes periodically gossip their membership list
- On receipt, the local membership list is updated
- If any heartbeat is older than T_{last} , node is marked as failed

Accrual Failure Detector:

- Adaptively sets the timeout based on underlying network and failure behavior
- Outputs a value **PHI (φ)** representing suspicion
- $$PHI(t) = -\frac{\log(CDF \text{ or Probability}(t_{now} - t_{last}))}{\log} 10$$
- PHI determines the detection timeout, taking into account historical inter-arrival time variations for gossiped heartbeats
- In practice, $\varphi = 5 \rightarrow 10\text{--}15$ second detection time

17.3.13. Eventual Consistency and ACID vs. BASE



Eventual Consistency

If all writes stop to a key, then all its values (across replicas) will **converge eventually**. If writes continue, the system always tries to keep converging: a moving “wave” of updated values lagging behind the latest values sent by clients, but always trying to catch up. May still return **stale values** to clients (e.g., if many back-to-back writes). Works well when there are periods of low writes; system converges quickly.

Property	ACID (RDBMS)	BASE (NoSQL)
A	Atomicity	Basically Available
C	Consistency	Soft-state
I / E	Isolation	Eventual Consistency
D	Durability	-
Priority	Consistency over availability	Availability over consistency

17.3.14. Cassandra Consistency Levels

Cassandra allows clients to **choose a consistency level per operation** (any read/write):

- **ANY**: any server (may not be replica): fastest; coordinator caches write and replies quickly
- **ALL**: all replicas: slowest, but ensures strong consistency
- **ONE**: at least one replica: faster than ALL, but cannot tolerate a failure
- **QUORUM**: quorum across all replicas in all DCs: global consistency, but still fast
- **LOCAL_QUORUM**: quorum in coordinator DC: faster, only waits for quorum in first DC client contacts
- **EACH_QUORUM**: quorum in every DC: lets each DC do its own quorum, supports hierarchical replies



17.3.15. Quorums in Detail

Quorum

A **quorum** is a majority: $> 50\%$ of replicas. Any two quorums **intersect**: this guarantees that at least one node in each quorum has seen the latest write, so a read quorum always finds the most recent value.

Two necessary conditions for correct quorum operation:

1. $W + R > N$ (write count + read count $>$ total replicas)
2. $W > N/2$ (write majority)

Usage patterns:

- (W=1, R=1): very few writes and reads
- (W=N, R=1): great for read-heavy workloads
- (W=N/2+1, R=N/2+1): great for write-heavy workloads
- (W=1, R=N): great for write-heavy workloads with mostly one client writing per key

17.3.16. Performance: Cassandra vs. MySQL



Cassandra vs. MySQL on Large Data (> 50 GB)

- **MySQL**: Writes 300 ms avg, Reads 350 ms avg
- **Cassandra**: Writes 0.12 ms avg, Reads 15 ms avg

Cassandra is **orders of magnitude faster**. The catch: you lose ACID guarantees, strict consistency, and join support. The trade-off is explicit: Cassandra is a **BASE** system.

17.4. MongoDB

MongoDB

MongoDB is a **document-oriented NoSQL** database. Open source (written in C++), it provides in-memory access to data (data is stored on disk always/optionally, but loaded in memory for performance), native replication toward reliability and high availability (CAP), and **collection partitioning** by using a sharding key to keep information fast, available, and replicated.

17.4.1. MongoDB Data Model

MongoDB is based on **collections of documents**:

- A **collection** is a group of related documents with a shared common index
- Stores data in form of **BSON** (Binary JSON: Binary JavaScript Object Notation) documents
- BSON optimizes JSON (like trimming spaces, carriage return...): kind of a zip format

```
{
  name: "travis",
  salary: 30000,
  designation: "Computer Scientist",
```



```
teams: [ "front-end", "database" ]
}
```

Documents are schema-less: each document can have different fields.

17.4.2 MongoDB Queries

MongoDB uses a chainable query API:

- **Find:** `db.employee.find({salary:{$gt:18000}}, {name:1}).sort({salary:1})` : query all employees with salary > 18000, sorted ascending. The result of the query is a JSON array.
- **Insert:** `db.employee.insert({ name: "sally", salary: 15000, designation: "MTS", teams: ["cluster-management"] })`
- **Update:** `db.employee.update({salary:{$gt:18000}}, {$set:{designation:"Manager"}}, {multi:true})` : multi-option allows multiple document update
- **Remove:** `db.employee.remove({salary:{$lt:18000}})` : can accept a flag to limit the number of documents removed

👉 **Note:** Indexes speed up queries but slow down writes (every write updates all indexes associated with the collection; done in real time).

17.4.3 MongoDB Distributed Architecture

MongoDB uses a **sharded cluster** architecture:

- **Router (mongos):** accepts and routes incoming requests, coordinating with Config Server
- **Config Server:** stores collection-level metadata (which chunks are on which shards)
- **Shard:** stores data. Each shard is a **replica set** (typically 3 mongod servers that are mirrors of each other: one primary, others secondaries)

Sharding:

- Data split into **chunks** based on shard key (≈ primary key)
- Either use hash or range partitioning
- A shard is assigned to a replica set
- Shards are virtual: multiple shards can be on the same physical node (many techniques)

Pros: adding/removing shards, automatic balancing. **Cons:** max document size 16 MB; sharding and re-sharding are costly operations.

MongoDB uses ETCD (Raft) to keep consistency through routers: via metadata server. The real difference from Cassandra: Raft has a single leader while MongoDB has a leader per shard.

17.4.4 MongoDB Replication

Uses an **oplog (operation log)** for data sync:

- Oplog maintained at primary; delta transferred to secondaries continuously/every once in a while
- When needed, leader **Election Protocol** elects a master (Raft algorithm)
- Some MongoDB servers do not maintain data but can vote: called **Arbiters**

17.4.5 MongoDB Read Preferences and Write Concern

Read Preferences determine where to route read operations:

- **Primary (default):** read from primary (strongly consistent)
- **Primary-preferred:** primary if available, else secondary
- **Secondary:** read from secondary (may return stale data: eventually consistent)



- **Nearest:** lowest latency (useful to colocate application and DB on the same physical node; access in RAM)

Write Concern determines the guarantee MongoDB provides on write success:

- **Acknowledged** (default): primary returns answer immediately
- **Journalled:** write-ahead logging to an on-disk journal for durability
- **Replica-acknowledged:** quorum with a value of W

Weaker write concern → faster write time.

17.4.6. MongoDB Balancing and Consistency

Balancing: Over time chunks may grow larger than others:

- **Splitting:** upper bound on chunk size; when hit, chunk is split
- **Balancing:** migrates chunks among shards if there is an uneven distribution

Consistency:

- **Strongly Consistent:** Read Preference is Primary
- **Eventually Consistent:** Read Preference is Secondary (or Tertiary)
- **CAP Theorem:** with Strong Consistency, under partition, MongoDB becomes write-unavailable (thereby ensuring consistency)

Prof. Question: What happens if a partition grows too large in MongoDB? Is it a problem?

MongoDB **shards** by a shard key. When a **chunk** grows too big it is handled automatically: **Splitting** (the chunk is split in two) and **Balancing** (the balancer migrates chunks from overloaded shards to lighter ones). It becomes a **problem** if the **shard key is poor** (e.g. a monotonic timestamp sends every new chunk to one shard, a hot-spot) or if balancing floods the network. The shard key **cannot be changed** after the collection is sharded.

17.5. ACID vs. BASE: A Fundamental Trade-Off

The key architectural trade-off in distributed data storage is between **ACID** (correctness-first, RDBMS) and **BASE** (availability-first, NoSQL): NoSQL systems prefer availability over strict consistency, accepting stale reads that converge eventually.

Note: The full treatment of [ACID properties](#) (atomicity, 2PC, nested transactions) and [BASE](#) (basically available, soft-state, eventual consistency), including the CAP theorem and eBay's internet-scale principles, is in the **Cloud and Data Center Global Strategies** chapter.

17.6. Big Data Infrastructure Properties

The resource management taxonomy (runtime/static issues, technical and user-facing requirements) is shared across all Big Data infrastructure and is consolidated in the Data Batching chapter.

TiKV (a distributed key-value store used in TiDB) represents an advanced evolution in this space:

- **ACID guarantees**
- **RAFT distributed for leader election** → quorum
- **Geographic distribution**
- **Key-value with sequential keys** → fragmentation
- **Region: partitioning on nodes (continuous key intervals), automatic via PD (Placement Driver):** single thread, possible bottleneck with many regions
- **CAP: Consistency and Partitioning chosen**
- **Two-phase commit**



18. DATA BATCHING

Package: Data Batching - 12 - Data_batching 24.pdf

Processing **Big Data** requires a fundamental **shift in mindset**: moving from small-scale, single-machine analysis toward distributed, parallel approaches designed for massive scale. The central questions are: what are the requirements of Big Data systems, and what operations must they support?

Big Data analysis settings that are **very common and easy to offer as services** include:

- **Big Data storage, access and management**
- **Data batch processing**
- **Stream data processing**

Data and any service operating on it are treated **as the input** to processing pipelines.

18.1. Big Data Characteristics and Requirements



Big Data System Properties

Big Data environments are characterized by **enormous data volume** and must satisfy:

- **Distribution and Decentralization**: data lives across many nodes
- **Scalability**: must grow with data and users
- **Efficiency**: high throughput at low cost
- **Quality of Service**: reliable, observable processing



System Requirements for Big Data Support

- **Long life cycle** (tending to infinity): the system runs continuously
- **Open source**: community-driven, no vendor lock-in
- **Interoperability and standards**: no lock-in
- **Remote control**: dashboard for monitoring
- **Transparency and visibility**: black-box simplicity for users, observability for operators

18.2. Batch Data Processing in Large Clusters



Why Why Batch??

It is often paramount to **automatically process a very large set of data of a specified dimension** so as to provide **fast results for a search**. This is very common in big data installations. Map-Reduce batch, published in 2004, is an excellent mechanism for obtaining a **high-throughput result in a scalable, reliable, and maintainable way**.

Pioneer examples - **UNIX batch**: simple log analysis at system level. UNIX batch uses input as immutable and produces output on demand. Parallelism was not a primary goal in UNIX batch (data not so large).



18.2.1. Data Parallelism in Today's Large Clusters

Modern workloads exhibit excellent data parallelism:

- Data (e.g., web pages crawled by Google for indexing, documents) can be analyzed **independently**
- One program commonly runs on **thousands of nodes** processing enormous amounts of data



The Bottleneck

Unlike traditional HPC, in Big Data workloads:

- **Communication overhead is not the dominant cost** compared to overall execution time
- Tasks access **disks frequently** and run complex algorithms
- Access to data and computation time dominates execution time
- **Data access rate can become the bottleneck**: disk I/O, not CPU, limits throughput

Analogy: HPC vs. Big Data

*Traditional HPC (High-Performance Computing) focused on raw compute parallelism with special-purpose languages. Big Data flipped the problem: the bottleneck is **getting data to compute**, not the computation itself. This is why frameworks like MapReduce emerged: to solve the data-access and distribution problem, not just the CPU parallelism problem.*

18.3. MapReduce: Programming Model



MapReduce

MapReduce is a **programming framework** that provides a high-level API to specify parallel tasks while the runtime automatically handles:

- Automatic parallelization and scheduling
- Load balancing
- Fault tolerance
- I/O scheduling
- Monitoring and status updates

Everything runs on top of a **distributed file system** (GFS/HDFS).

The key promise: engineers can **focus only on the application logic and parallel tasks**, without dealing with scheduling, fault-tolerance, or synchronization.

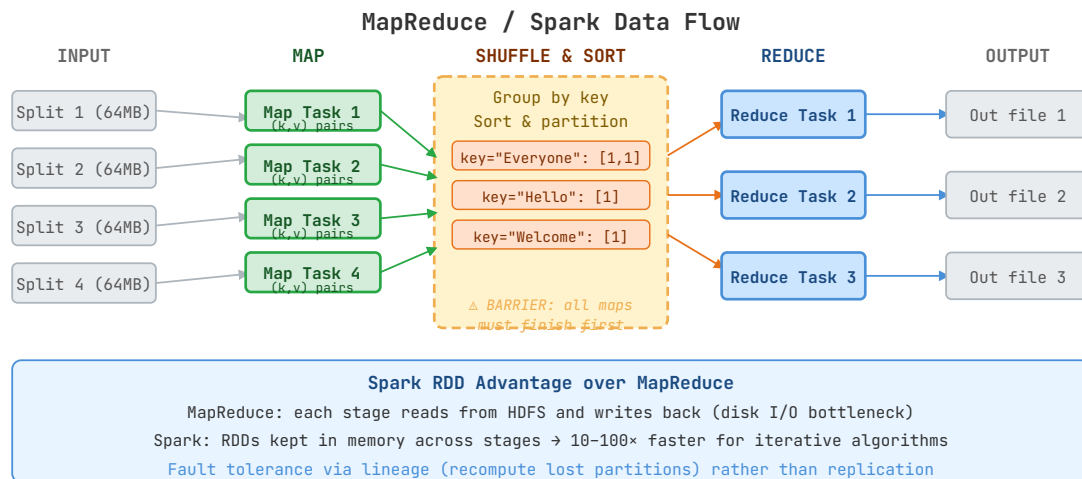


Figure 95 – MapReduce data flow: input splits → parallel Map tasks → Shuffle & Sort (global barrier) → parallel Reduce tasks → output. Spark avoids the disk I/O bottleneck by keeping RDDs in memory across stages.

18.3.1. Functional Language Origins

MapReduce borrows its semantics from **functional languages** (LISP, Scheme): a sequence of two complementary steps for parallel exploration and result harvesting. The same concepts appear in Python, Perl, Java, and others as built-in `map` and `reduce` functions.

Map (distribution phase)

1. **Input:** a list of data and one function
2. **Execution:** the function is applied to each list item independently
3. **Result:** a new list with all results of the function

In the Big Data context: map processes each record to generate **intermediate key/value pairs**.

Reduce (result harvesting phase)

1. **Input:** a list and one function
2. **Execution:** the function combines/aggregates the list items
3. **Result:** one new final item

In the Big Data context: reduce merges all intermediate values **associated per key**.



Sum of Squares

```
map(square, [1, 2, 3, 4]) ⇒ [1, 4, 9, 16]
reduce(add, [1, 4, 9, 16]) ⇒ 30
```

`map` processes each record **sequentially and independently**; `reduce` processes the **set of all records in batches**.



18.3.2. The Google MapReduce Definition

In the Google formulation:

- `map(String key, String val)` runs on each item in the set: input is a set of files, keys are **file names**, values are **file contents**. The function emits (**new-key, new-val**) pairs. The size of the output set can differ from the input.
- `reduce(String key, Iterator vals)` runs for each **unique key** emitted by map. It is possible to have more values for one key. Emits **final output pairs** (possibly smaller than the intermediate set).
- The runtime aggregates the output of map by key (the **shuffle and sort** phase) before calling reduce.

👉 **Note:** Keys and values can have different types: the programmer converts between Strings and appropriate types inside `map()`. The runtime takes care of grouping: all values for the same key are sent to the same reducer.

18.3.3. Map Phase in Detail

The MAP phase runs **in parallel** across a large number of records:

- Each **Map Task** processes a subset of the input data
- Each record is processed independently (**no data dependencies between records**)
- Output: intermediate `(key, value)` pairs



Word Count: Map

Input: `<filename, file text>` containing "Welcome Everyone / Hello Everyone"

Map emits: `(Welcome, 1), (Everyone, 1), (Hello, 1), (Everyone, 1)`

18.3.4. Reduce Phase in Detail

The REDUCE phase merges all intermediate values per key:

- Each key is assigned to exactly one Reduce task
- Reduce tasks run **in parallel by partitioning keys**
- Popular splitting: **hash partitioning** - `reduce# = hash(key) % number_of_reduce_tasks`



Word Count: Reduce

Intermediate pairs: `(Welcome,1), (Everyone,1), (Hello,1), (Everyone,1)`

After reduce: `(Everyone, 2), (Hello, 1), (Welcome, 1)`



Barrier Between Map and Reduce

Map and aggregation (shuffle+sort) must finish before Reduce can start. This is a global barrier. The runtime aggregates intermediate values by output key, then distributes key groups to reducers. This barrier is the fundamental constraint on latency: the entire map phase must complete before results begin to emerge.



18.3.5. Running a MapReduce Program

The user provides a **specification object** containing:

- **Input/output file names**
- **Optional tuning parameters** (e.g., size to split input/output into)

The user defines a **MapReduce function** and passes it the specification object. The **runtime system** calls `map()` and `reduce()`: the user only specifies the operations, not the parallelization.

18.3.6. Word Count Example (Full Code)

```
map(String input_key, String input_value):
    // input_key: document name
    // input_value: document contents
    for each word w in input_value:
        Emit.Intermediate(w, "1");

reduce(String output_key, Iterator intermediate_values):
    // output_key: a word
    // output_values: a list of counts
    int result = 0;
    for each v in intermediate_values:
        result += ParseInt(v);
    Emit(AsString(result));
```

18.3.7. Other MapReduce Applications



MapReduce is General-Purpose

- **Distributed grep**: `map()` emits a line if it matches a pattern; `reduce()` is an identity function
- **Distributed sort**: `map()` extracts a sorting key and outputs (key, record) pairs; `reduce()` is identity: the actual sort is done automatically by the runtime
- **Reverse web-link graph**: `map()` emits (target, source) pairs for each link to a target URL found in a source file; `reduce()` emits (target, list(source))
- **Machine learning** (clustering, classification)
- **Google news clustering, popular query extraction** (Zeitgeist)
- **Processing satellite imagery data**
- **Graph computations, language models for machine translation**
- **Google rewrote its indexing code in MapReduce** (used until 2011)

18.4. MapReduce Implementation and Architecture

18.4.1. Implementation at Google

- Large clusters of PCs connected with **Gigabit Links**
- 4–8 GB RAM per machine, dual x86 processors
- Network bandwidth often significantly less than 1 GB/s
- **Machine failures are common** due to the large number of machines



- **GFS** (Google File System): distributed file system managing data; storage provided by cheap IDE disks attached to machines
- **Job scheduling system**: jobs composed of tasks, scheduler assigns tasks to machines
- Implementation is a C++ library linked into user programs

18.4.2. Architecture: Master and Workers

The MapReduce execution follows a **Master/Worker** (Farm) pattern:

1. User program **forks** master and worker processes
2. **Master** assigns map and reduce tasks to free workers
3. Map workers **read input splits** from GFS and write intermediate results to local disk, divided into R regions
4. Reduce workers **remotely read** intermediate data from map workers
5. Reduce workers **write final output** to GFS

👉 **Note**: Input data is split into M map tasks (typically 64 MB per split). Reduce phase is partitioned into R reduce tasks. Typical values: M=200,000; R=4,000; workers=2,000.

18.4.3. Scheduling and Execution

Master assigns each map task to a free worker:

- Considers **locality of data**: prefers putting map tasks on the same machine (or rack) as the input replica
- Workers read input often from **local disk**: avoids network traffic
- Intermediate key/value pairs written to **local disk**, divided into R regions; region locations passed to master

Master assigns each reduce task to a free worker:

- Worker reads intermediate k/v pairs from map workers via **remote read**
- Worker applies the user reduce function and stores output in GFS

18.4.4. Favouring Data Locality

🤖 **Why** Why Data Locality Matters?

GFS stores data files divided into **64 MB blocks** with **3 replicas** on different machines. The master schedules map tasks **based on the location of the replicas**: placing map tasks **physically on the same machine** as one of the input replicas (or at least the same rack). This way, machines can read input at local disk speed. Otherwise, rack switches would limit read rate and waste network bandwidth.

18.4.5. Fault Tolerance

On master failure:

- State is checkpointed to GFS; new master recovers and continues

On worker failure: (detected via periodic heartbeats)

- Both **completed** and **in-progress** map tasks on that worker are re-executed (output stored on local disk: inaccessible after failure)
- Only **in-progress** reduce tasks need re-execution (completed reduce output is in GFS: globally accessible)



Robustness Example

Google ran a sort program on 1800 machines and lost 1600 of them partway through: the job still completed successfully.

18.4.6. Backup Tasks (Stragglers)



The Straggler Problem

Stragglers (slow workers finishing last) can significantly lengthen total completion time. Causes include: other jobs consuming resources, bad disks with soft errors (slow correctable transfers), disabled processor caches at machine init.

Solution: close to completion, spawn **backup copies** of the remaining in-progress tasks. Whichever copy finishes first wins. Additional cost: a few percent more resource usage. Result: a sort program **without backup tasks** was **44% longer**.

18.5. Hadoop: The Open-Source MapReduce



Apache Hadoop

Hadoop is an **open source platform for MapReduce by Apache**. It started as open source MapReduce written in Java but evolved to support other Apache languages such as Pig and Hive.

Core subprojects:

- **Hadoop Common:** set of utilities (FileSystem, RPC, serialization libraries)
- **HDFS** (Hadoop Distributed File System)
- **MapReduce:** the processing engine
- **YARN** (Yet Another Resource Negotiator): cluster resource management

18.5.1. YARN Resource Manager



YARN

YARN (Yet Another Resource Negotiator) provides **management for virtual Hadoop clusters over a large physical cluster**. It treats each server as a collection of **containers**, where a container = fixed CPU + fixed memory (think Linux cgroups but even lighter).

YARN responsibilities:

- Handles **node allocation** in a cluster
- Supplies new nodes with configuration
- Distributes Hadoop to allocated nodes
- Starts Map/Reduce and HDFS workers
- Includes management and monitoring

Other resource managers are available, such as **Apache MESOS**.



18.5.2. YARN Architecture

YARN has three main components:

1. **Global Resource Manager (GRM)**: single node that:
 - Globally allocates the required resources
 - Contains the **Scheduler** and **ApplicationsManager**
2. **Application Master (AM)**: per-application (per job):
 - Container negotiation with Resource Manager and Node Managers
 - Detecting task failures for that job
3. **Per-server Node Manager (NM)**:
 - Daemon with server-specific functions that manage local resources
 - Instantiates containers to run tasks
 - Monitors container resource usage

18.5.3. YARN Workflow

A typical YARN job submission follows these steps:

1. Client submits job (copies job resources to HDFS)
2. Client requests application from Resource Manager
3. Resource Manager requests container for Application Master
4. Node Manager starts Application Master
5. AM retrieves input data from HDFS
6. AM requests resource allocation from Scheduler
7. Node Manager starts container with Map/Reduce task
8. Task retrieves job resources from HDFS
9. Results stored back to HDFS

Hadoop extensions (out of primary scope): Avro (serialization), Chukwa (log collection), HBase (structured data storage for large tables), Hive (data warehousing, Facebook), Pig (parallel SQL-like, Yahoo), ZooKeeper (coordination), Mahout (ML/data mining), Sahara (deployment on OpenStack).

18.5.4. Hadoop on OpenStack (Sahara)

Hadoop can exploit **OpenStack virtualization** for more flexible clusters and better resource utilization. OpenStack's **Sahara** service allows deploying and configuring Hadoop clusters in a Cloud environment, adding:

- **Cluster scaling functions**
- **Analytics as a Service (AaaS) functions**

Sahara is accessible via dashboard, CLI, or RESTful API.

18.6. Apache Spark

18.6.1. Why Spark?

MapReduce greatly simplified Big Data analysis, but as it became popular, users wanted more:

- **More complex, multi-stage applications** (e.g., iterative graph algorithms and machine learning):
MapReduce chains require writing intermediate results to disk between every job
- **More interactive ad-hoc queries**

Both multi-stage and interactive apps require faster **data sharing across parallel jobs**.



⚠ **MapReduce Data Sharing Problem**

In MapReduce: iterative jobs require HDFS read → process → HDFS write → HDFS read → process → ... for every iteration. Interactive queries each need a fresh HDFS read. Both are **slow due to replication, serialization, and disk I/O**. This is the fundamental bottleneck MapReduce cannot solve.

📖 **Apache Spark**

Spark is not a modified version of Hadoop but a separate, fast, MapReduce-like engine, optimized for in-memory processing. It provides:

- In-memory data storage for very fast iterative queries
- General execution of graphs and powerful optimizations
- Up to 40× faster than Hadoop for iterative workloads
- Compatible with Hadoop storage APIs (HDFS, HBase, SequenceFiles, etc.)

Analogy: Spark vs. Hadoop Data Sharing

Hadoop is like passing notes by printing them, distributing copies, collecting them, shredding them, and printing new ones for each step. Spark keeps the notes in RAM, passing them directly between steps. First read from disk is unavoidable, but subsequent iterations are 10-100× faster.

18.6.2. Spark Basics

Spark offers various types of data processing computations in **one single tool**:

- Batch/streaming analysis, interactive queries, and iterative algorithms
- Previously these required several different and independent tools

Supports several storage options and streaming inputs for parsing. APIs available in Java, Scala, Python, R.

18.6.3. Resilient Distributed Datasets (RDDs)

The key data abstraction in Spark is the **RDD**:

📖 **Resilient Distributed Dataset (RDD)**

An **RDD** is a distributed, immutable collection of objects that is:

- Maintained in memory (when possible)
- Distributed across cluster nodes
- Immutable: transformations create new RDDs, not modified ones
- Can be cached in memory across cluster nodes for reuse

RDDs achieve fault tolerance through **lineage** rather than replication.

🎓 **Prof. Question:** What is an RDD? Why is it important?

An RDD (Resilient Distributed Dataset) is Spark's core data unit: a **distributed, immutable, fault-tolerant** collection partitioned across workers. **Resilient**: on a worker failure it is recomputed from its lineage (the chain of transformations), so no replication is needed. **Distributed**: partitioned for parallelism. **Dataset**: an abstraction over data wherever it lives. It enables lazy, optimized evaluation.



18.6.4. RDD Operations

Two kinds of operations on RDDs:



Transformations (lazy)

Act on existing RDDs by **creating new ones**. Similar to Hadoop map tasks. **Lazily evaluated**: no computation happens until an action is triggered. Examples: `map`, `filter`, `groupByKey`, `sort`, `join`, `union`, `reduceByKey`, `groupByKey`, `partitionBy`, `cogroup`, `cross`, `leftOuterJoin`, `rightOuterJoin`, `sample`.



Actions (eager)

Return **results** from input RDDs. Similar to Hadoop reduce tasks. **Force immediate evaluation** of all pending transformations in the input RDD. Examples: `reduce`, `count`, `first`, `take`, `save`, `pipe`.

100 Prof. Question: *Difference between a Transformation and an Action in Spark?*

A **Transformation** (`map`, `filter`, `flatMap`) is **lazy**: it builds a new RDD but runs nothing yet. An **Action** (`count`, `collect`, `save`) is **eager**: it triggers execution of the accumulated DAG and returns a result.



Lazy Evaluation

```
val lines = sc.textFile("data.txt")           // Transformation
val lineLengths = lines.map(s => s.length)    // Transformation
val totalLength = lineLengths.reduce((a, b) => a + b) // Action
```

Until the third line (the action), **no operation is performed**. The `reduce()` action forces a read from the text file and the `map()` transformation.

18.6.5. Persisting RDDs

By default, all transformations are recomputed on **every action requested**. This can be expensive. Using the `persist()` method, the RDD data (read and mapped) is **saved for future actions**:

```
val lines = sc.textFile("data.txt")
val lineLengths = lines.map(s => s.length)
lineLengths.persist()
```

👉 Note: Persisting is the key to Spark's speed advantage for iterative algorithms: load data once into memory, then iterate over it many times without re-reading from disk.



18.6.6. Fault Tolerance via Lineage

RDD Lineage

RDDs track the series of transformations used to build them (their **Lineage**) to re-compute lost data. If a partition is lost (node crash), Spark recomputes only that partition from the original source using the lineage graph: without needing to replicate all data.

Lineage Graph

```
messages = textFile(...).filter(_.contains("error"))
               .map(_.split('\t')(2))
```

Lineage: `HadoopRDD (path=hdfs://...)` ← `FilteredRDD (func=_.contains(...))` ← `MappedRDD (func=_.split(...))`

If MappedRDD is lost, Spark retraces from HadoopRDD.

18.6.7. Spark Performance: Why It Wins on Iterative Workloads

Logistic Regression: Spark vs. Hadoop

Iterative ML algorithm (gradient descent):

```
val data = spark.textFile(...).map(readPoint).cache()
var w = Vector.random(D)
for (i ← 1 to ITERATIONS) {
  val gradient = data.map(p =>
    (1 / (1 + exp(-p.y*(w dot p.x))) - 1) * p.y * p.x
  ).reduce(_ + _)
  w -= gradient
}
```

Data is loaded once and cached. Each iteration reuses the in-memory RDD.

Results: Hadoop 127s/iteration; Spark 174s first iteration (loading), then 6s for further iterations: 20× speedup at 30 iterations.

PageRank performance: Hadoop 171s/iteration, Basic Spark 72s/iteration, Spark + Controlled Partitioning 23s/iteration. Controlled data partitioning avoids unnecessary shuffling by keeping related data co-located across iterations.

18.6.8. Other Spark Engine Features

- General graphs of operators (e.g., map-reduce-reduce chains), not just two-phase pipelines
- Hash-based reduces: faster than Hadoop's sort-based approach
- Controlled data partitioning adapted to lower communication overhead



18.6.9. Spark Architecture

Spark Architecture

Spark programs create **Directed Acyclic Graphs (DAGs)** of all transformations and actions, internally optimized for execution. The graph is split into **stages**, composed by **tasks** (the smallest unit of work).

The support is a **master/slave system**:

- **Driver**: central coordinator node running the `main()` method of the program, dispatching tasks
- **Cluster Master**: launches and manages actual executors
- **Executors**: responsible for running tasks; each spawns at least one dedicated JVM with an assigned share of CPU threads and RAM memory

 **Prof. Question**: *What is the base architecture of Apache Spark?*

A **master/worker** architecture. The **Driver** builds the **DAG** of lazy transformations on RDDs. When an **Action** is called, the DAG is optimized and split into **Stages**, each split into **Tasks** (one per partition). The **Cluster Manager** allocates **Executors** (JVMs), which run the tasks and return results to the Driver.

18.6.10. Spark Deployment Modes

Spark can be deployed:

- **Standalone cluster**: its own cluster master independently launches and manages executors
- **Hadoop YARN**: relies on YARN for resource management (already seen above)
- **Apache MESOS**: fine-grained sharing, richer scheduling queues

External resource managers provide richer functionalities (scheduling queues, multi-tenancy) not available in standalone mode.

18.6.11. Spark Provisioning and Output

Spark can produce **very large results**. Managing large aggregations (e.g., document at a URL) requires automation. The experience points toward storing produced batch data in **NoSQL repositories** for scalable access.

18.7. The Big Data Tools Ecosystem

The Big Data processing landscape organizes along two dimensions: the **computational model** and the **use case**:

Use Case	DAG Model	MapReduce Model	Graph Model	BSP/Collective
Iterations / Learning	Spark, Dryad/ DryadLINQ	Hadoop, HaLoop, Twister, Spark	Gi- raph, GraphLab, GraphX, Hama	MPI, Harp
Query	Drill, Dryad	Pig/PigLatin, Hive, Tez, Shark, MRQL	-	-



Use Case	DAG Model	MapReduce Model	Graph Model	BSP/Collective
Streaming	S4, Samza	Storm, Spark Streaming	-	-

A layered architecture view shows these tools sitting above distributed file systems (HDFS), resource managers (YARN, MESOS), and cloud infrastructure (OpenStack, AWS, Azure), with cross-cutting capabilities like in-memory databases, NoSQL stores, ORM/mapping, and DevOps/deployment tooling.

18.8. Big Data Resource Analysis

18.8.1. Resources as Unifying Concepts

All Big Data systems ultimately deal with the same **resource management issues**, categorized by when they must be handled:

Runtime issues (dynamic, in-band):

- Resource Sharing (multicast)
- Resource Distribution (events)
- Resource Synchronization
- Resource Replication
- Resource Control

Static issues (before runtime):

- Resource Configuration
- Resource Timing

18.8.2. IT Properties Required

For Big Data and cloud systems to function reliably at scale, the following technical properties are required:

- **Dynamicity and adaptability:** systems must react to changing conditions
- **Fault tolerance or Replication:** availability and reliability
- **Loose Consistency:** CAP theorem trade-offs accepted for scale
- **Group communication:** coordinating many nodes
- **Data configuration and access**
- **Resource life cycle support**

And cross-cutting concerns:

- **Transparency:** hide distribution complexity from users
- **Low intrusion:** management overhead must be minimal
- **Time awareness:** event ordering, timeouts, expiry
- **Simplicity:** the dominant design goal; complexity kills adoption

The MapReduce/Spark Lesson

The success of MapReduce and Spark is not just about performance: it is about **abstraction and simplicity**. By hiding fault tolerance, scheduling, data movement, and parallelism behind a clean API, these frameworks allow engineers to write Big Data programs without expertise in distributed systems. The internal complexity is real; the user-visible model is not.